

香港中文大學
The Chinese University of Hong Kong

Formal Languages and Automata Theory

Lecture 2

何宗易
Tsung-Yi Ho
tyho@cse.cuhk.edu.hk

Outline



Deterministic Finite Accepters (DFA)



Nondeterministic Finite Accepters (NFA)



Equivalence of DFA and NFA

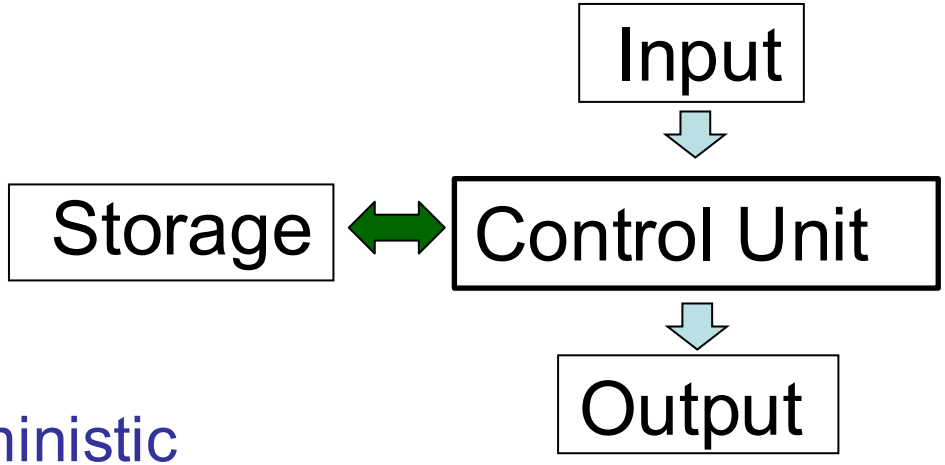


Reduction of the Number of States in FA*

Automata

Automaton:

An abstract model of a digital computer



•Deterministic V.S. Nondeterministic

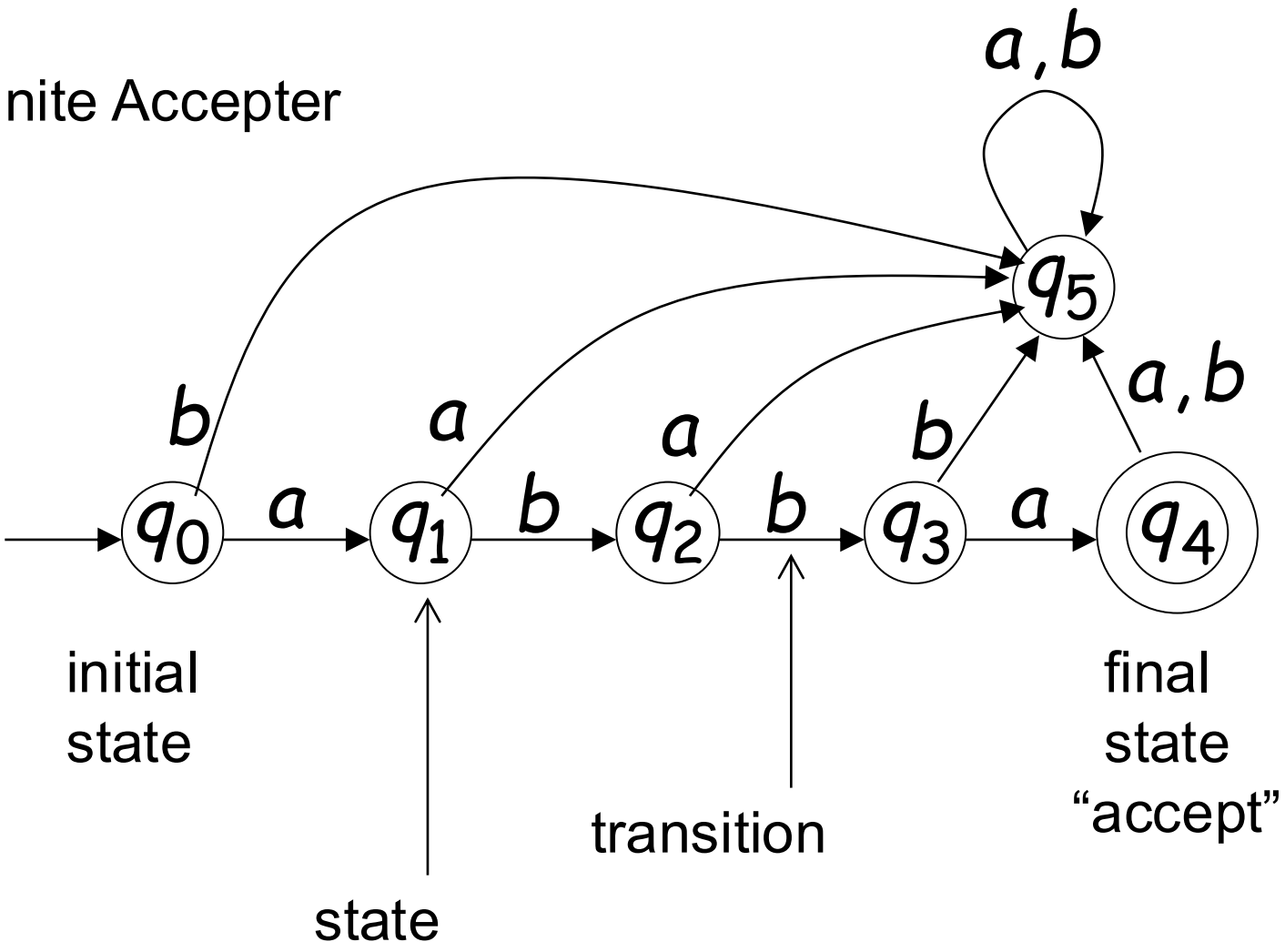
•An automaton whose output is YES or NO

Accepter

•An automaton whose output are strings of symbols

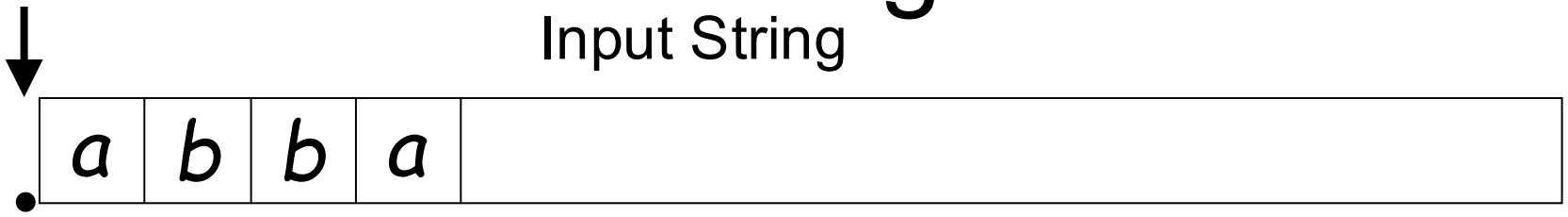
Transducer

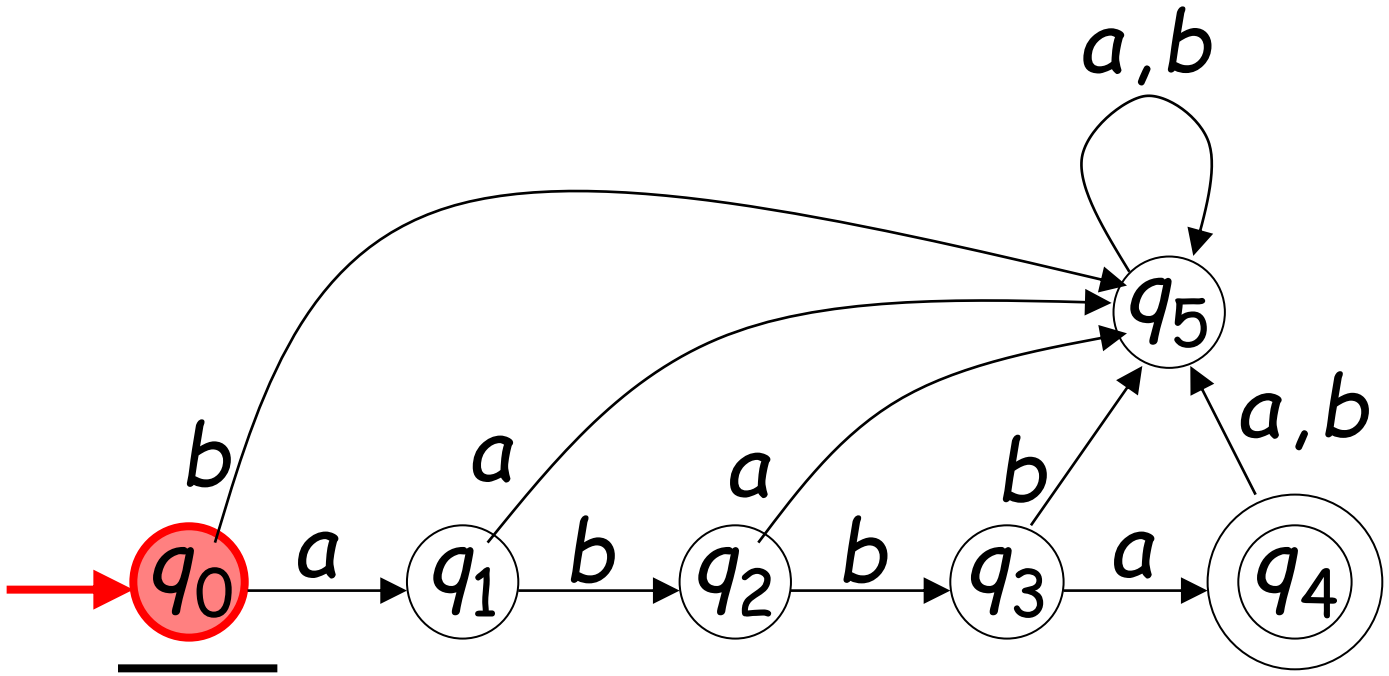
Transition Graph



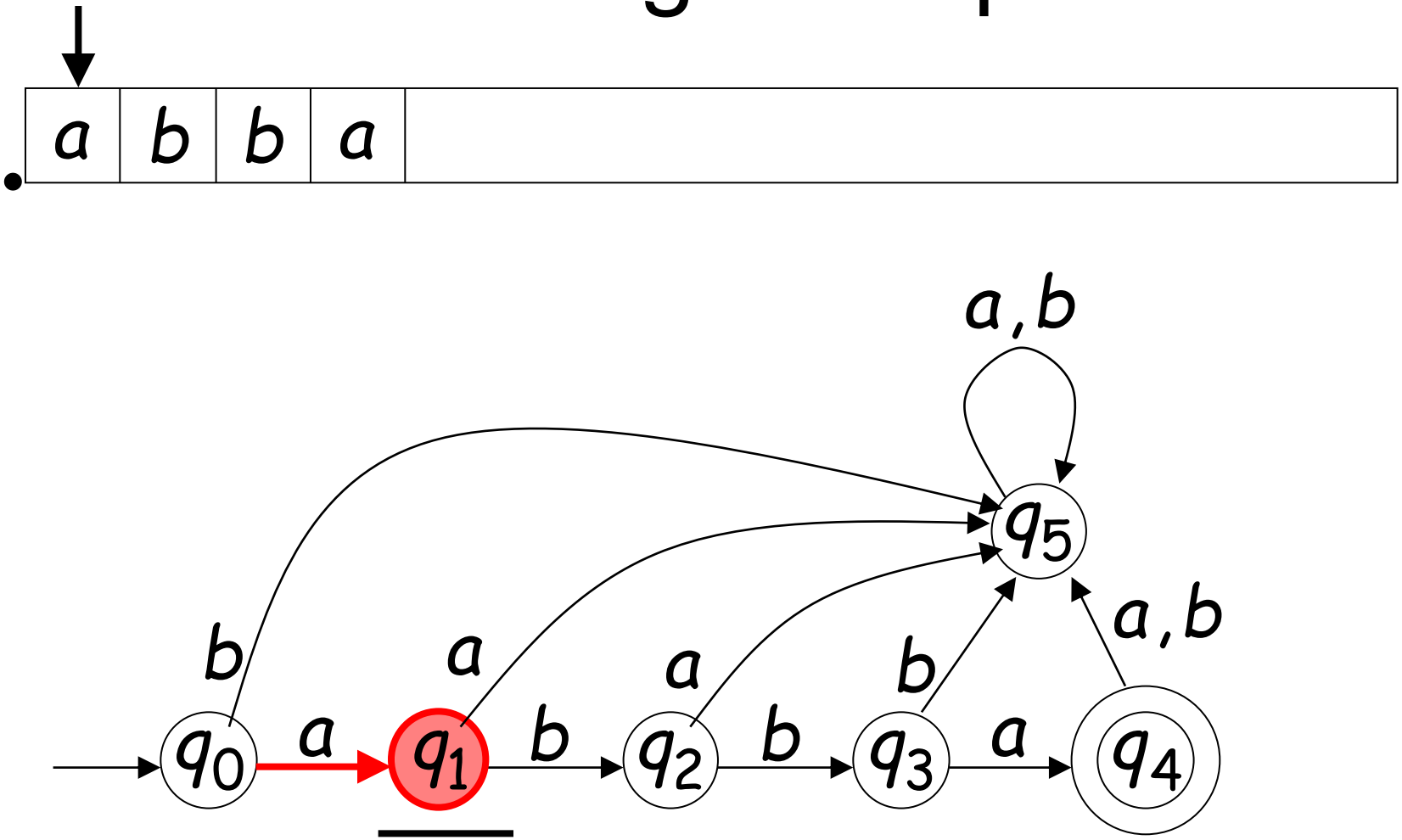
Initial Configuration

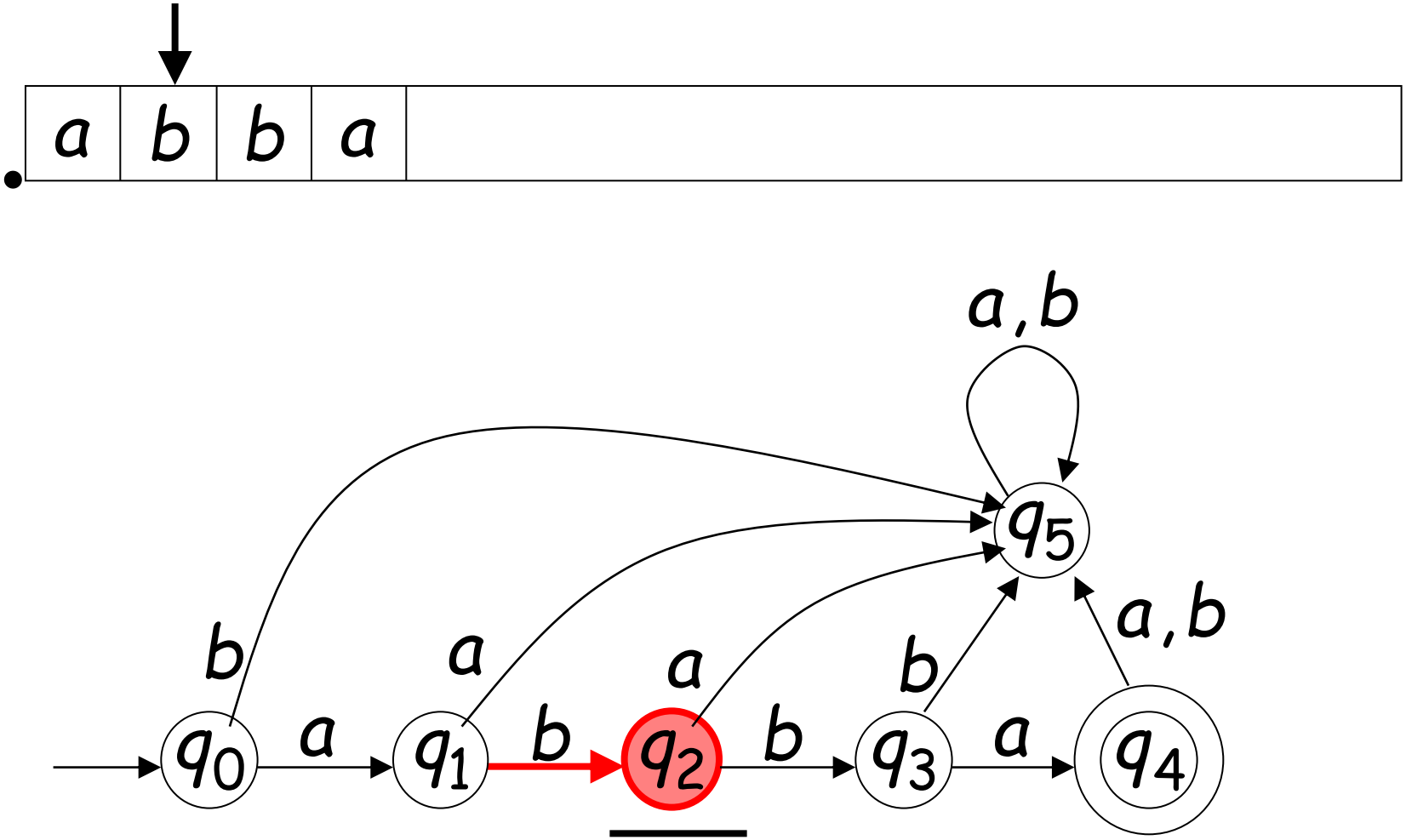
Input String

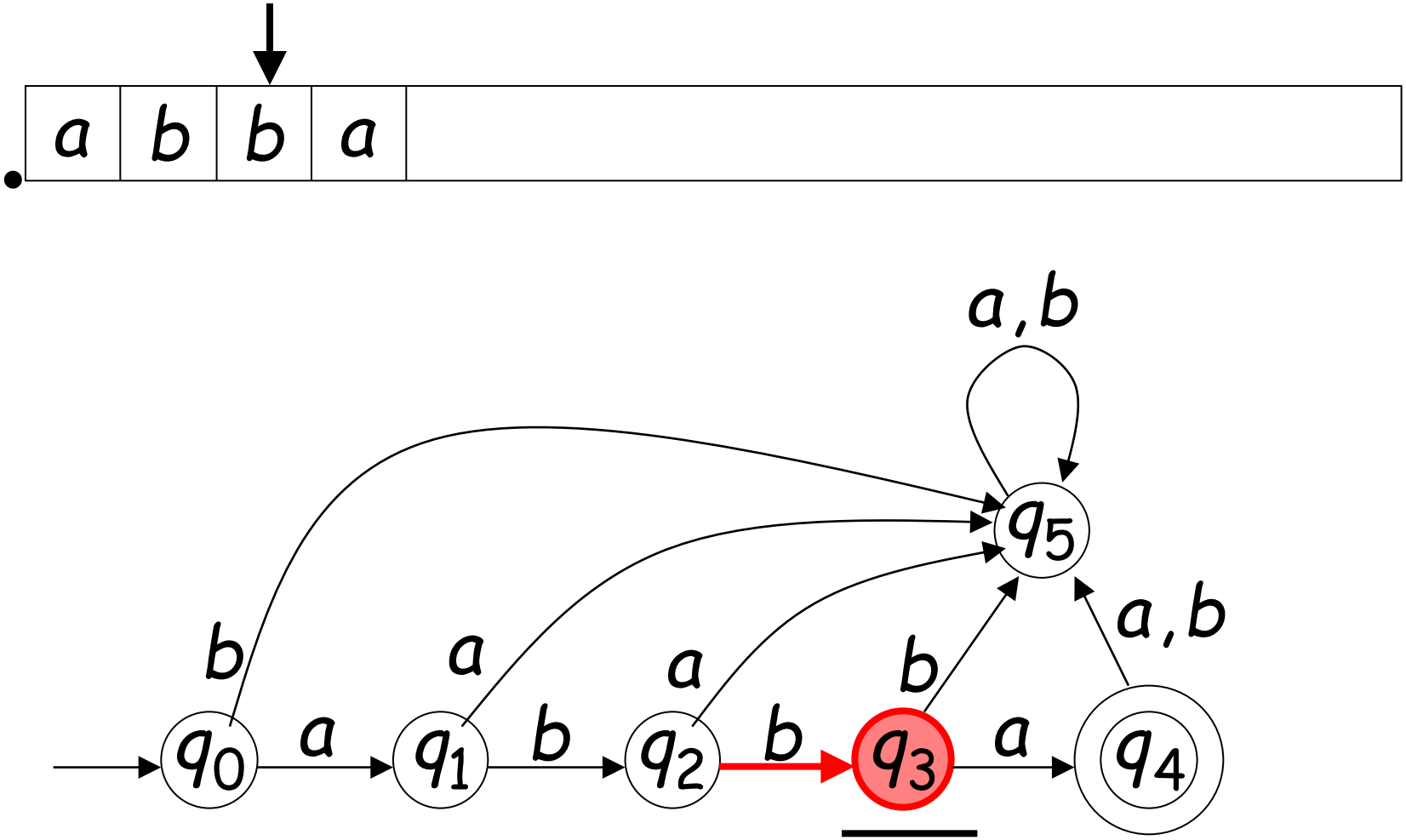


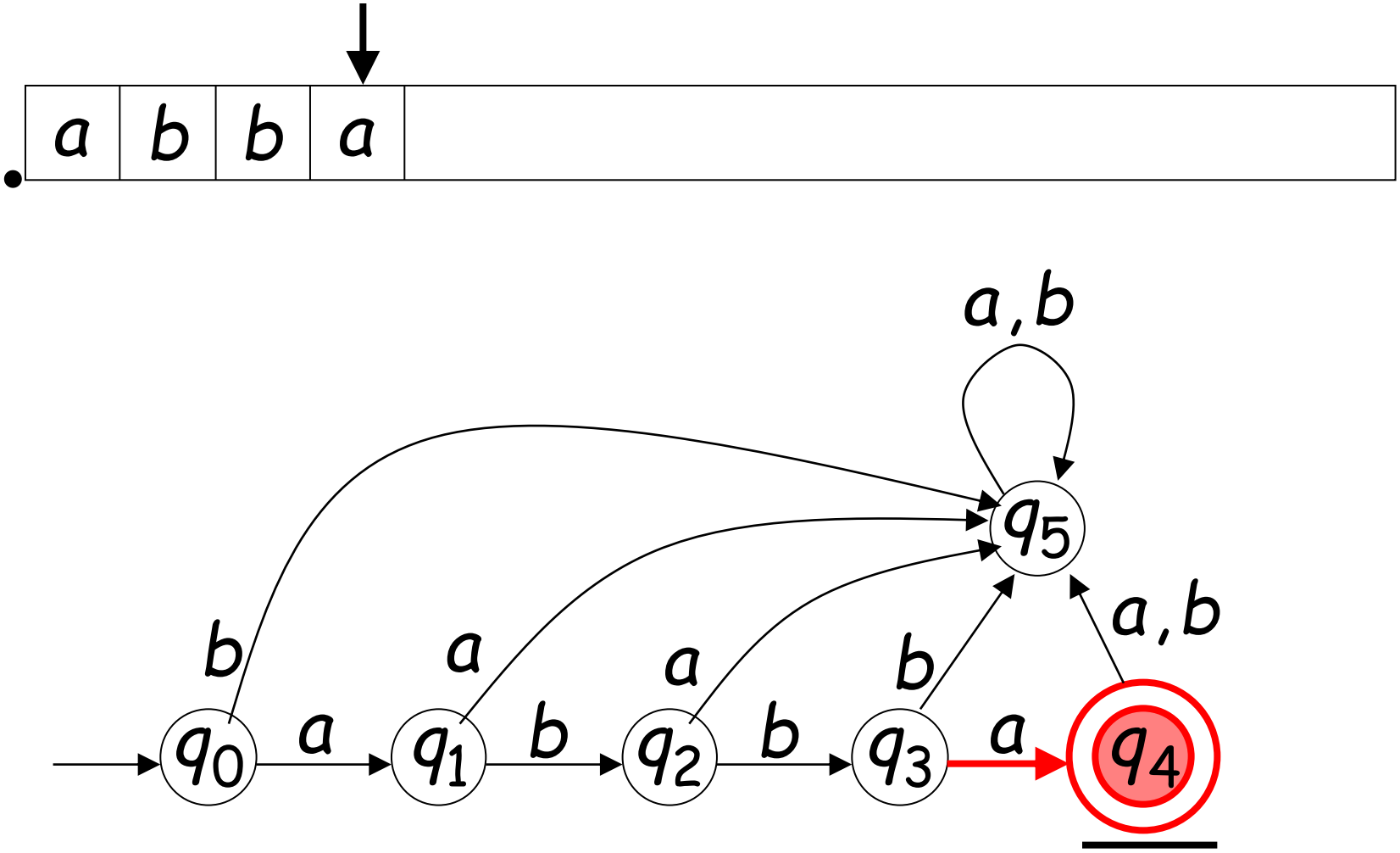


Reading the Input

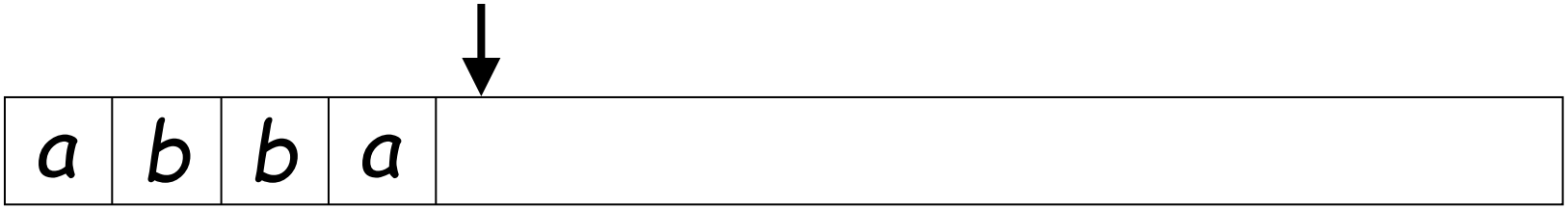


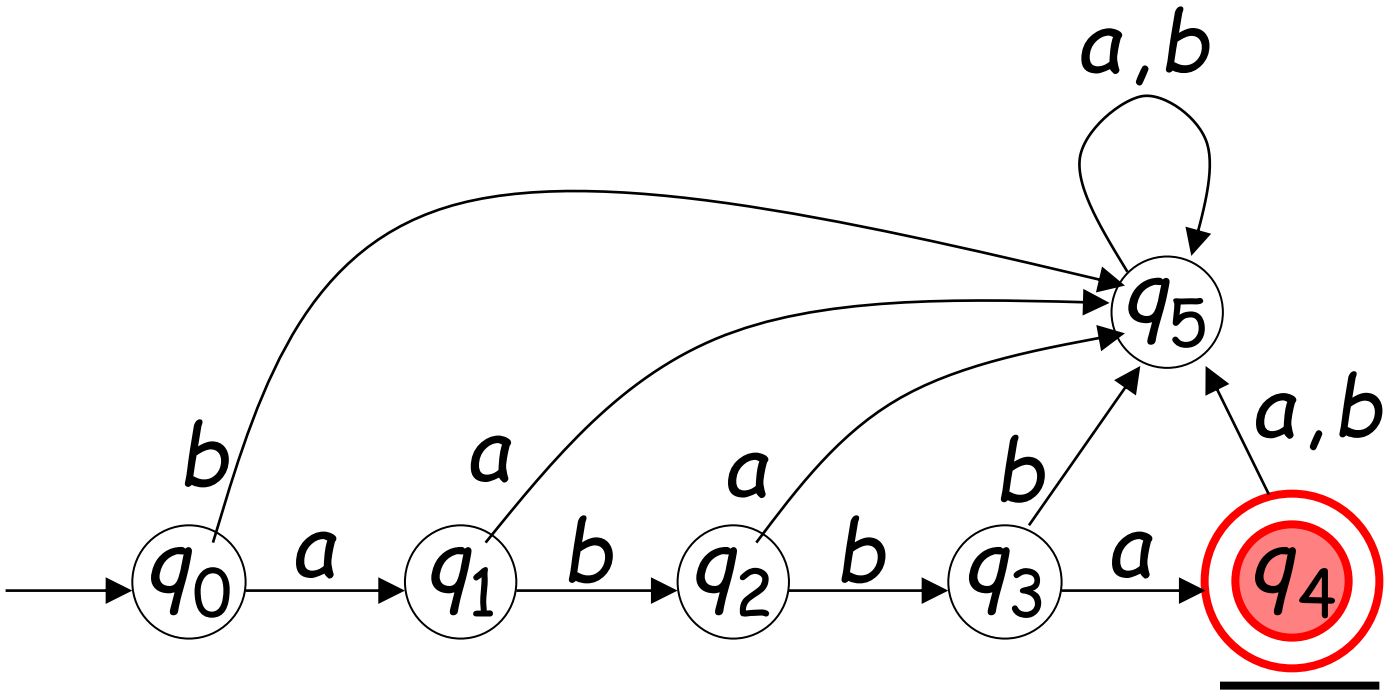

a b b a
a,b
q5
b
a
a
b
a,b
q0
a
q1
b
q2
b
q3
a
q4


a b b a
a,b
q5
b
a
a
b
a,b
q0
a
q1
b
q2
b
q3
a
q4


a b b a
a,b
q5
b
a
a
b
a,b
q0
a
q1
b
q2
b
q3
a
q4

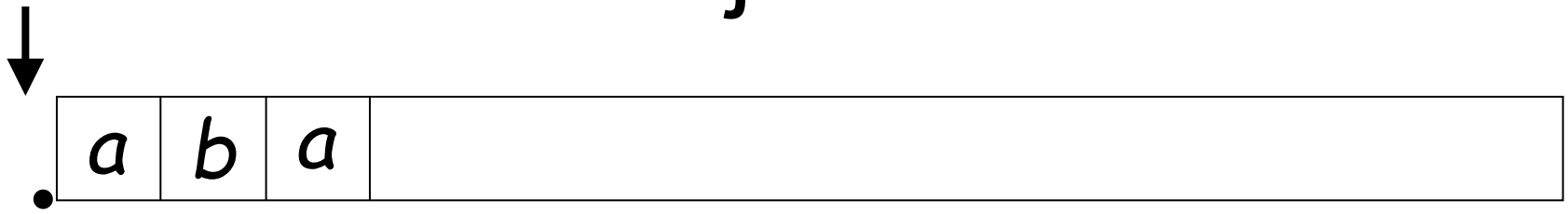
Input finished

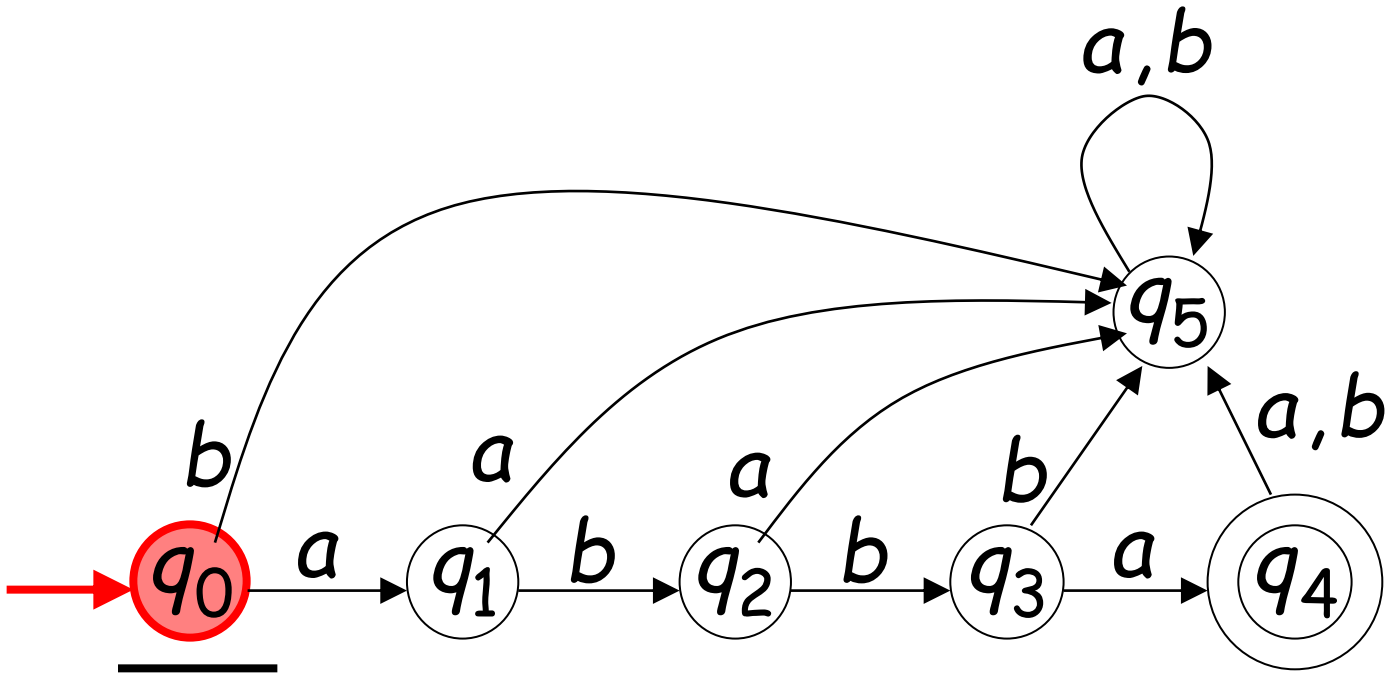


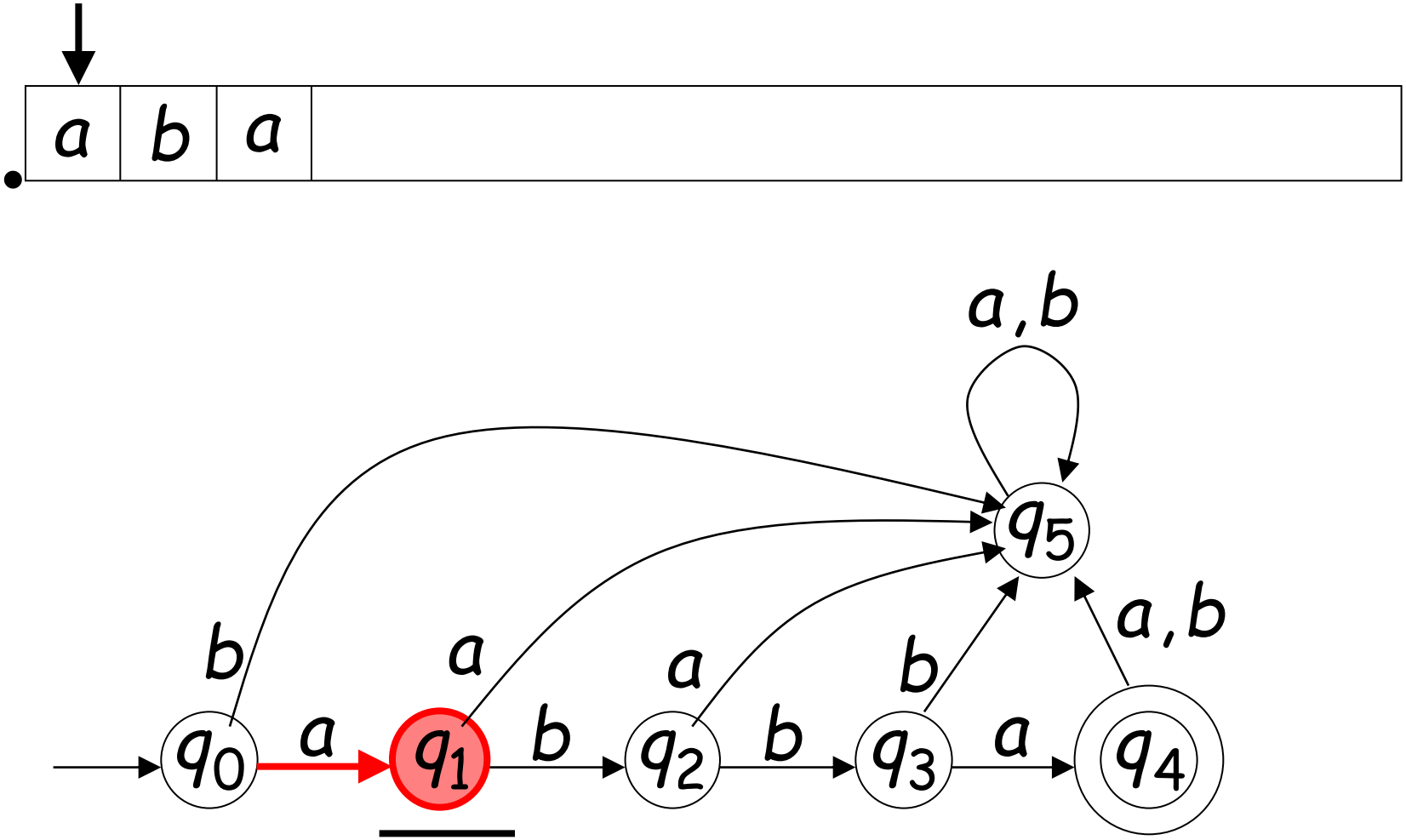


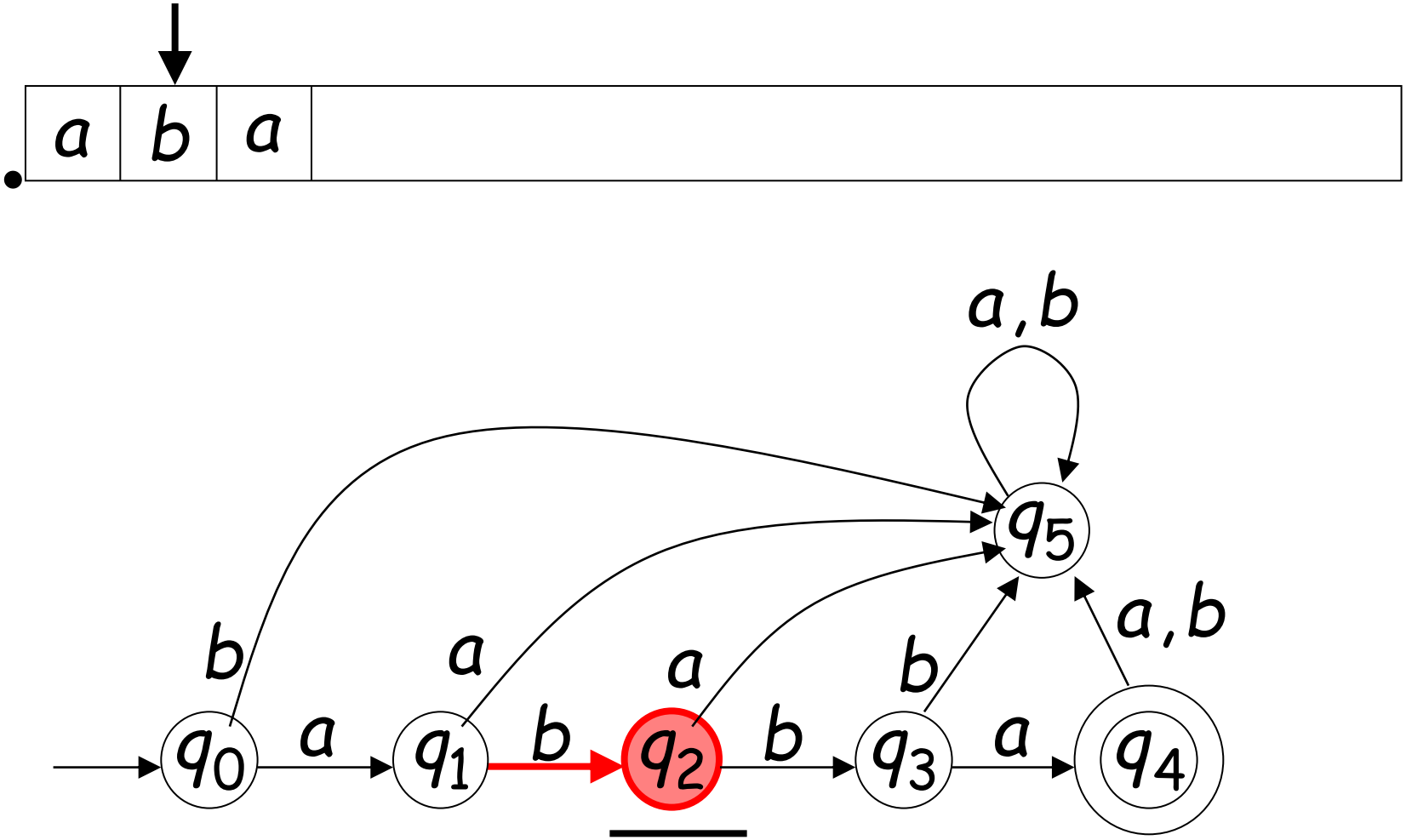
Output: “accept”

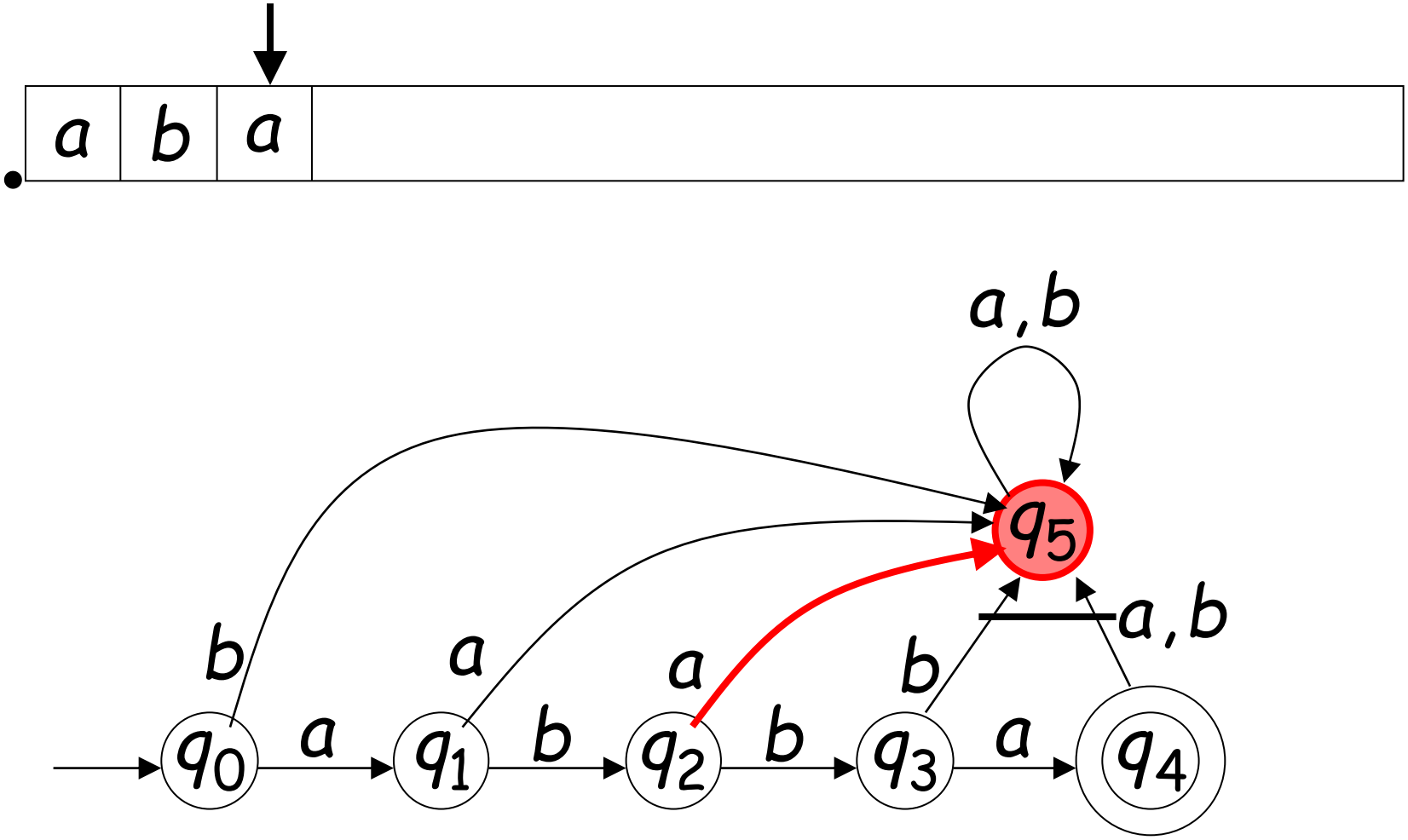
Rejection



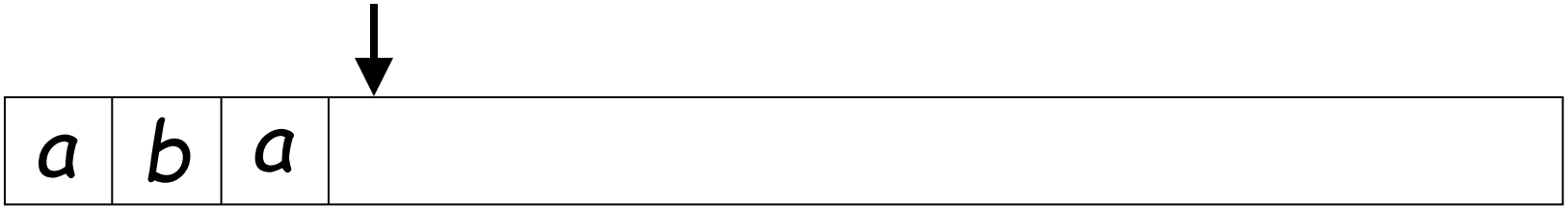


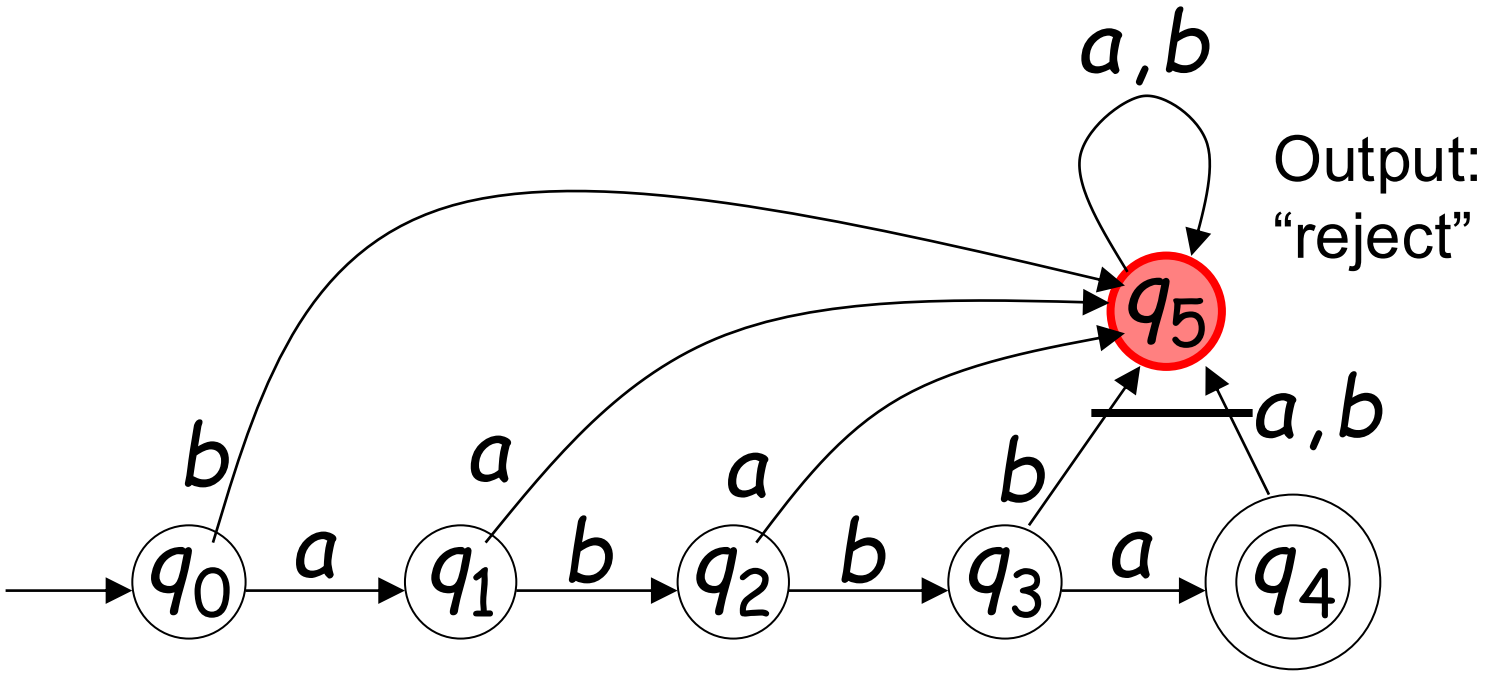

a b a
a,b
q5
b
a
a
b
a,b
q0
a
q1
b
q2
b
q3
a
q4


a
b
a
a,b
q5
b
a
a
b
a,b
q0
a
q1
b
q2
b
q3
a
q4

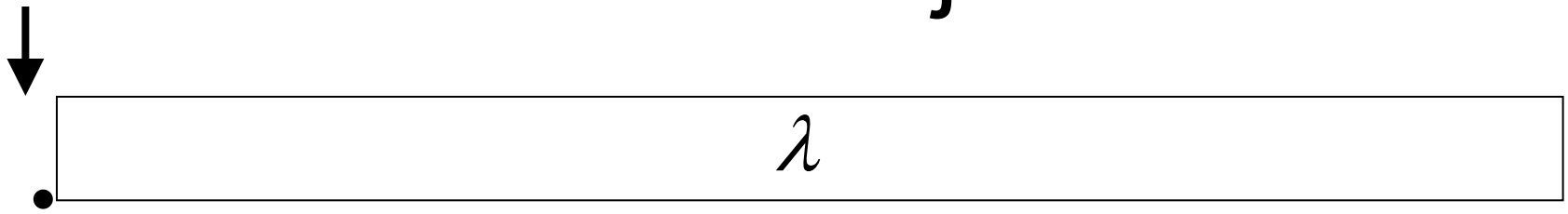

a
b
a
a,b
q5
b
a
a
b
a,b
q0
a
q1
b
q2
b
q3
a
q4

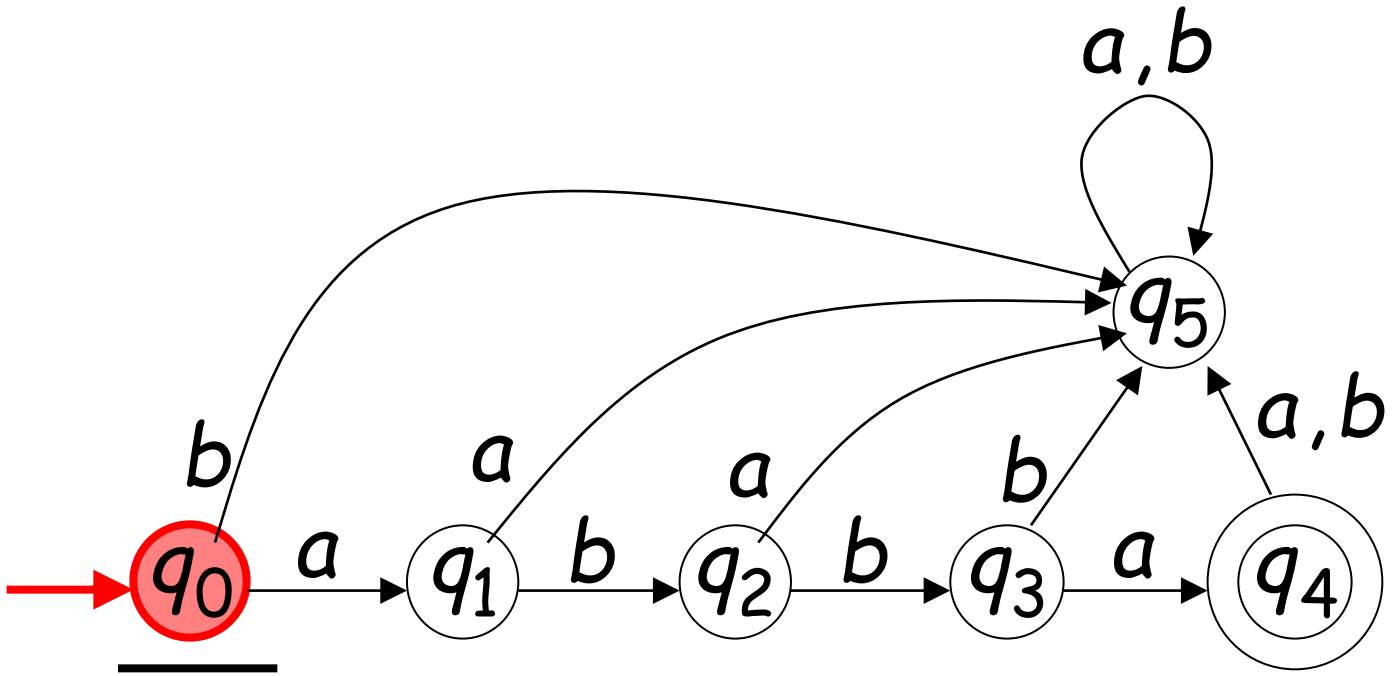
Input finished

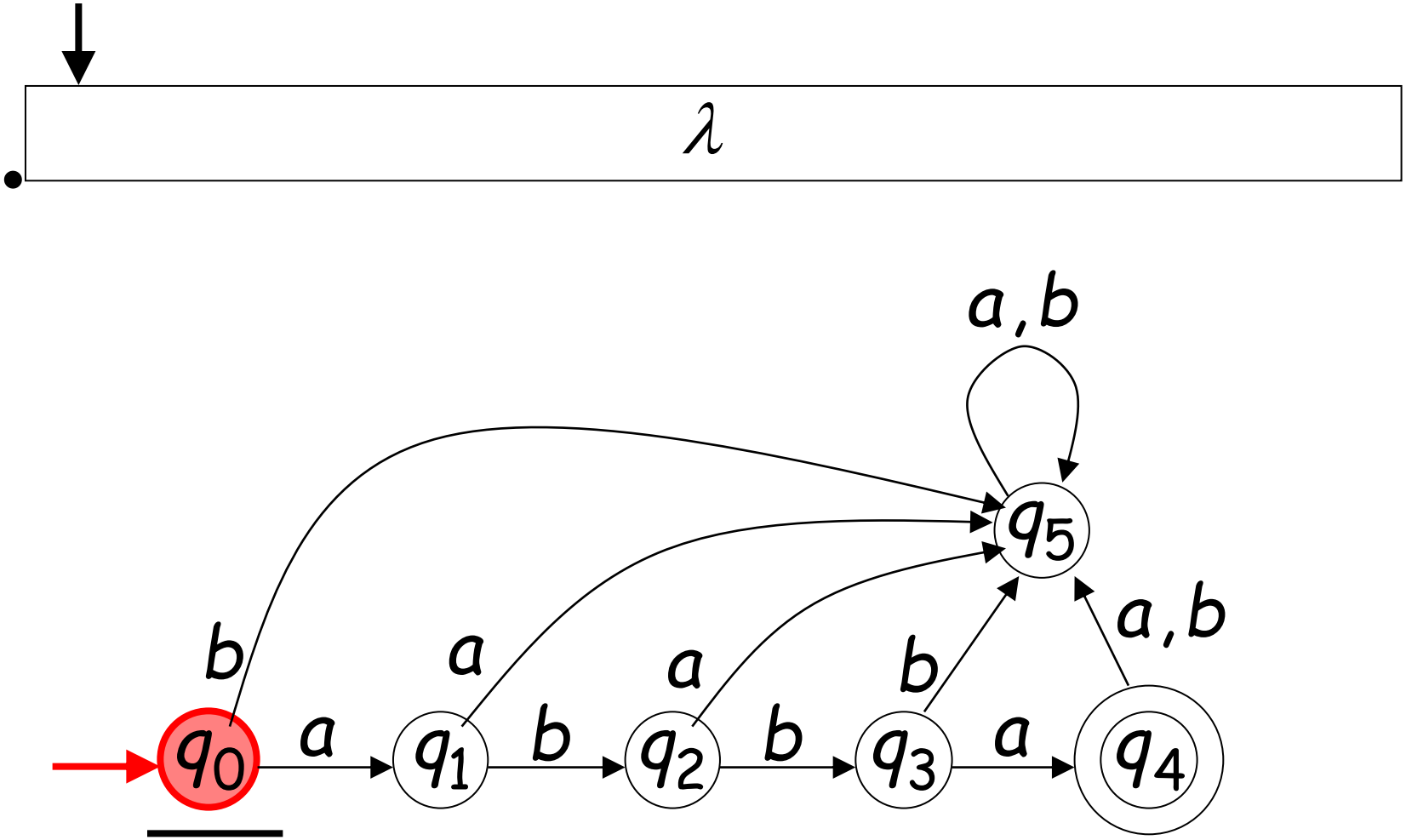




Another Rejection

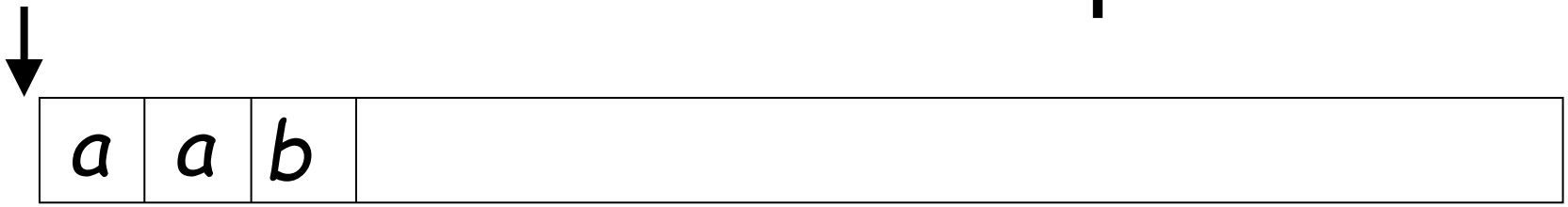


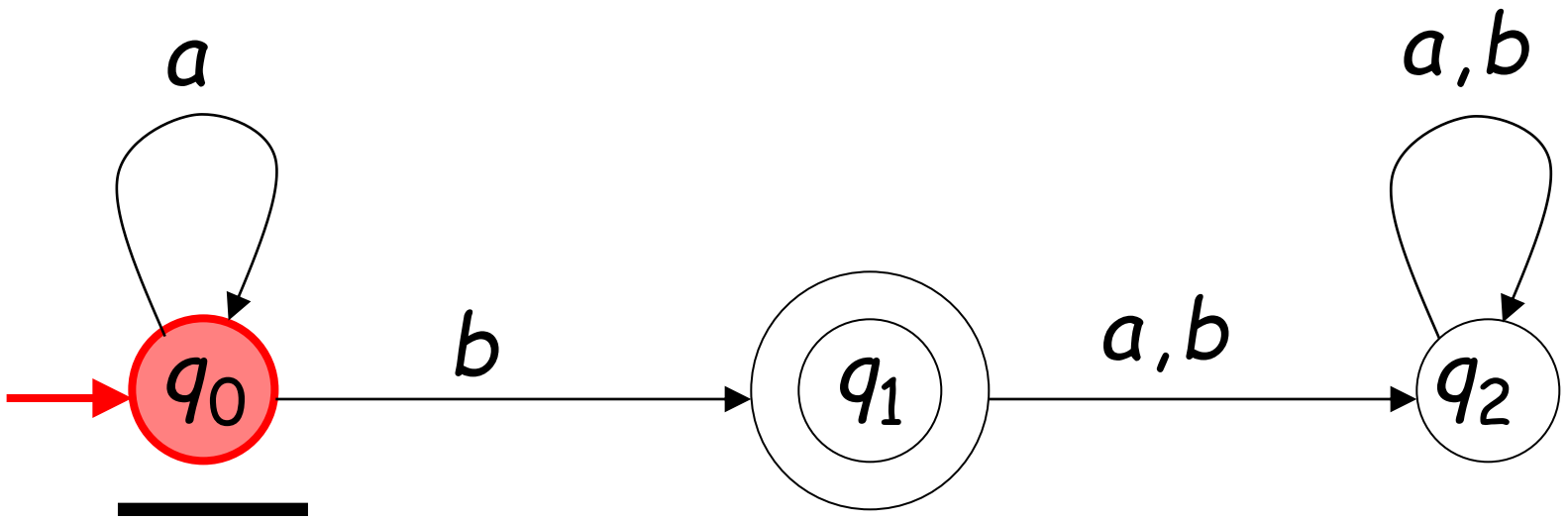


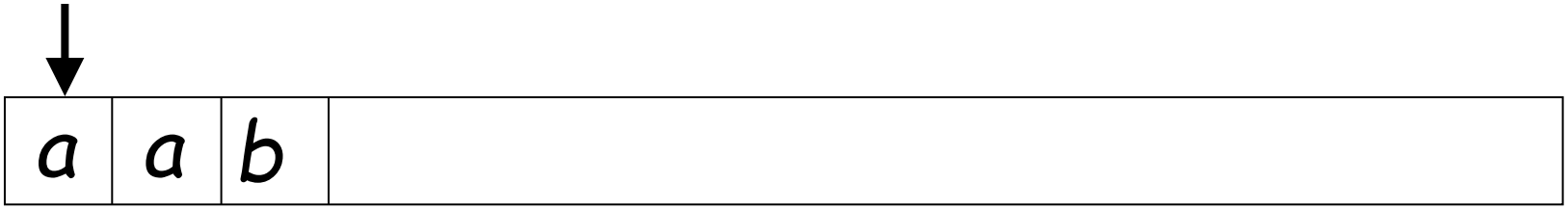


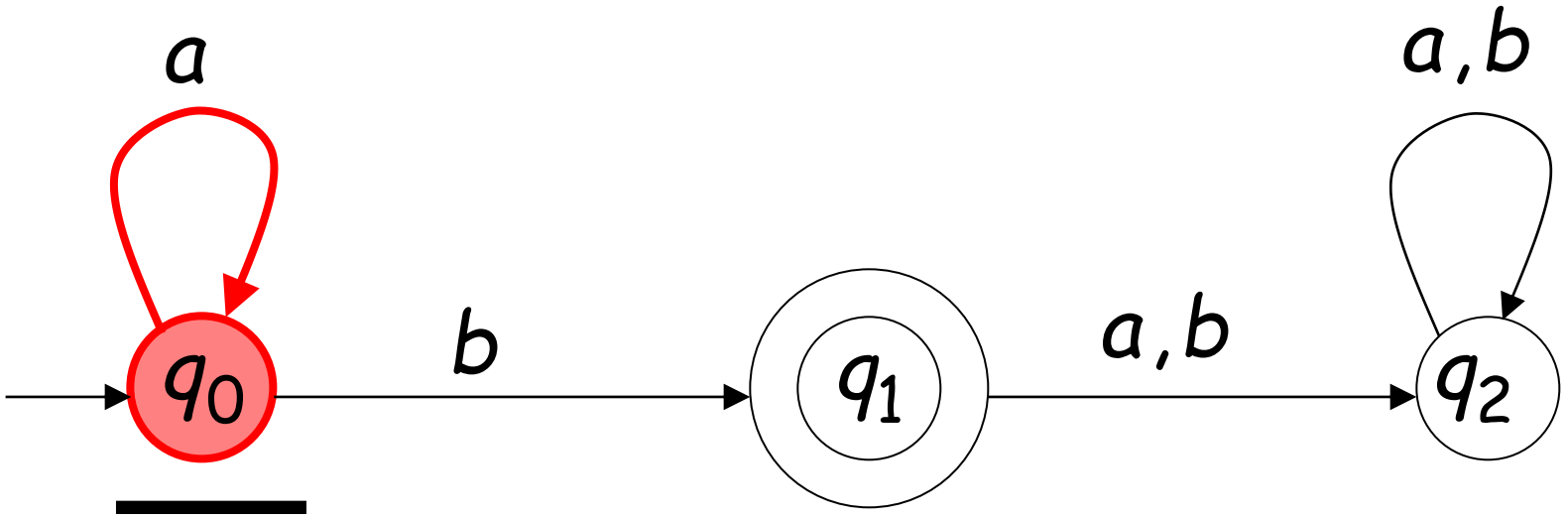
Output:
"reject"

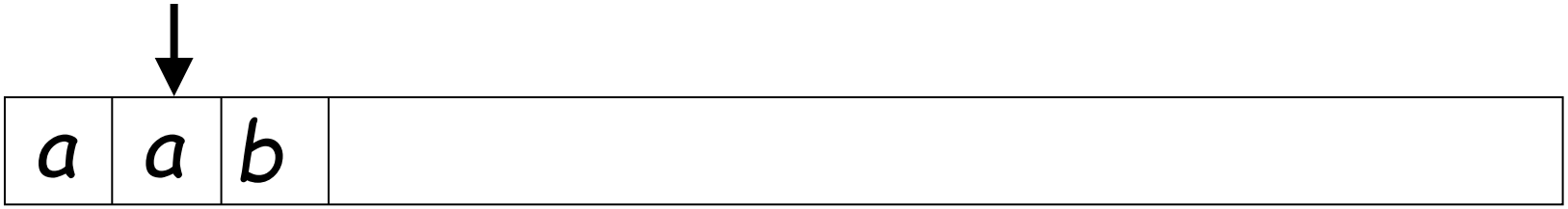
Another Example

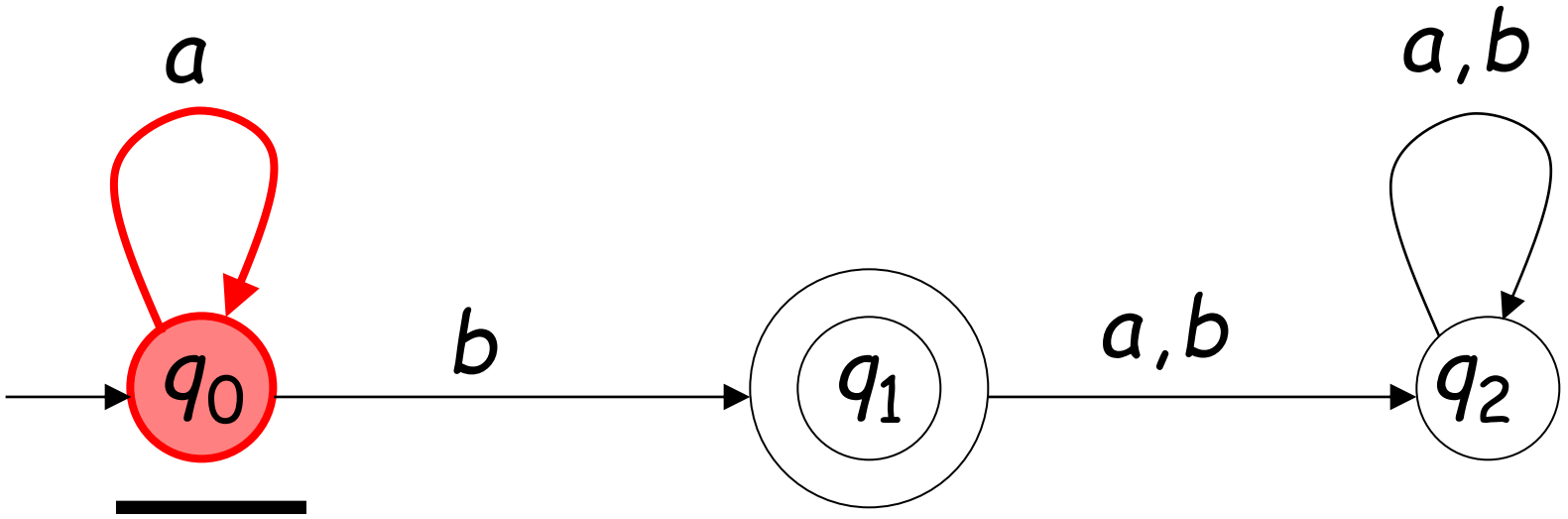


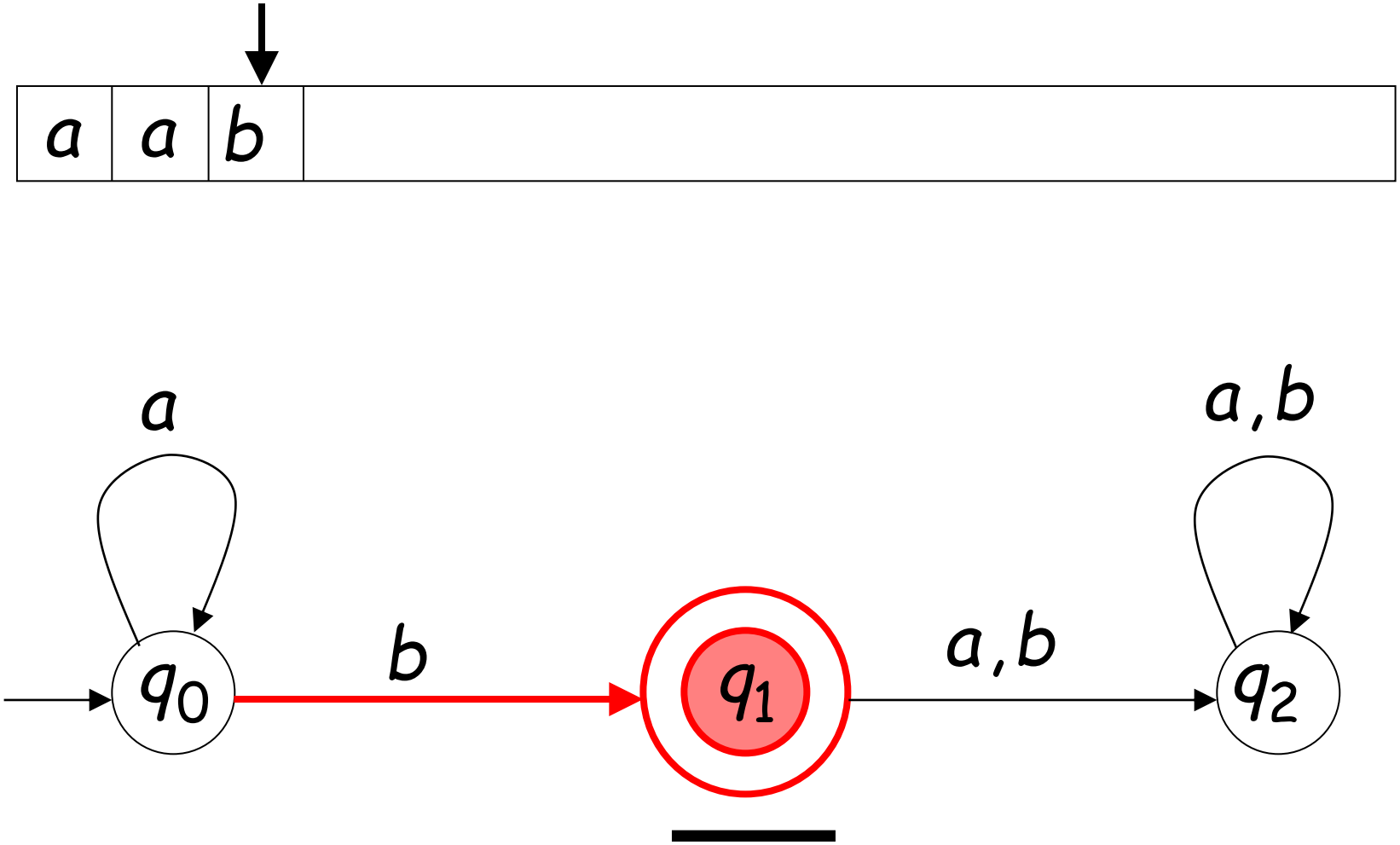


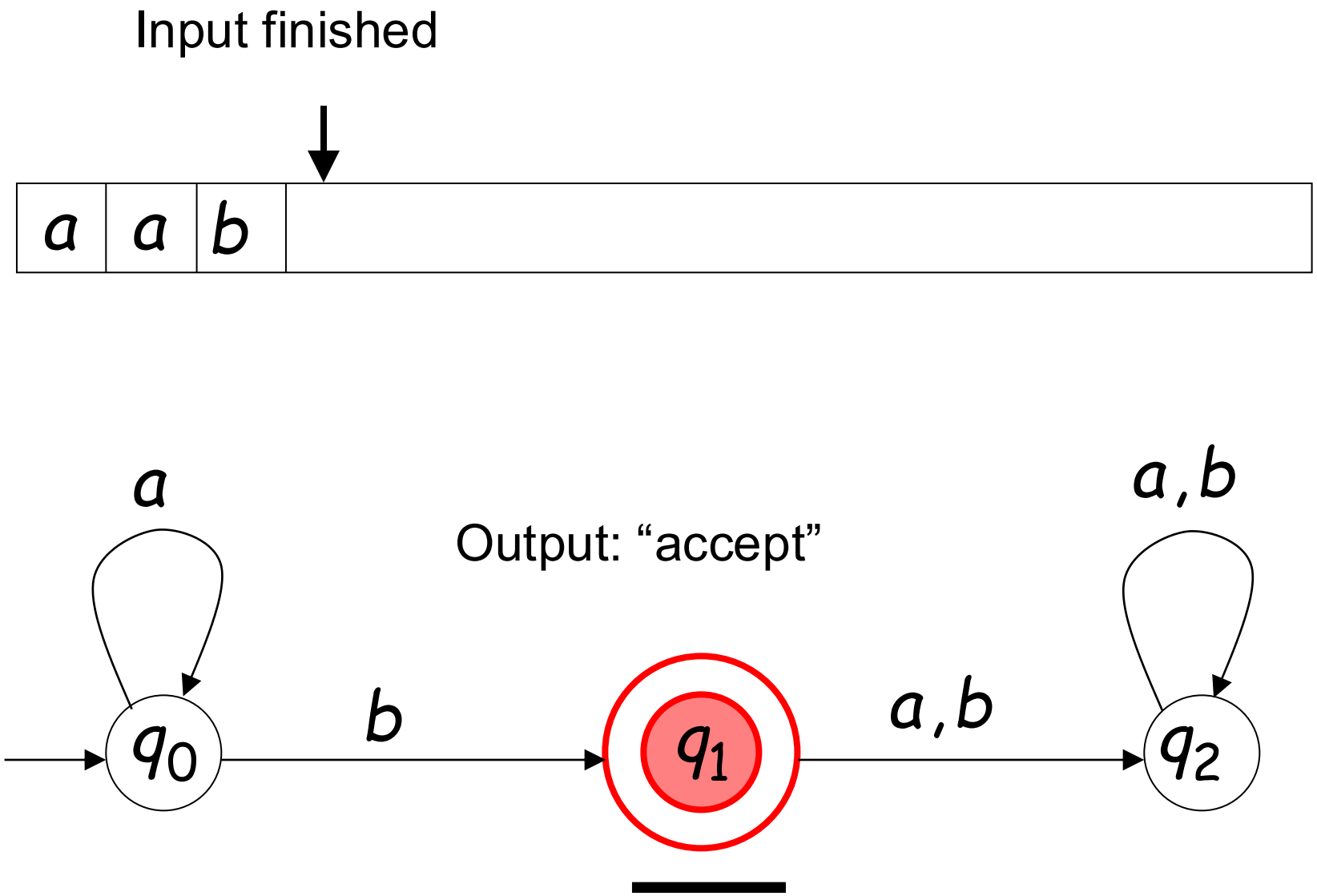

a
a
b


a
a, b
b
a, b
q_0
q_1
q_2

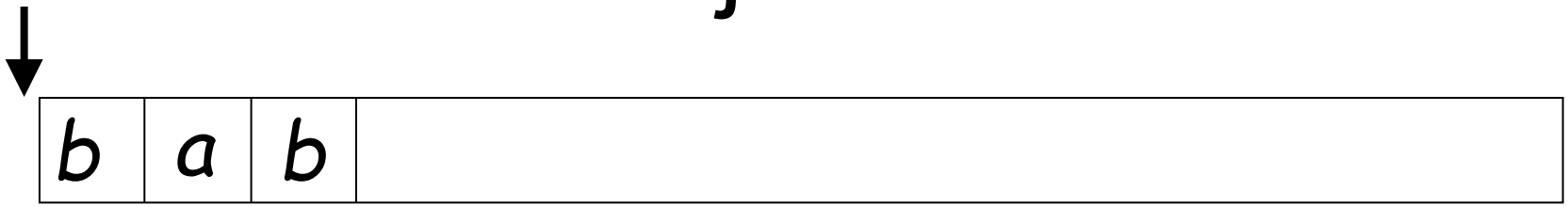

a
a
b

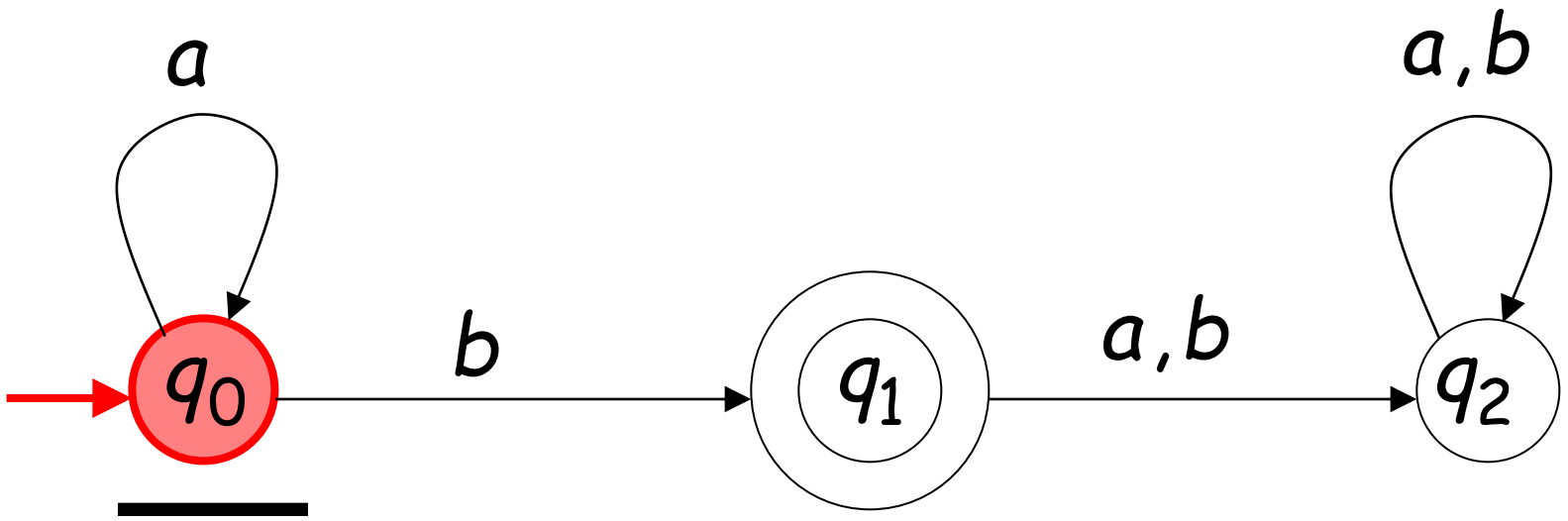

a
a, b
b
a, b
q0
q1
q2

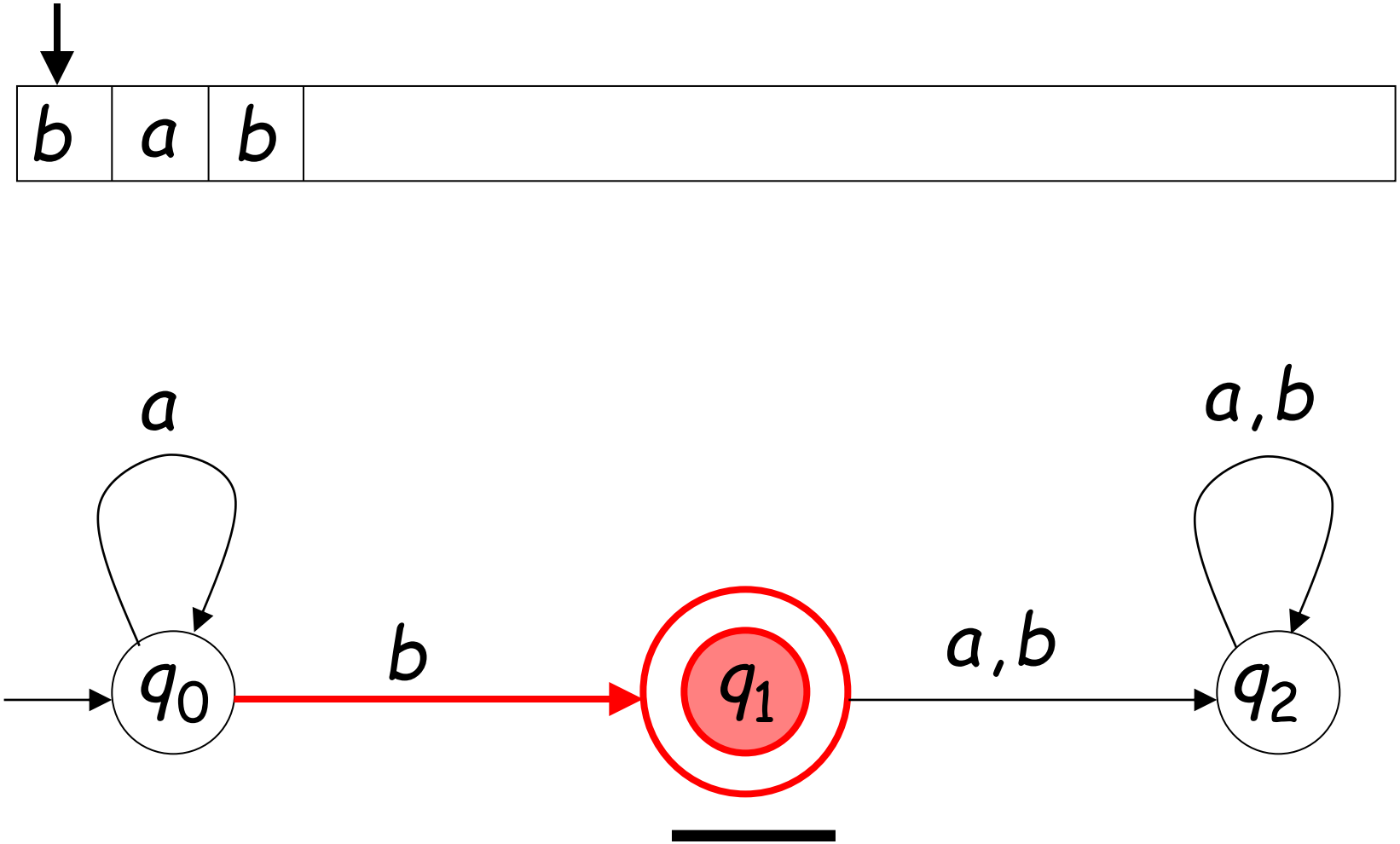

a
a
b
a
a, b
a, b
q_0
b
q_1
q_2

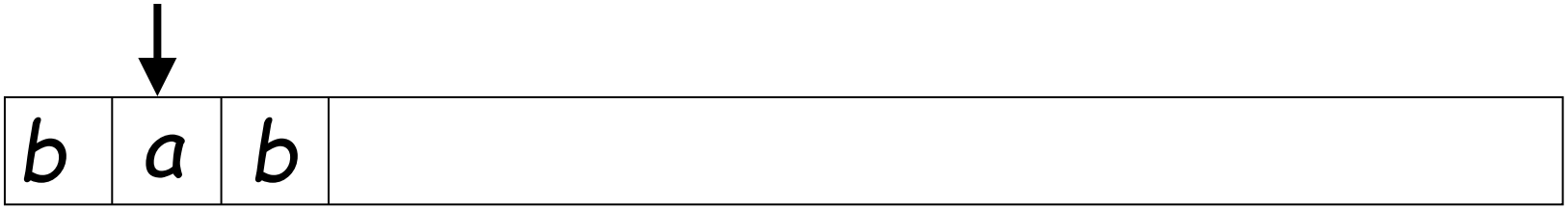

Input finished
a
a
b
a
a, b
Output: “accept”
q0
b
q1
a, b
q2

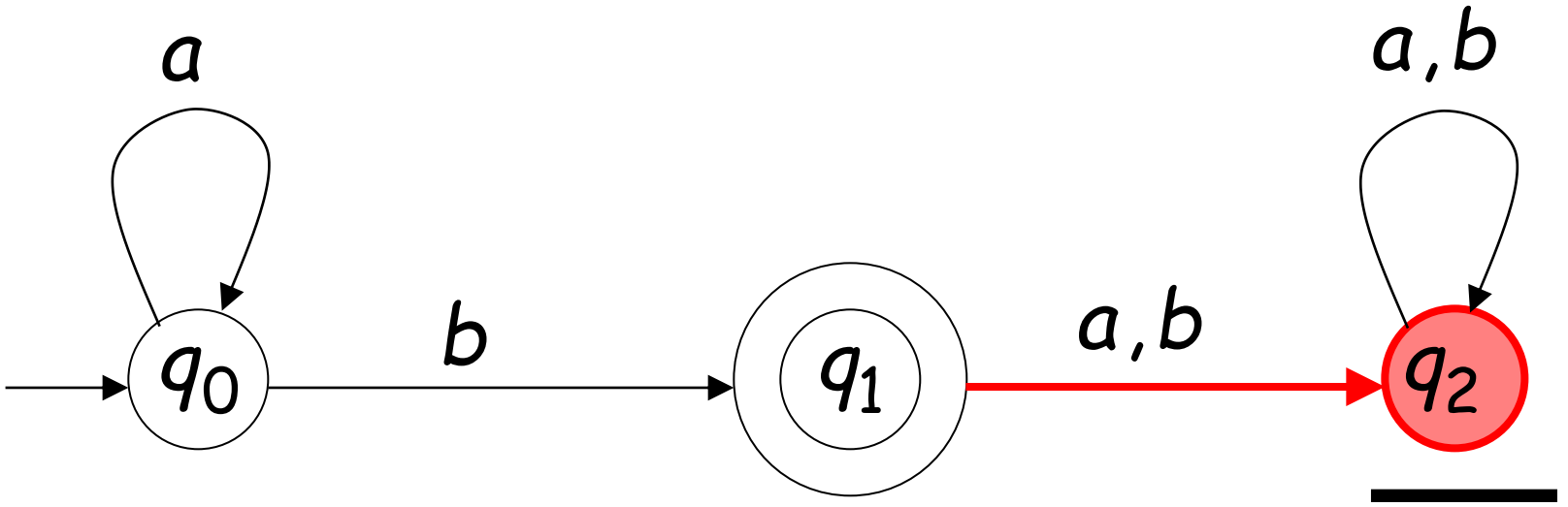
Rejection

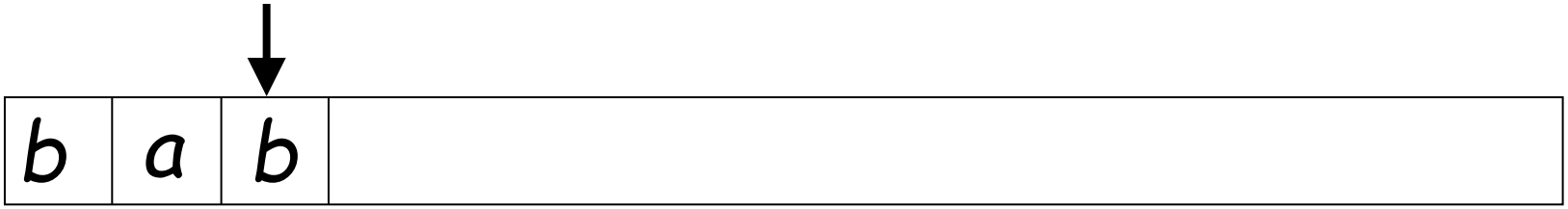


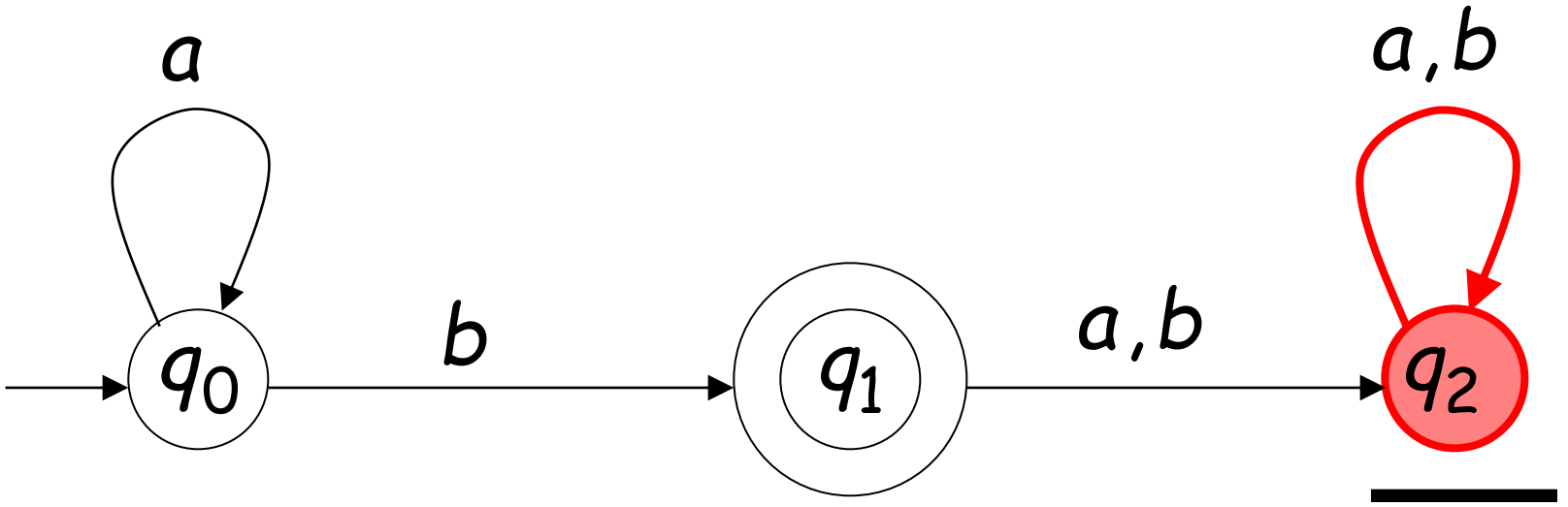



b
a
b
a
a, b
a, b
b
q_0
q_1
q_2

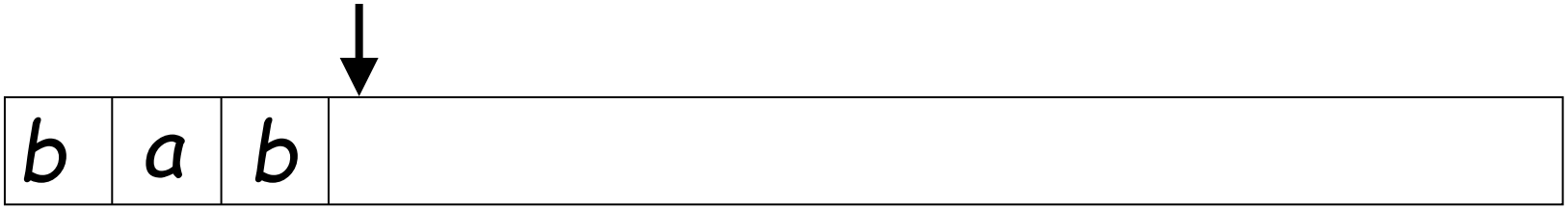

b
a
b

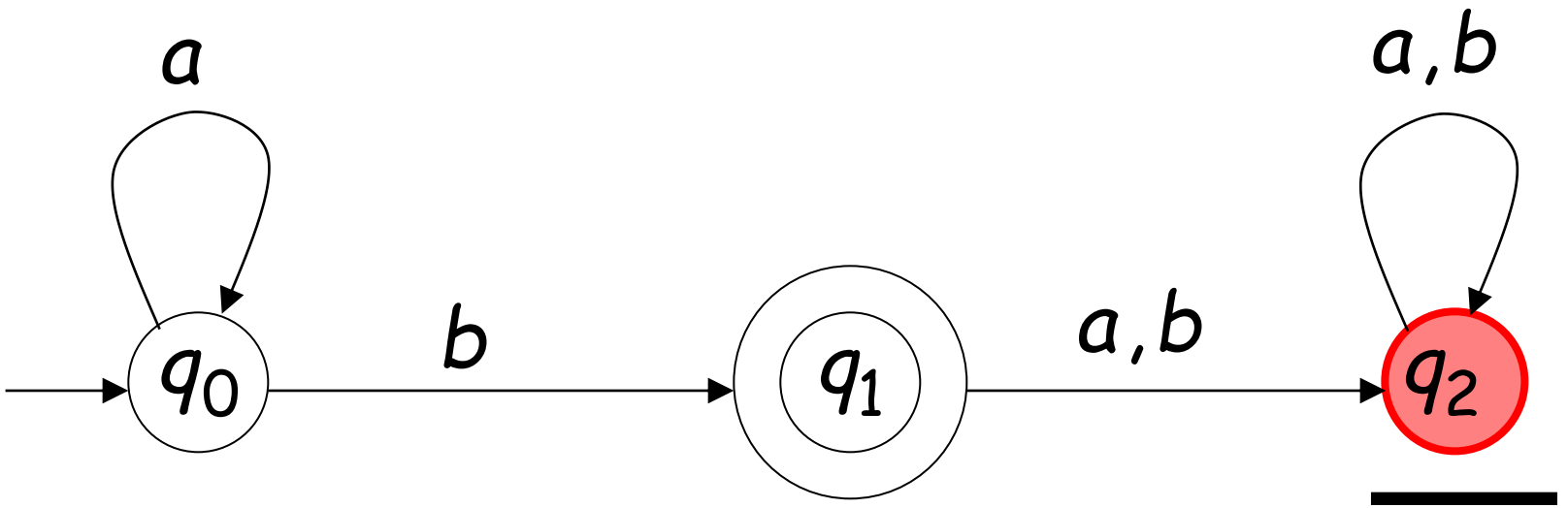

a
a, b
b
a, b
q_0
q_1
q_2


b
a
b


a
a, b
b
a, b
q_0
q_1
q_2

Input finished





Output: “reject”

Trap state

Definition 2.1

Deterministic Finite Accepter (DFA) is defined by the 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : a finite set of **internal states**

Σ : a finite set of symbols called **input alphabet**

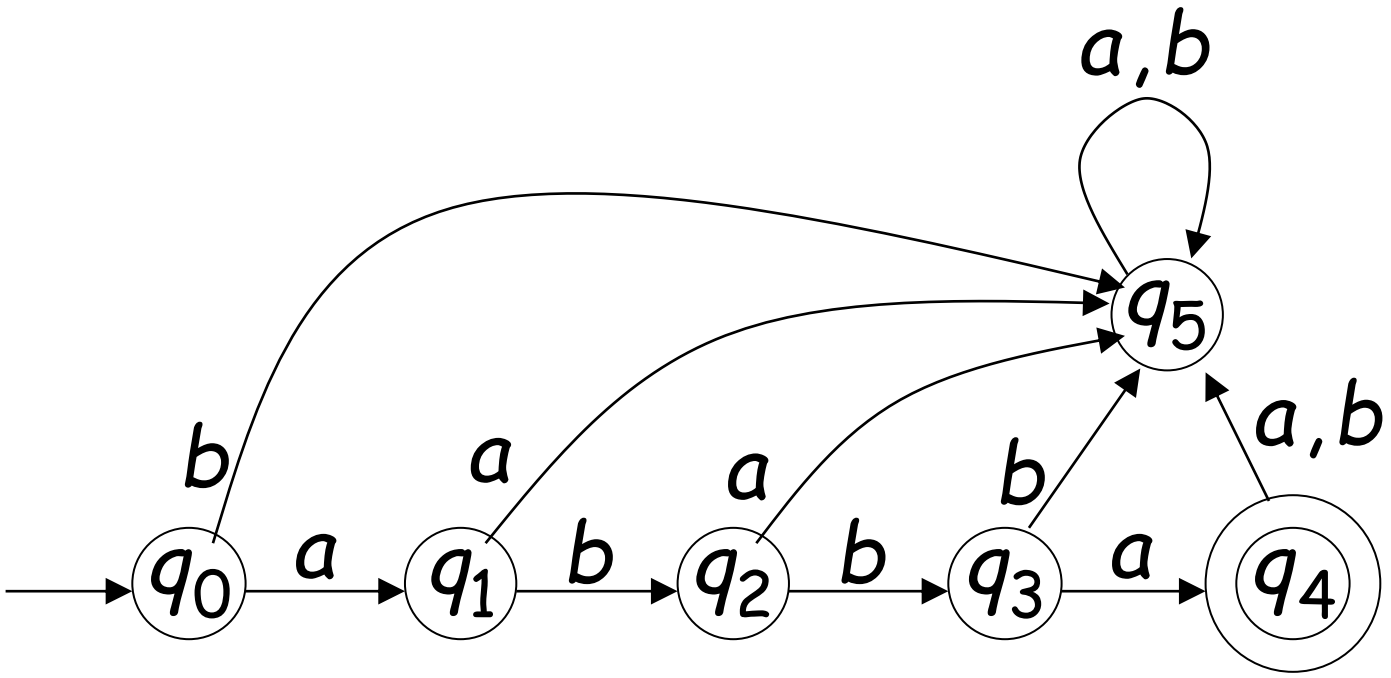
δ : Q x Σ → Q called **transition function** (Total function)

q_0 : $q_0 \in Q$ is the **initial state**

F : $F \subseteq Q$ is a set of **final states**

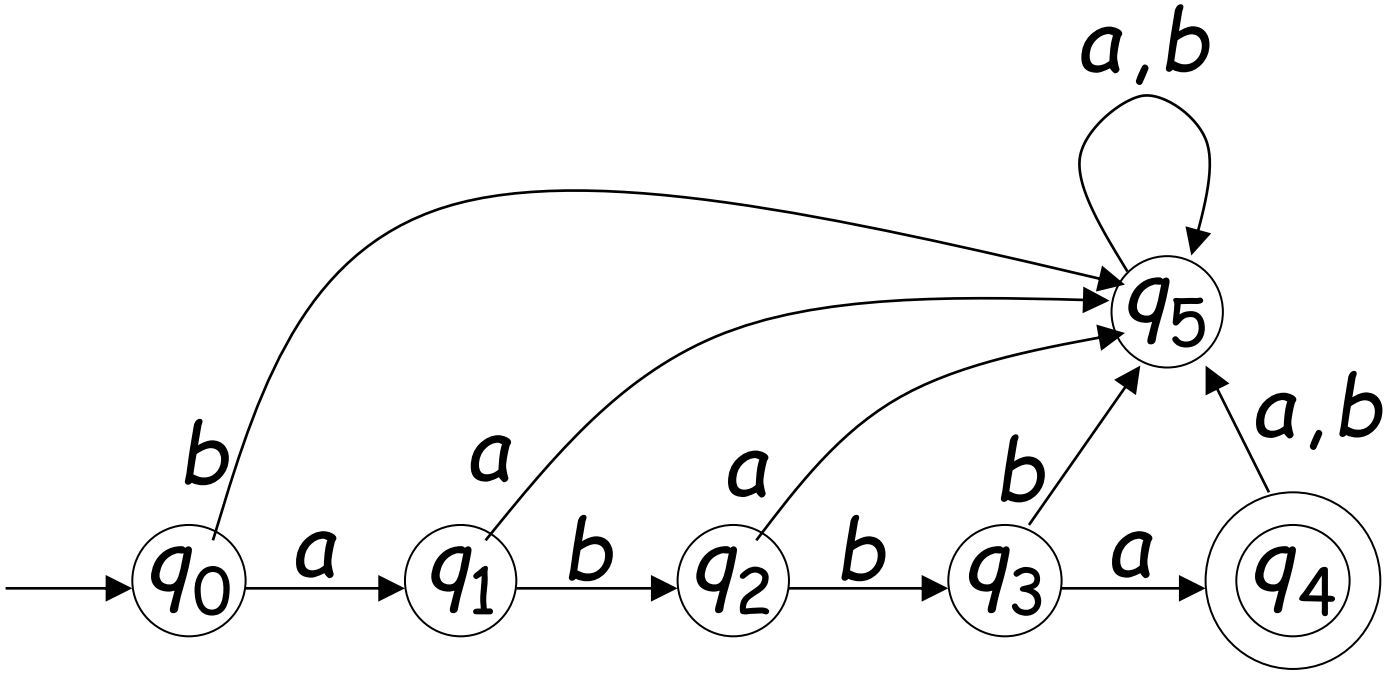
Input Alphabet Σ

$$\Sigma = \{a, b\}$$

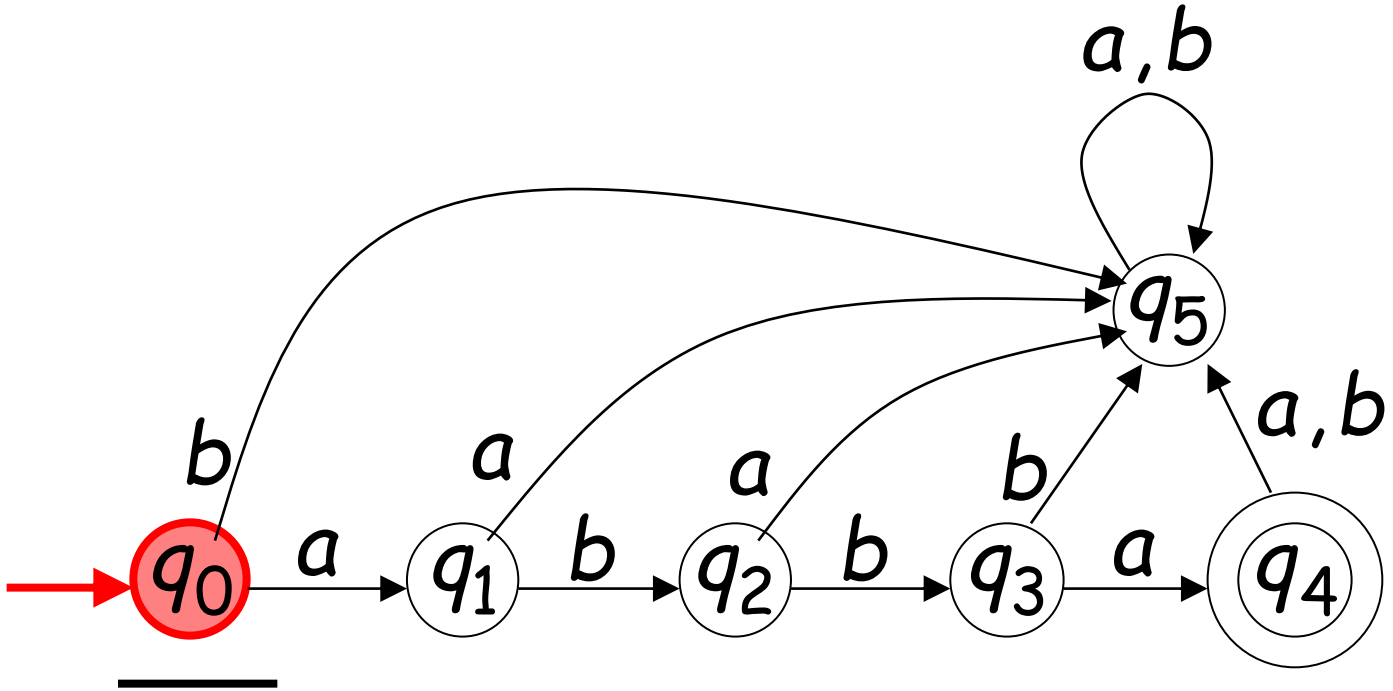


Set of States Q

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

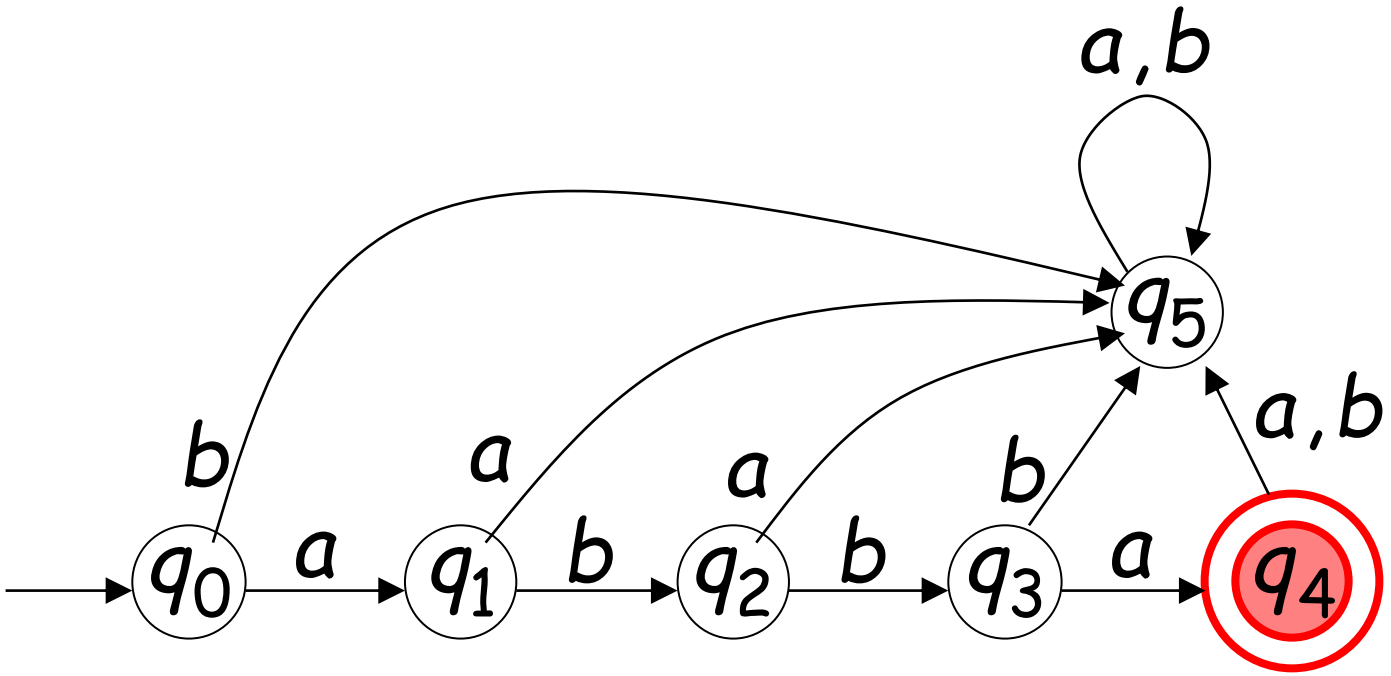


Initial State q_0



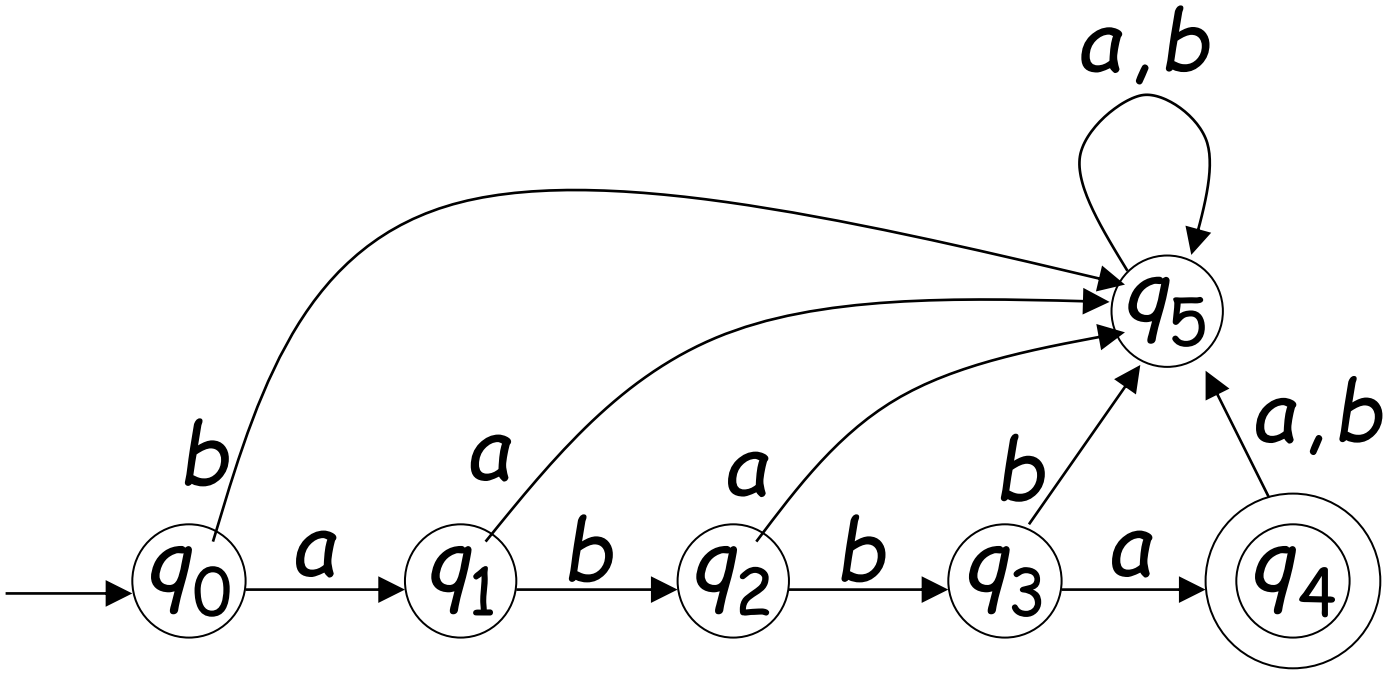
Set of Final States F

$$F = \{q_4\}$$

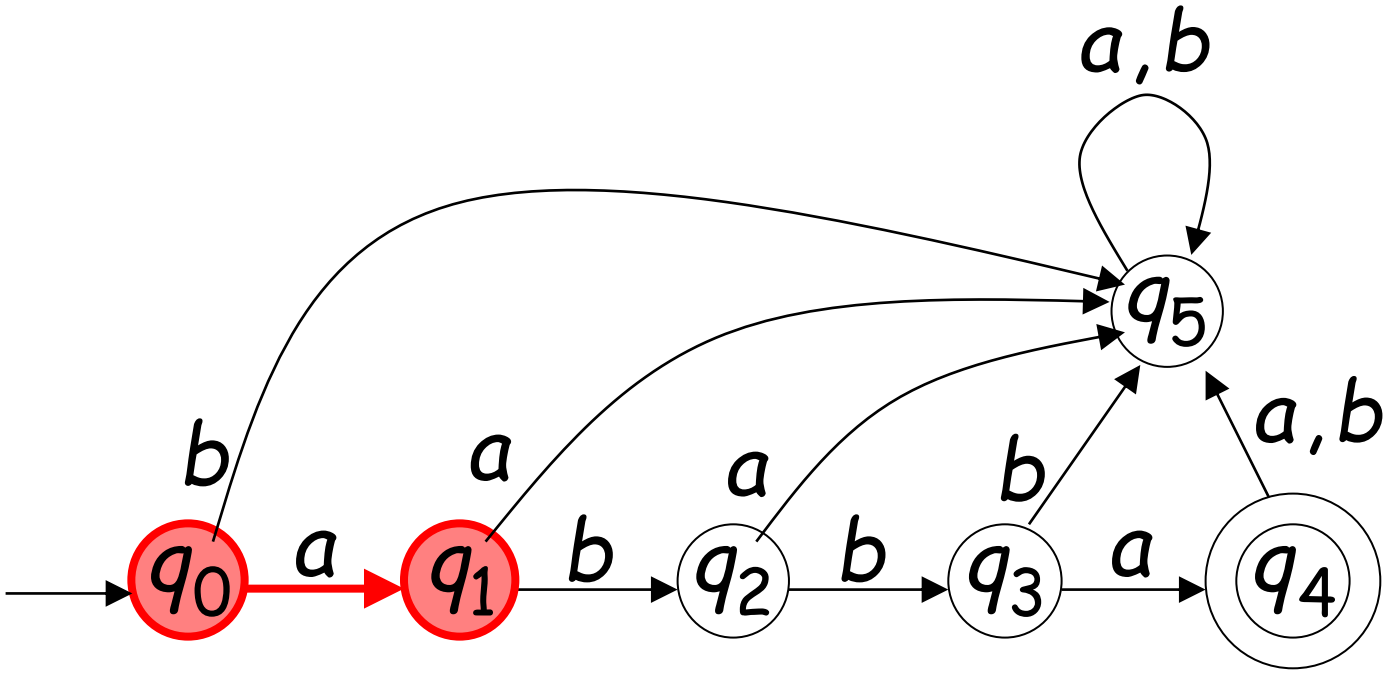


Transition Function δ

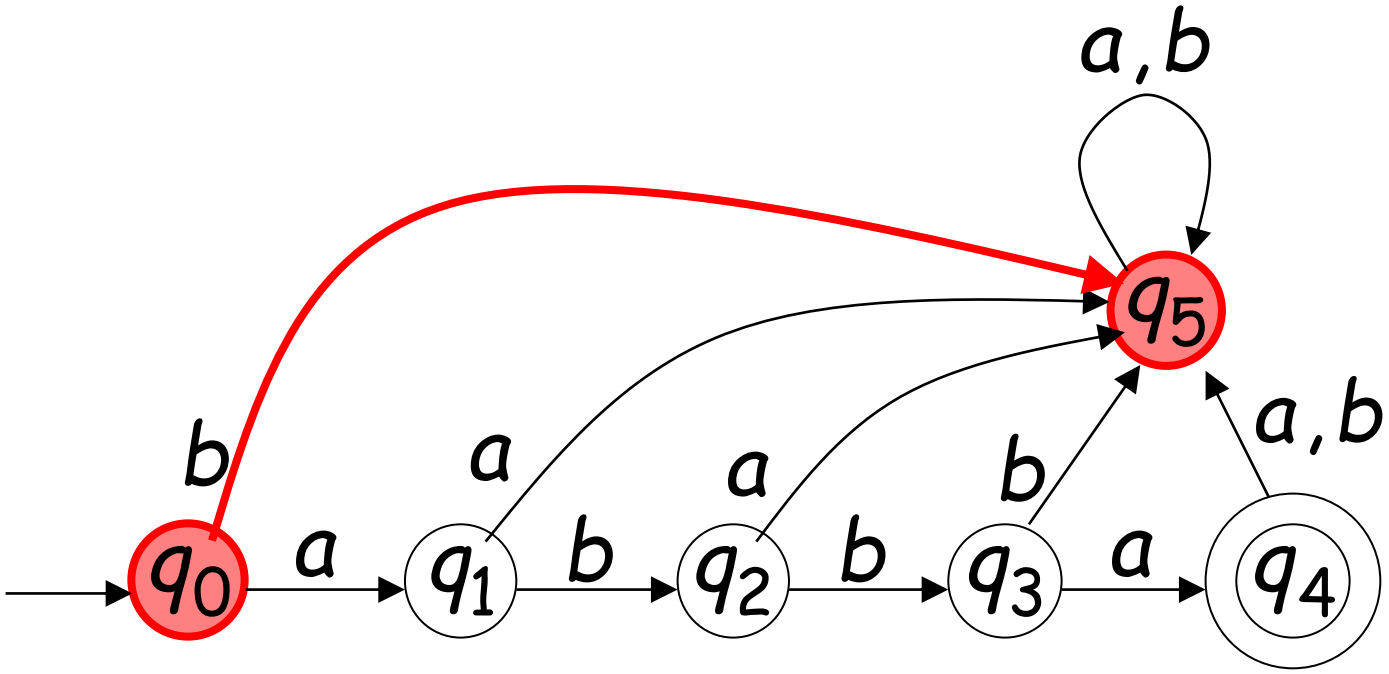
$$\delta : Q \times \Sigma \to Q$$



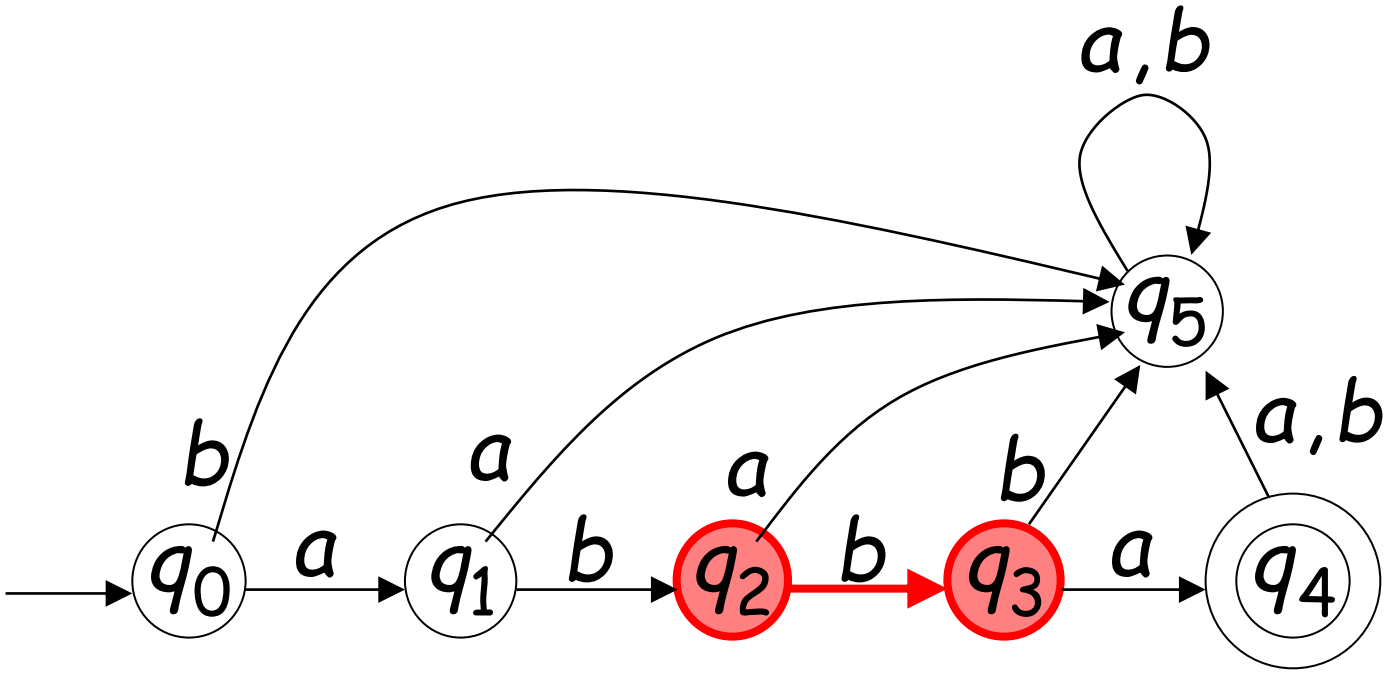
$$\delta(q_0, a) = q_1$$



$$\delta(q_0, b) = q_5$$

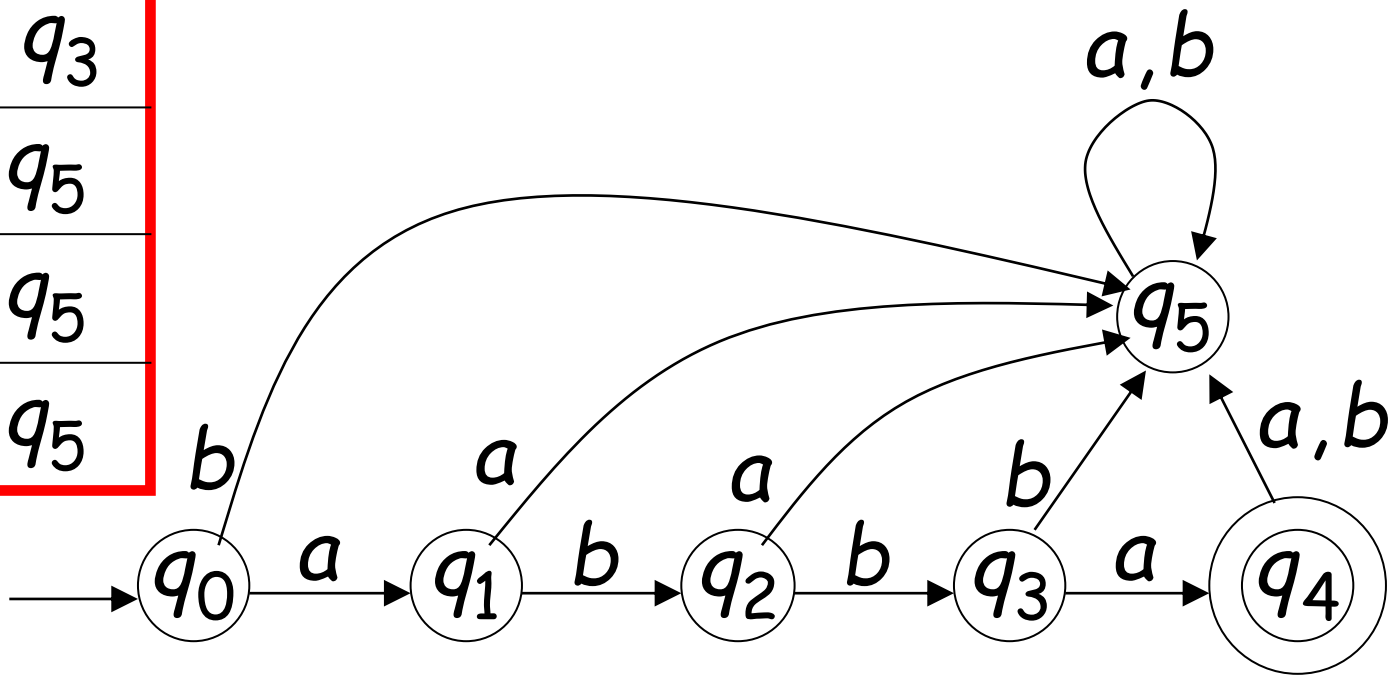


$$\delta(q_2, b) = q_3$$



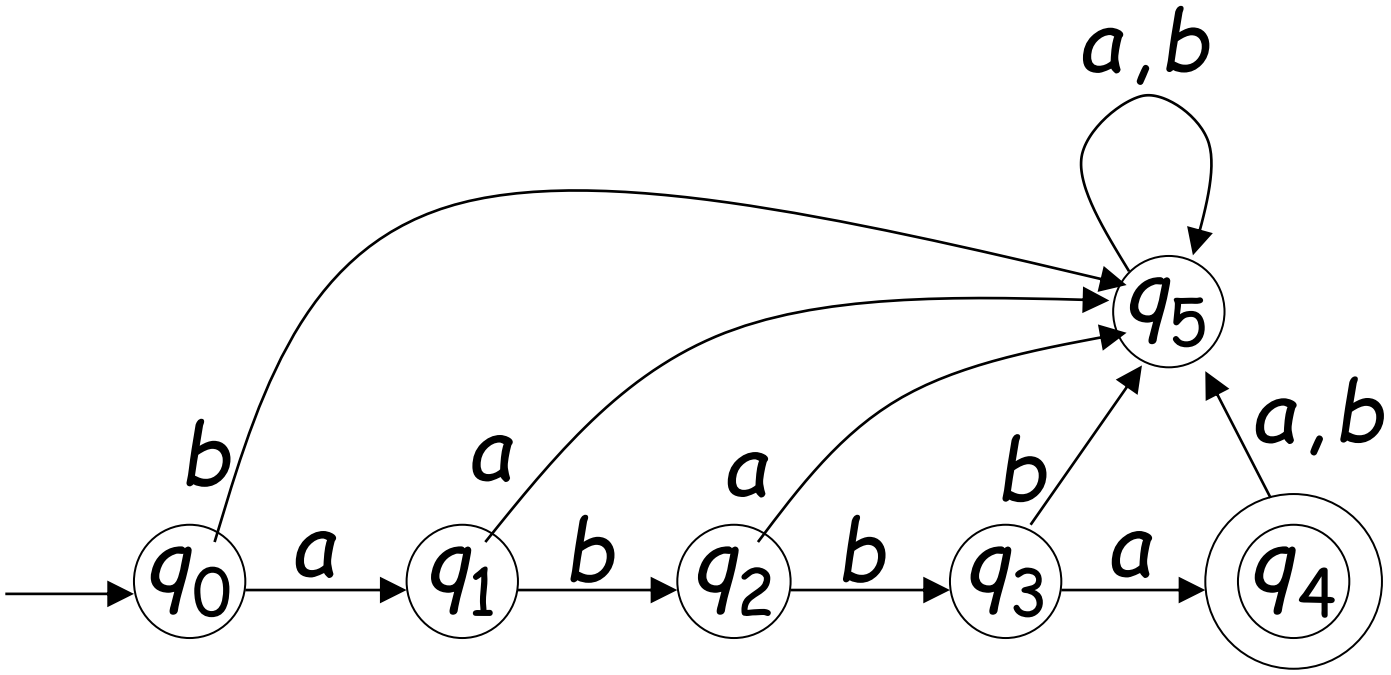
Transition Function δ

δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

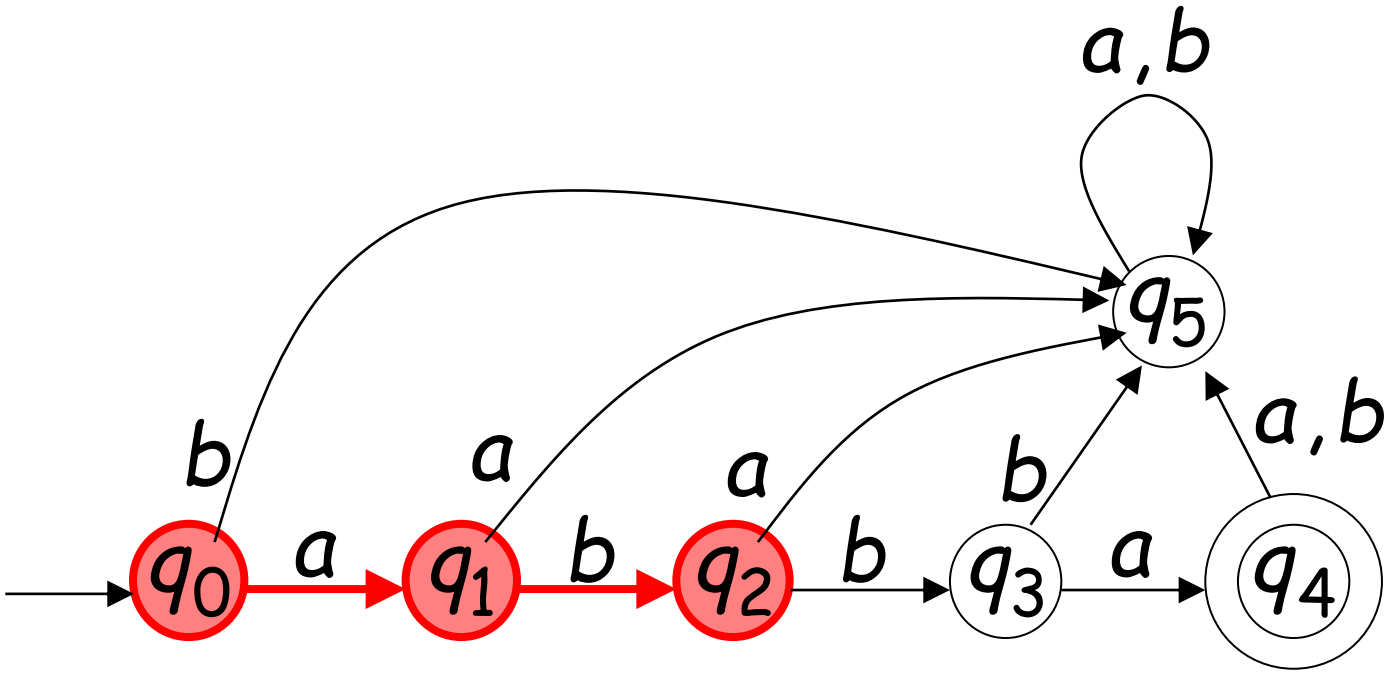


Extended Transition Function δ^*

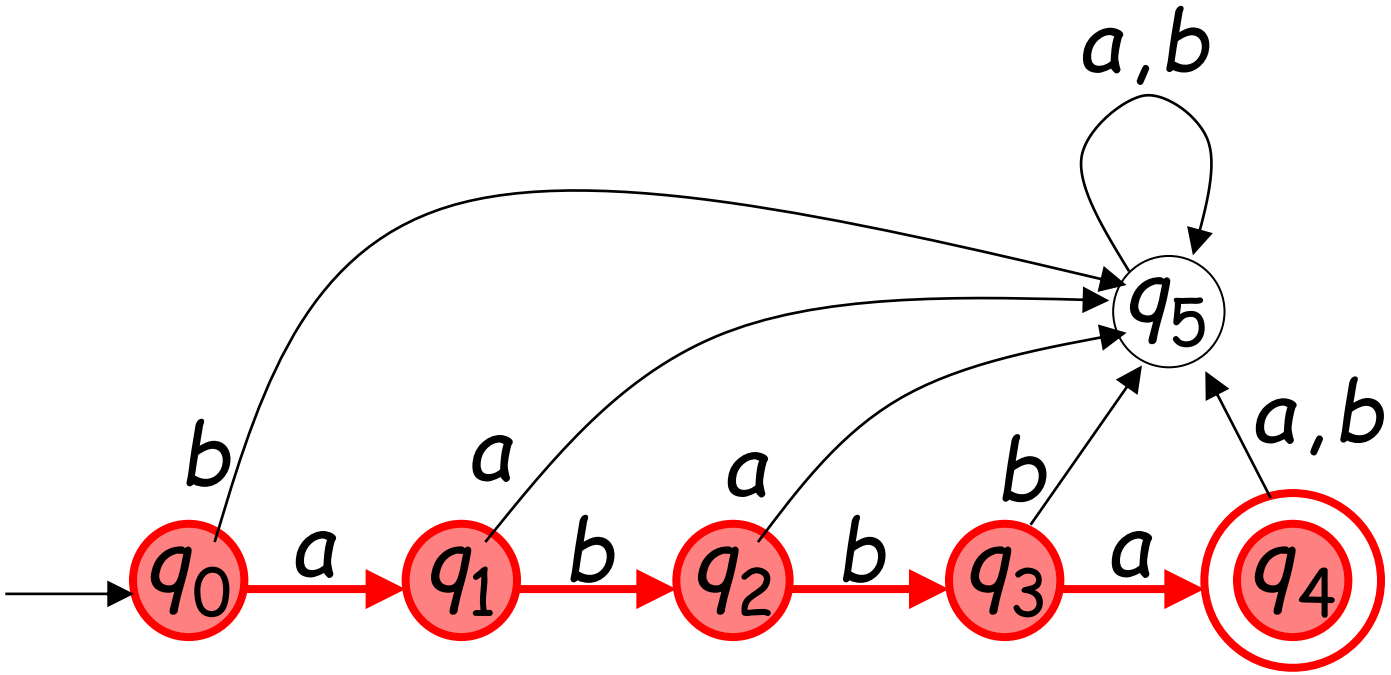
$$\delta^* : Q \times \Sigma^* \to Q$$



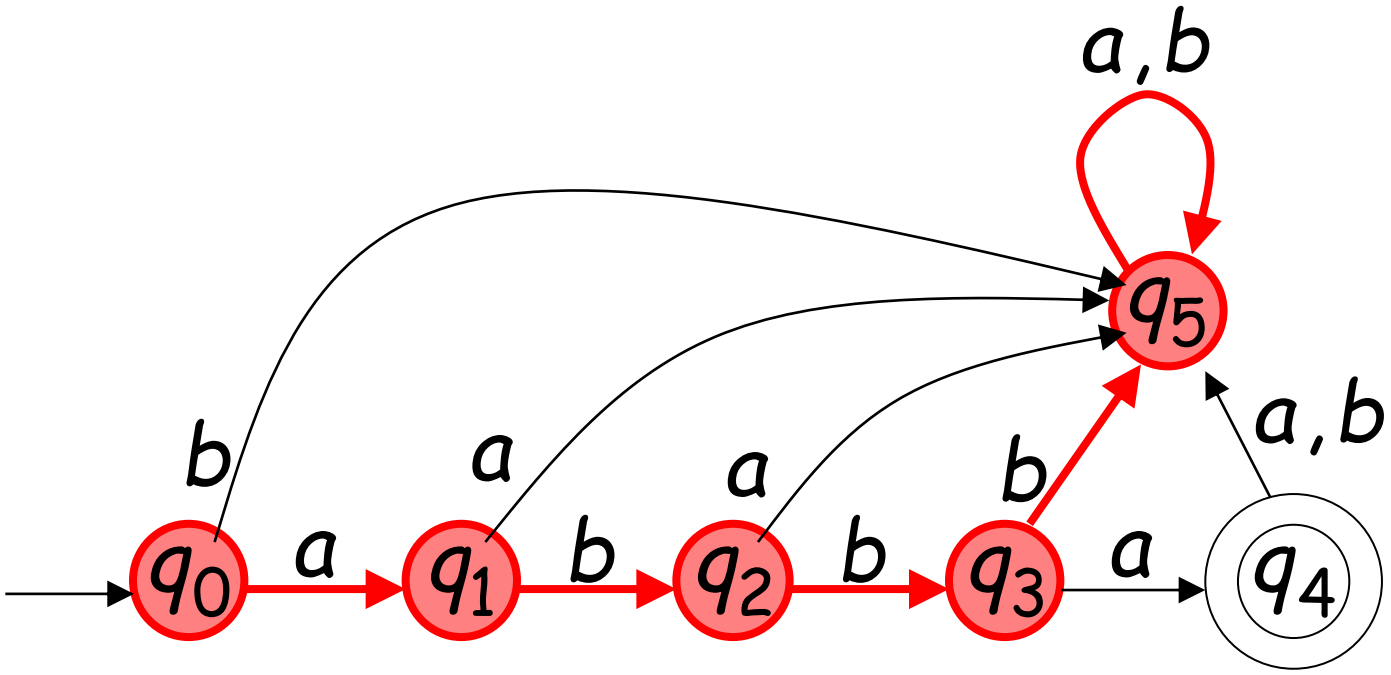
$$\delta^*(q_0, ab) = q_2$$



$$\delta^*(q_0, abba) = q_4$$



$$\delta^*(q_0, abbbaa) = q_5$$



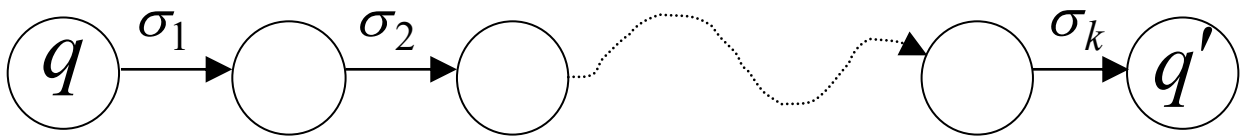
Observation: If there is a walk from q to q' with label w

■Theorem 2.1

iff $\delta^*(q,w)=q'$

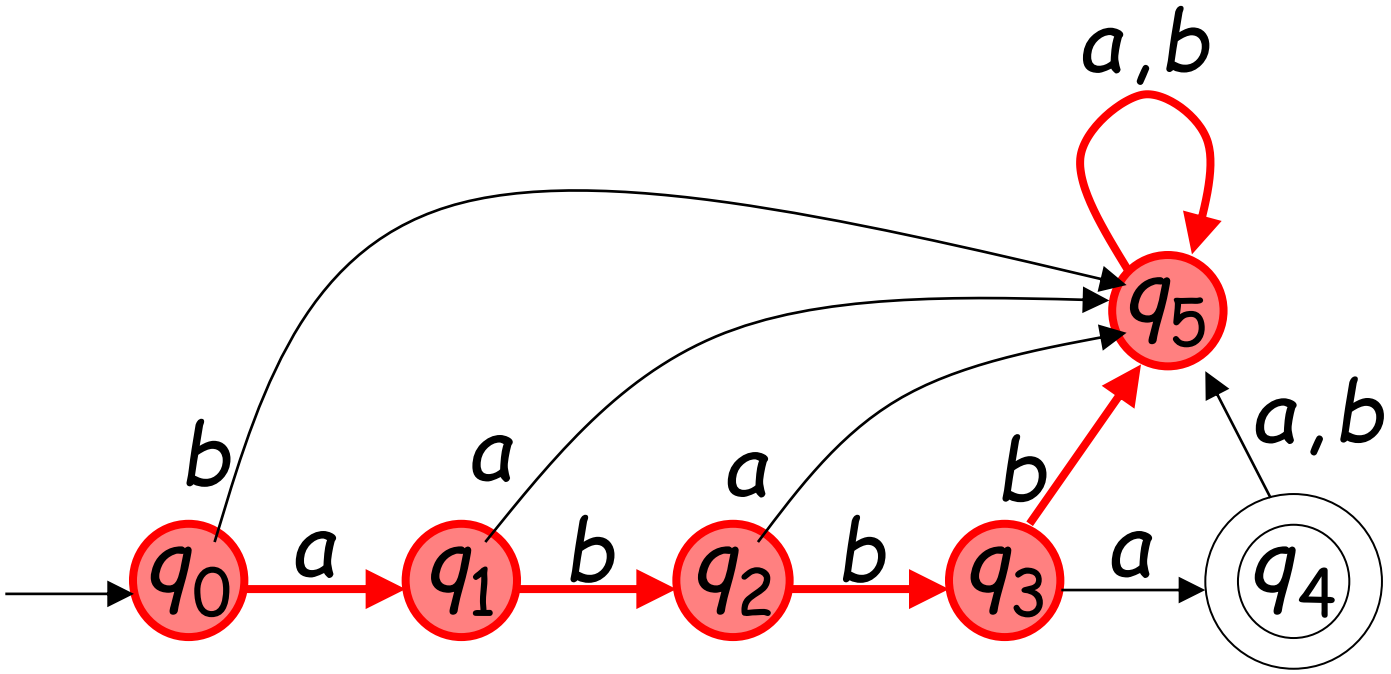


$w=\sigma_1\sigma_2\cdots\sigma_k$



Example: There is a walk from q_0 to q_5 with label $abbbaa$

$$\delta^*(q_0, abbbaa) = q_5$$



Recursive Definition

$$\delta^*(q,\lambda) = q$$

$$\delta^*(q,w\sigma) = \delta(\delta^*(q,w),\sigma)$$



$\delta^*(q,w\sigma) = q'$

$\delta(q_1,\sigma) = q'$

$\delta^*(q,w\sigma) = \delta(q_1,\sigma)$

$\delta^*(q,w) = q_1$

$\delta^*(q,w\sigma) = \delta(\delta^*(q,w),\sigma)$

$$\delta^*(q_0, ab) =$$

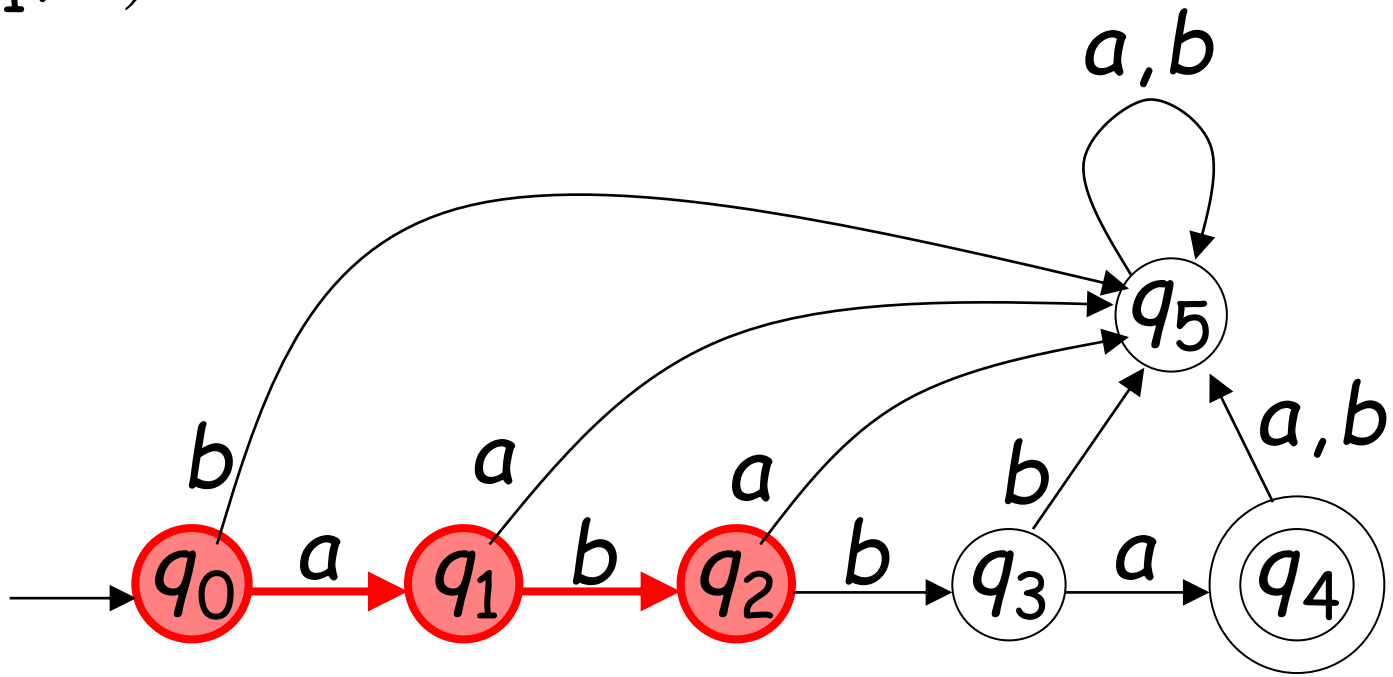
$$\delta(\delta^*(q_0, a), b) =$$

$$\delta(\delta(\delta^*(q_0, \lambda), a), b) =$$

$$\delta(\delta(q_0, a), b) =$$

$$\delta(q_1, b) =$$

$$q_2$$



Languages Accepted by DFAs

Take DFA M

Definition:

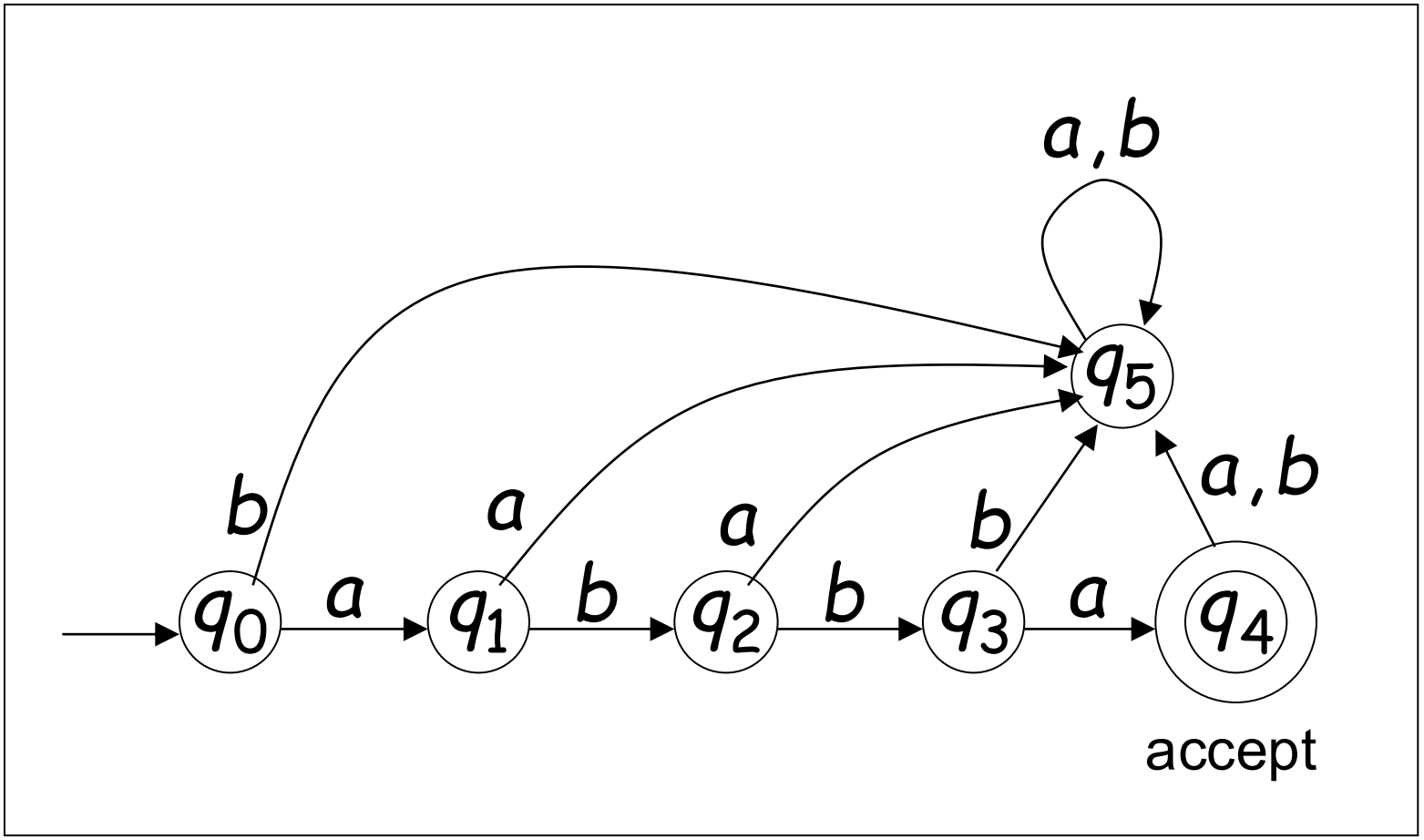
- The language $L(M)$ contains all input strings accepted by M

- $L(M)$= { strings that drive M to a final state}

Example

$L(M) = \{abba\}$

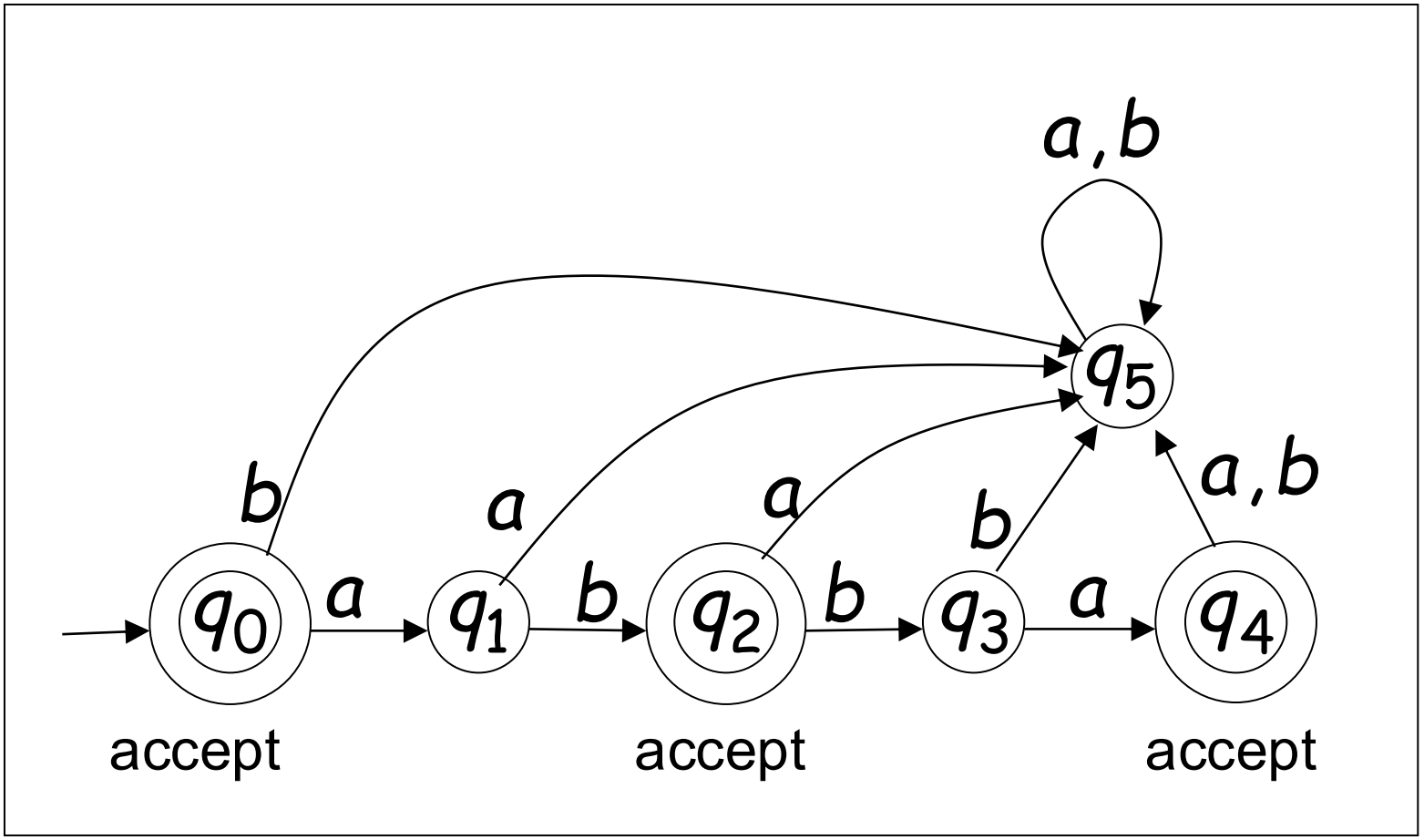
M



Another Example

$L(M) = \{\lambda, ab, abba\}$

M

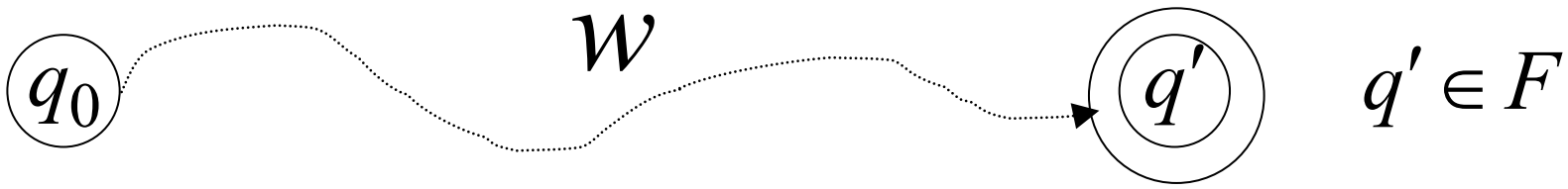


Formally

For a DFA $M = (Q, \Sigma, \delta, q_0, F)$

Language accepted by M :

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$$



Observation

- Language rejected by M :

$$\overline{L(M)} = \{w \in \Sigma^* : \delta^*(q_0, w) \notin F\}$$

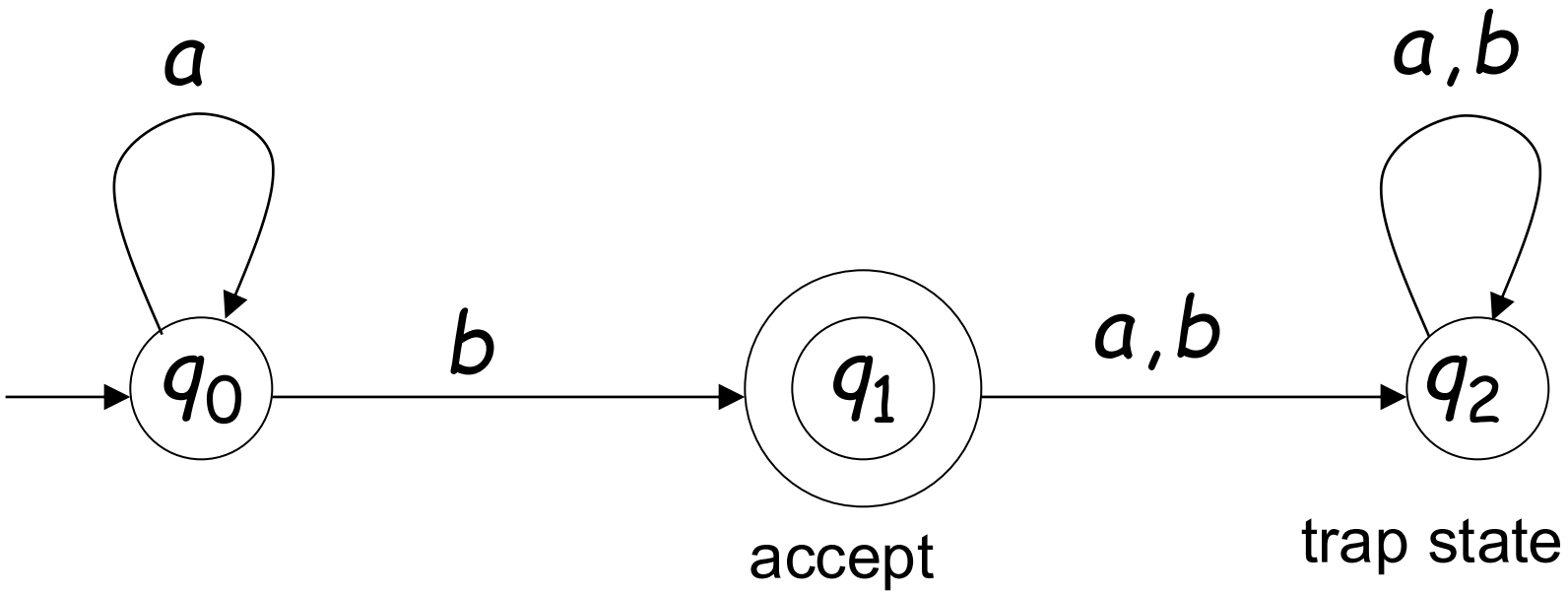


Example 2.2

$$M = (Q, \Sigma, \delta, q_0, F)$$

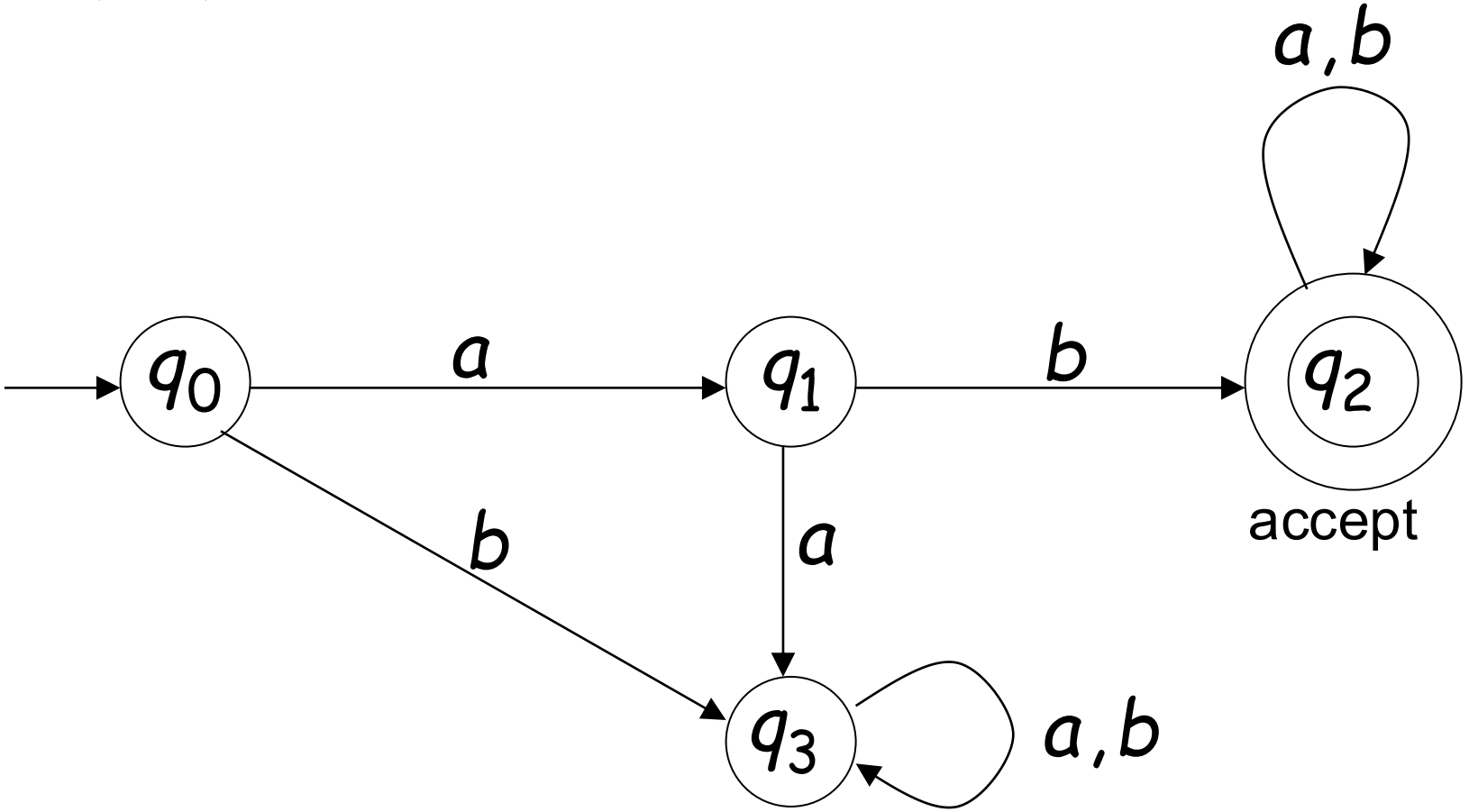
$$\Sigma = \{a, b\}$$

$$L(M) = \{a^n b : n \geq 0\}$$



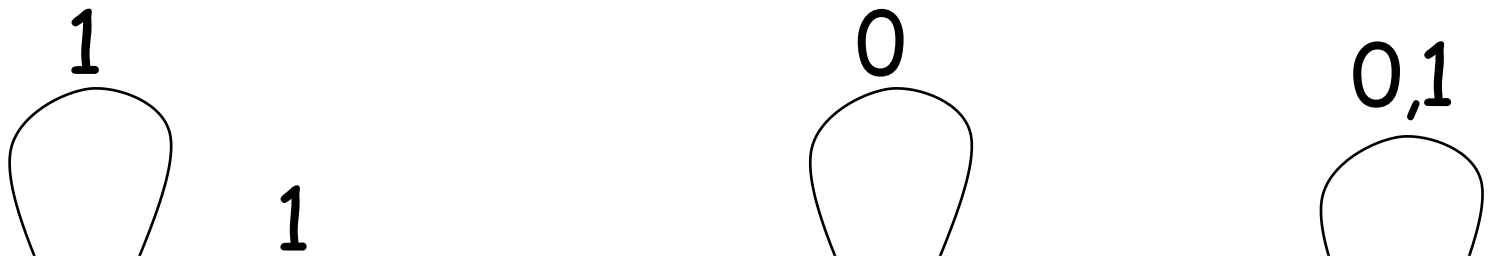
Example 2.3

$L(M)$ = { all strings with prefix ab } $\Sigma = \{a, b\}$



Example 2.4

$L(M)$ = { all strings without substring 001 }



Regular Languages

A language L is regular iff there exists some DFA M such that $L = L(M)$

All regular languages form a language family

Examples of regular languages:

$$\{abba\} \qquad \{\lambda, ab, abba\} \qquad \{a^n b : n \geq 0\}$$

{ all strings with prefix ab }

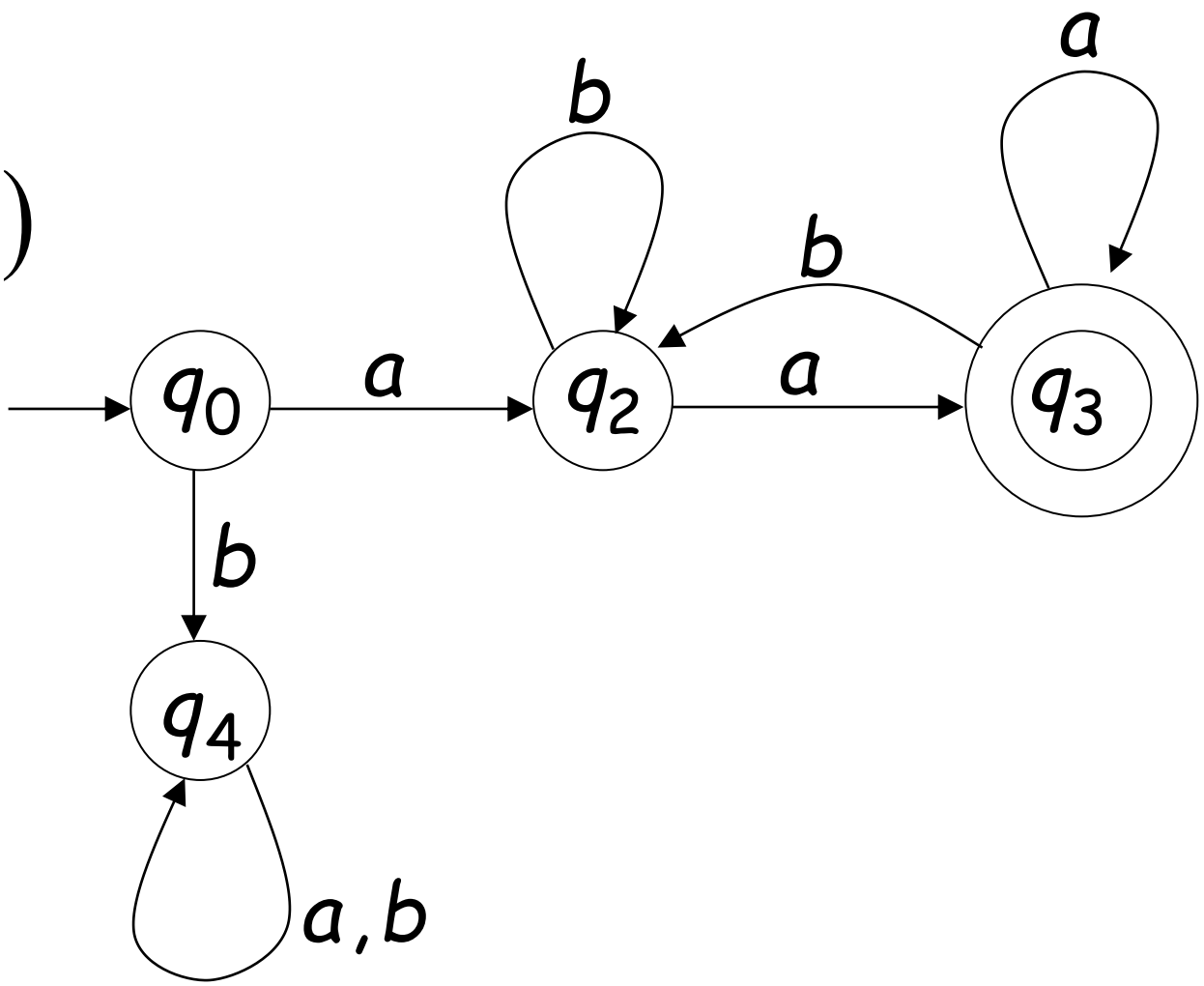
{ all strings without substring 001 }

There exists DFA that accept these Languages

Example 2.5

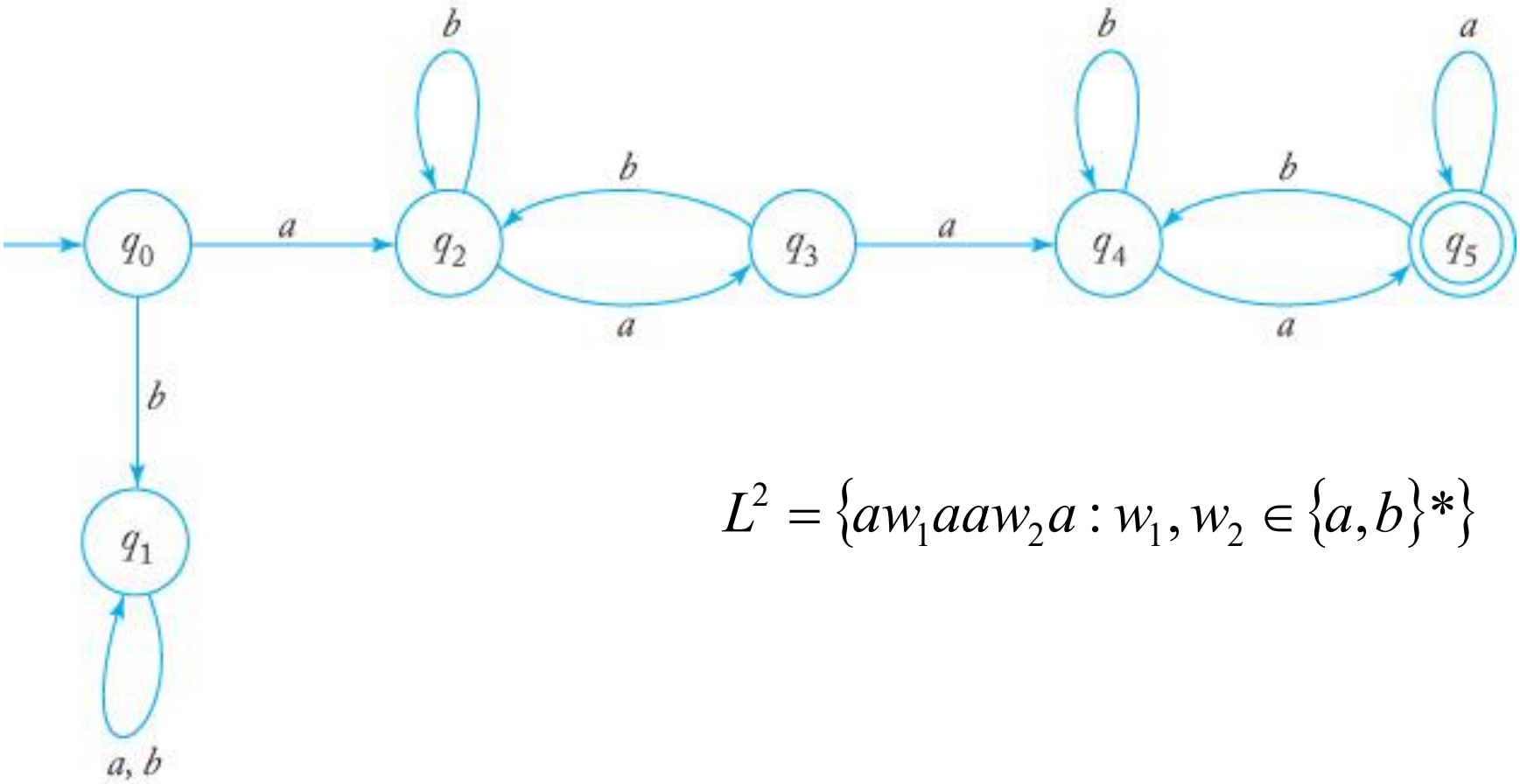
The language $L = \{awa : w \in \{a,b\}^*\}$ is regular:

$$L = L(M)$$



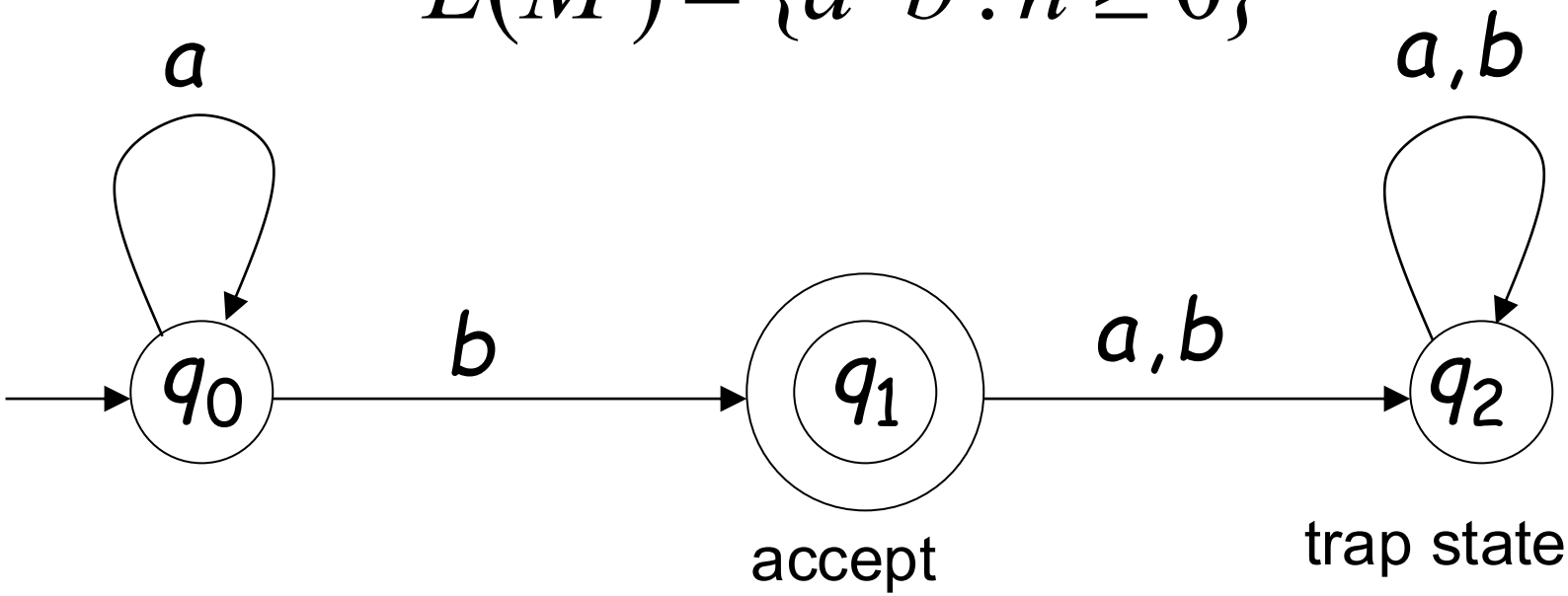
Example 2.6

The language $L = \{awa : w \in \{a,b\}^*\}$ is regular, how about L^2?



$$L^2 = \{aw_1aaw_2a : w_1, w_2 \in \{a,b\}^*\}$$

$$L(M) = \{a^n b : n \geq 0\}$$



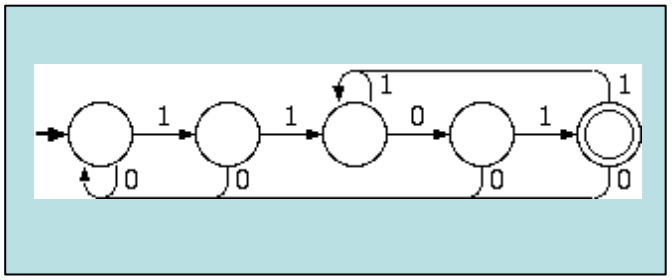
$$L = \{a^n b^n : n \geq 0\} \; ?$$

There exist languages which are not Regular:

There is no DFA that accepts such a language (we will prove this later in the class)

DFA Recap

Input String



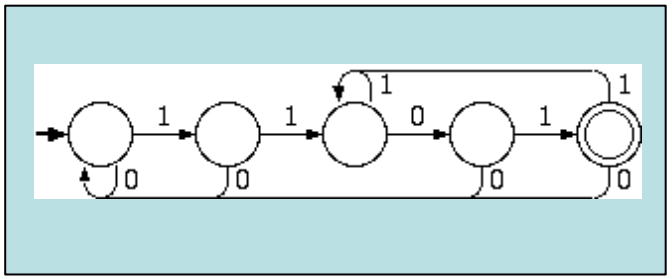
DFA

Accept or Reject

- A machine with finite number of states, some states are accepting states, others are rejecting states
- At any time, it is in one of the states
- It reads an input string, one character at a time

DFA Recap

Input String



DFA

Accept or Reject

- After reading each character, it moves to another state depending on what is read and what is the current state
- If reading all characters, the DFA is in an accepting state, the input string is accepted.
- Otherwise, the input string is rejected.

Definition 2.1

Deterministic Finite Accepter (DFA) is define by the 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : a finite set of **internal states**

Σ : a finite set of symbols called **input alphabet**

δ : Q x Σ → Q called **transition function**

q_0 : $q_0 \in Q$ is the **initial state**

F : $F \subseteq Q$ is a set of **final states**

Regular Languages

A language L is regular iff there exists some DFA M such that $L = L(M)$

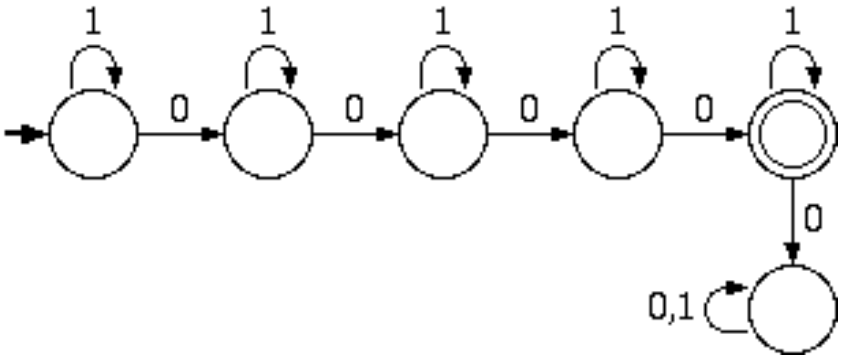
Definition:

- The language $L(M)$ contains all input strings accepted by a DFA M

- $L(M)$= { strings that drive M to a final state}

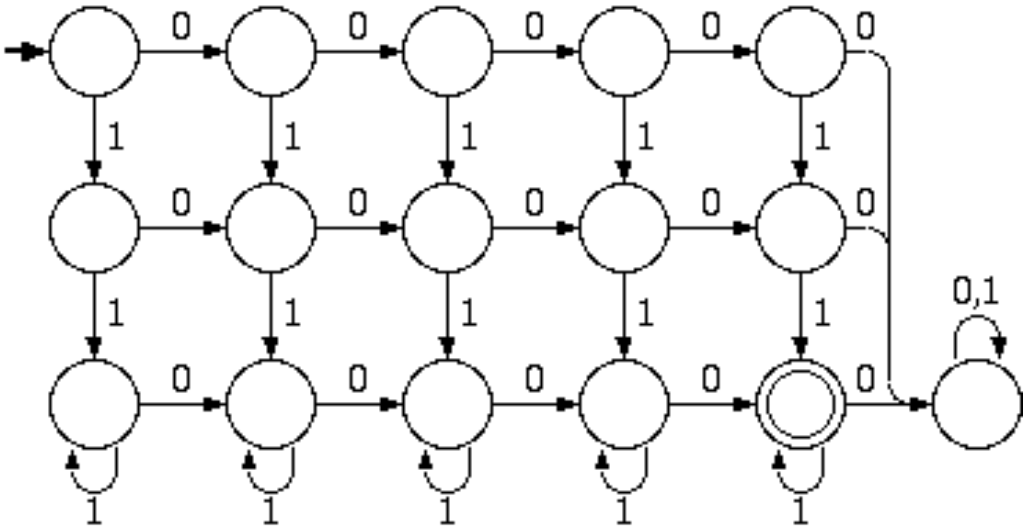
More Examples

- All strings that contain exactly 4 "0"s.
- All strings containing exactly 4 "0" s and at least 2 "1" s.
- All strings of length at most five.
- All strings ending in "1101".
- All strings whose binary interpretation is divisible by 5.
- All strings that contain the substring 0101.
- All strings that don’t contain the substring 110.



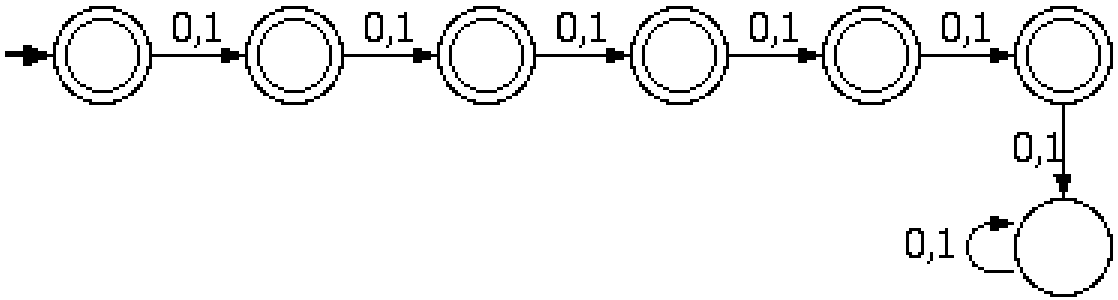
More Examples

- All strings that contain exactly 4 "0"s.
- All strings containing exactly 4 "0" s and at least 2 "1" s.
- All strings of length at most five.
- All strings ending in "1101".
- All strings whose binary interpretation is divisible by 5.
- All strings that contain the substring 0101.
- All strings that don't contain the substring 110.



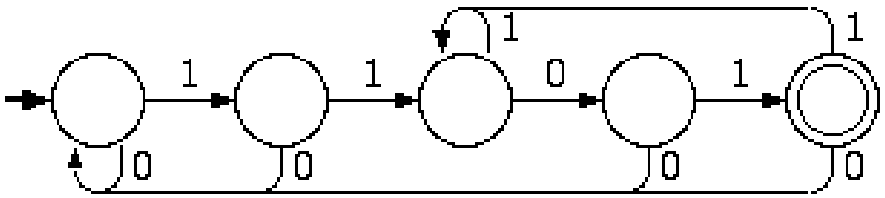
More Examples

- All strings that contain exactly 4 "0"s.
- All strings containing exactly 4 "0" s and at least 2 "1" s.
- All strings of length at most five.
- All strings ending in "1101".
- All strings whose binary interpretation is divisible by 5.
- All strings that contain the substring 0101.
- All strings that don’t contain the substring 110.



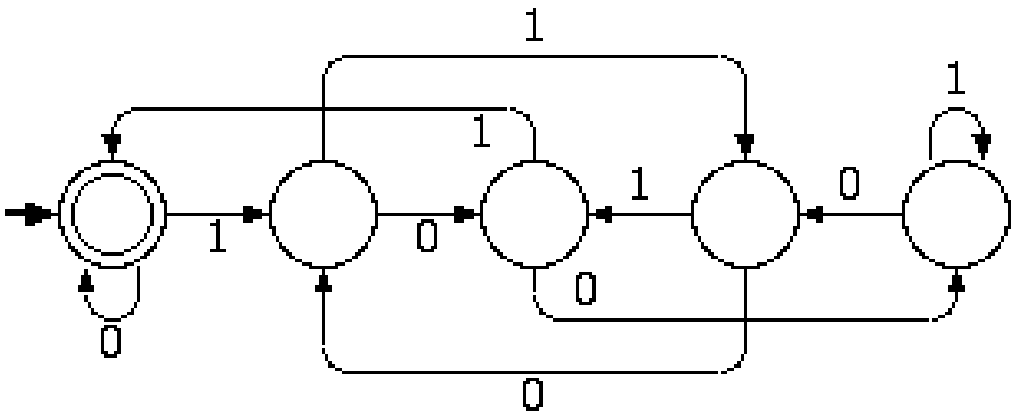
More Examples

- All strings that contain exactly 4 "0"s.
- All strings containing exactly 4 "0" s and at least 2 "1" s.
- All strings of length at most five.
- All strings ending in "1101".
- All strings whose binary interpretation is divisible by 5.
- All strings that contain the substring 0101.
- All strings that don’t contain the substring 110.



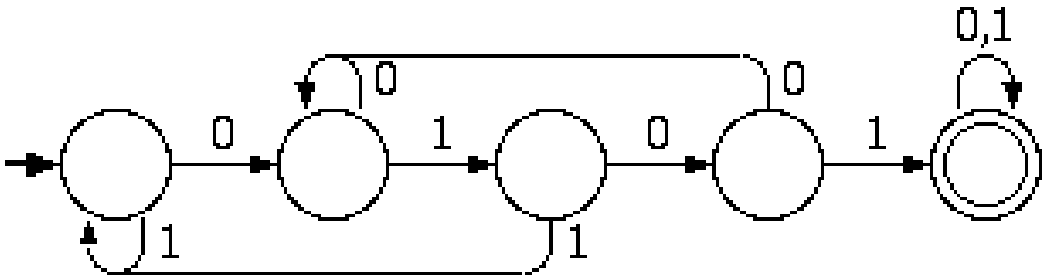
More Examples

- All strings that contain exactly 4 "0"s.
- All strings containing exactly 4 "0" s and at least 2 "1" s.
- All strings of length at most five.
- All strings ending in "1101".
- All strings whose binary interpretation is divisible by 5.
- All strings that contain the substring 0101.
- All strings that don’t contain the substring 110.



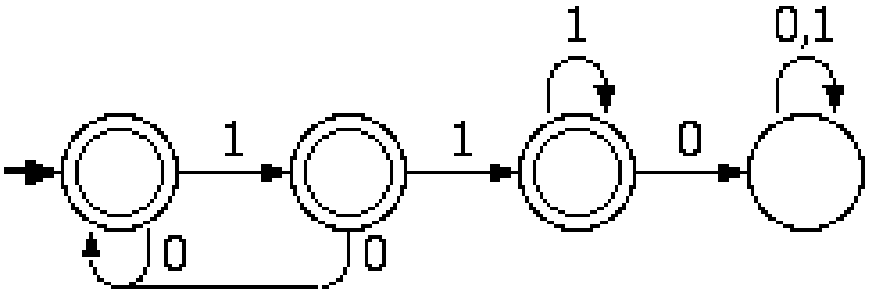
More Examples

- All strings that contain exactly 4 "0"s.
- All strings containing exactly 4 "0" s and at least 2 "1" s.
- All strings of length at most five.
- All strings ending in "1101".
- All strings whose binary interpretation is divisible by 5.
- All strings that contain the substring 0101.
- All strings that don’t contain the substring 110.



More Examples

- All strings that contain exactly 4 "0"s.
- All strings containing exactly 4 "0" s and at least 2 "1" s.
- All strings of length at most five.
- All strings ending in "1101".
- All strings whose binary interpretation is divisible by 5.
- All strings that contain the substring 0101.
- All strings that don’t contain the substring 110.

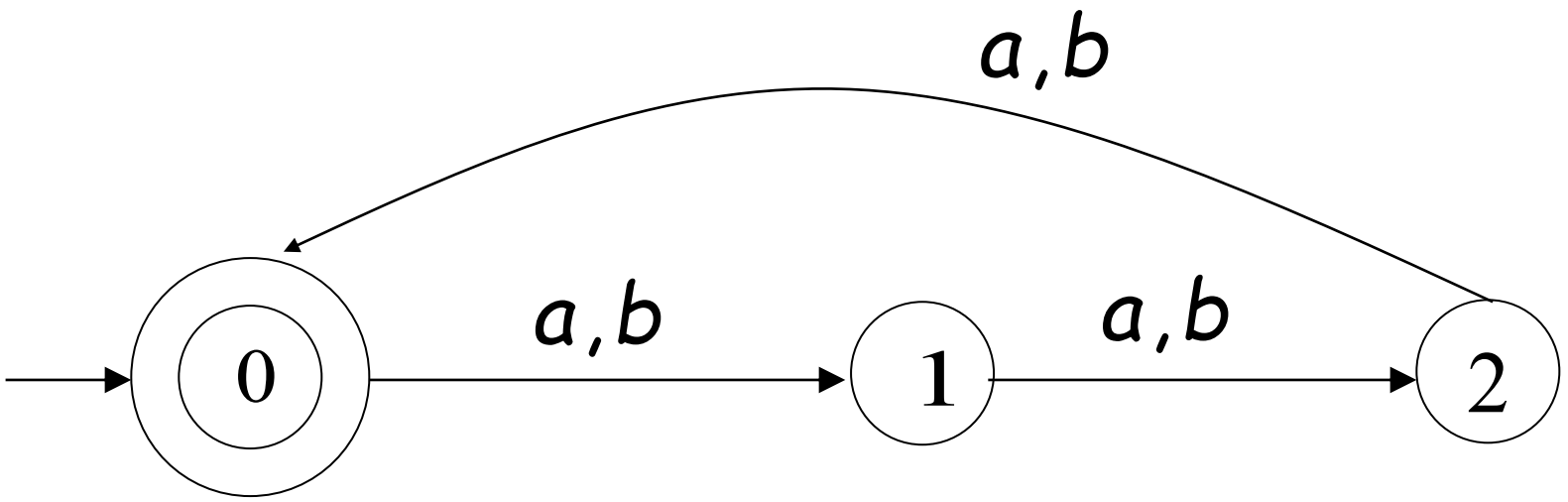


Exercise 2.1.7

Find DFAs for the following languages on ∑={a,b}

(a) L = {w: |w| mod 3 = 0}

(b) L = {w: $n_a(w)$ mod 3 > $n_b(w)$ mod 3}

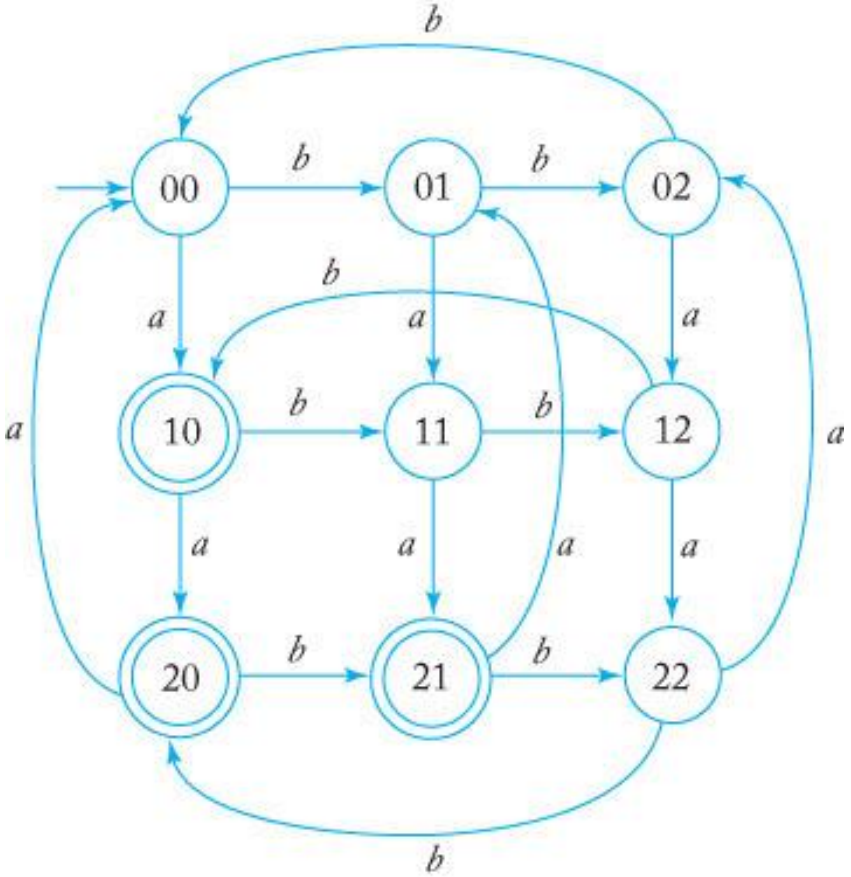


Exercise 2.1.7

Find DFAs for the following languages on ∑={a,b}

(a) L = {w: |w| mod 3 = 0}

(b) L = {w: $n_a(w)$ mod 3 > $n_b(w)$ mod 3}



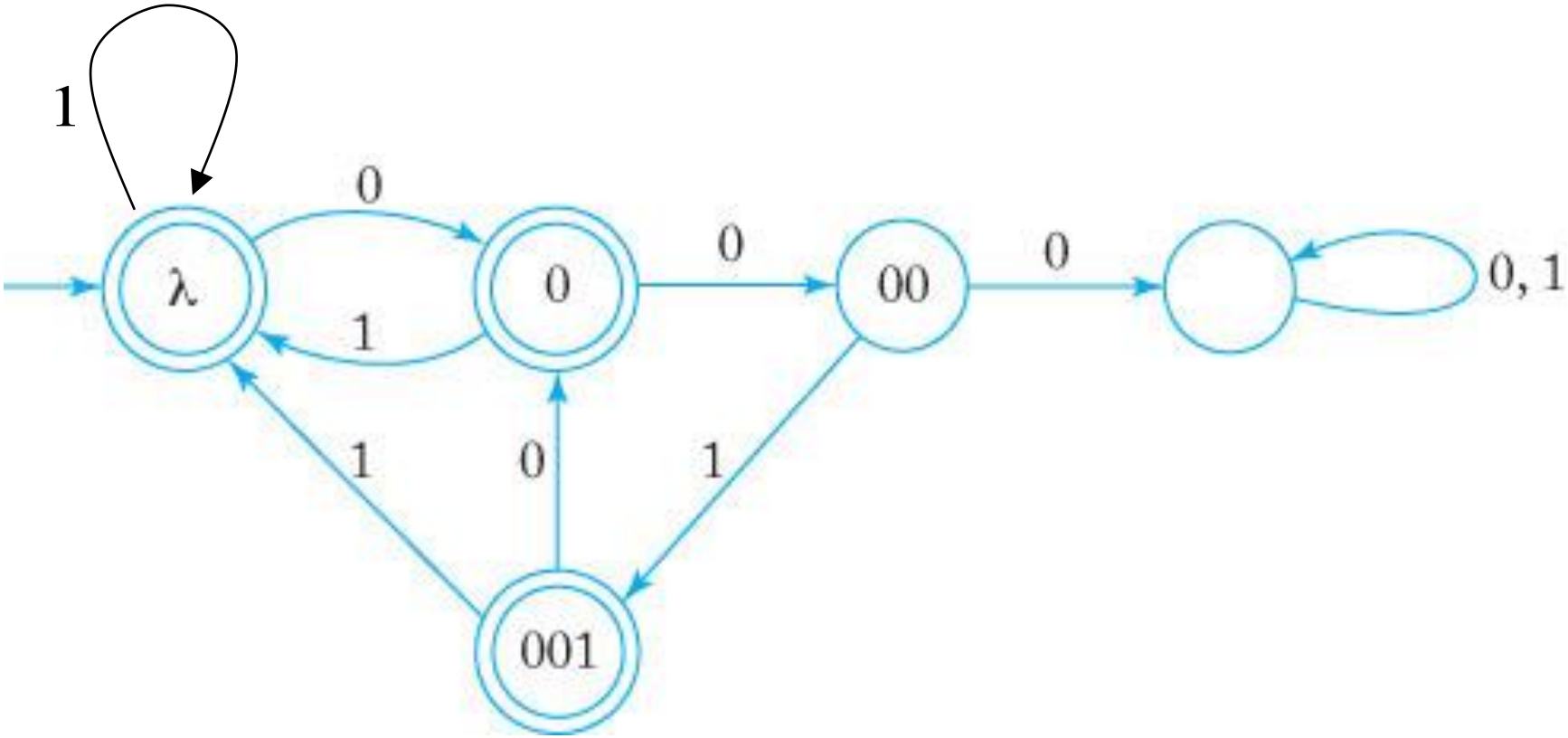
Exercise 2.1.9

(a)Every 00 is followed immediately by a 1.

Ex: 101, 0010,0010011001 ϵ L

0001 and 00100 ∉ L

$\Sigma = \{0,1\}$

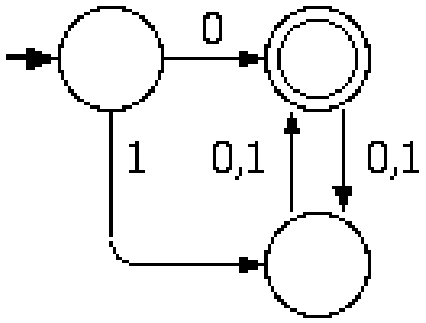


More Examples

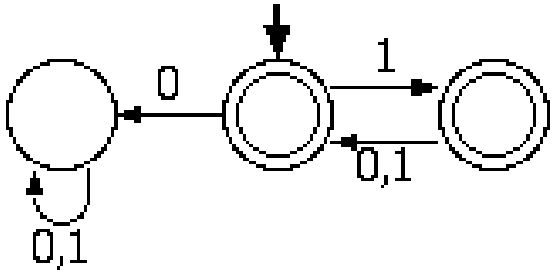
- All strings that start with 0 and have odd length or start with 1 and have even length.
- All strings where every odd position is a 1.

More Examples

- All strings that start with 0 and have odd length or start with 1 and have even length.



- All strings where every odd position is a 1.



Outline



Deterministic Finite Accepters (DFA)



Nondeterministic Finite Accepters (NFA)



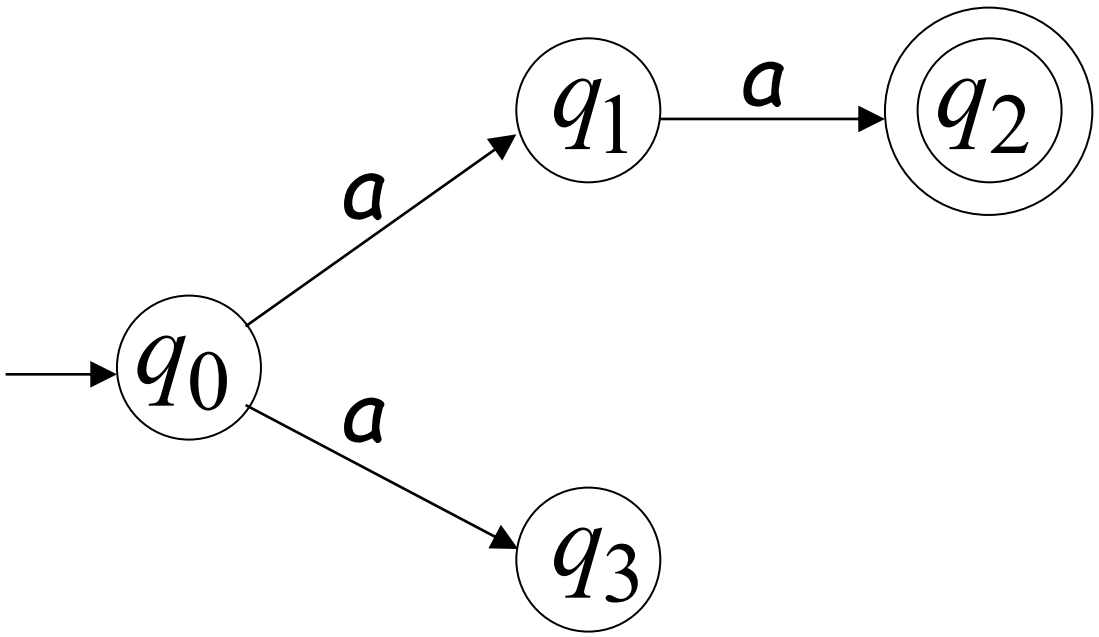
Equivalence of DFA and NFA



Reduction of the Number of States in FA*

Nondeterministic Finite Accepter (NFA)

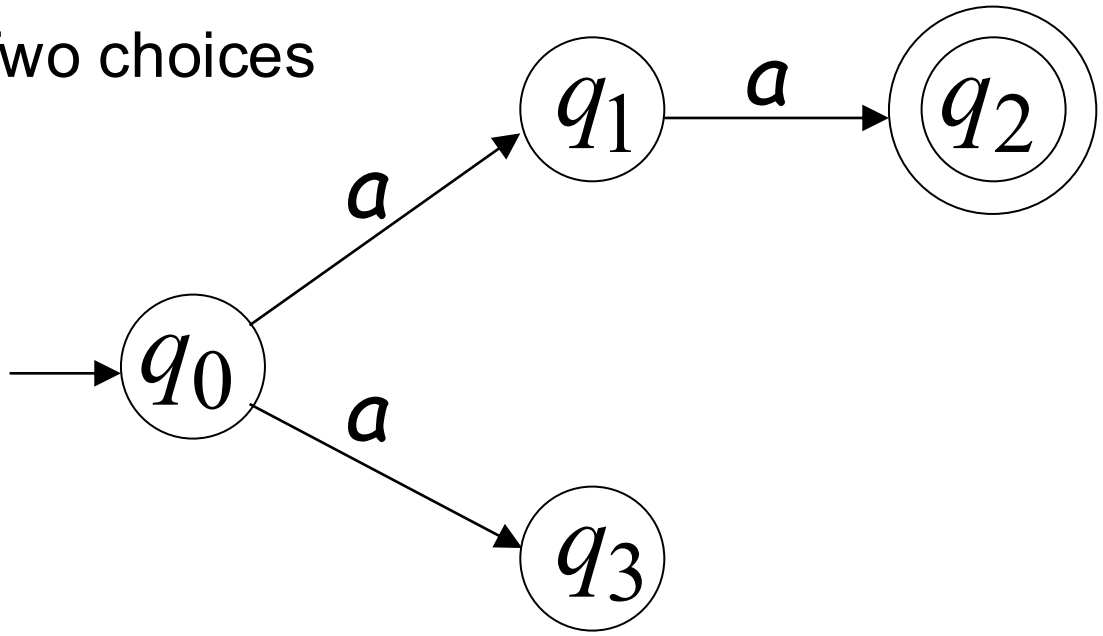
Alphabet = $\{a\}$



Nondeterministic Finite Accepter (NFA)

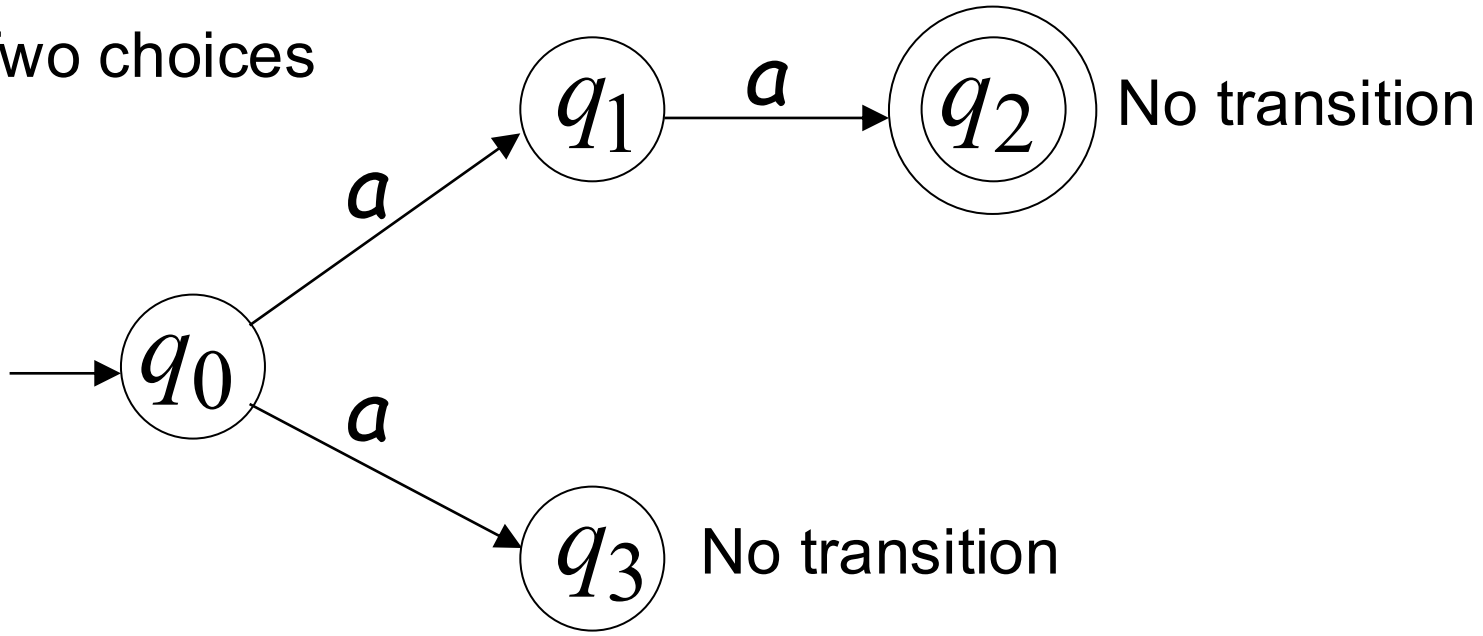
Alphabet = $\{a\}$

Two choices

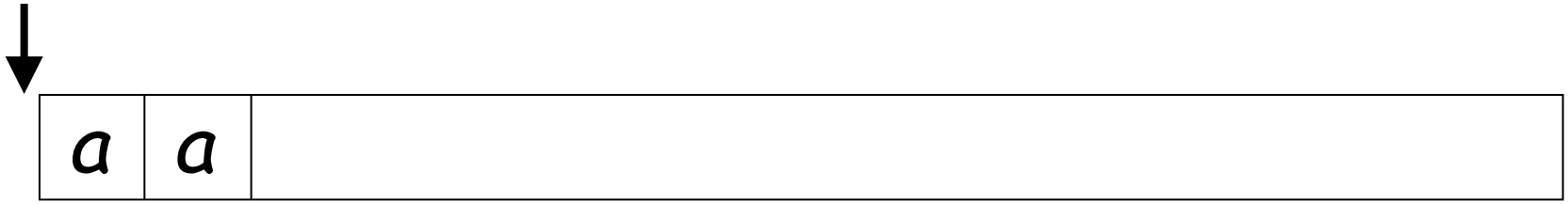


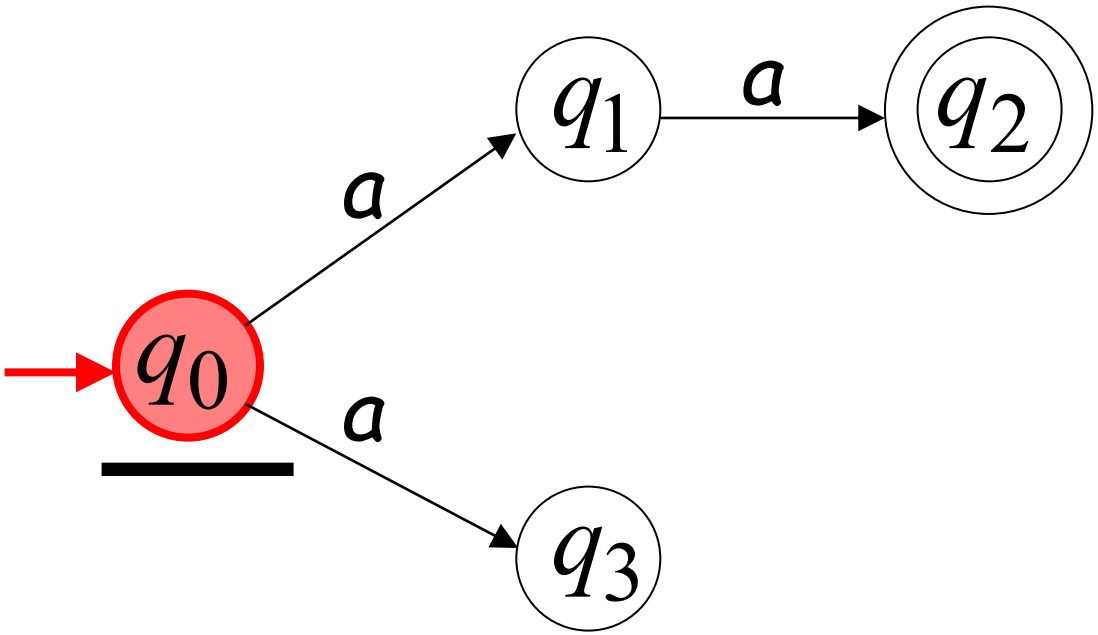
Nondeterministic Finite Accepter (NFA)

Alphabet = $\{a\}$

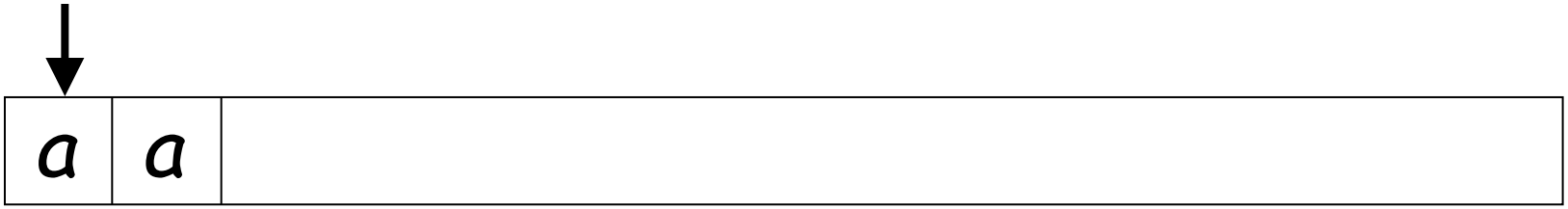


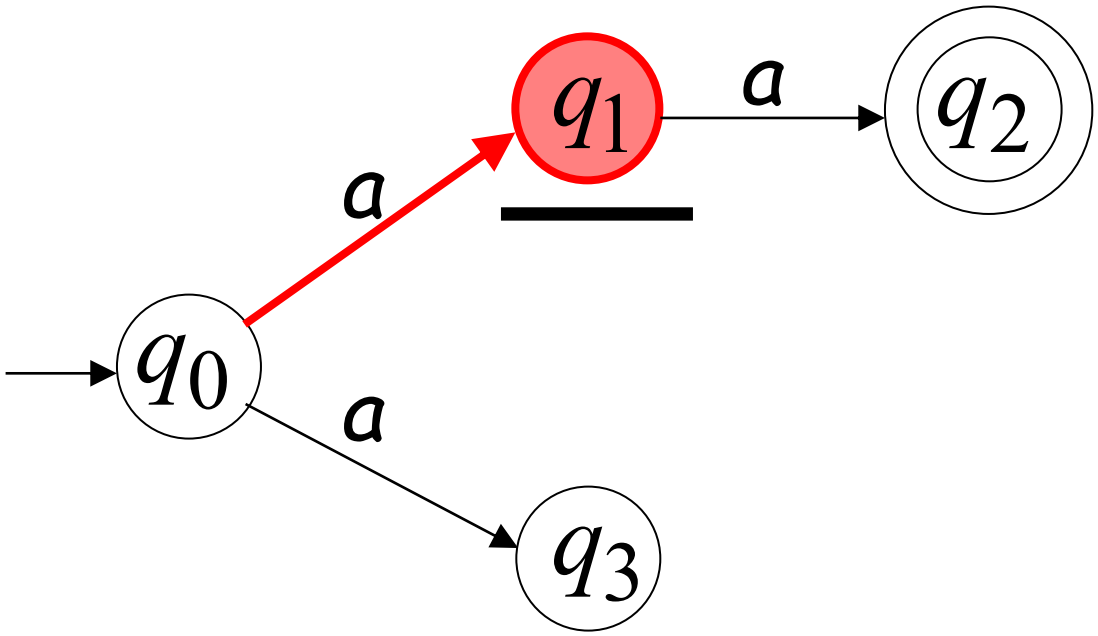
First Choice



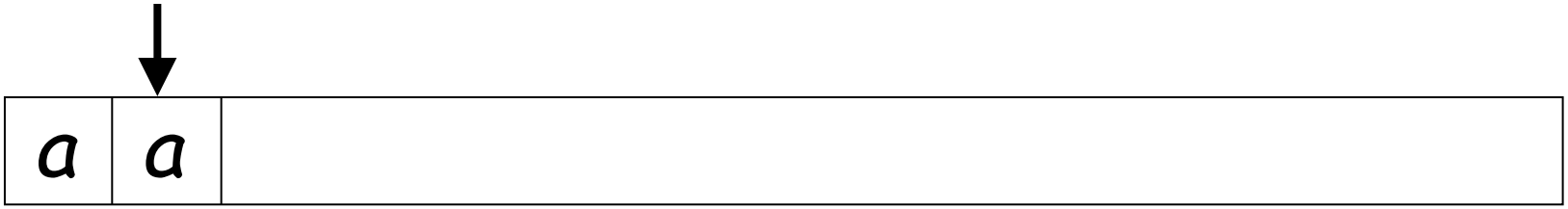


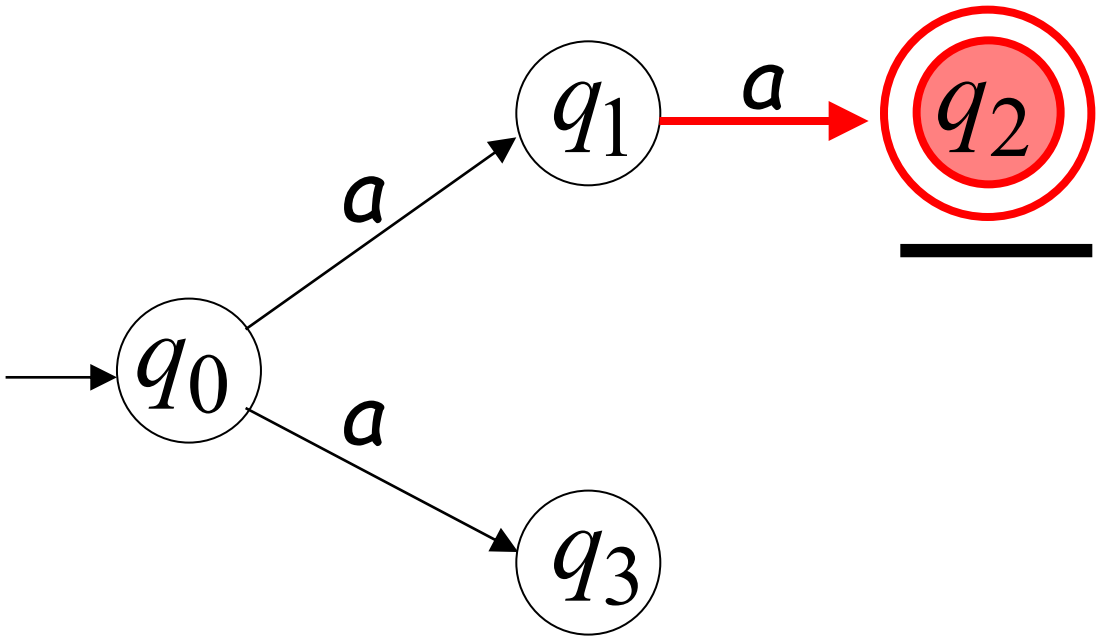
First Choice



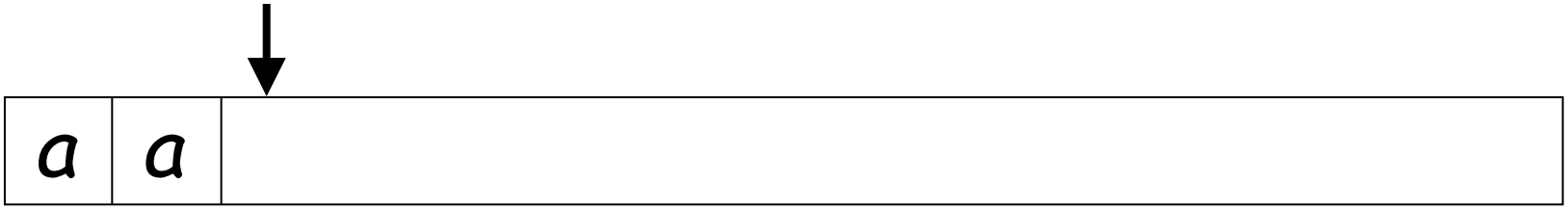


First Choice

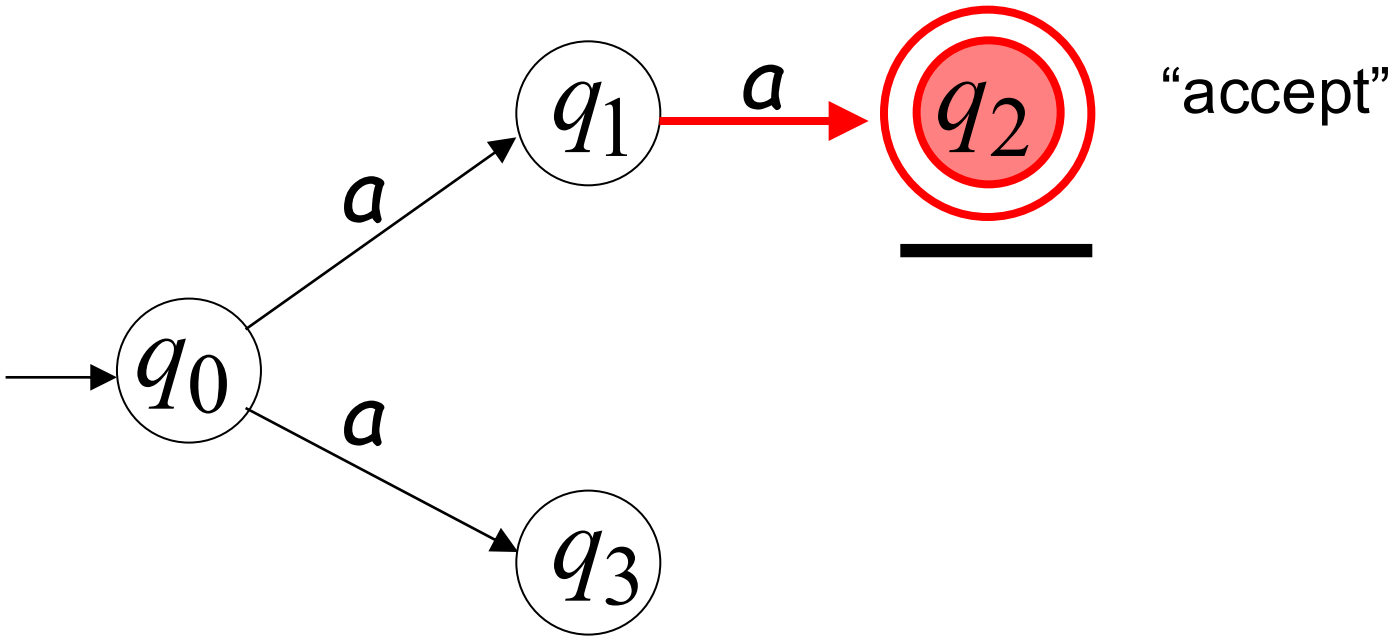




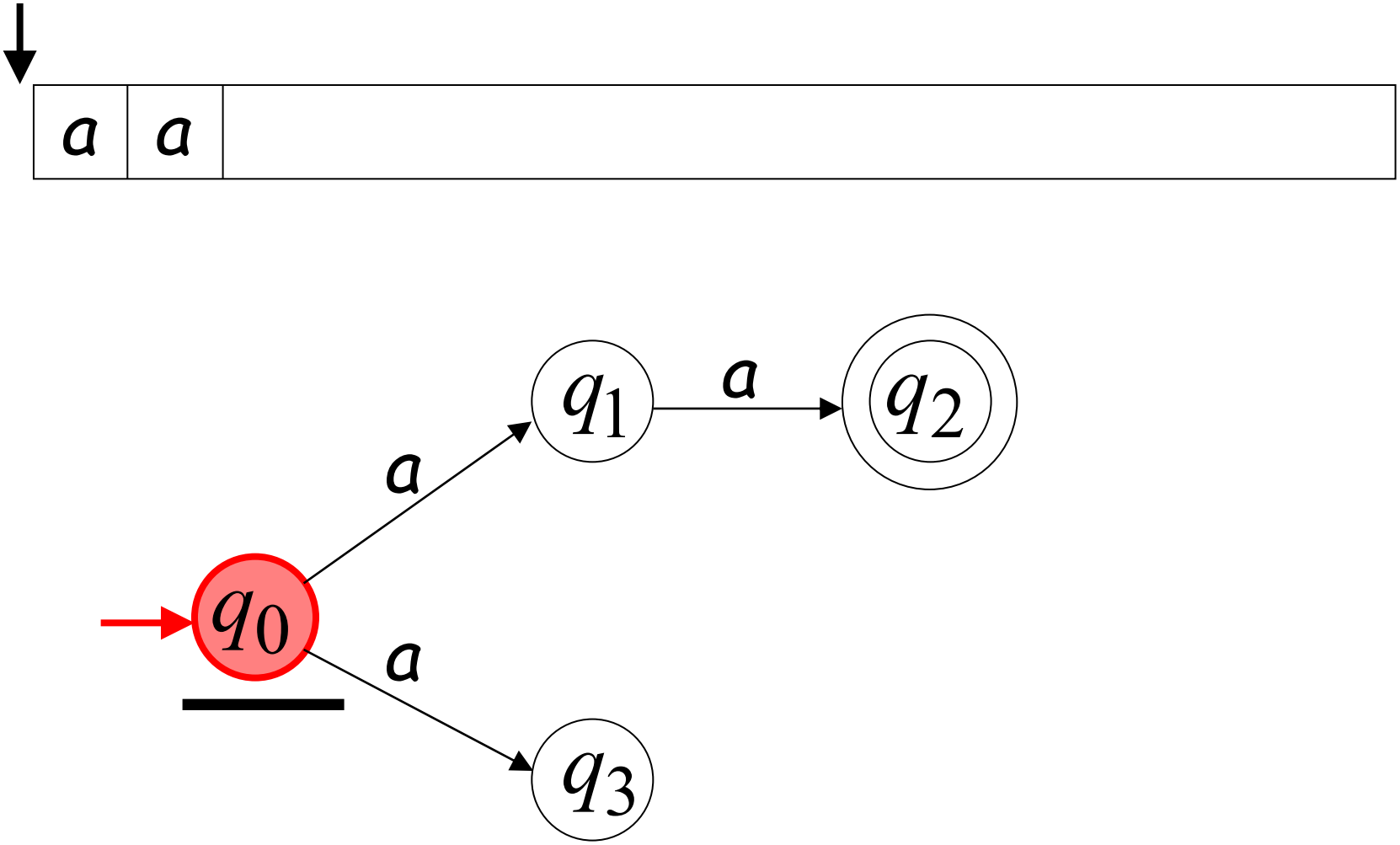
First Choice



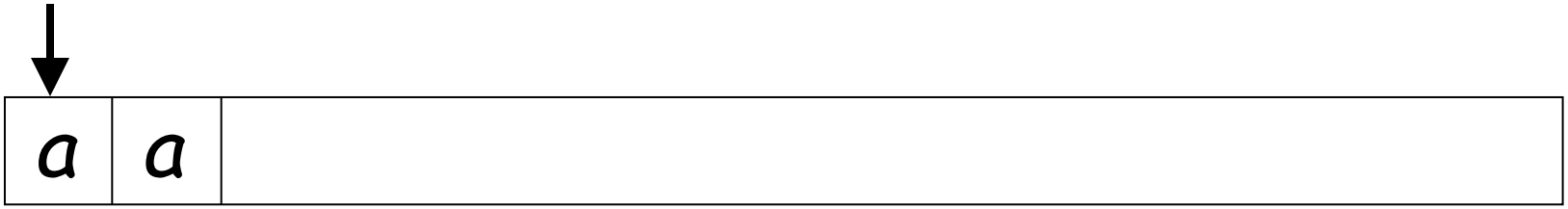
All input is consumed

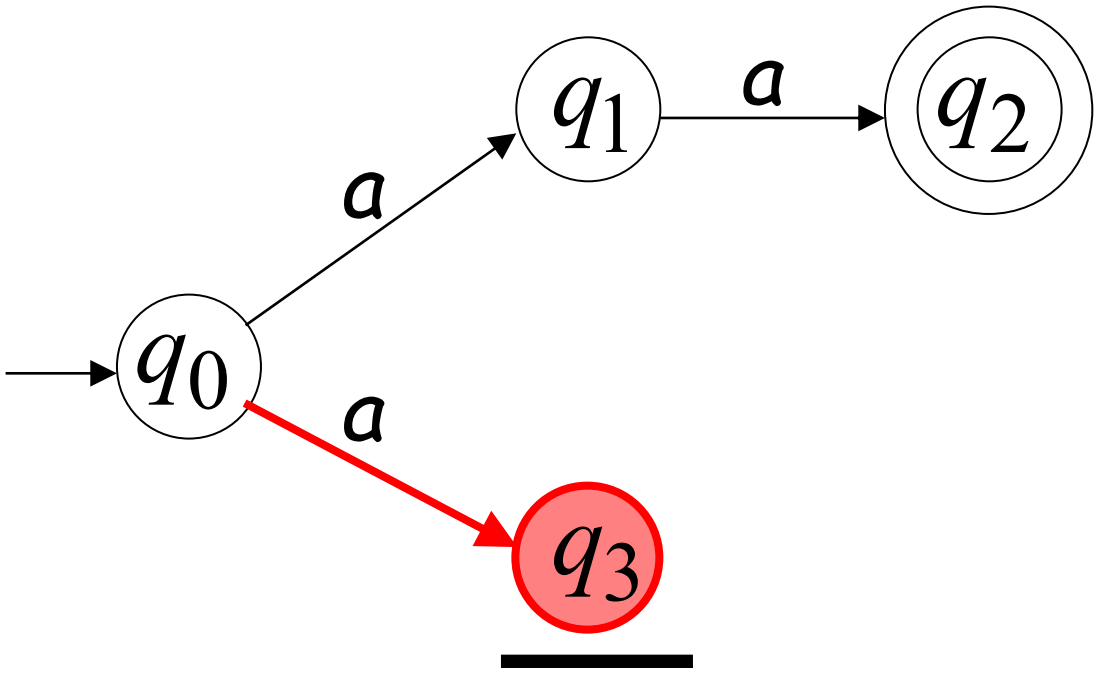


Second Choice

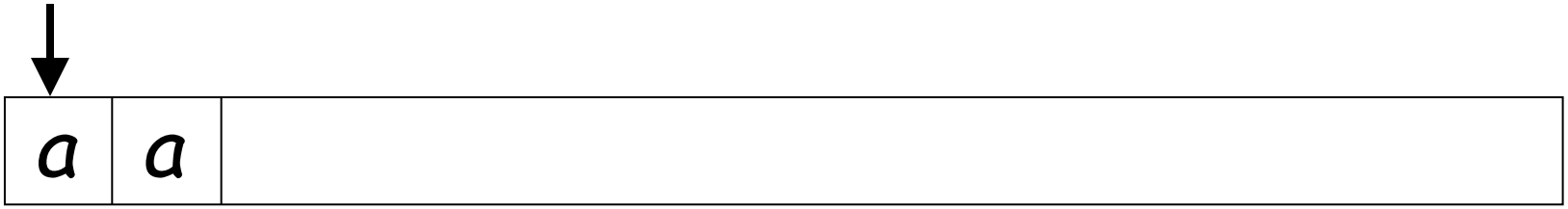


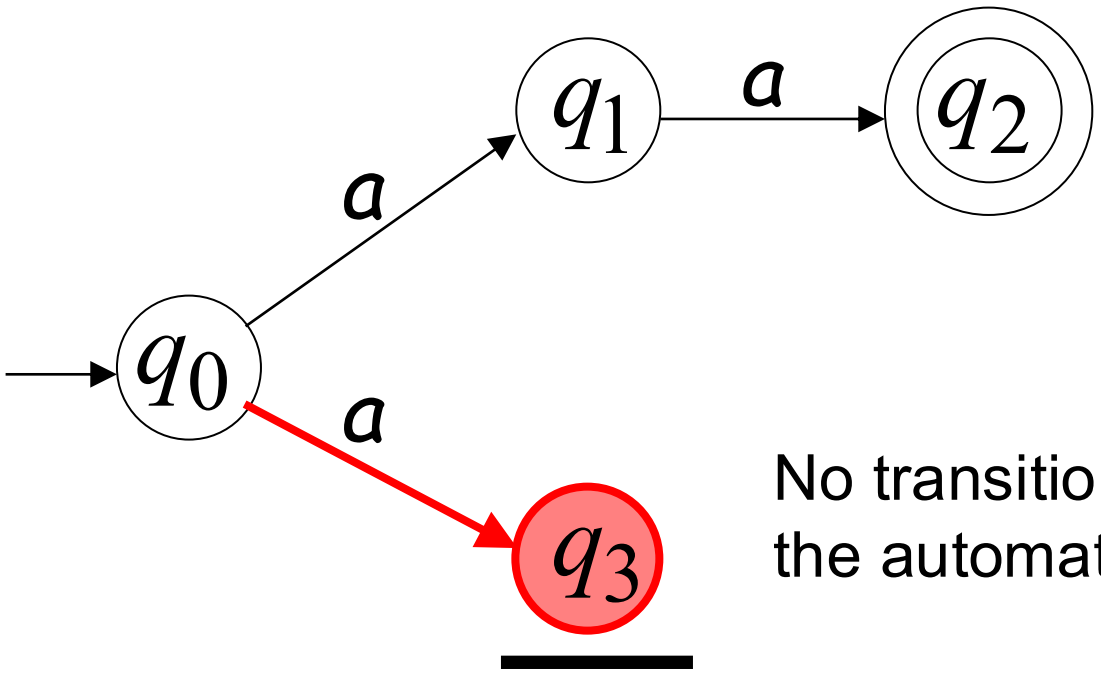
Second Choice





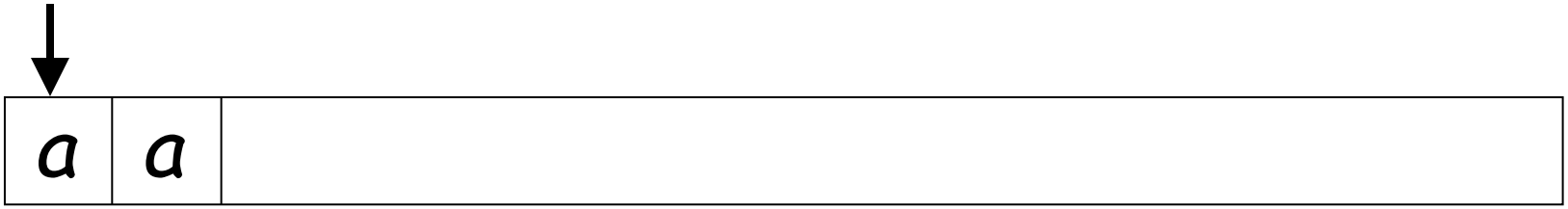
Second Choice



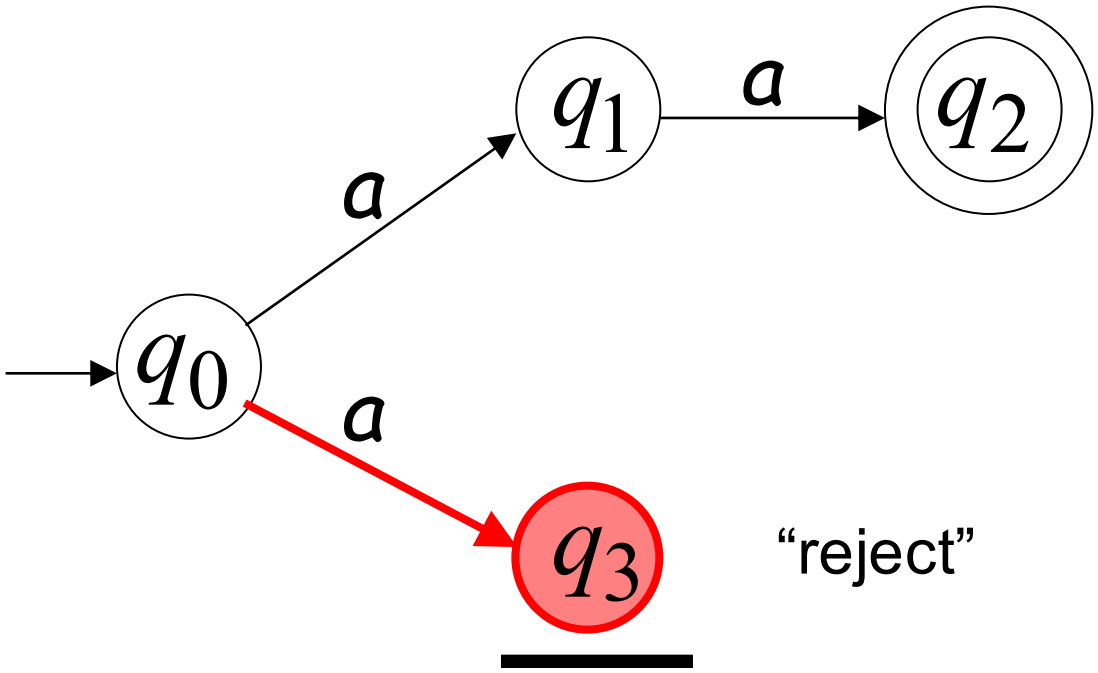


No transition:
the automaton hangs

Second Choice



Input cannot be consumed



"reject"

An NFA accepts a string:

when there is a computation of the NFA that accepts the string

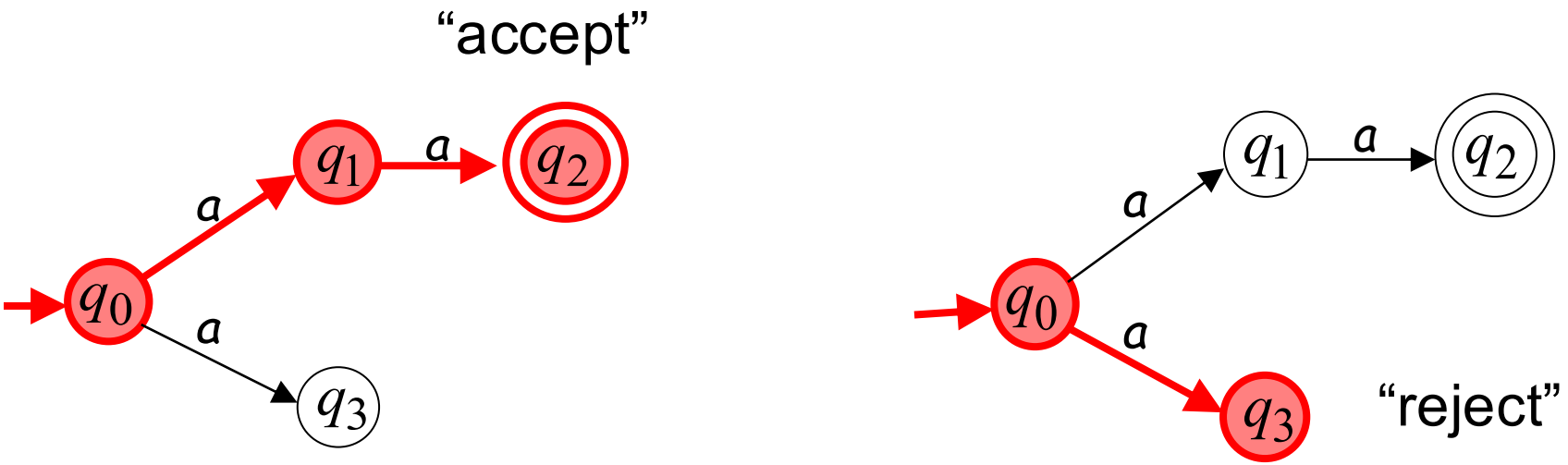
all the input is consumed

AND

the automaton is in a final state

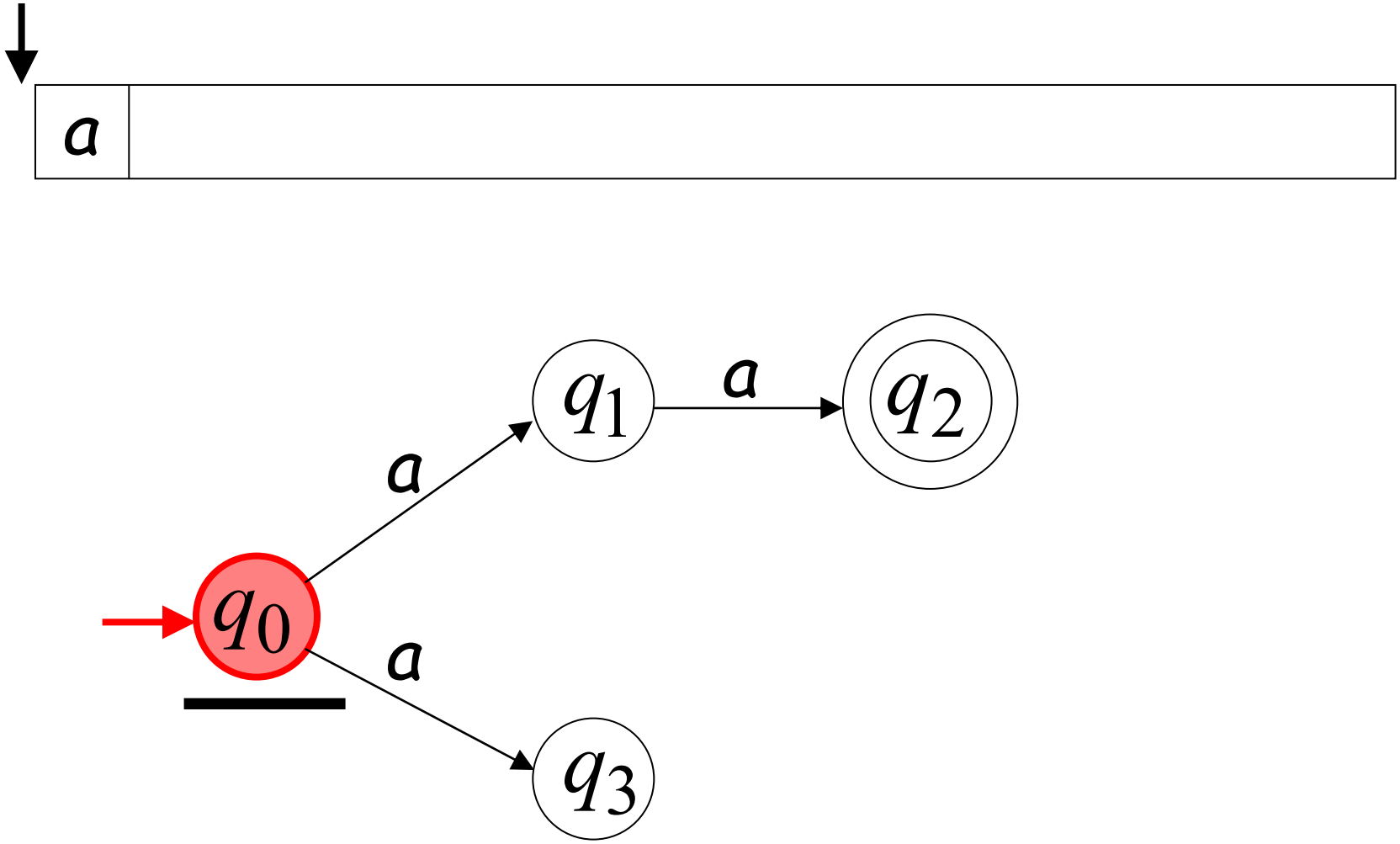
Example

aa is accepted by the NFA:

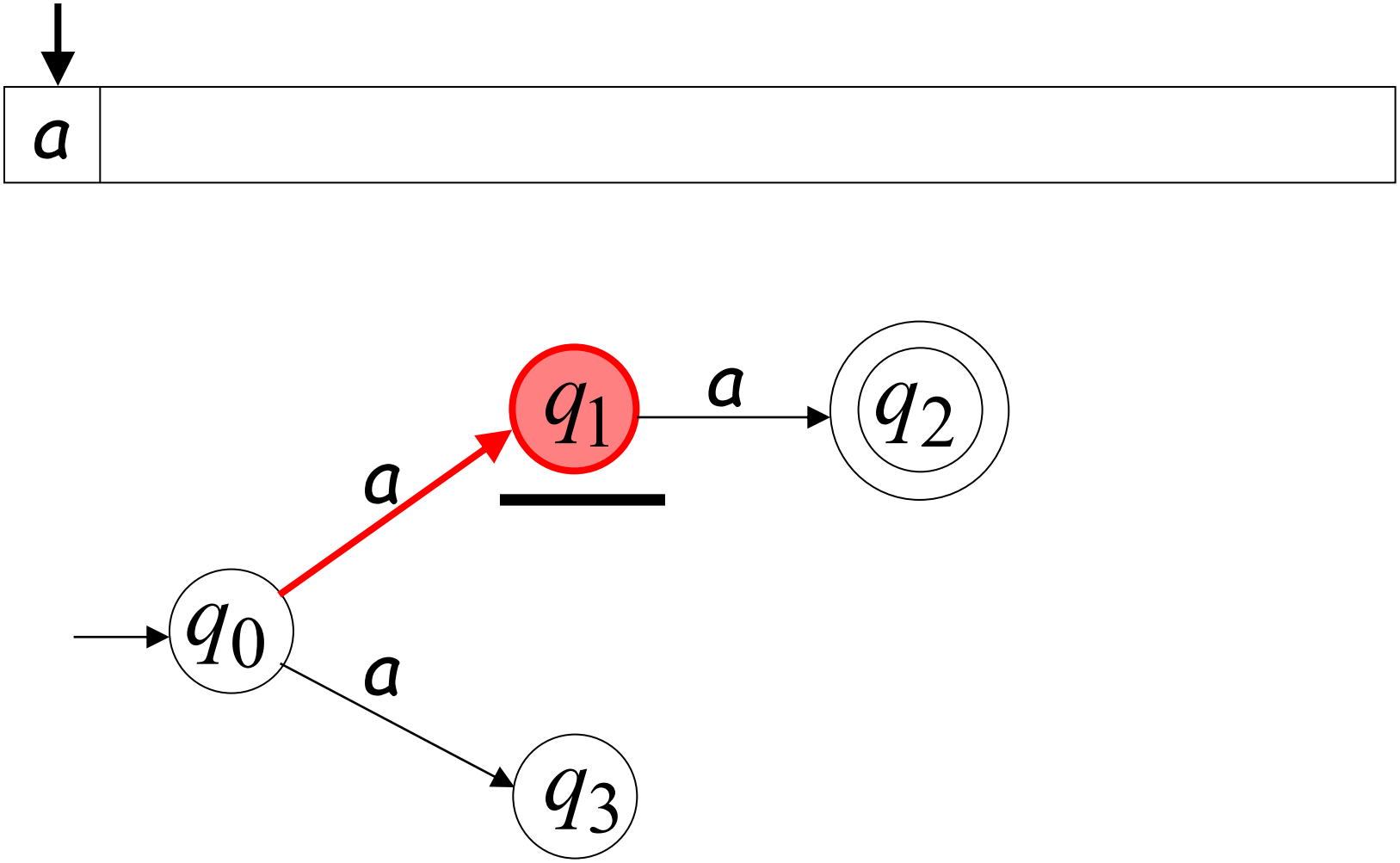


because this computation accepts aa

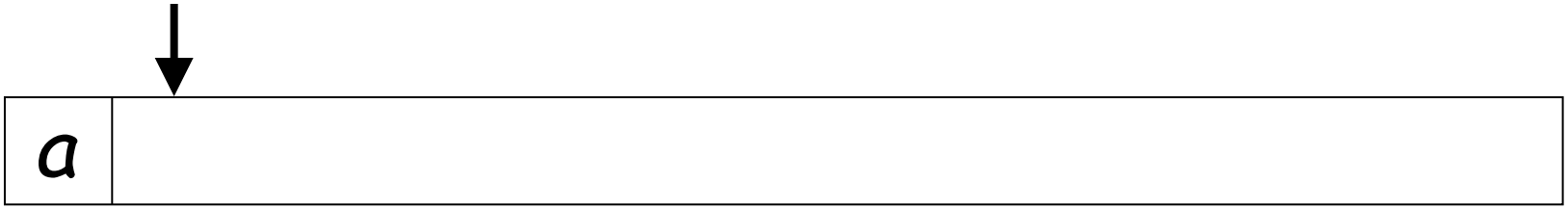
Rejection example



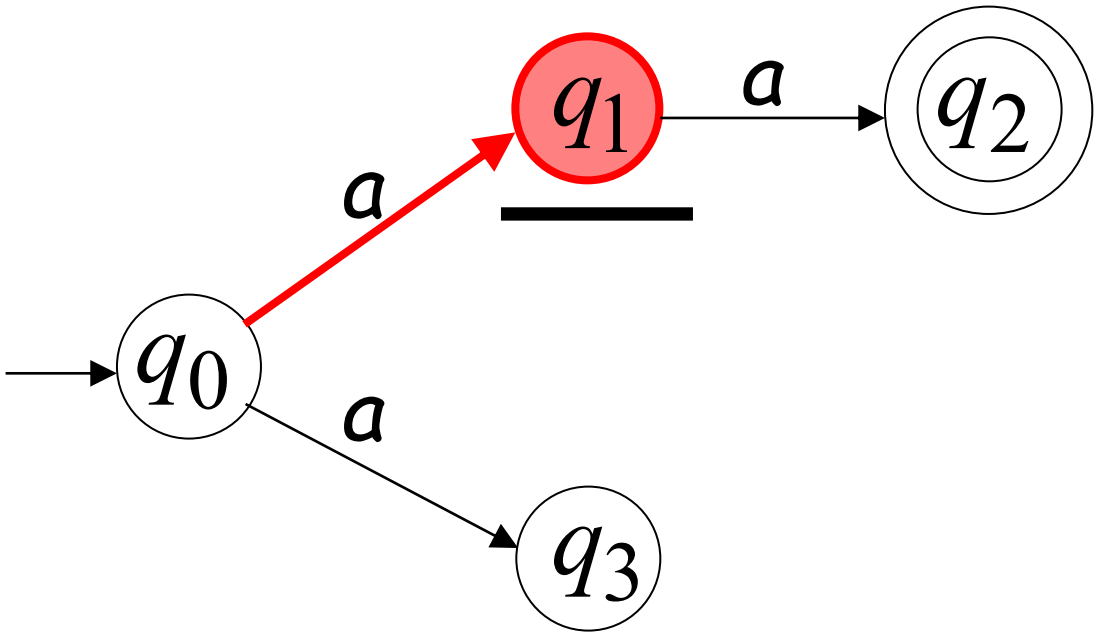
First Choice



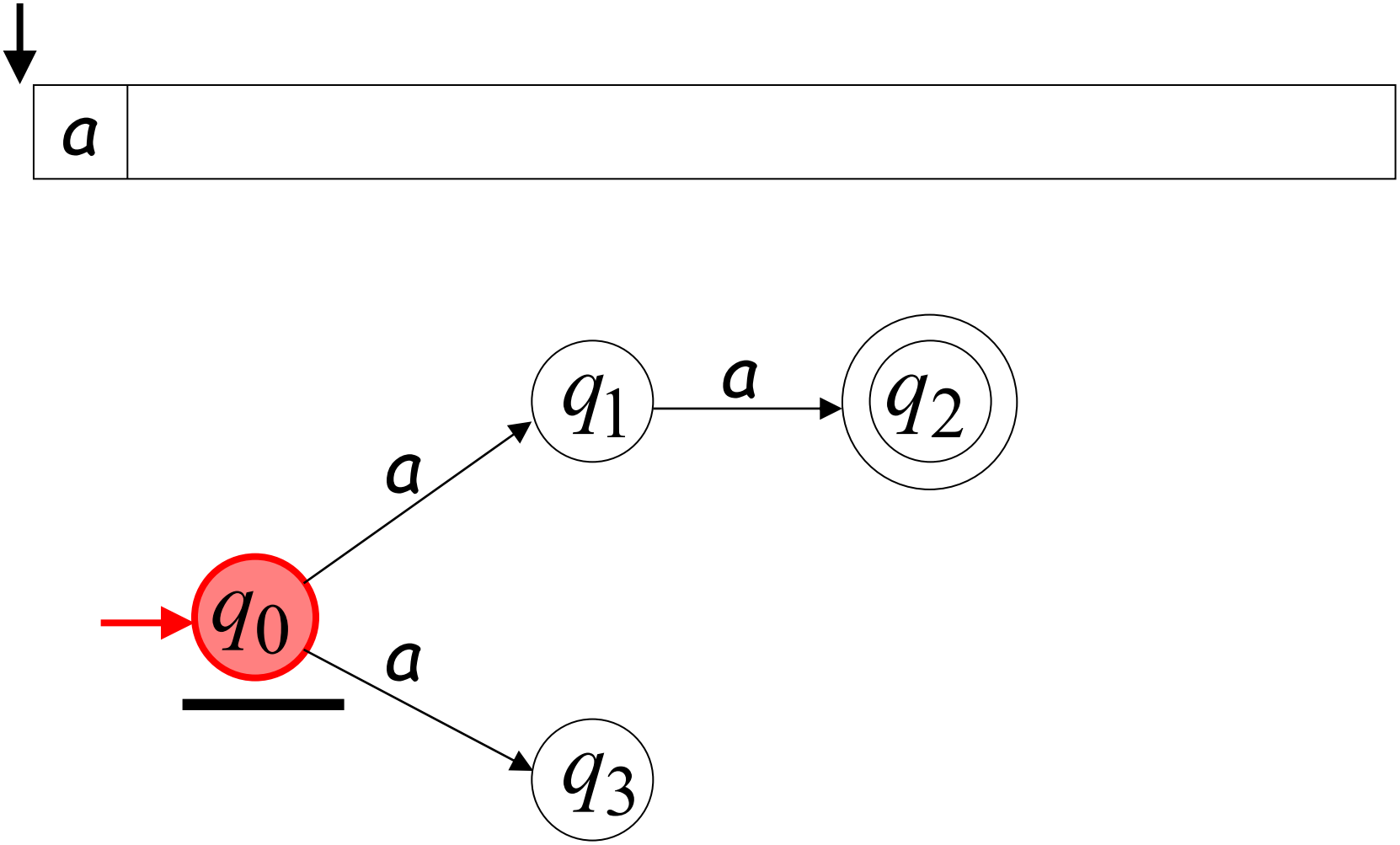
First Choice



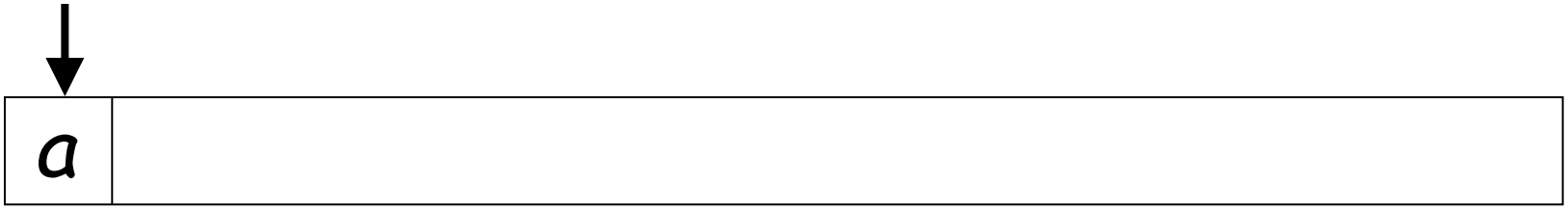
"reject"

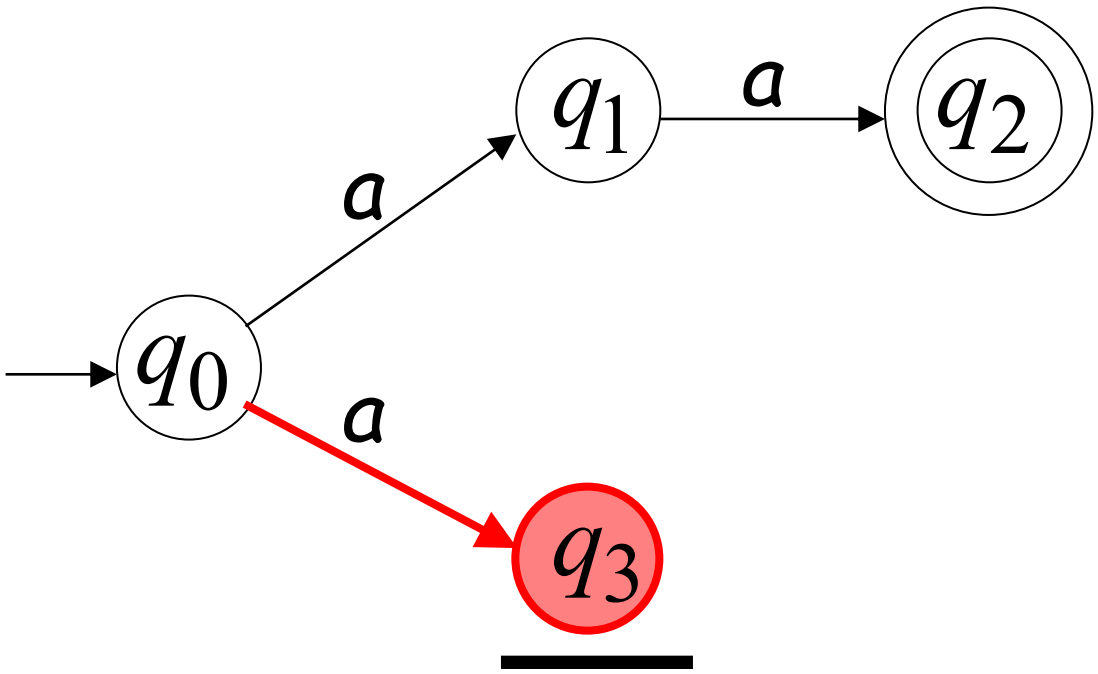


Second Choice

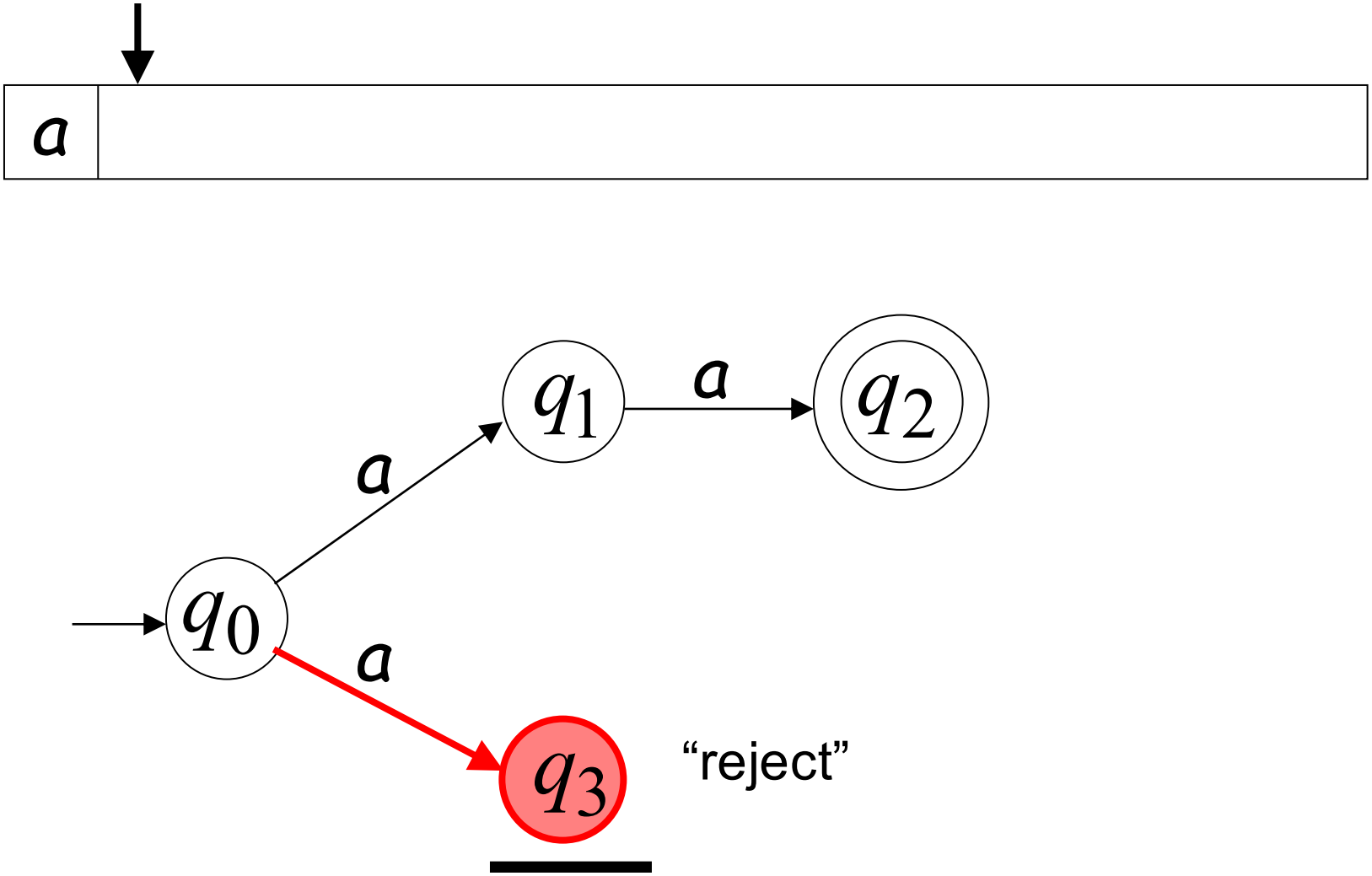


Second Choice





Second Choice



An NFA rejects a string:

when there is no computation of the NFA that accepts the string:

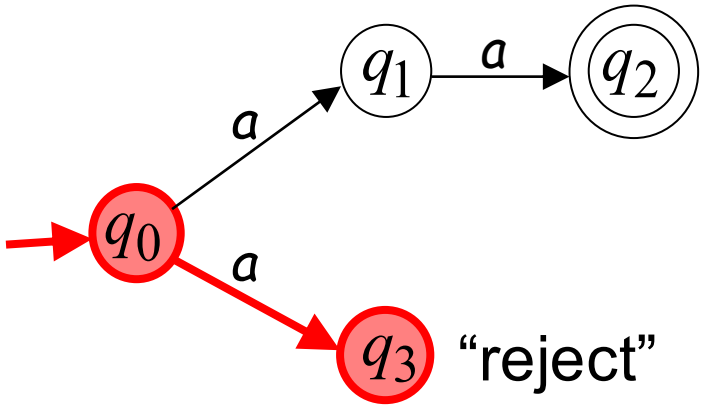
All the input is consumed and the automaton is in a non-final state

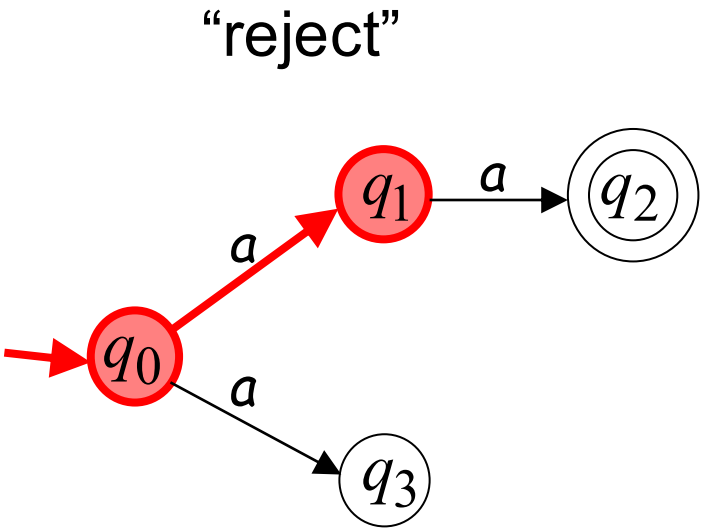
OR

The input cannot be consumed

Example

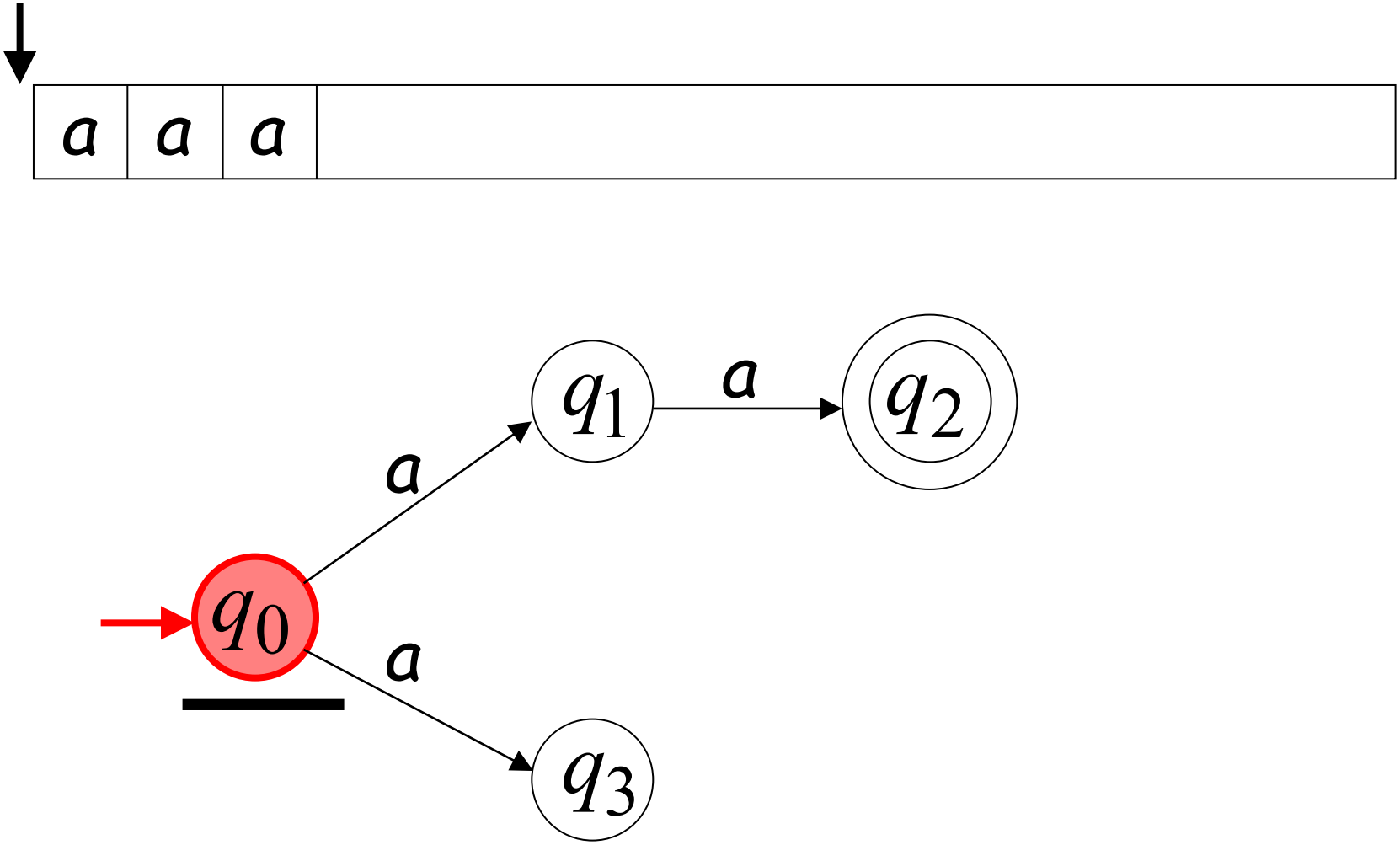
a is rejected by the NFA:



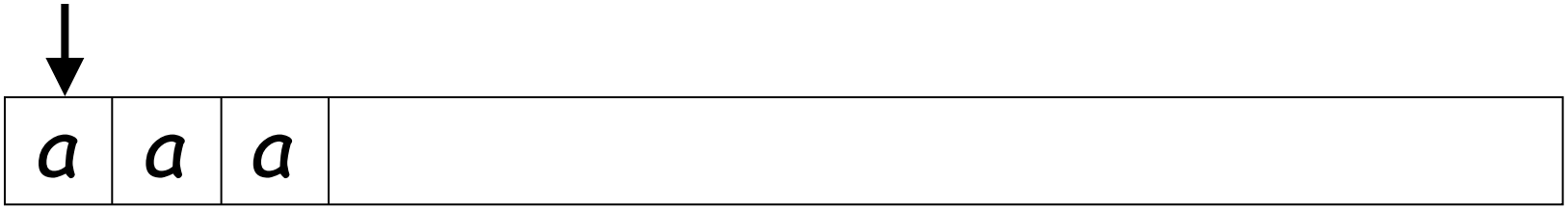


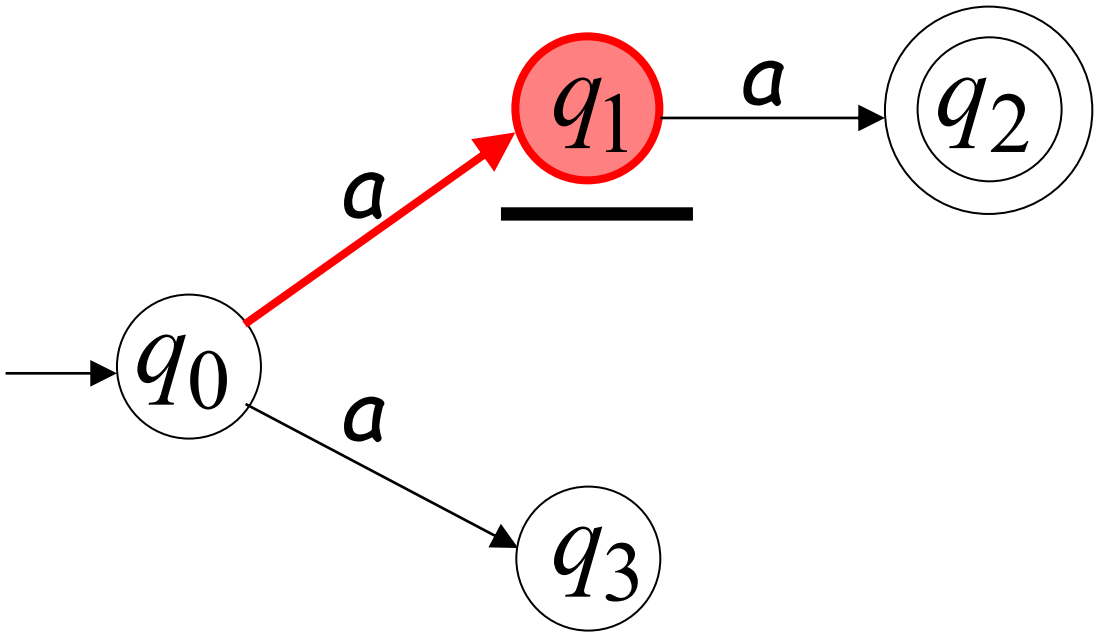
All possible computations lead to rejection

Rejection example

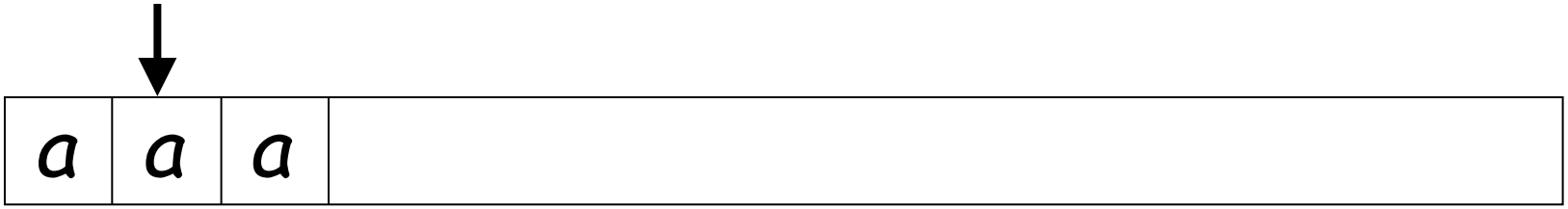


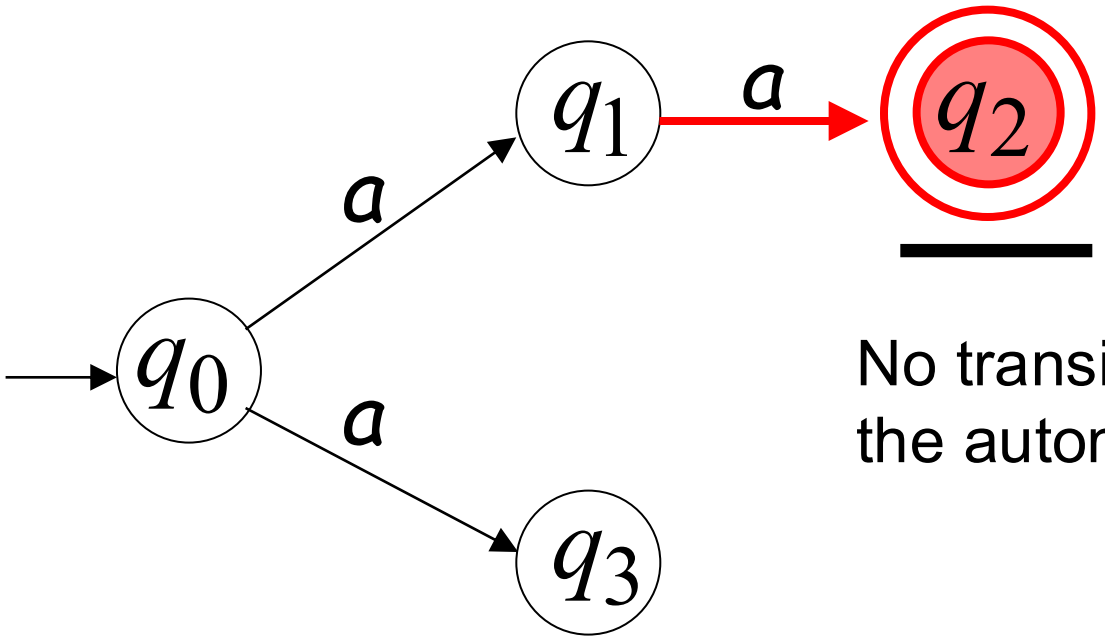
First Choice





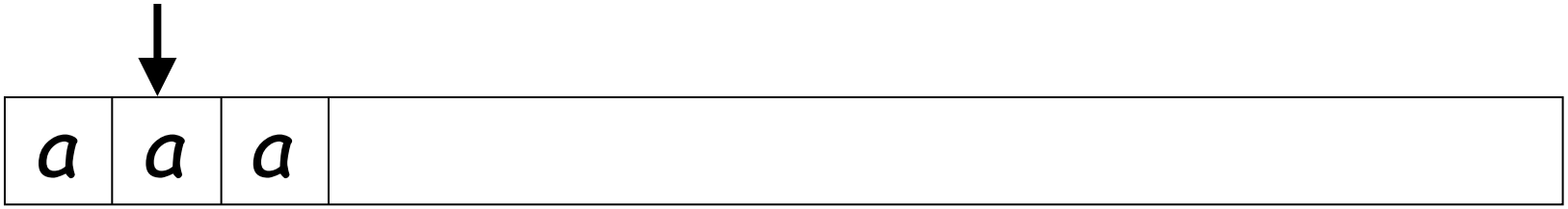
First Choice



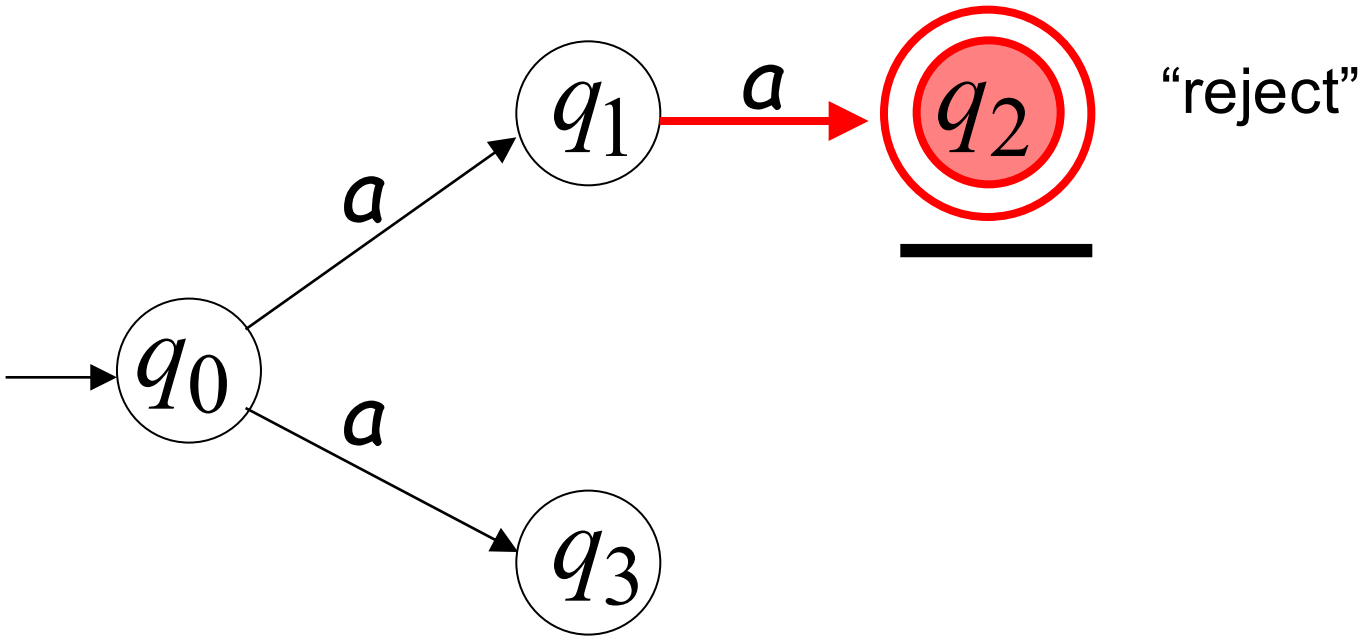


No transition:
the automaton hangs

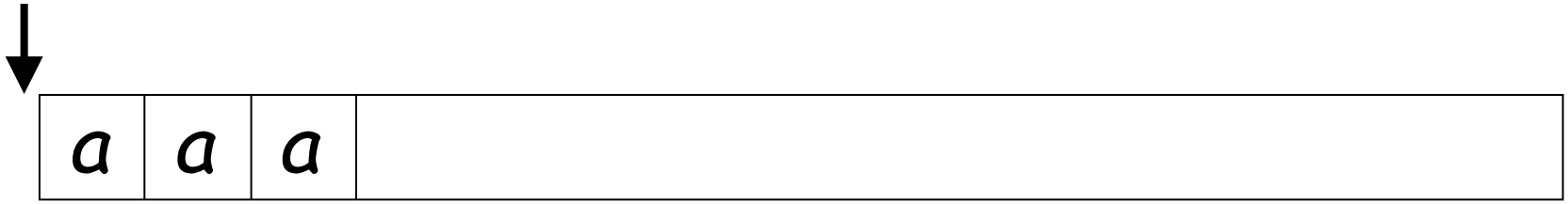
First Choice

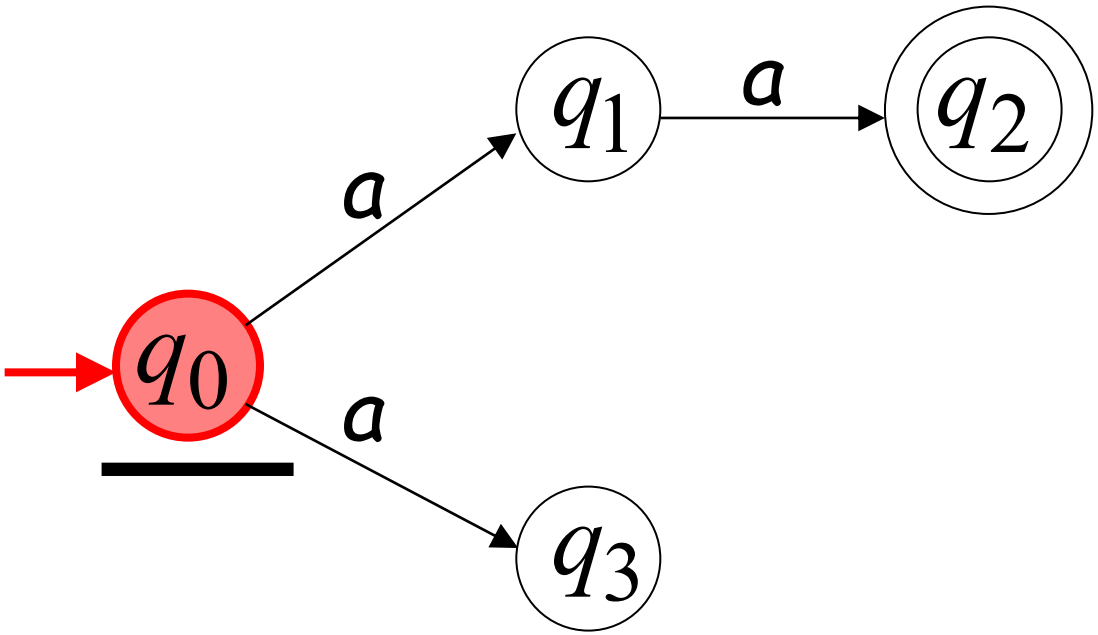


Input cannot be consumed

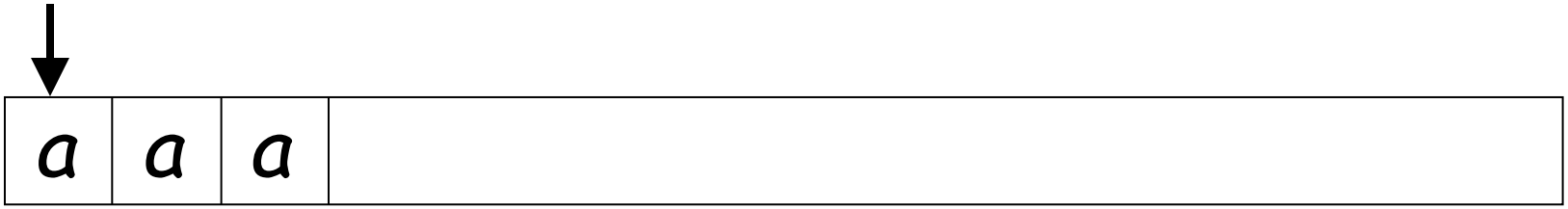


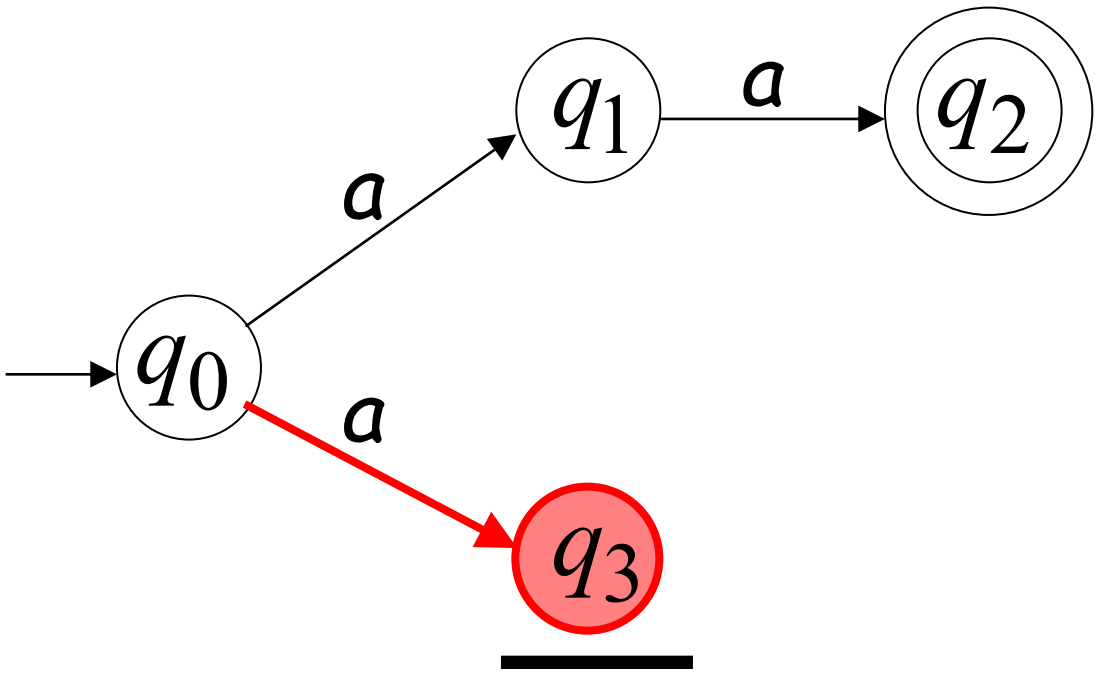
Second Choice



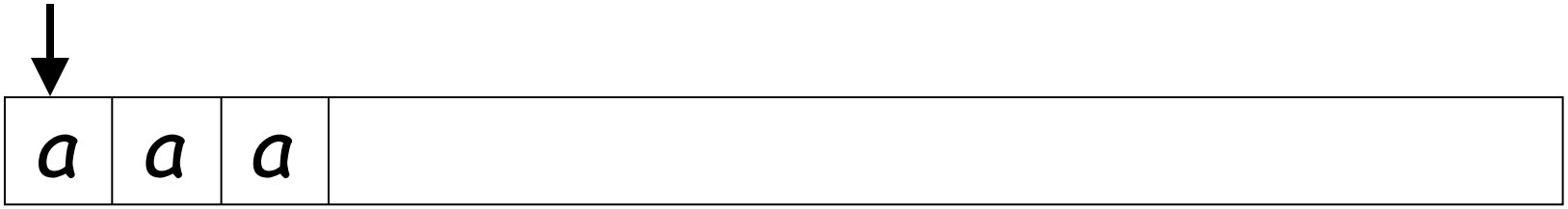


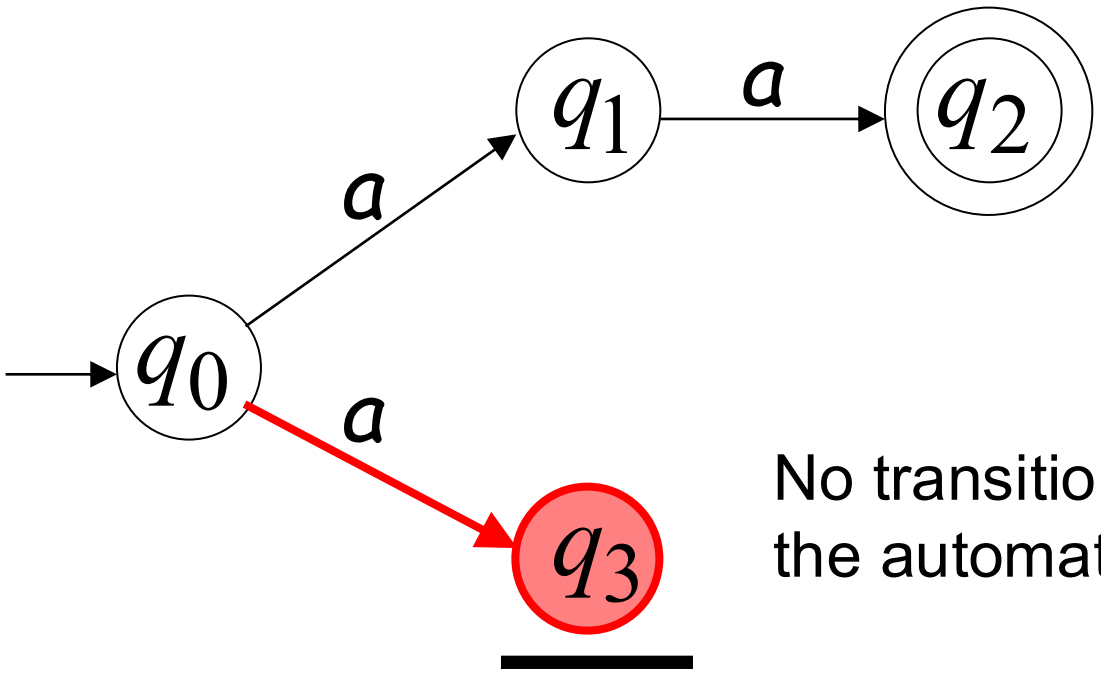
Second Choice





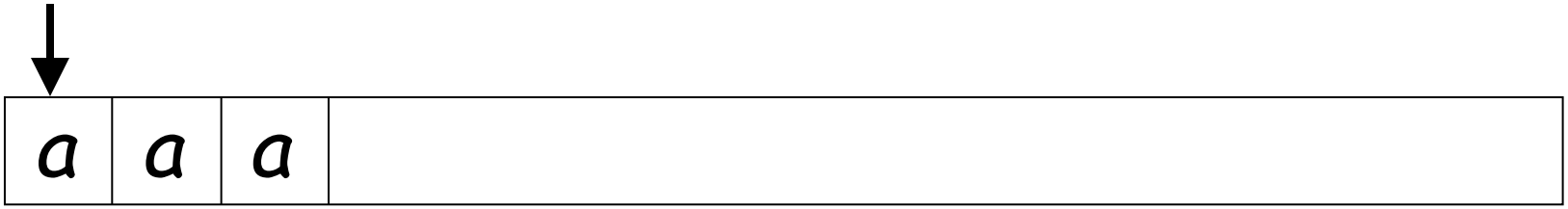
Second Choice



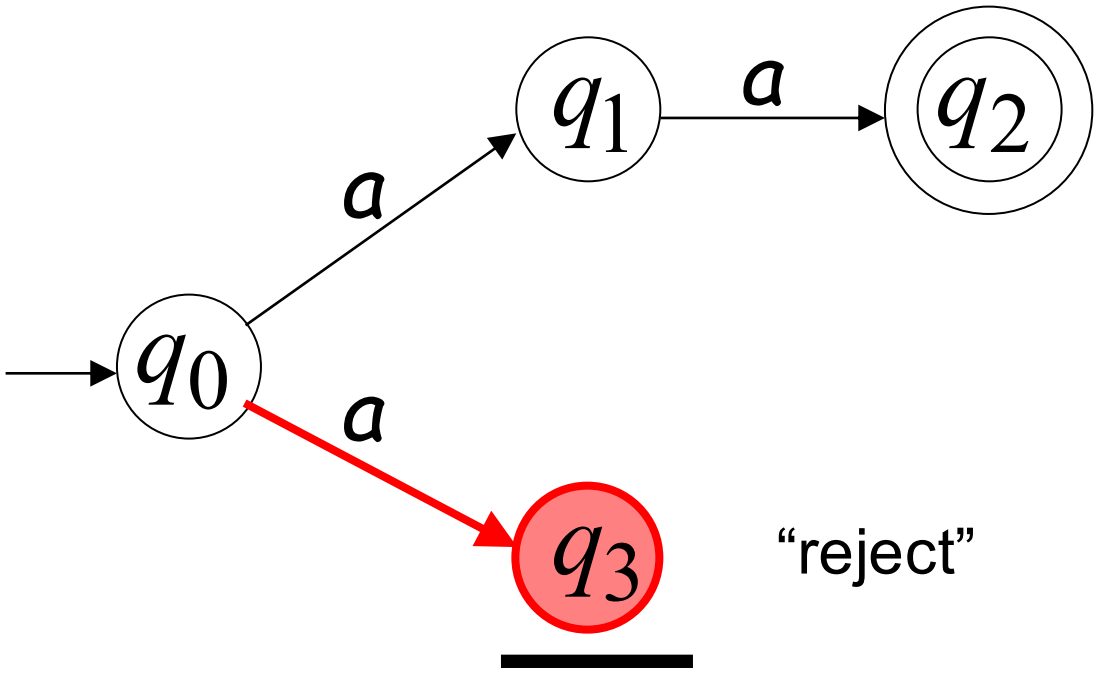


No transition:
the automaton hangs

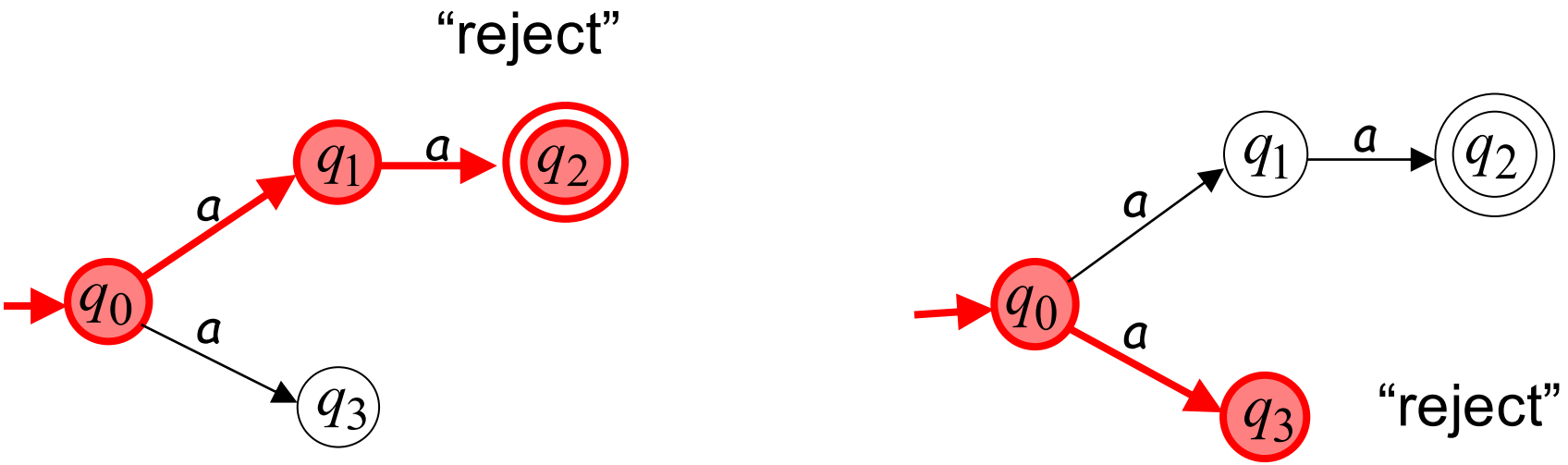
Second Choice



Input cannot be consumed

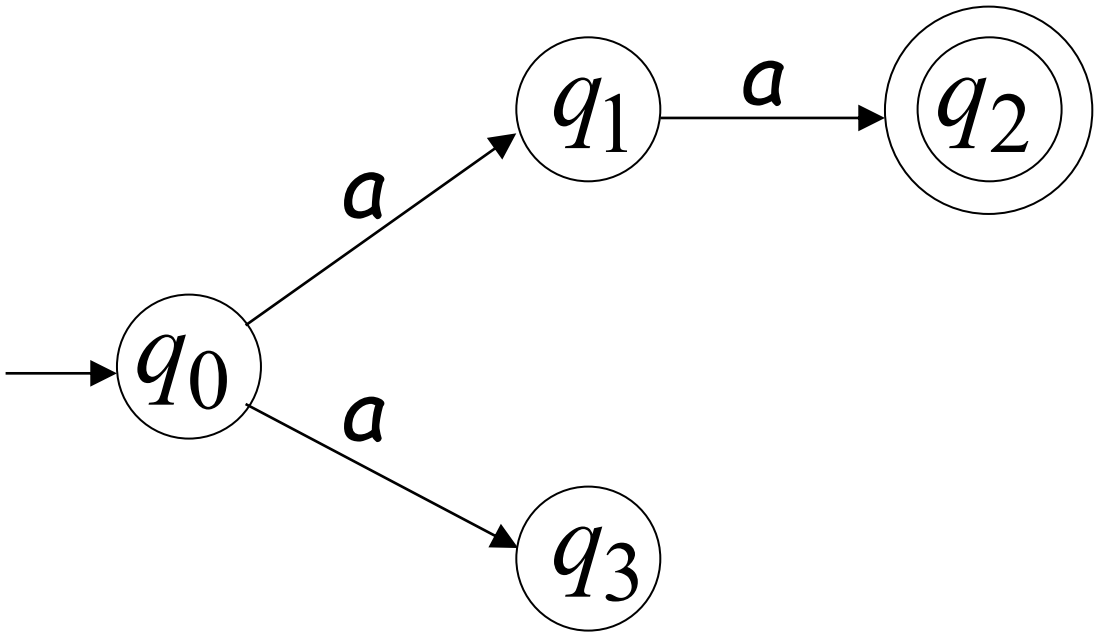


aaa is rejected by the NFA:

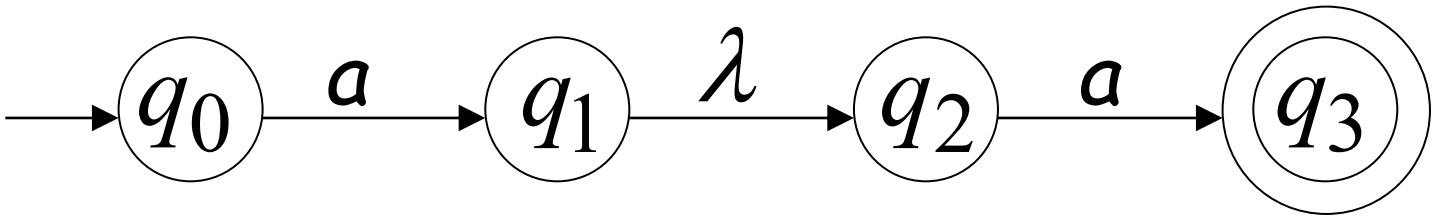


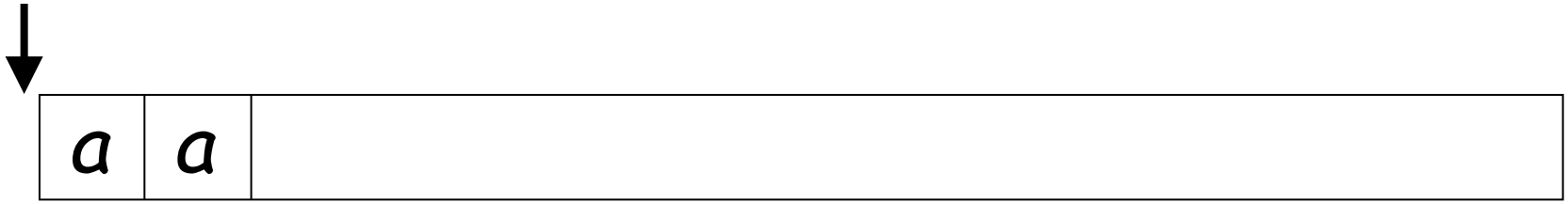
All possible computations lead to rejection

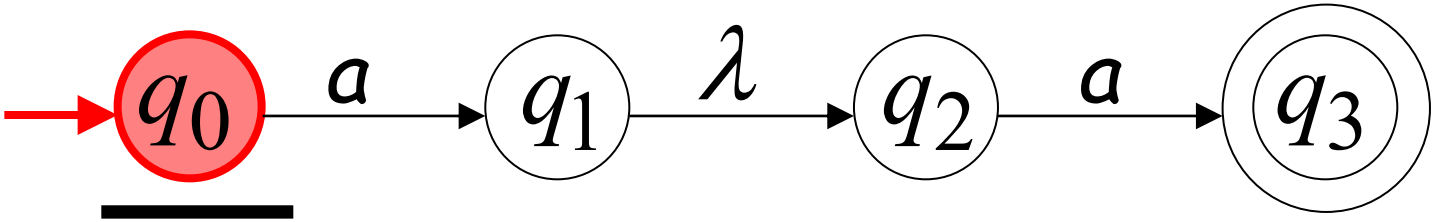
Language accepted: $L = \{aa\}$

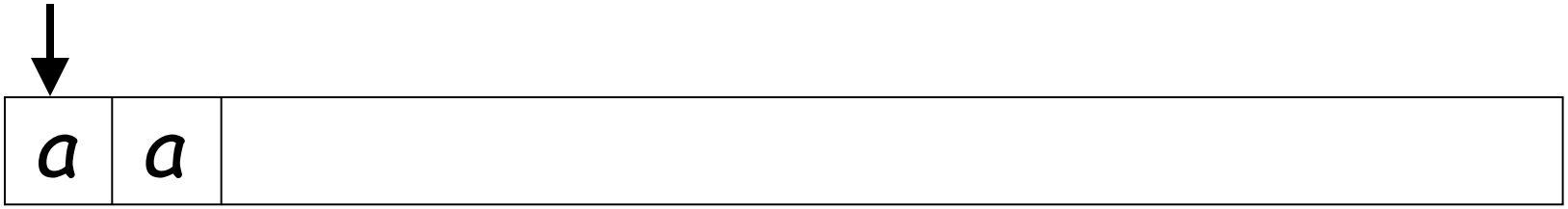


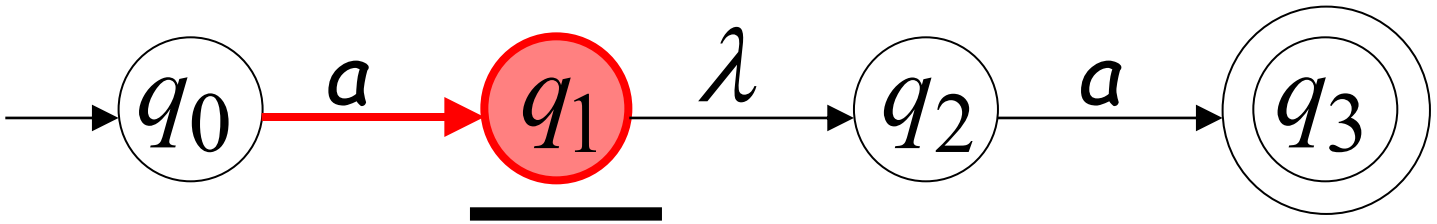
Lambda(λ) Transitions



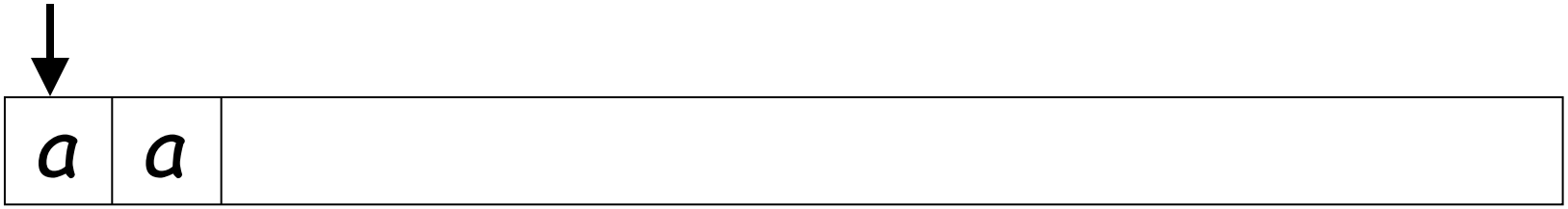

a
a

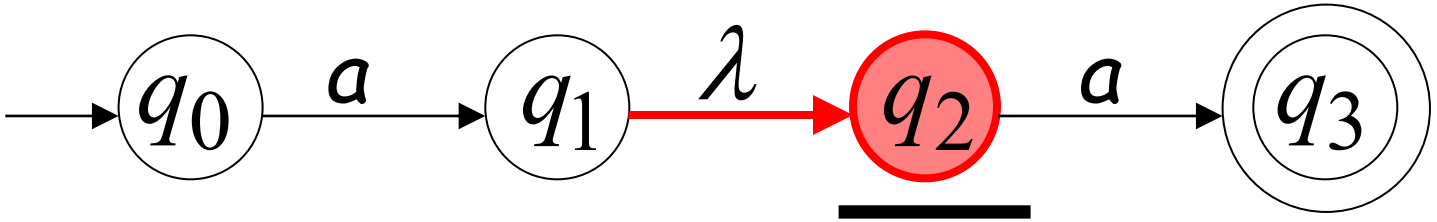

q_0
a
q_1
λ
q_2
a
q_3

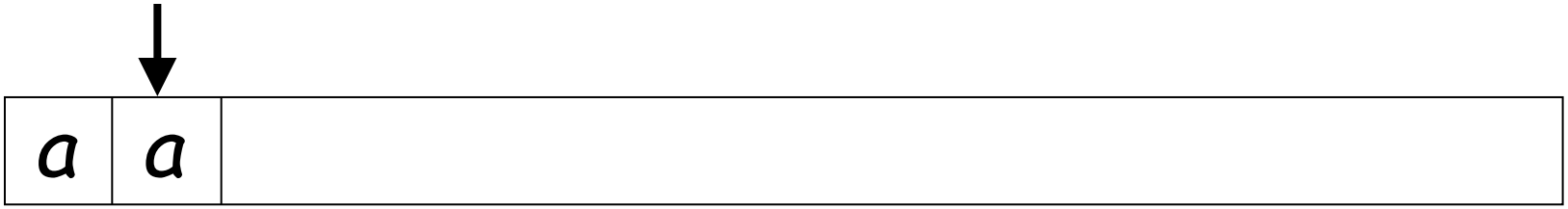

a
a

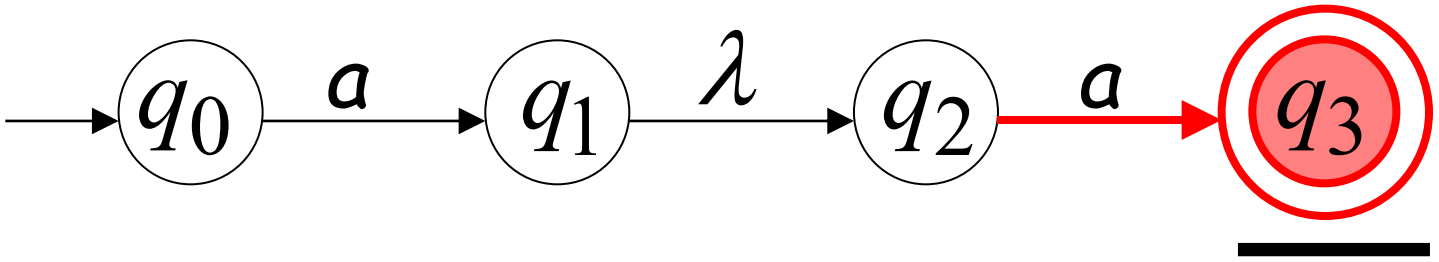

q_0
a
q_1
λ
q_2
a
q_3

(read head does not move)

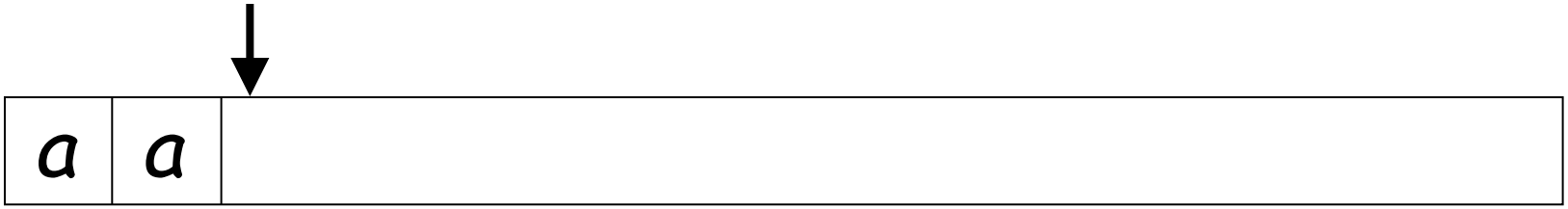


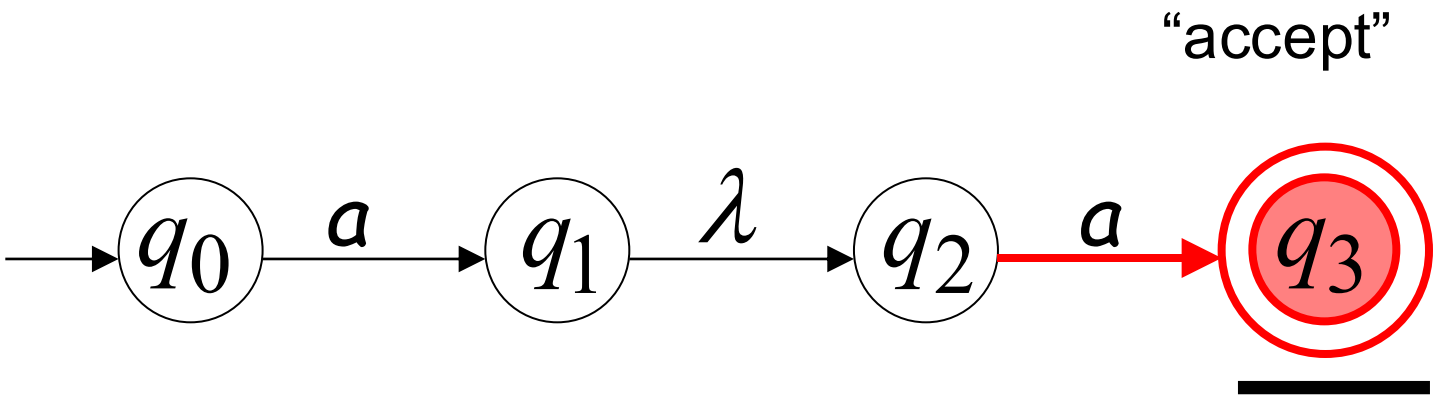



a
a


q_0
a
q_1
λ
q_2
a
q_3

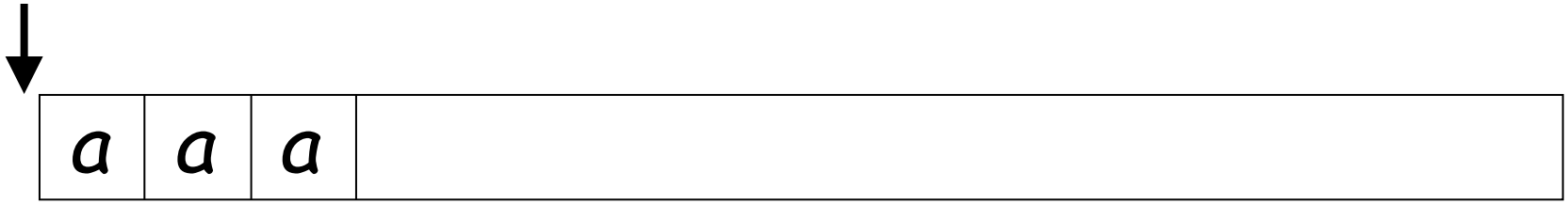
all input is consumed

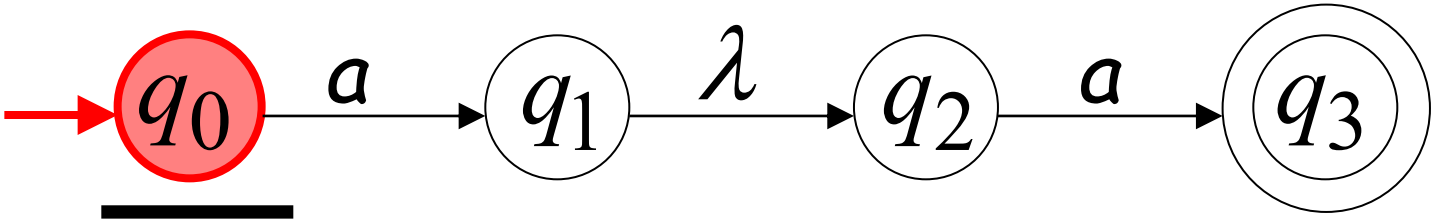


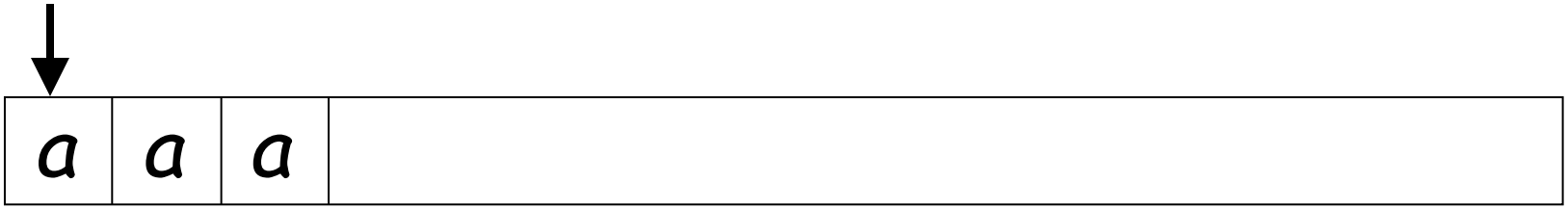


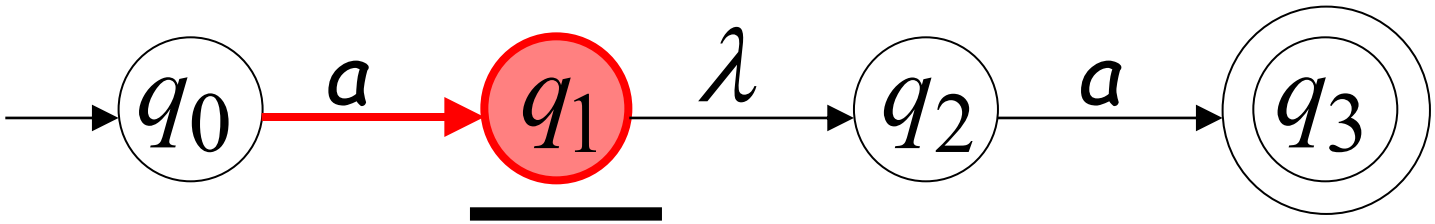
String aa is accepted

Rejection Example

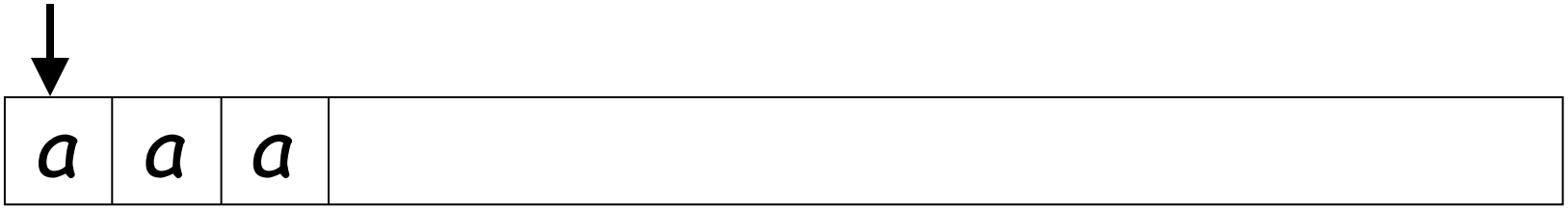


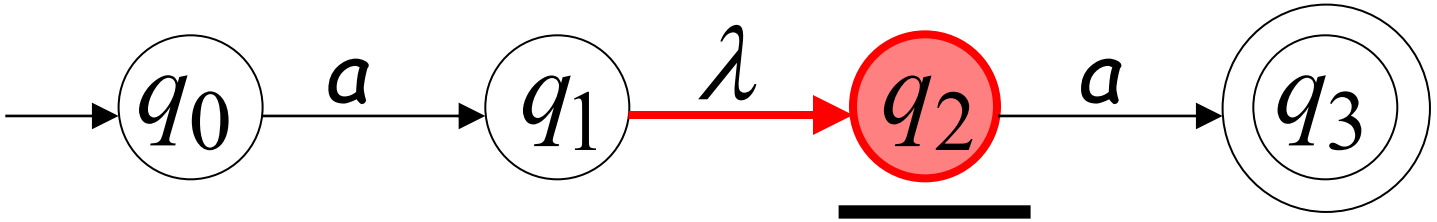


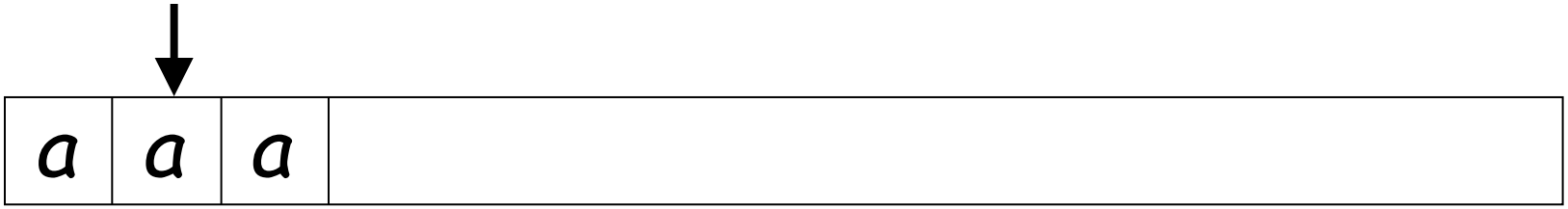

a
a
a

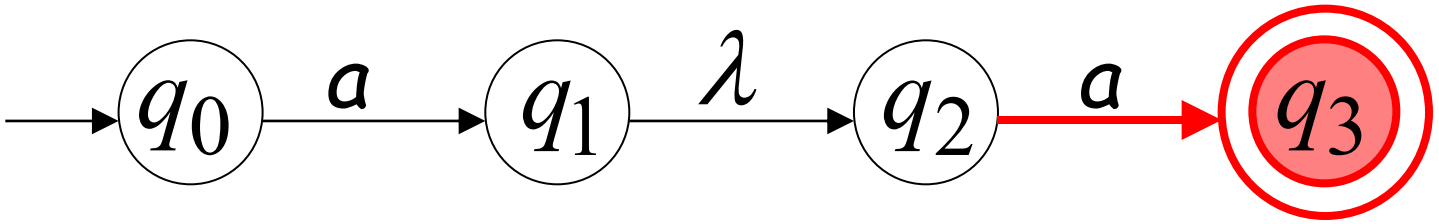

q_0
a
q_1
λ
q_2
a
q_3

(read head doesn't move)



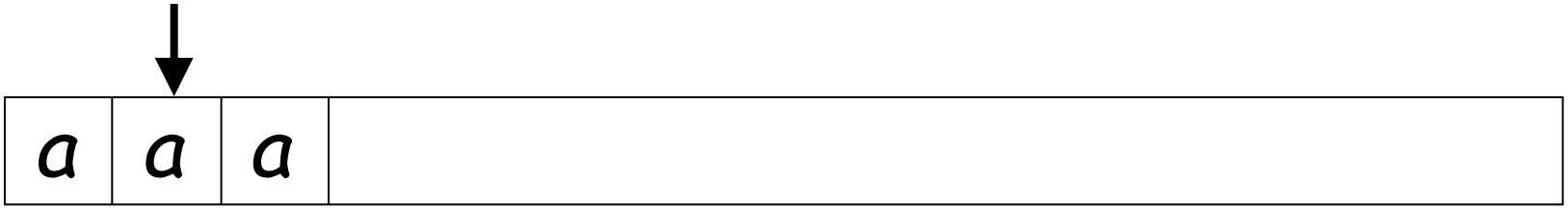


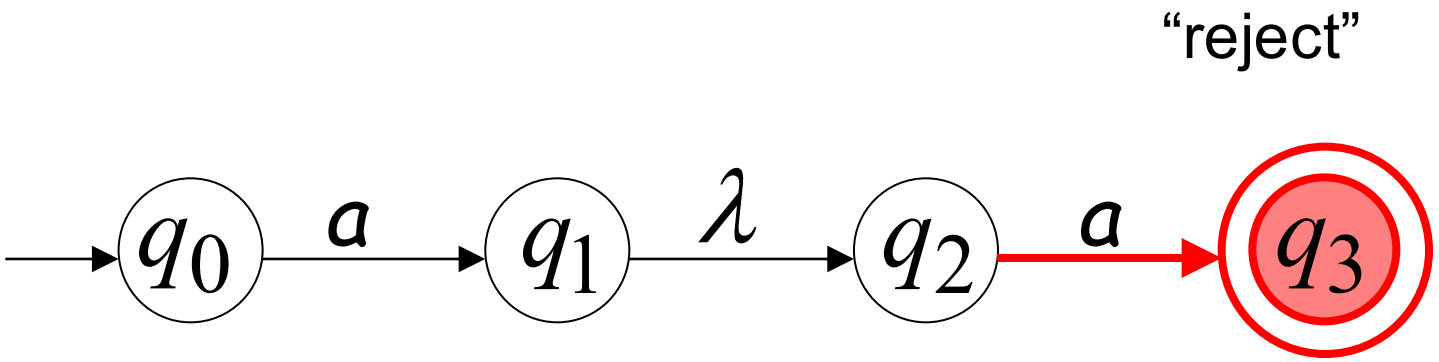




No transition:
the automaton hangs

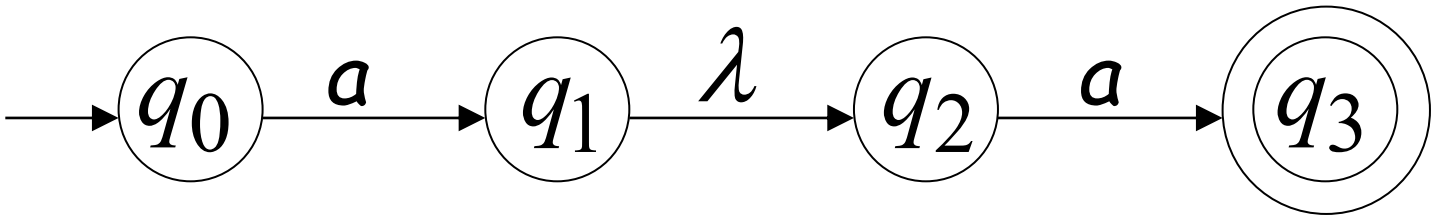
Input cannot be consumed



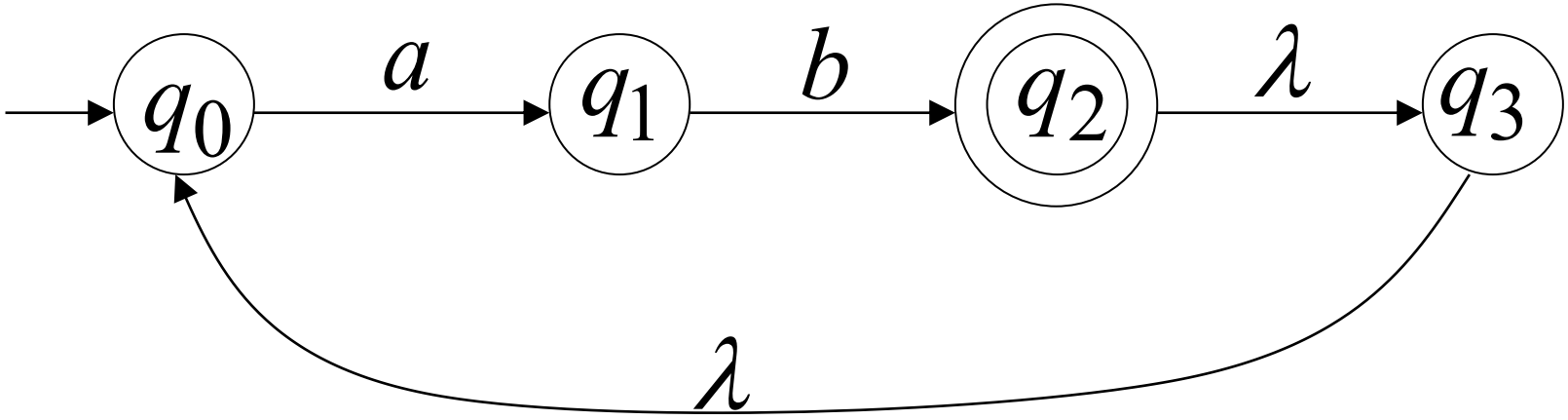


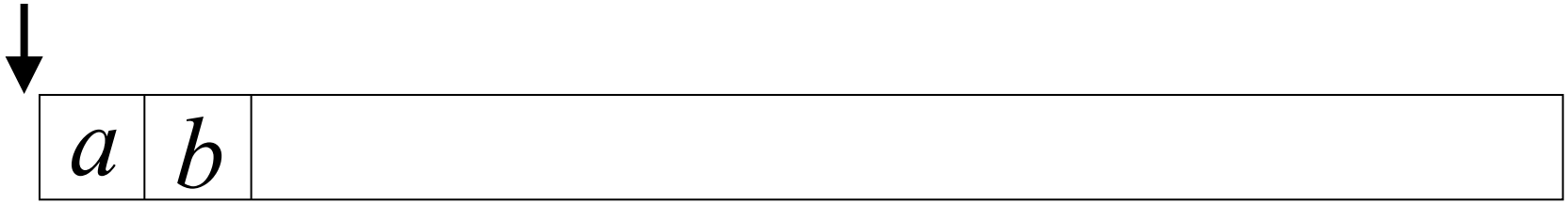
String **aaa** is rejected

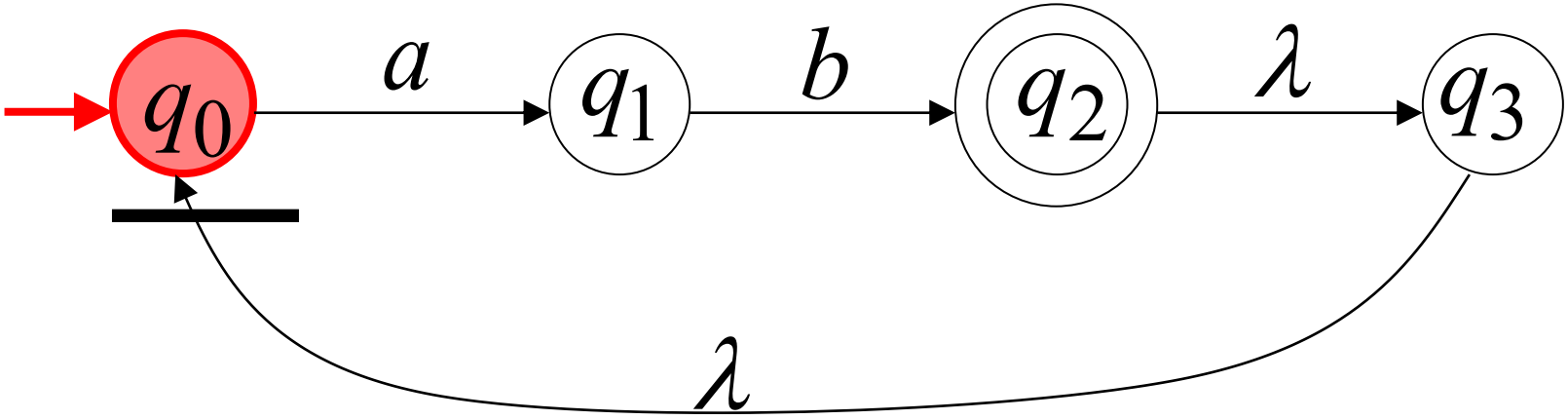
Language accepted: $L = \{aa\}$

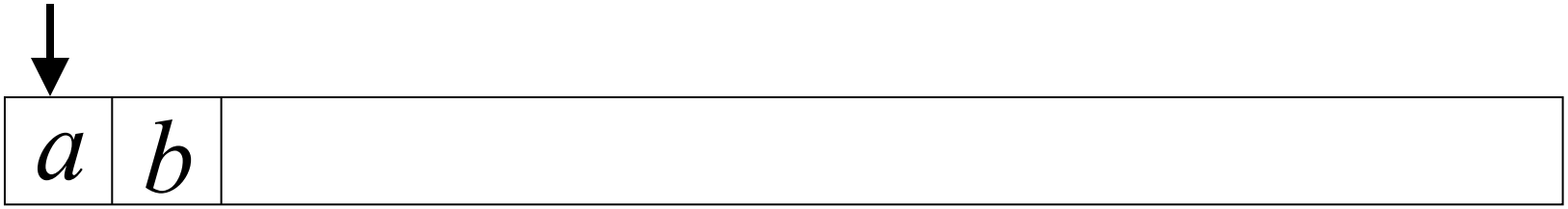


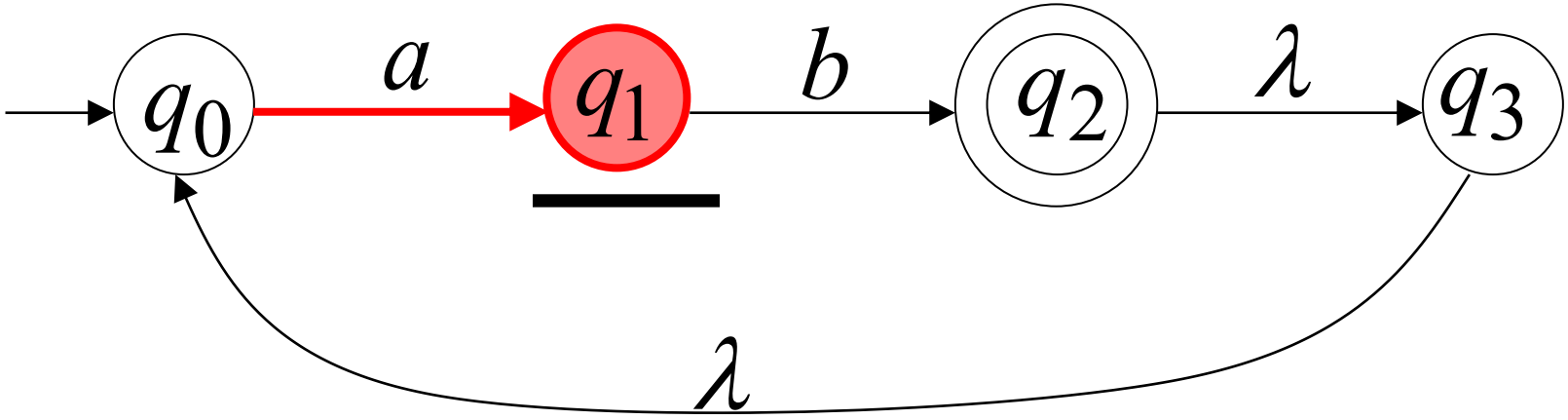
Another NFA Example

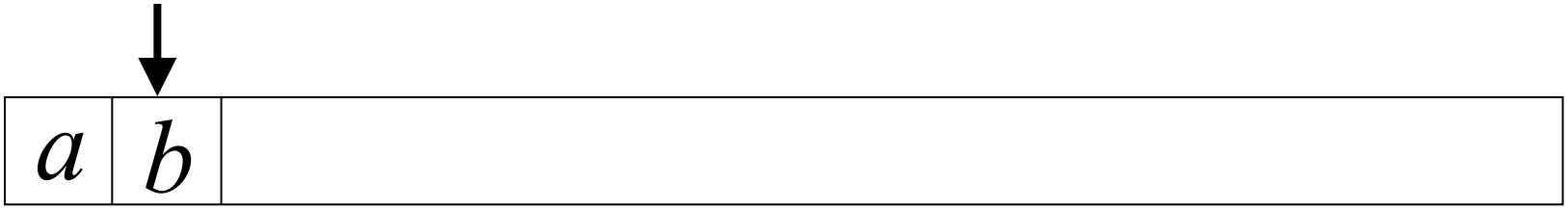


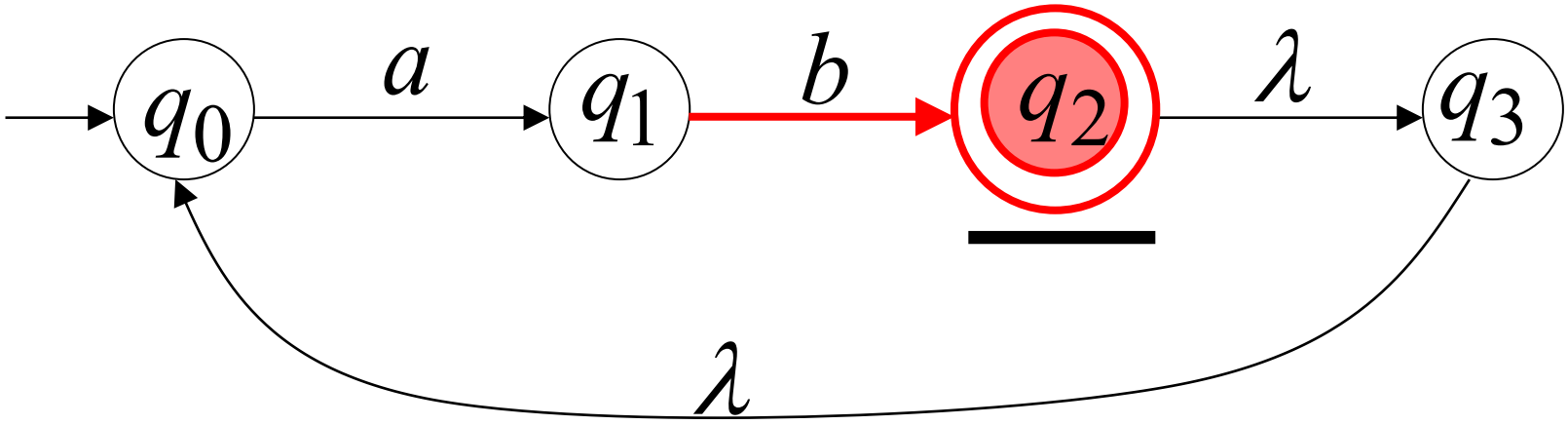

a
b

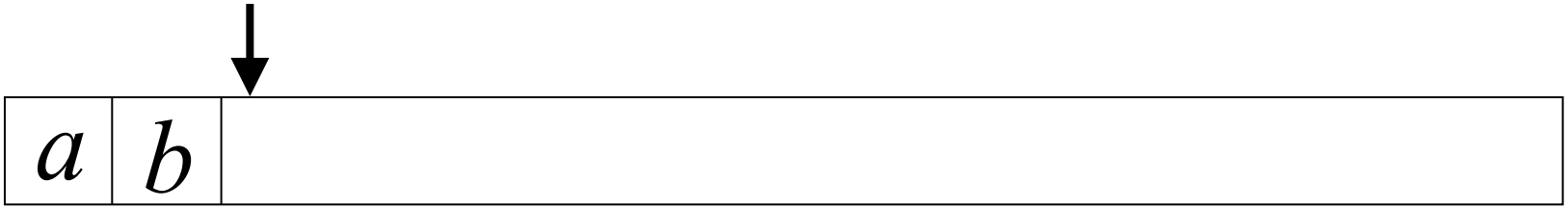

q_0
a
q_1
b
q_2
λ
q_3
λ

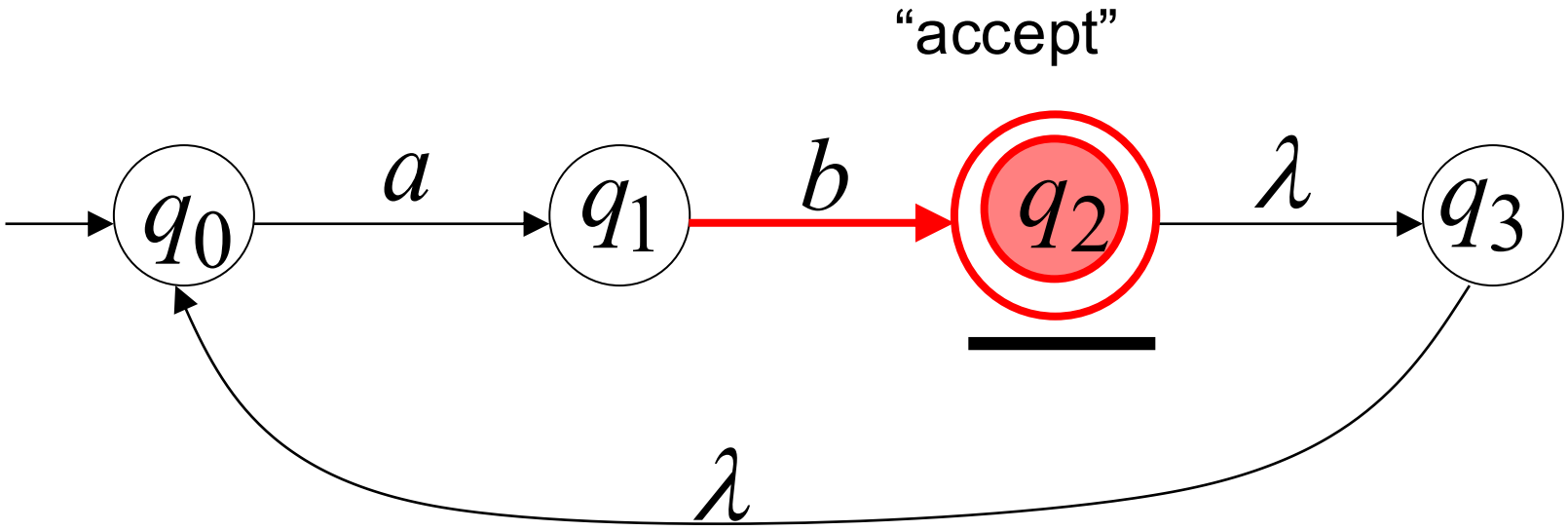

a
b


q_0
a
q_1
b
q_2
λ
q_3
λ

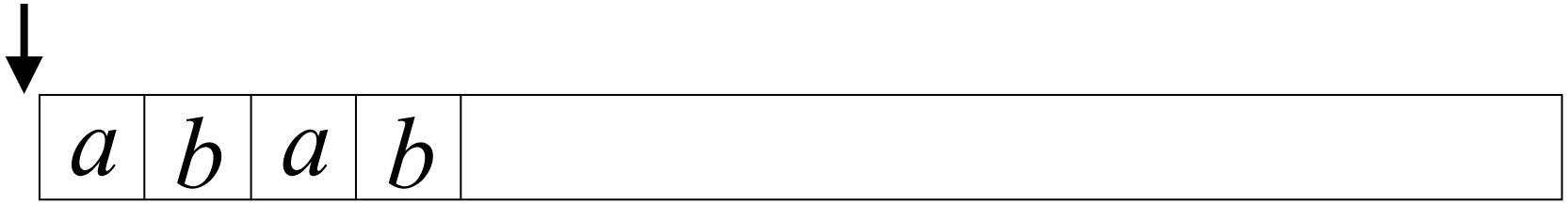

a
b

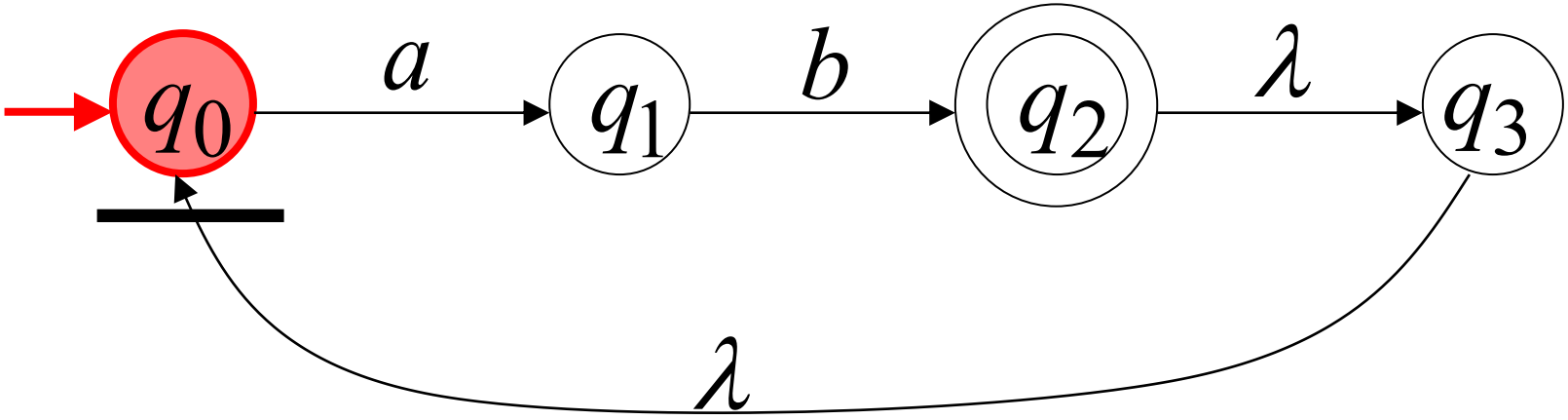

q_0
a
q_1
b
q_2
λ
q_3
λ

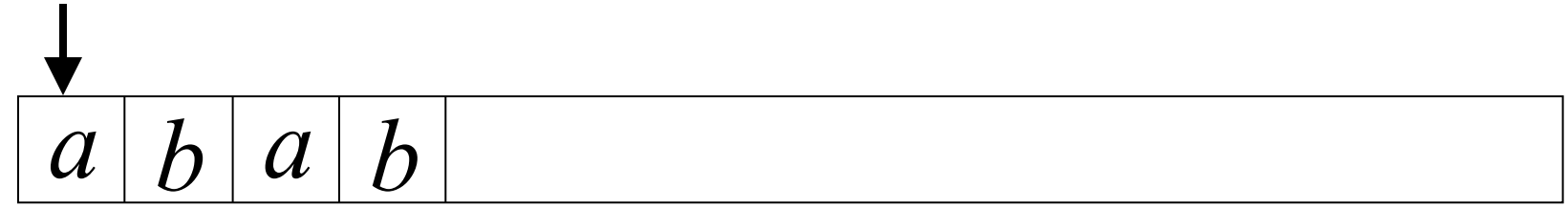

a
b

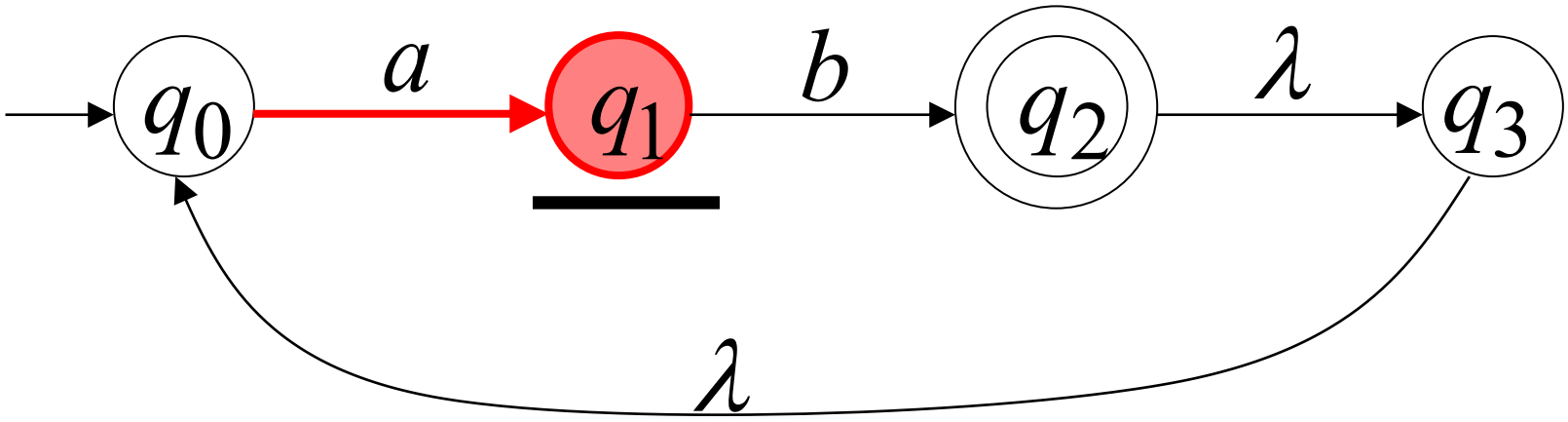

"accept"
q_0
q_1
q_2
q_3
a
b
λ
λ

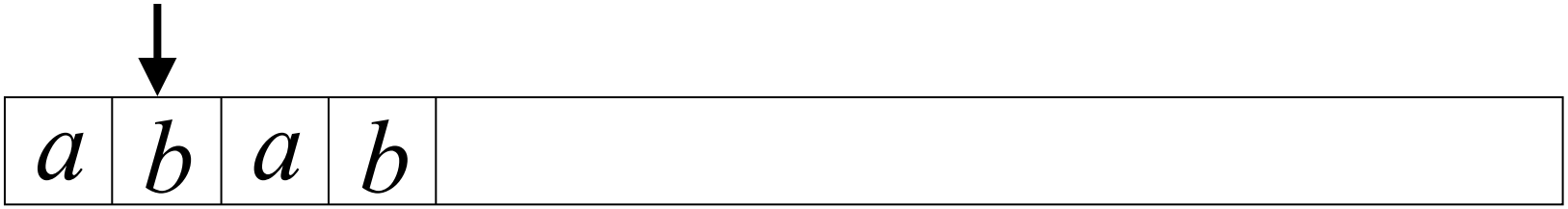
Another String

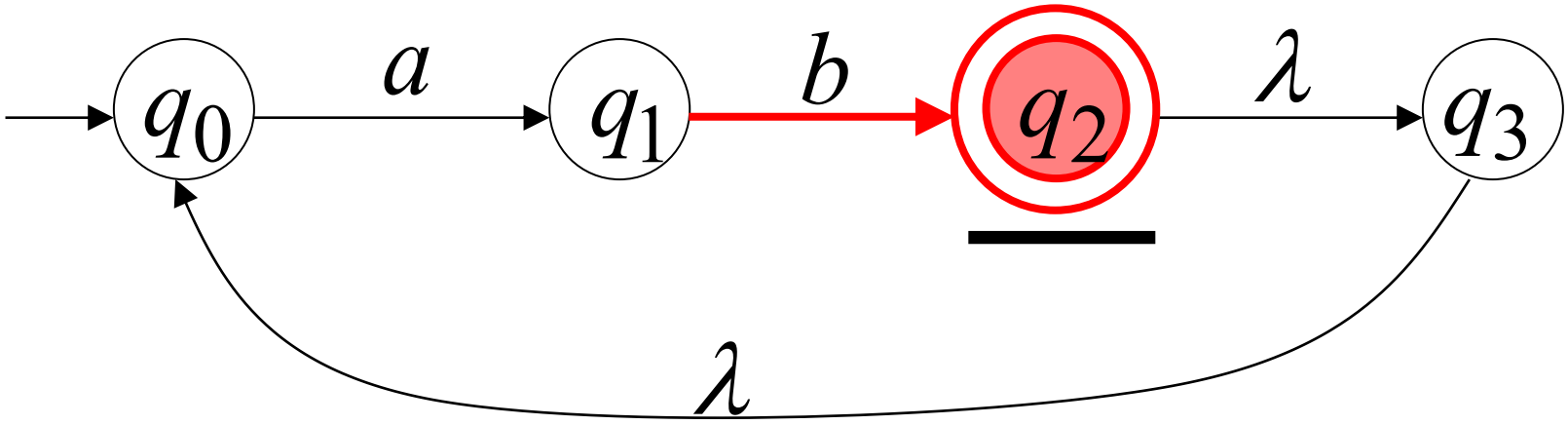


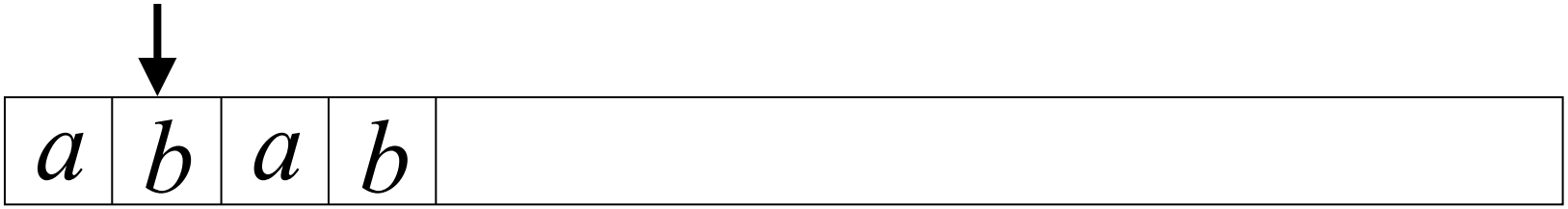


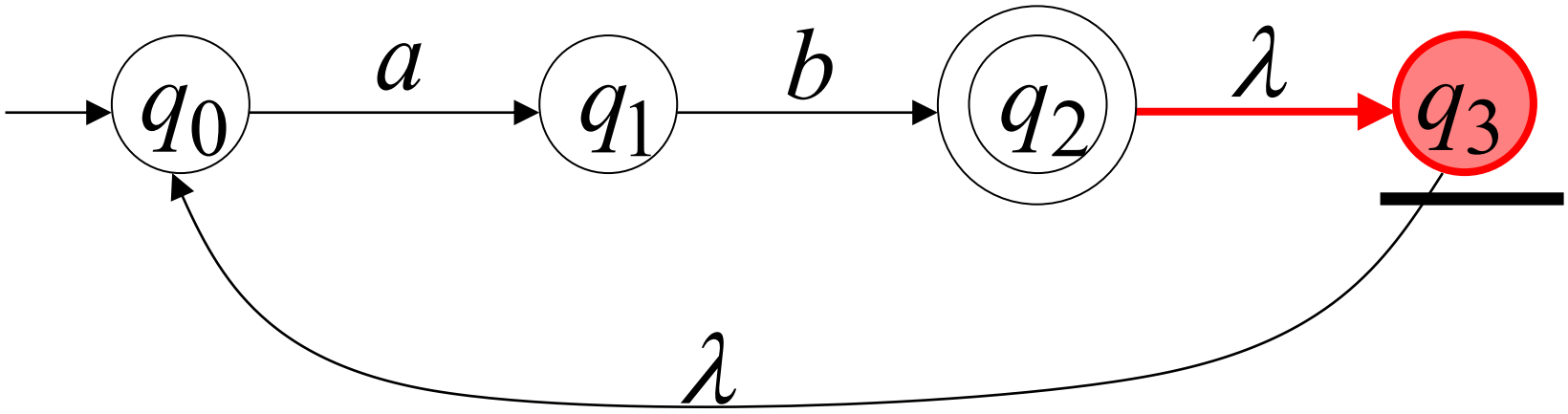

a
b
a
b

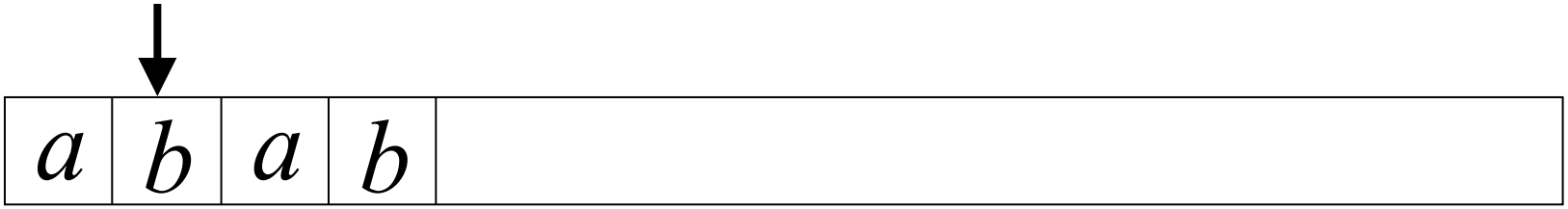

q_0
a
q_1
b
q_2
λ
q_3
λ

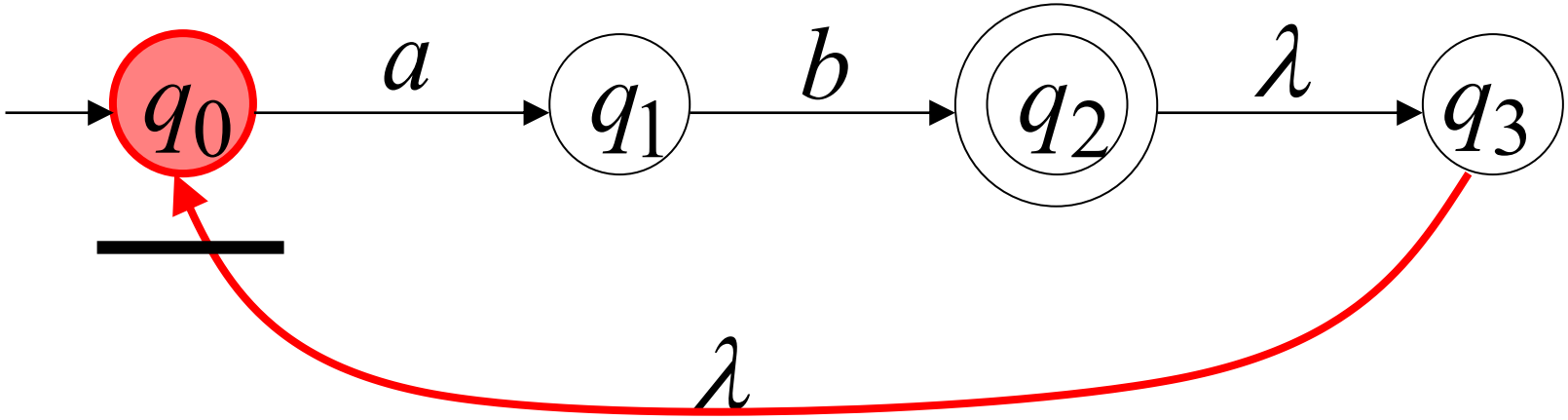

a
b
a
b

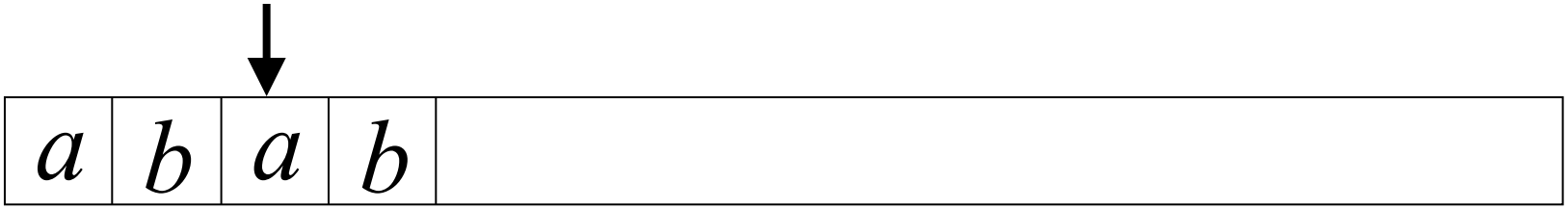

q_0
a
q_1
b
q_2
λ
q_3
λ

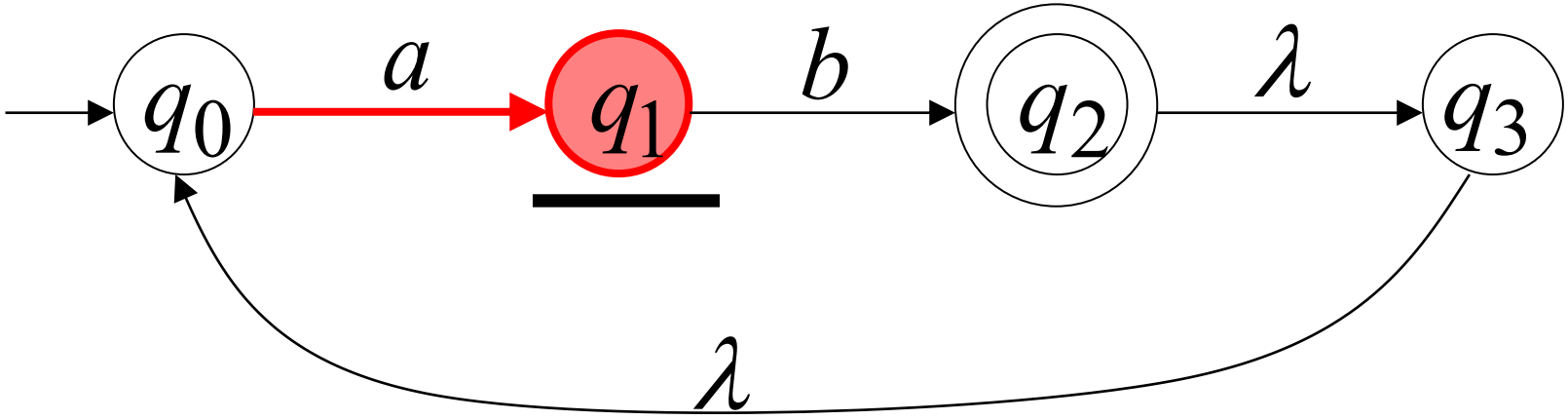

a
b
a
b

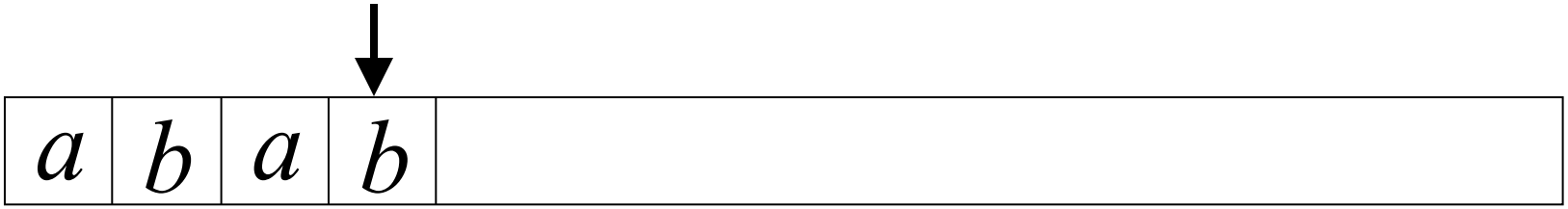

q_0
a
q_1
b
q_2
λ
q_3
λ

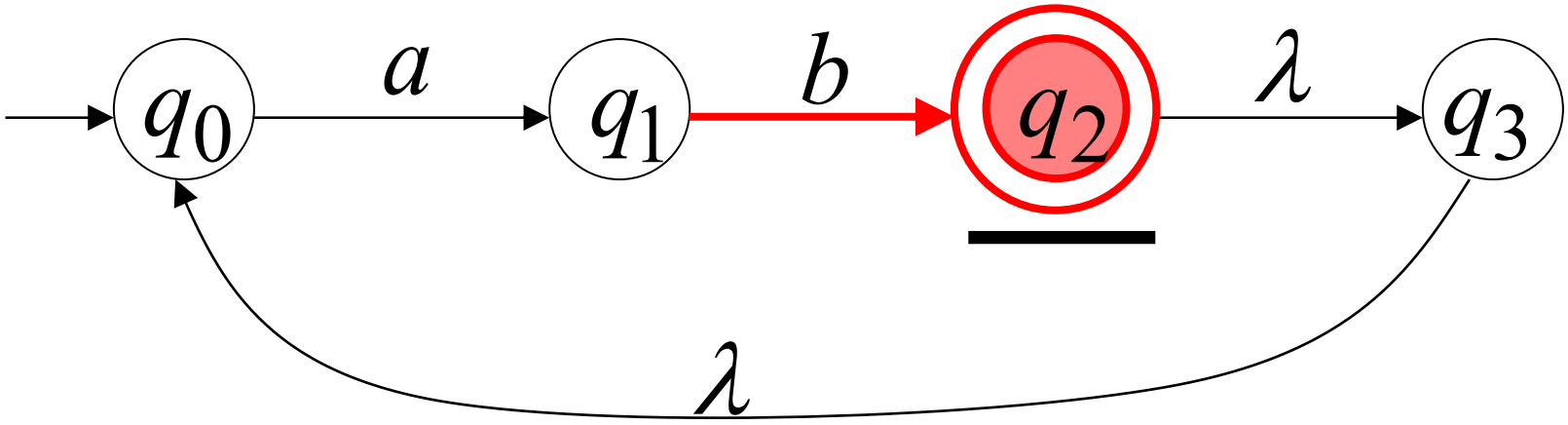

a
b
a
b

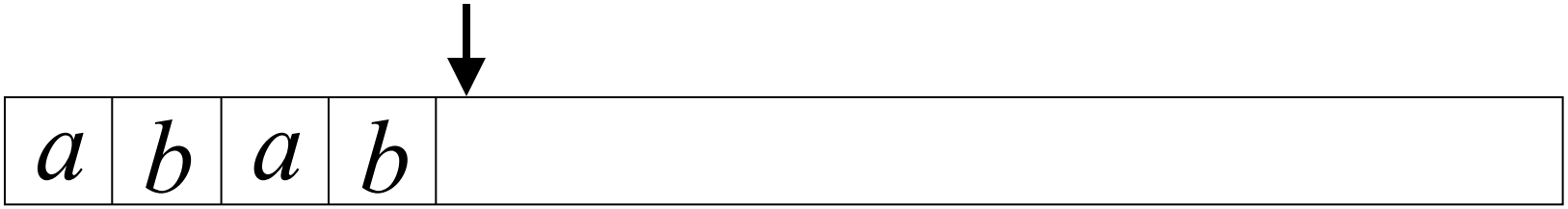

q_0
a
q_1
b
q_2
λ
q_3
λ

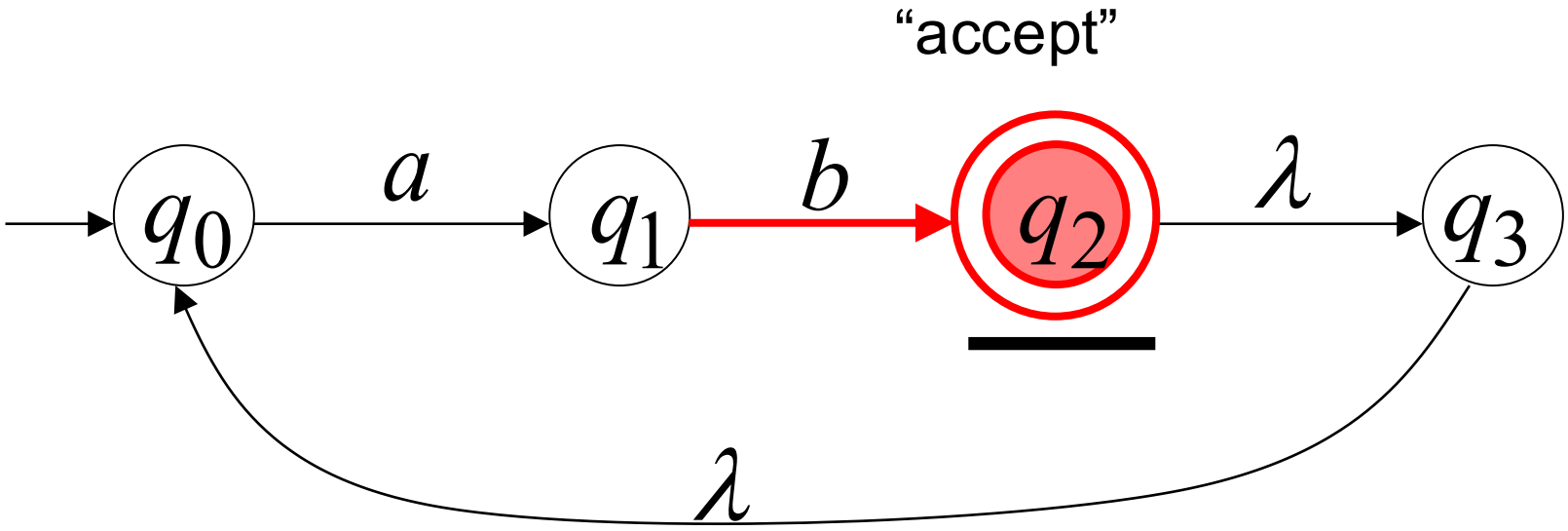

a
b
a
b


q_0
a
q_1
b
q_2
λ
q_3
λ


a
b
a
b


q_0
a
q_1
b
q_2
λ
q_3
λ

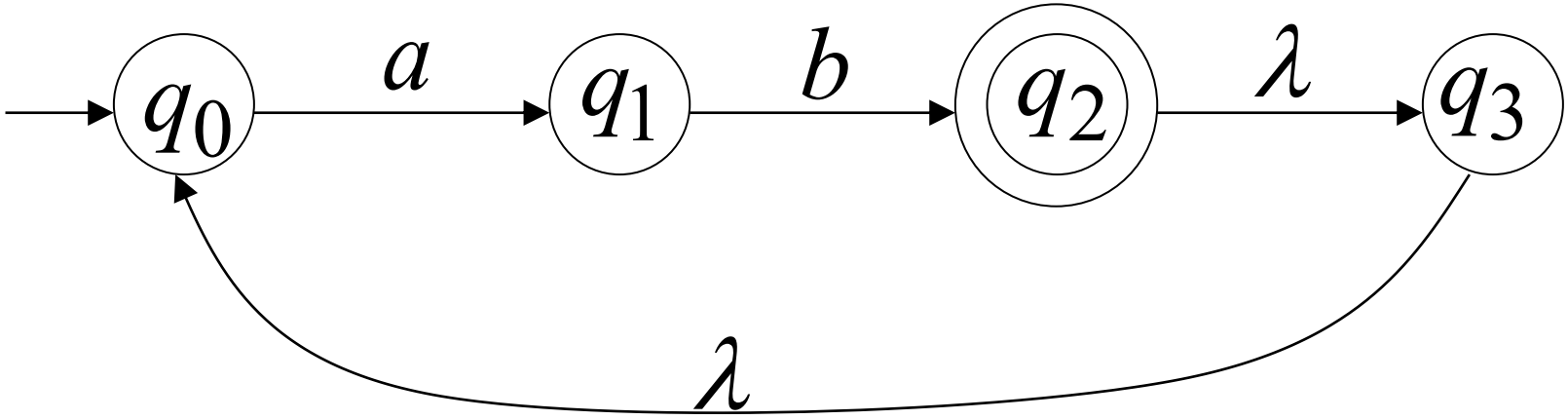

a
b
a
b


"accept"
q_0
q_1
q_2
q_3
a
b
λ
λ

Language accepted

$$L = \{ab,\ abab,\ ababab,\ ...\}$$

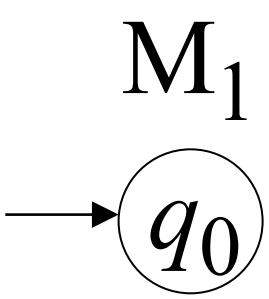
$$= \{ab\}^{+}$$

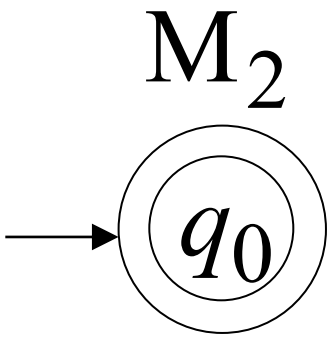


Remarks:

- The λ symbol never appears on the input tape

- Simple automata:



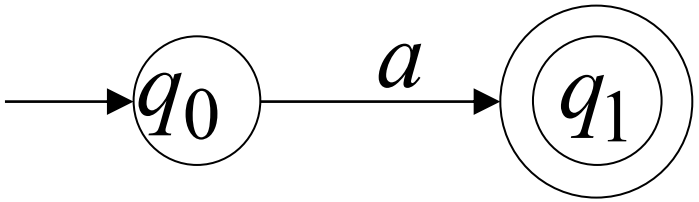


$$L(M_1) = \{\}$$

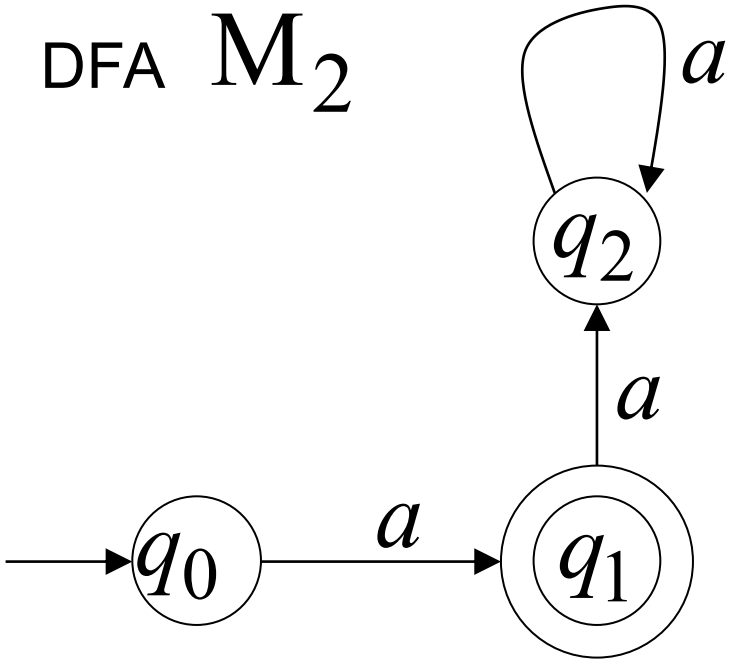
$$L(M_2) = \{\lambda\}$$

- NFAs are interesting because we can express languages easier than DFAs

NFA M_1



DFA M_2



$$L(M_1) = \{a\}$$

$$L(M_2) = \{a\}$$

Formal Definition of NFAs

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : a finite set of **internal states**

Σ : a finite set of symbols called **input alphabet**

δ : Q x (Σ U {λ}) → 2^Q called **transition function**
: Q x Σ → Q (DFA)

q_0 : q_0 ϵ Q is the **initial state**

F : F ⊆ Q is a set of **final states**

Difference Between DFA and NFA

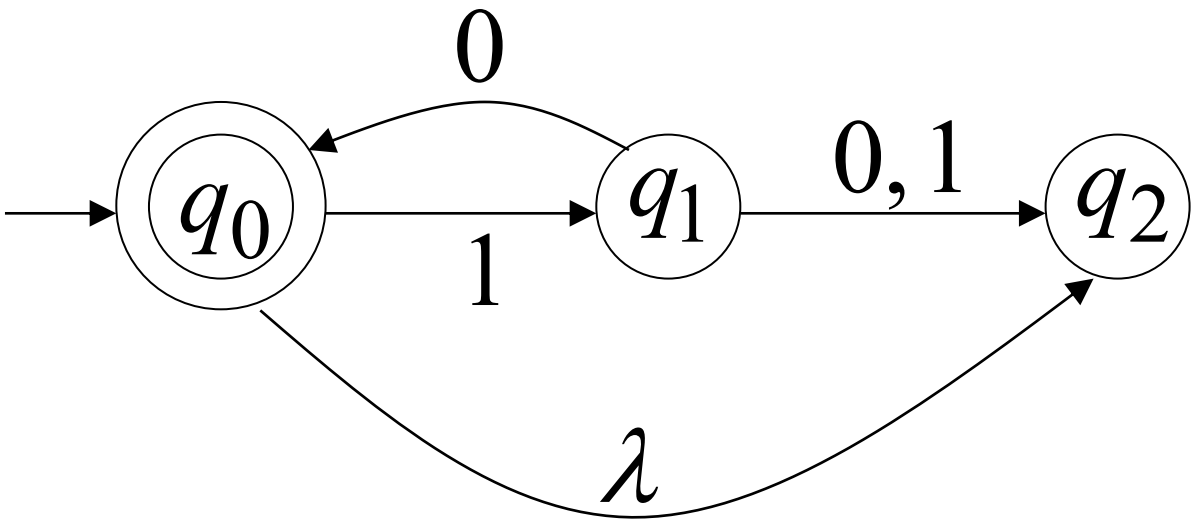
$\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q$

In NFA

- The range of δ is in the powerset 2^Q
- It allows λ as the second argument of δ
- The set $\delta(q_i, a)$ may be empty

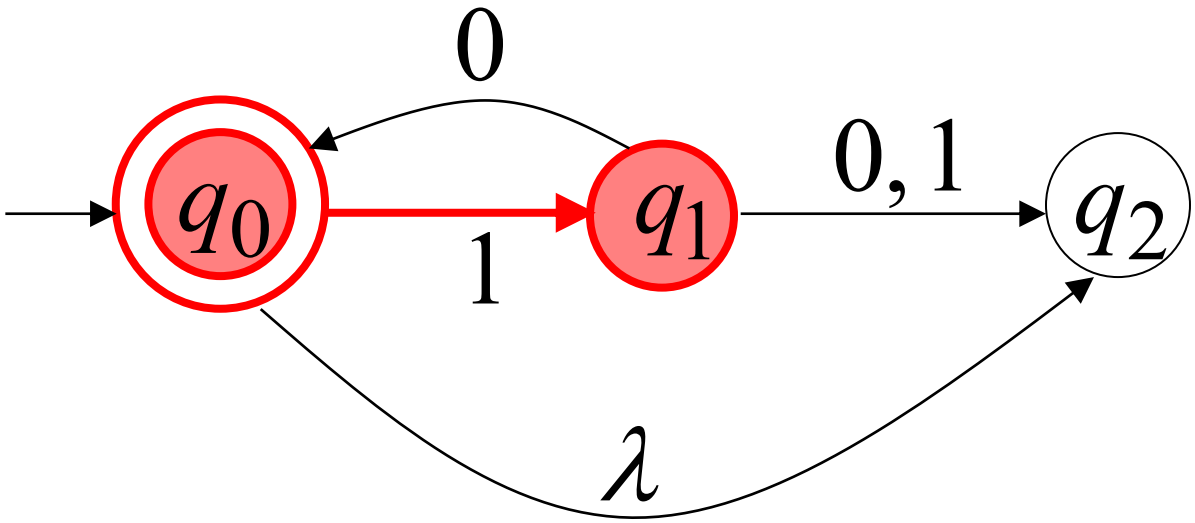
Assume NFA wants to accept every string (try the best move)

Example 2.8

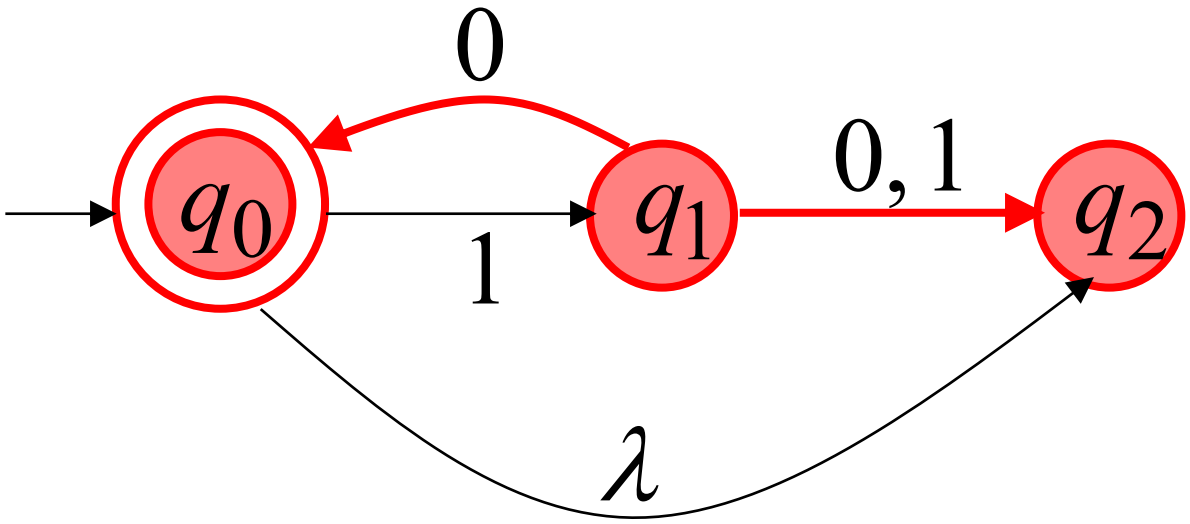


Transition Function δ

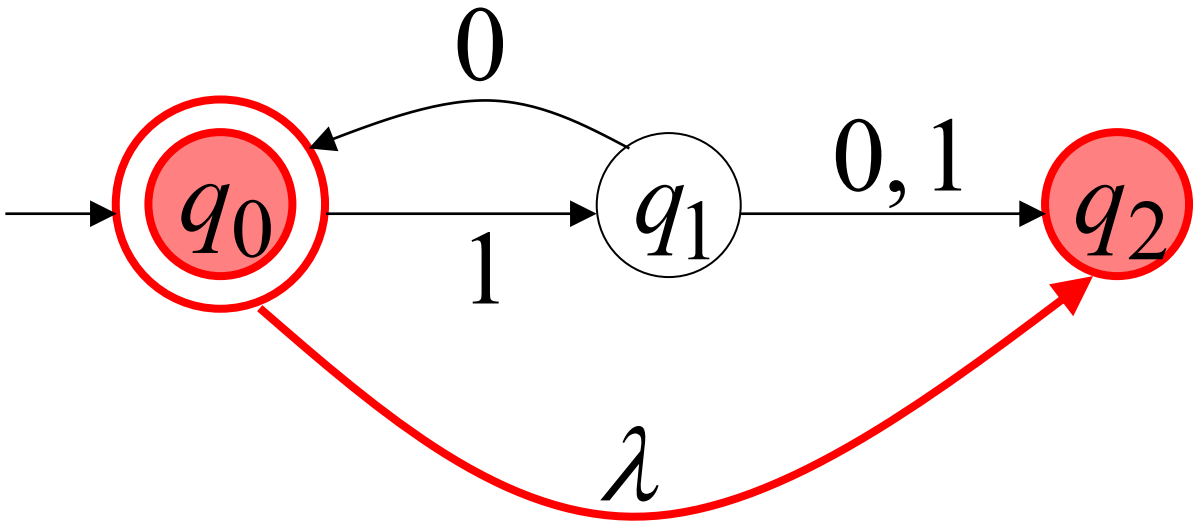
$$\delta(q_0, 1) = \{q_1\}$$



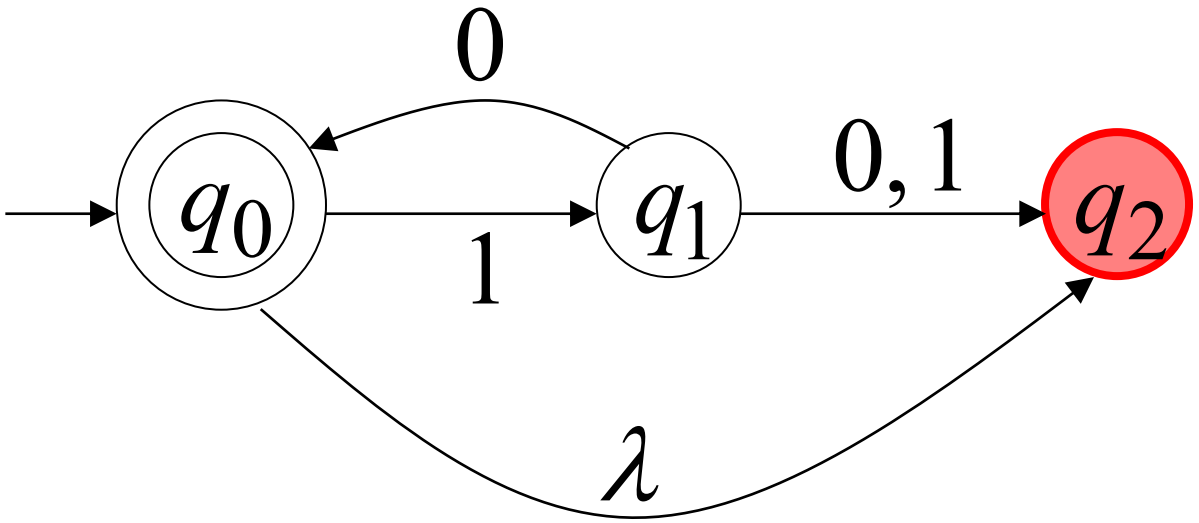
$$\delta(q_1,0) = \{q_0, q_2\}$$



$$\delta(q_0, \lambda) = \{q_0, q_2\}$$



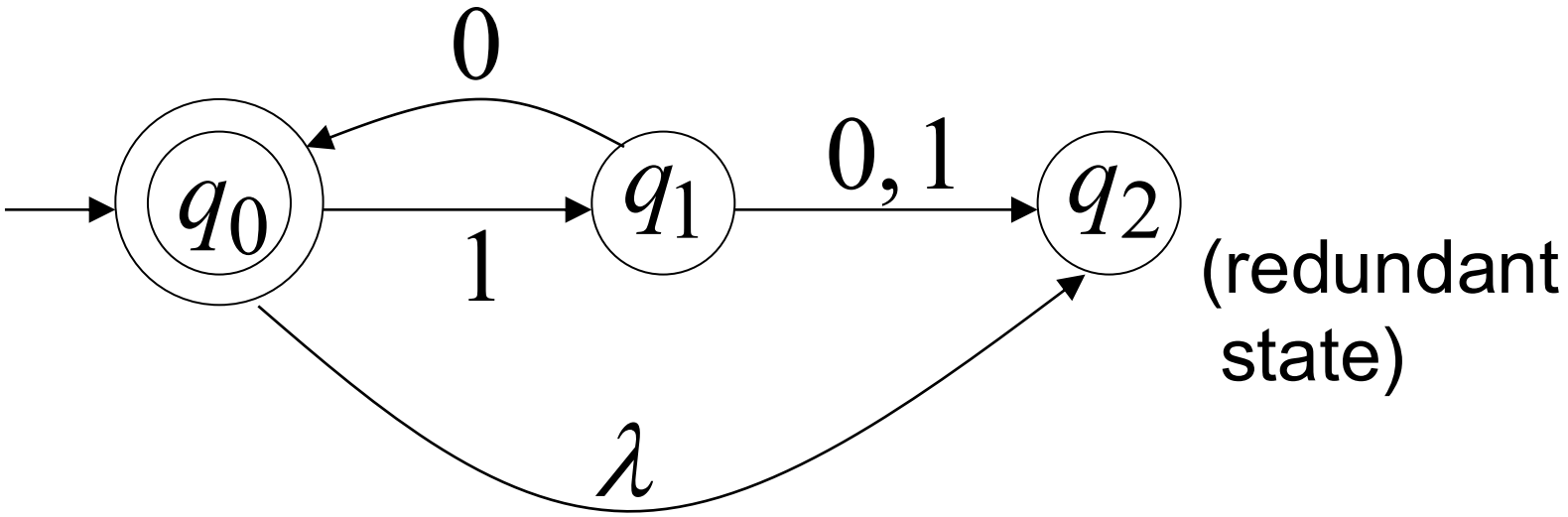
$$\delta(q_2, 1) = \varnothing$$



Language accepted

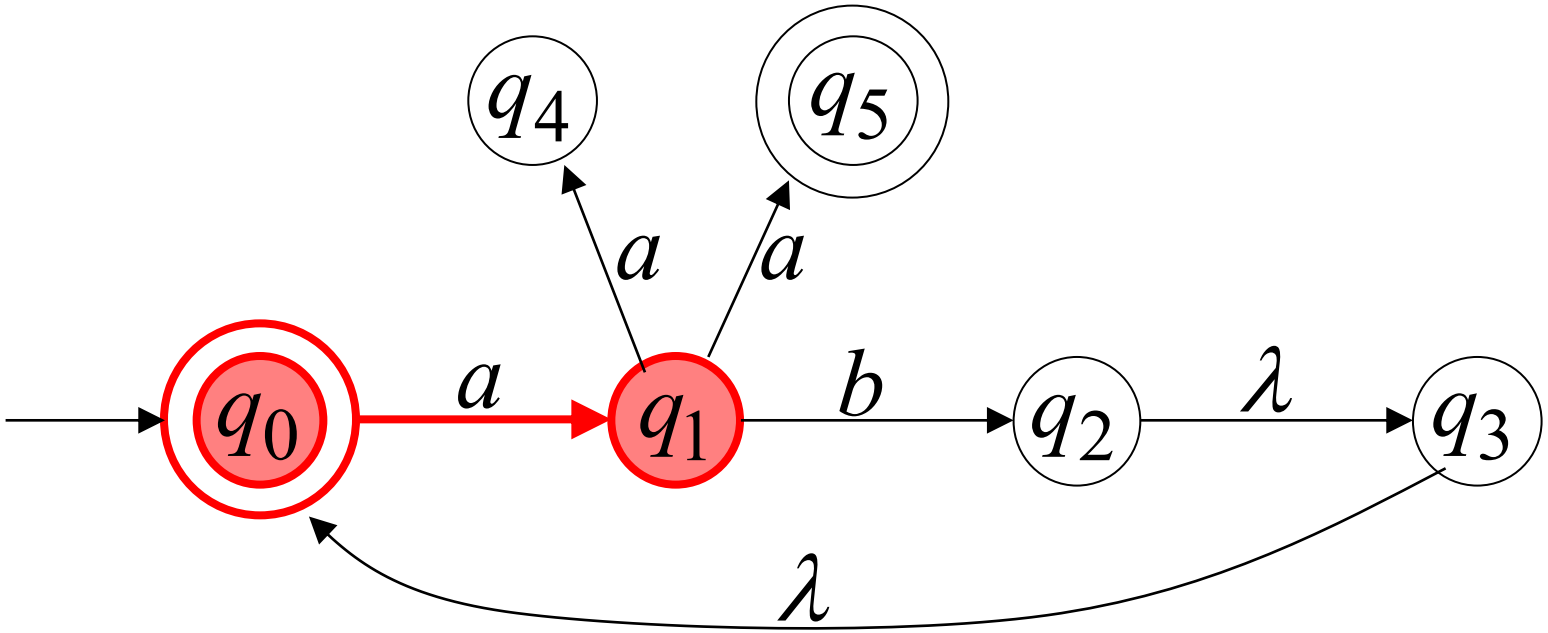
$$L(M) = \{\lambda,\ 10,\ 1010,\ 101010,\ \ldots\}$$

$$= \{10\}^*$$

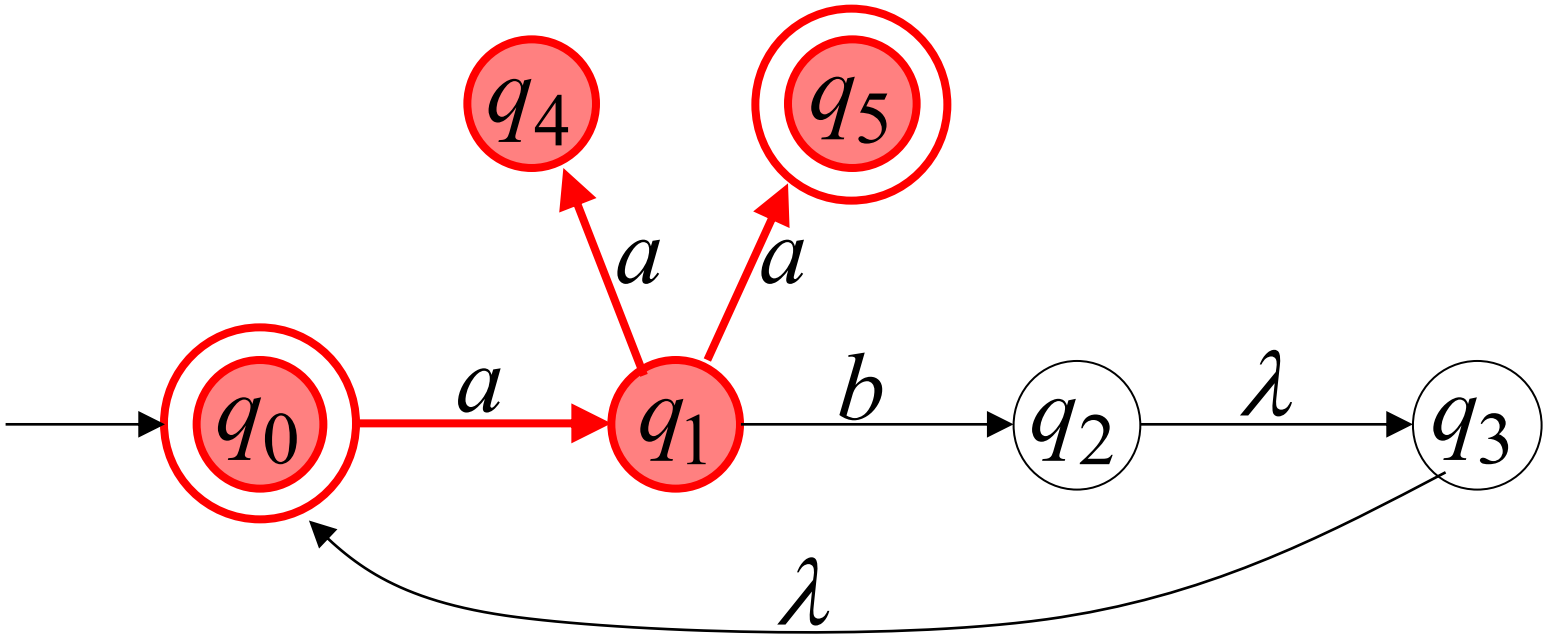


Extended Transition Function δ^*

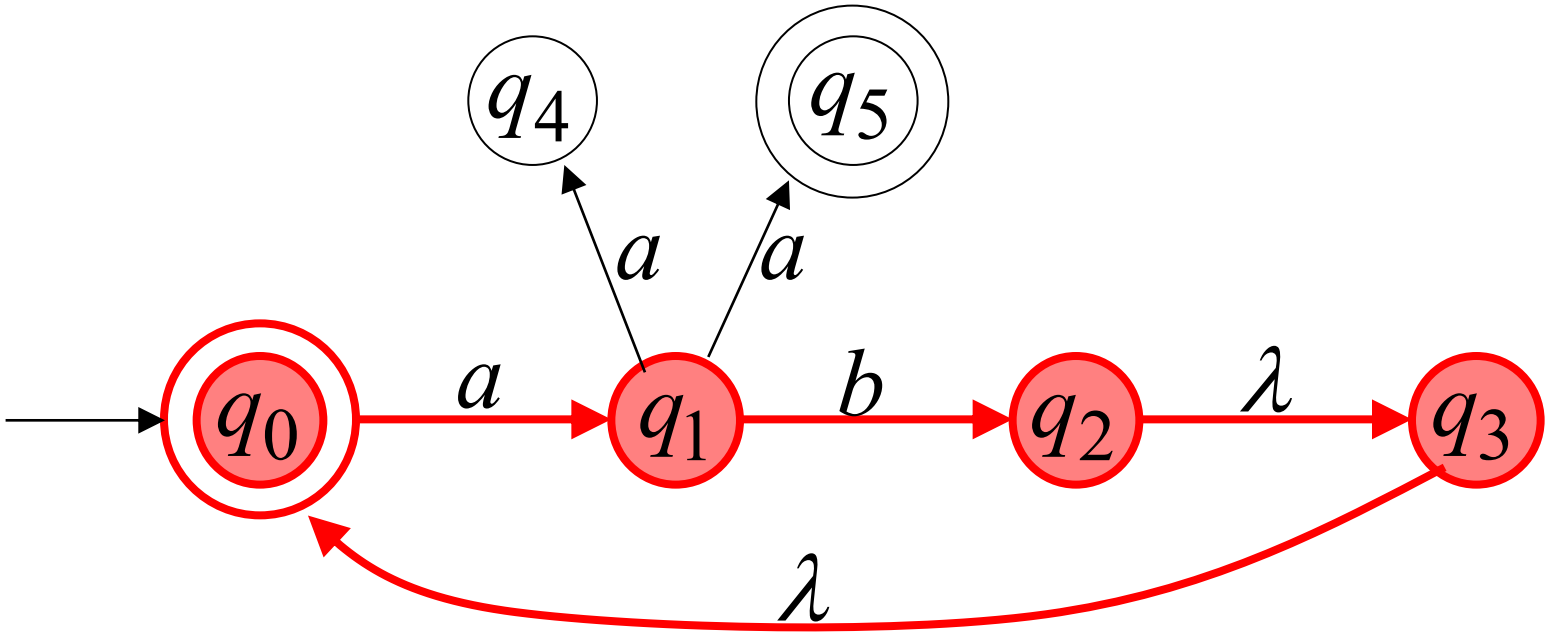
$$\delta^*(q_0, a) = \{q_1\}$$



$$\delta^*(q_0, aa) = \{q_4, q_5\}$$



$$\delta^*(q_0, ab) = \{q_2, q_3, q_0\}$$

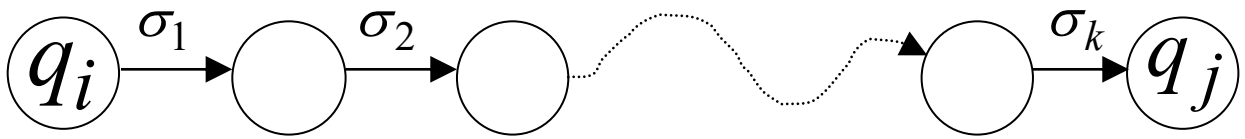


Formally

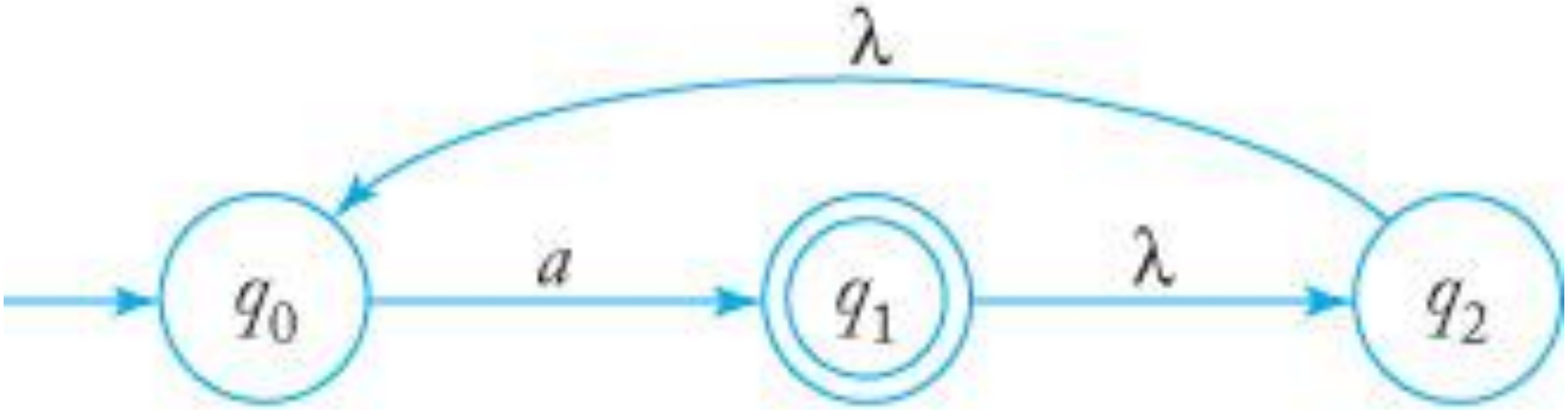
$q_j \in \delta^*(q_i, w)$: there is a walk from q_i to q_j with label w



$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$



Example 2.9

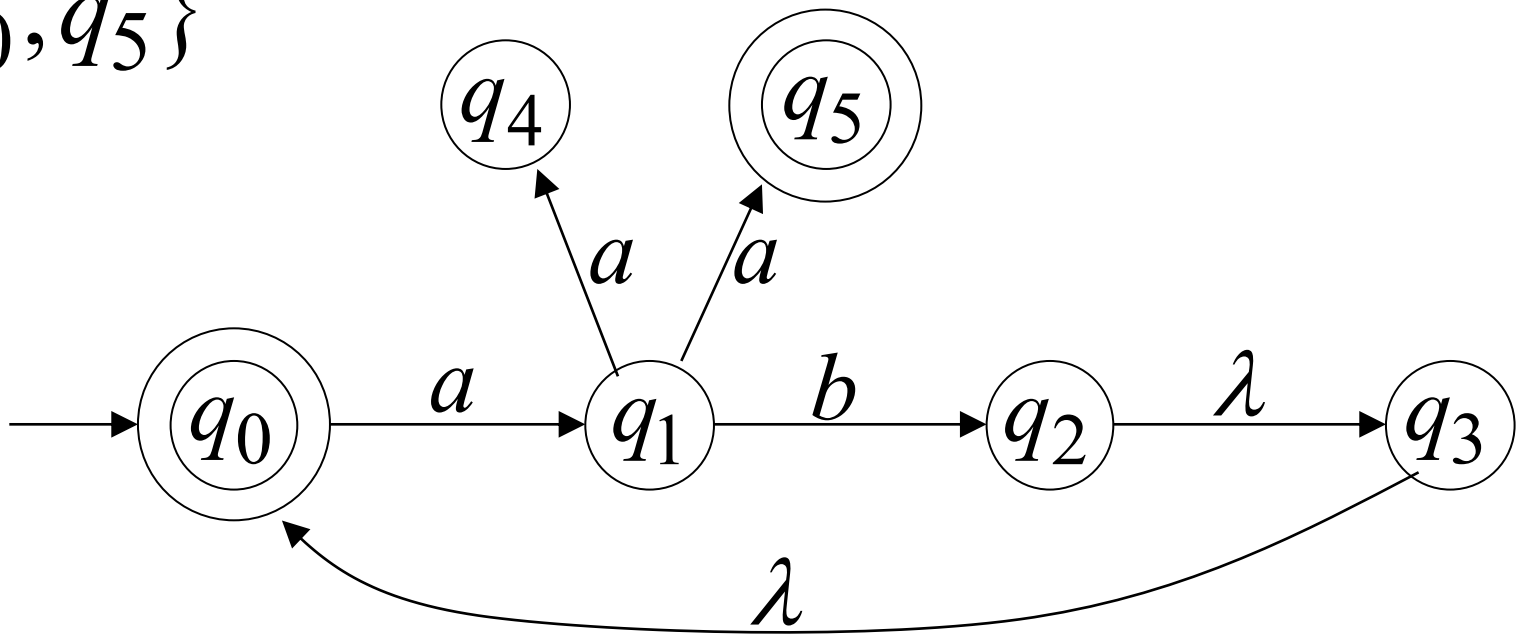


$$\delta^*(q_1, a) = \{q_0, q_1, q_2\}$$
$$\delta^*(q_2, \lambda) = \{q_0, q_2\}$$
$$\delta^*(q_2, aa) = \{q_0, q_1, q_2\}$$

The length of a walk labeled *a* between q_1 and q_2 is 4

The Language of an NFA M

$F = \{q_0, q_5\}$

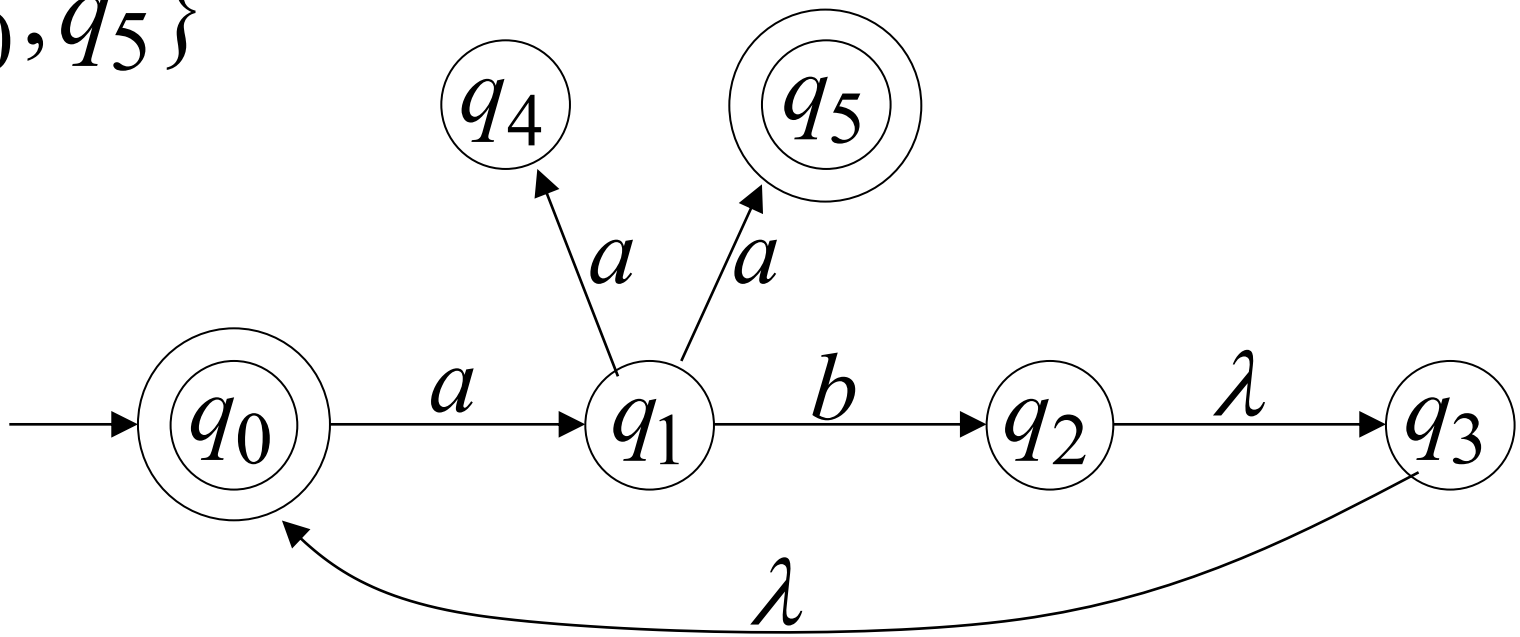


$\delta^*(q_0, aa) = \{q_4, \underline{q_5}\}$

$\searrow \in F$

$aa \in L(M)$

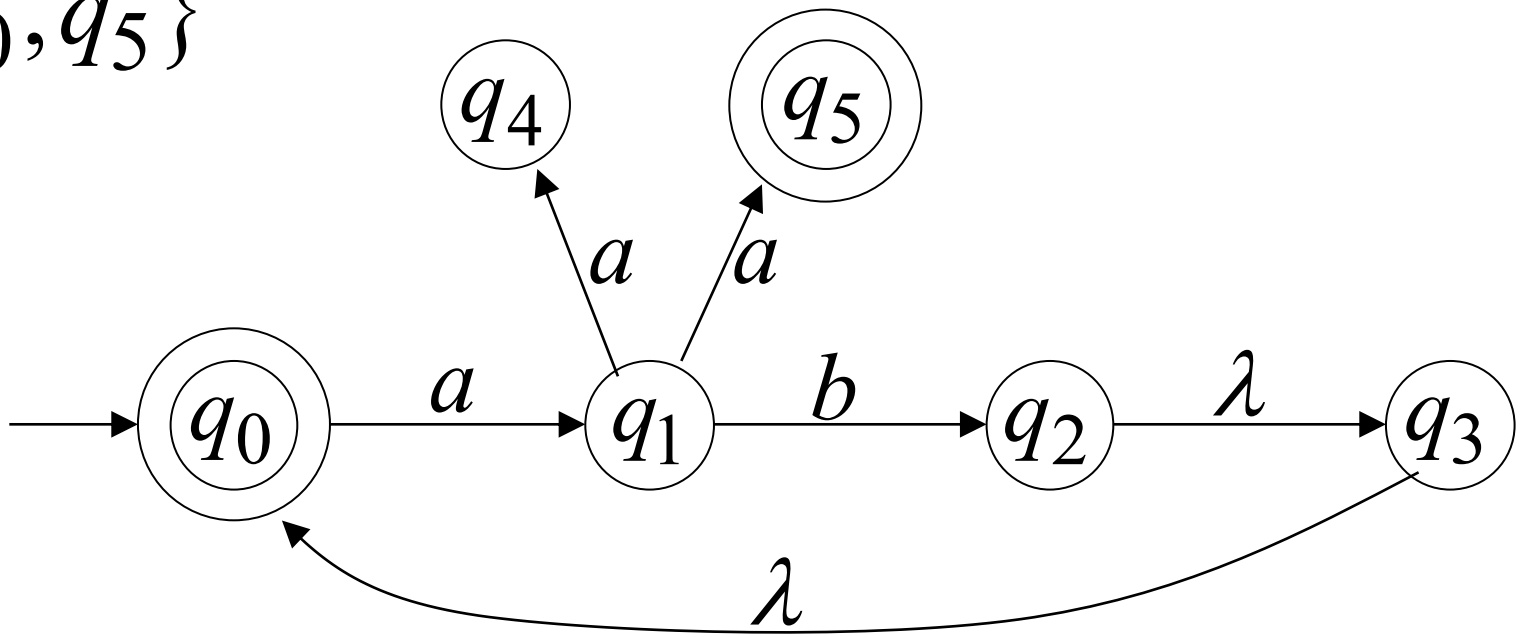
$F = \{q_0, q_5\}$



$\delta^*(q_0, ab) = \{q_2, q_3, \underline{q_0}\}$ $\quad ab \in L(M)$

$\searrow \in F$

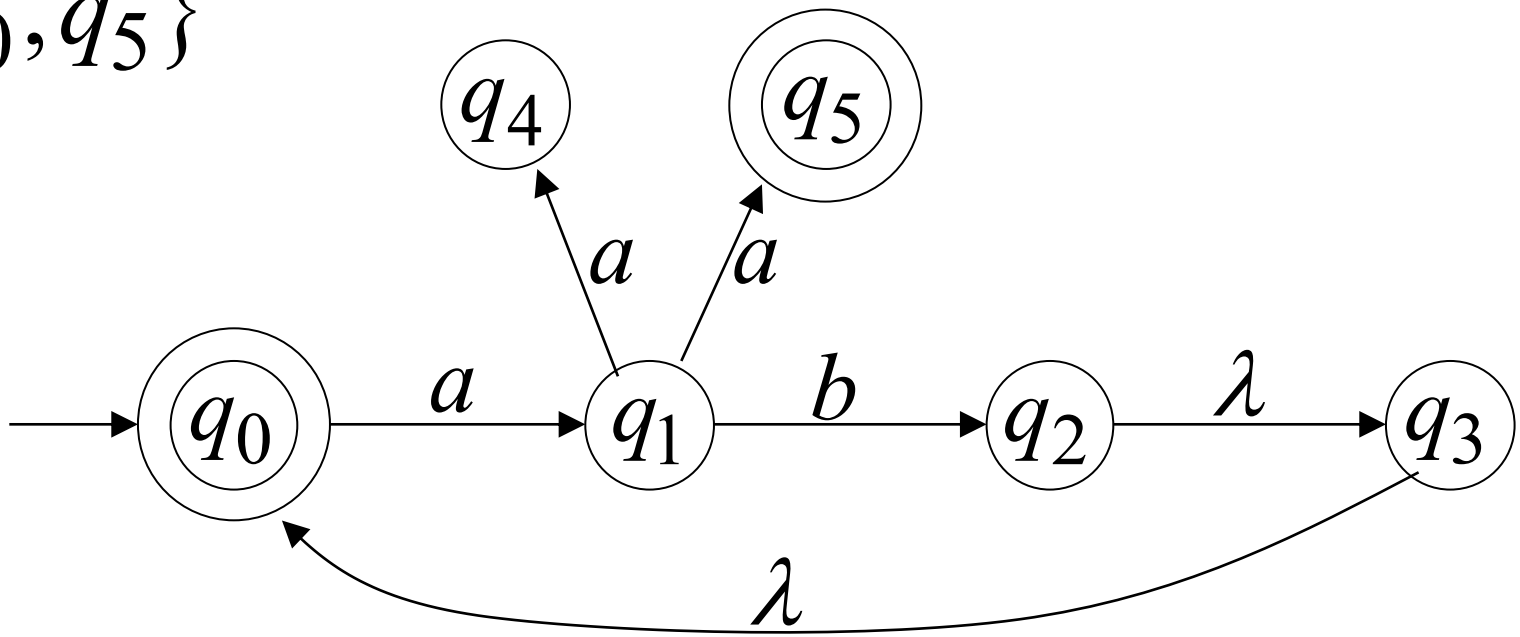
$F = \{q_0, q_5\}$



$\delta^*(q_0, abaa) = \{q_4, \underline{q_5}\}$ $\in F$

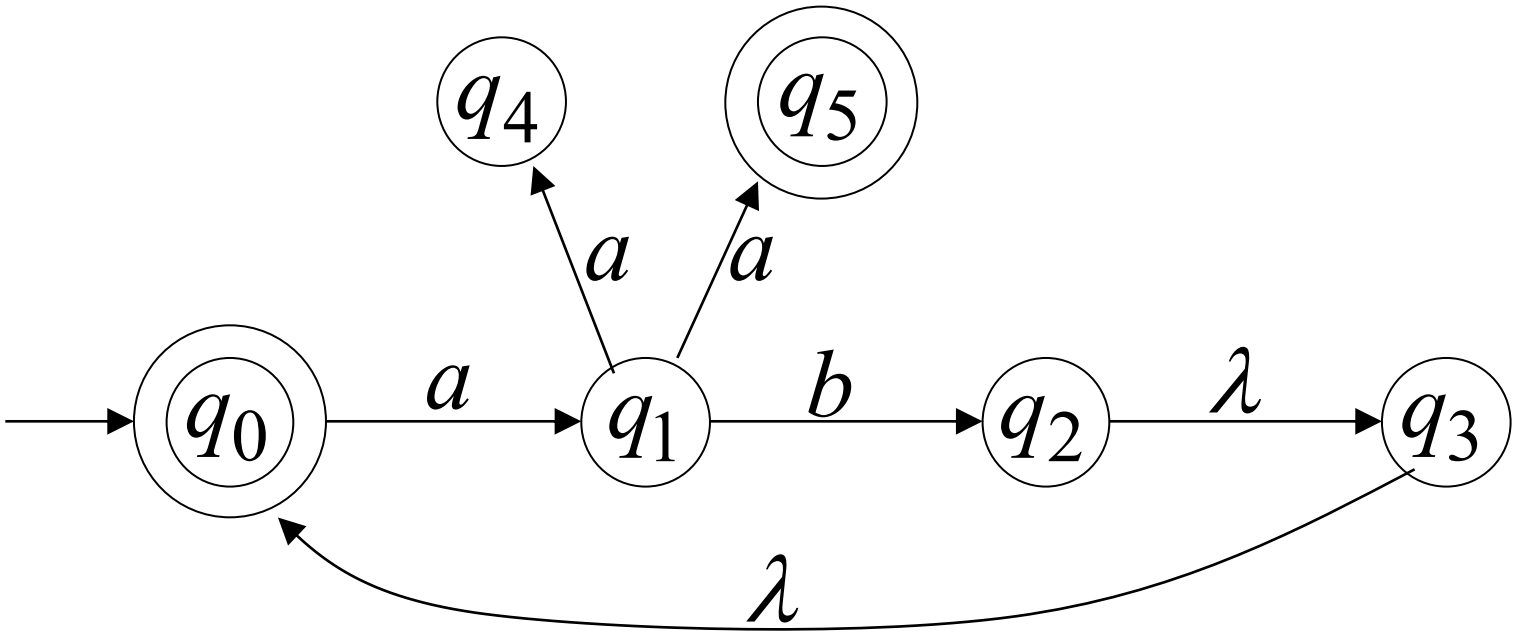
$abaa \in L(M)$

$$F = \{q_0, q_5\}$$



$$\delta^*(q_0, aba) = \{q_1\} \qquad aba \notin L(M)$$

$$\searrow \notin F$$



$$L(M) = \{ab\}^* \ \{aa\} \ \cup \ \{ab\}^*$$

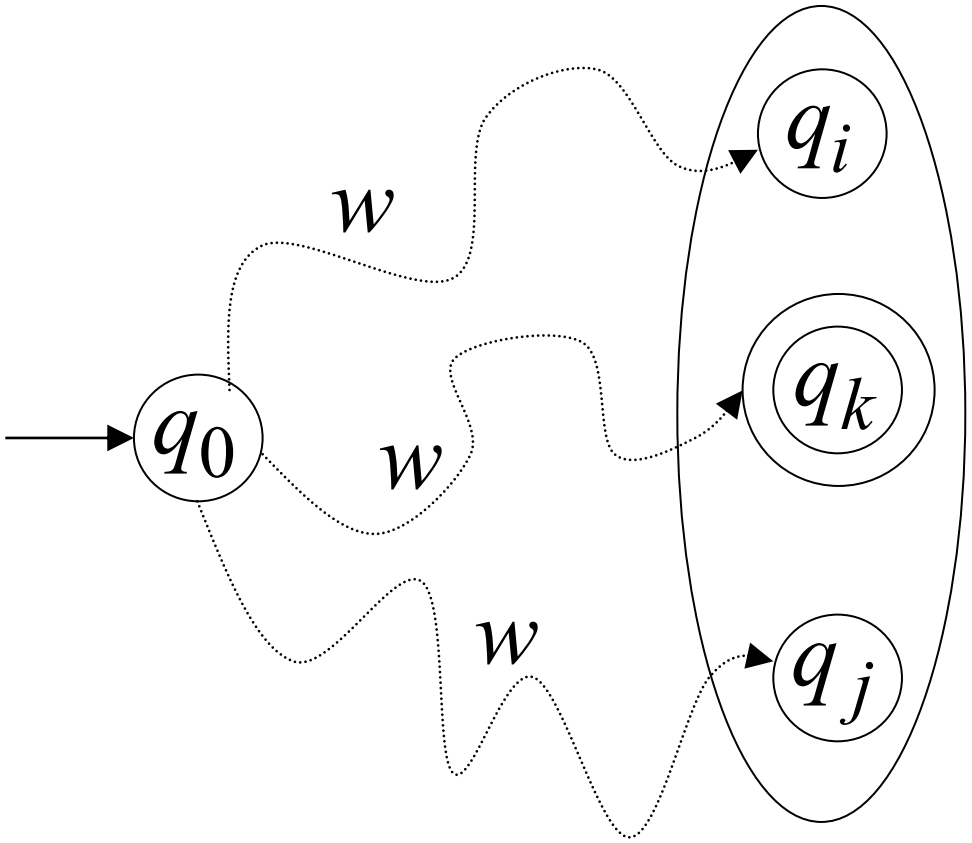
Definition 2.6

The language L accepted by an NFA M is defined as the set of all accepted strings :

$$L(M) = \left\{ w \in \sum^* : \delta^*(q_0, w) \bigcap F \neq \phi \right\}$$

$w \in L(M)$

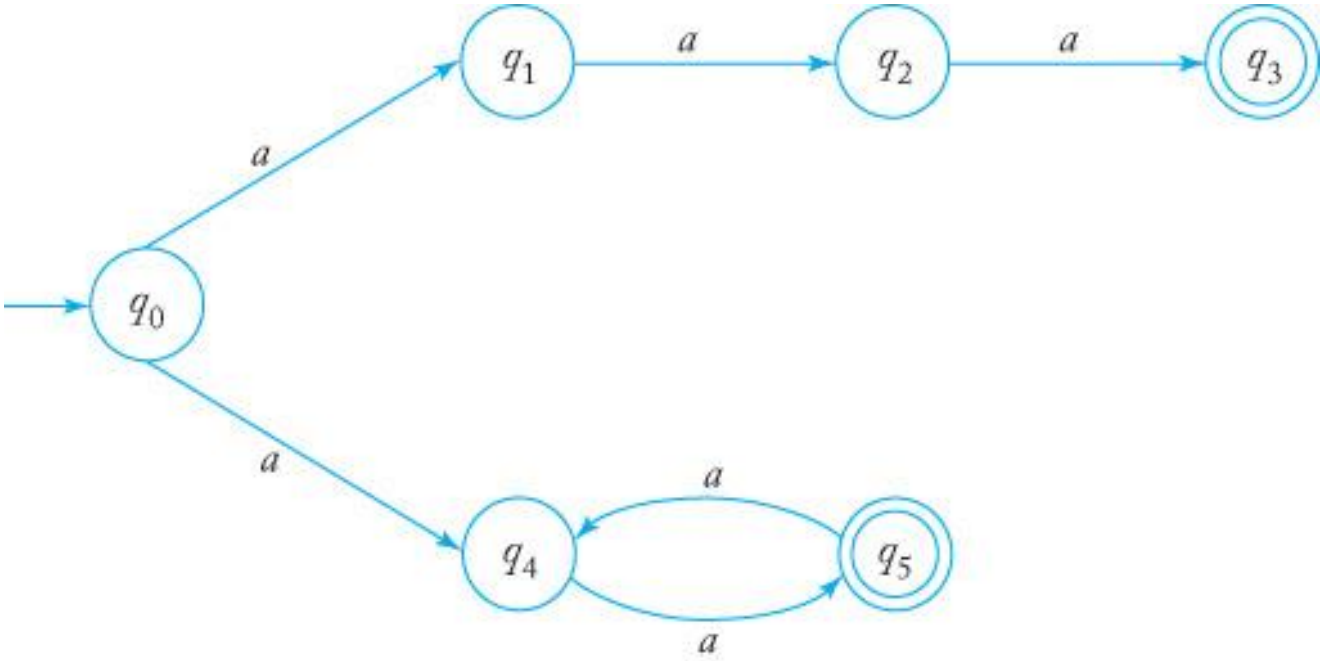
$\delta^*(q_0, w)$



$q_k \in F$

Why Nondeterminism?

- Many deterministic algorithms require that one make a choice at some stage (game-playing program, TSP, etc)
- Nondeterminism is sometimes helpful in solving problems easily



The language accepted by the NFA is

$\{a^3\} \cup \{a^{2n}: n \geq 1\}$

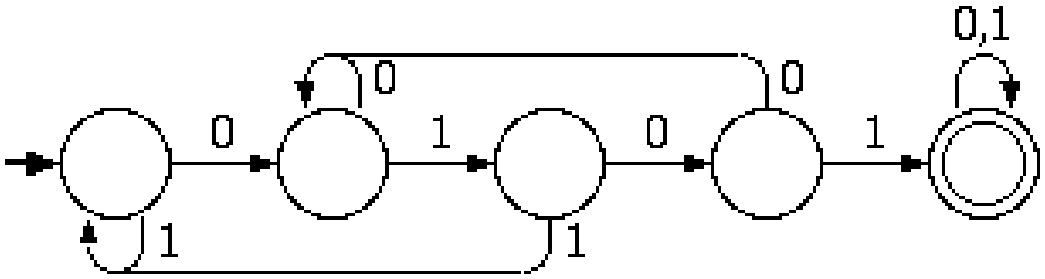
Why Nondeterminism?

- Nondeterminism is an effective mechanism for describing some complicated languages concisely.

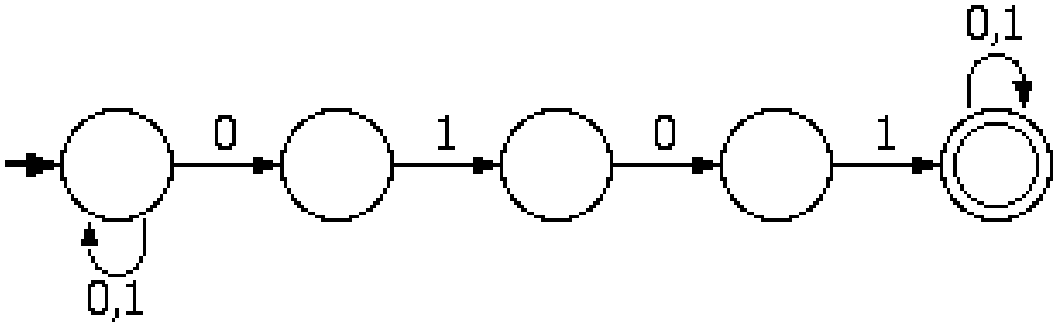
  Ex: $S \rightarrow aSb \mid \lambda$

More Examples

– All strings that contain the substring 0101.



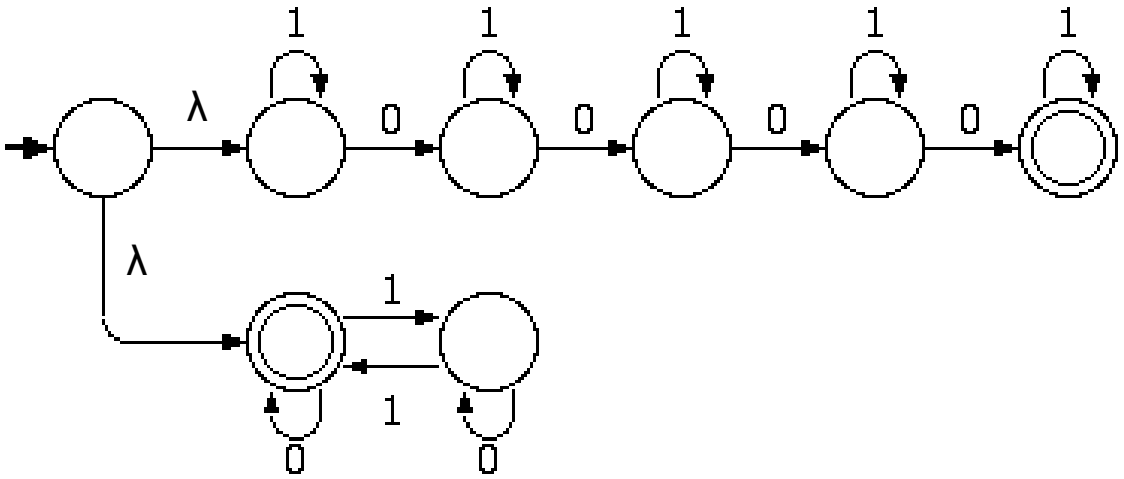
DFA



NFA
(5 states)

More Examples

– All strings containing exactly 4 0s or an even number of 1s. (8 states)



NFA

Outline



Deterministic Finite Accepters (DFA)



Nondeterministic Finite Accepters (NFA)



Equivalence of DFA and NFA



Reduction of the Number of States in FA*

Equivalence of Machines

- Definition 2.7:

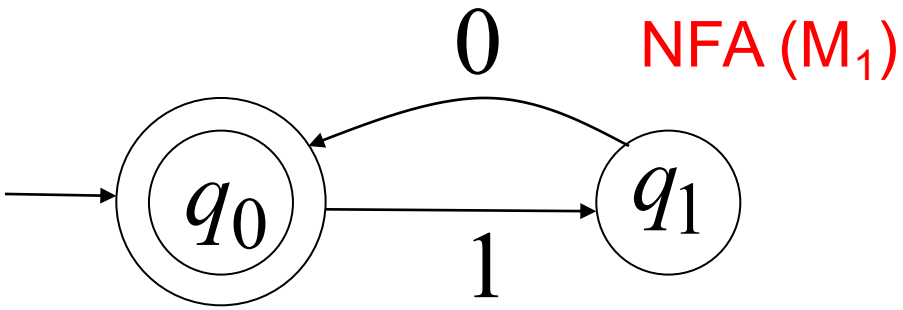
Two finite accepters M_1 and M_2 are said to be equivalent if

$$L(M_1) = L(M_2),$$

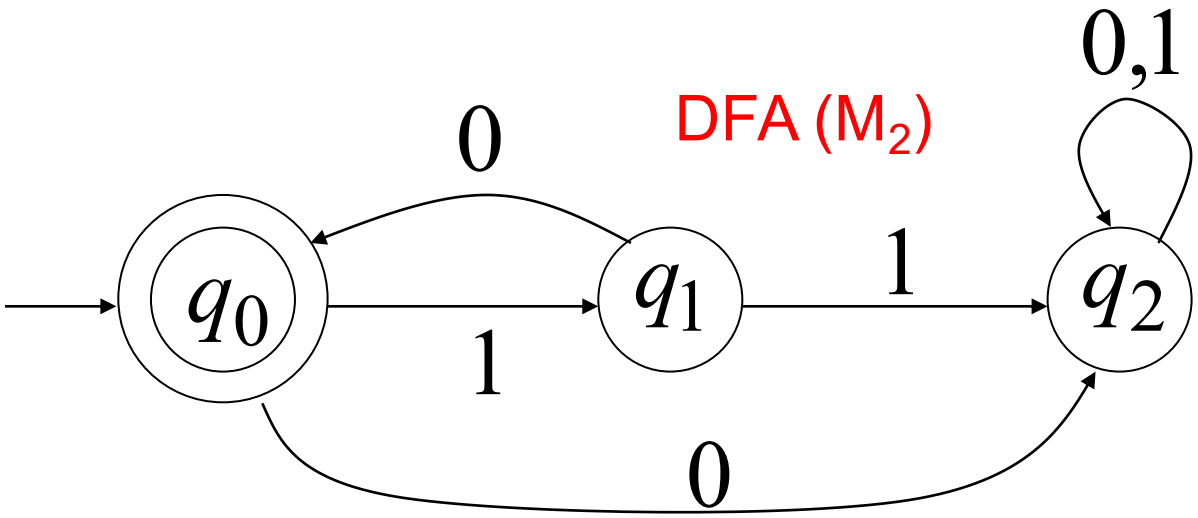
that is, if they both accept the same language.

Example of equivalent machines

$L(M_1) = \{10\}^*$



$L(M_2) = \{10\}^*$

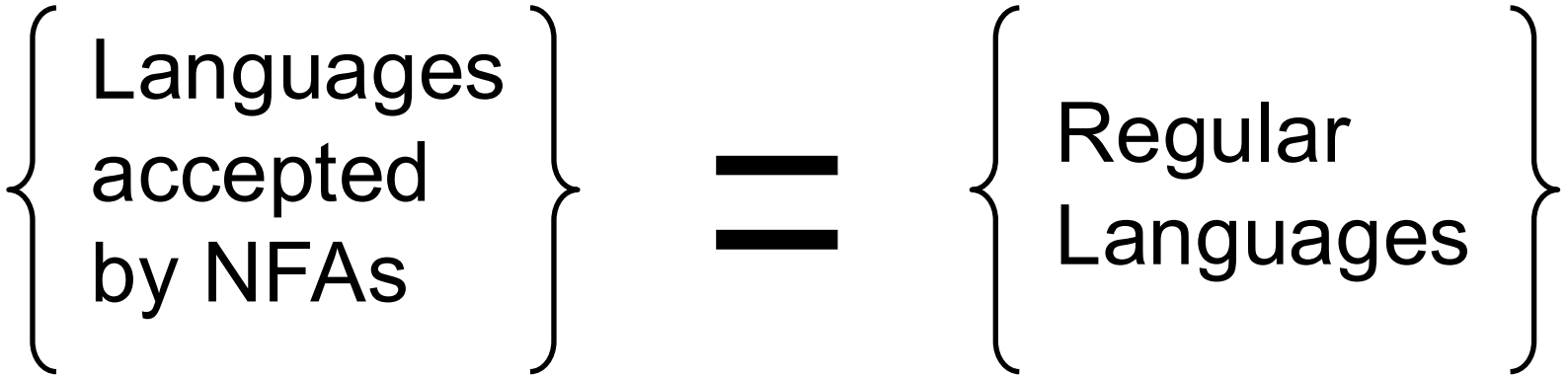


DFA v.s. NFA

- Which one is more powerful?
- "More powerful" means
  - An automaton of one kind can achieve something that cannot be done by any automaton of the other kind
- Trivially, DFA is a restricted kind of NFA

NFAs and DFAs have the same computation power

We will prove:



Languages
accepted
by DFAs

Step 1

$$\left\{\begin{array}{l}\text{Languages}\\ \text{accepted}\\ \text{by NFAs}\end{array}\right\} \supseteq \left\{\begin{array}{l}\text{Regular}\\ \text{Languages}\end{array}\right\}$$

Proof: Every DFA is trivially an NFA

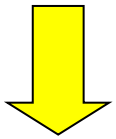


Any language L accepted by a DFA is also accepted by an NFA

Step 2

$$\left\{\begin{array}{l}\text{Languages}\\ \text{accepted}\\ \text{by NFAs}\end{array}\right\} \subseteq \left\{\begin{array}{l}\text{Regular}\\ \text{Languages}\end{array}\right\}$$

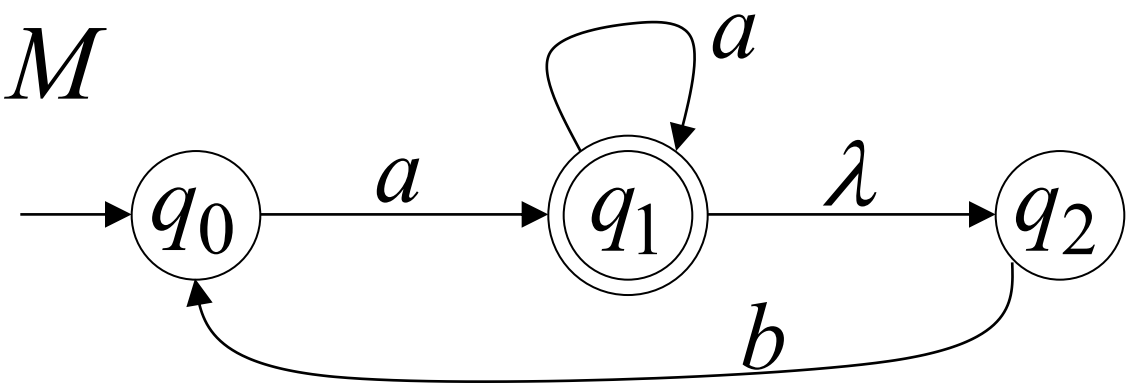
Proof: Any NFA can be converted to an equivalent DFA



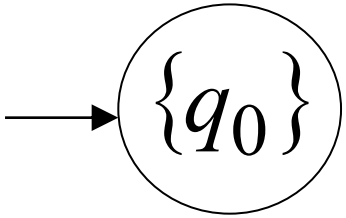
Any language L accepted by an NFA is also accepted by a DFA

Convert NFA to DFA

NFA M

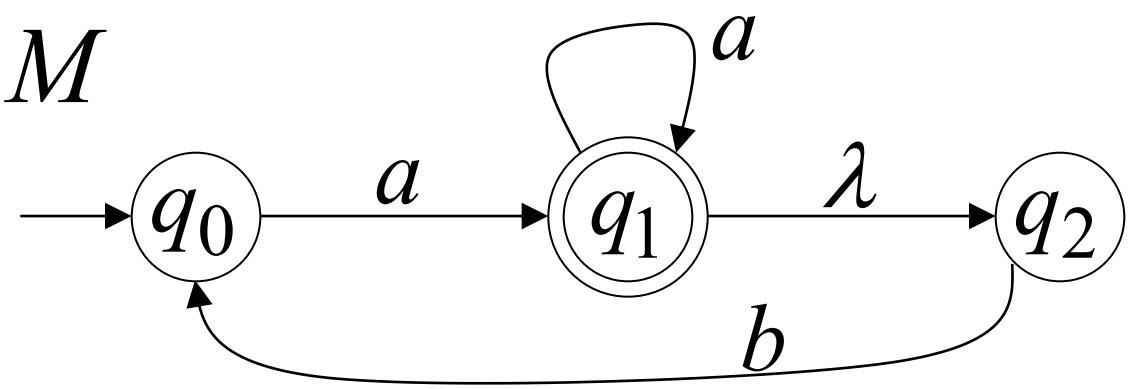


DFA M'

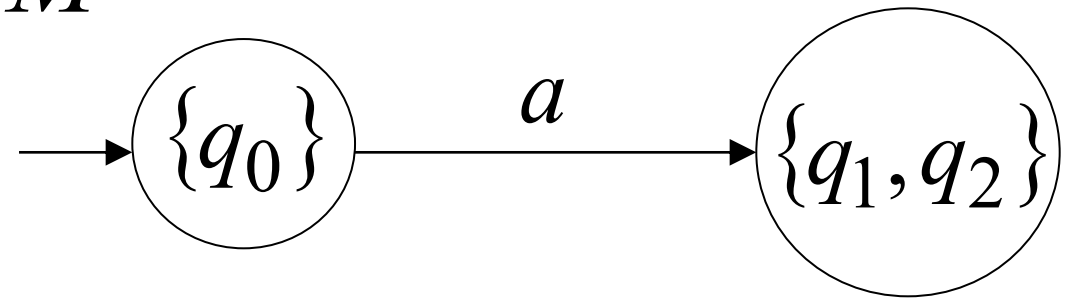


Convert NFA to DFA

NFA M

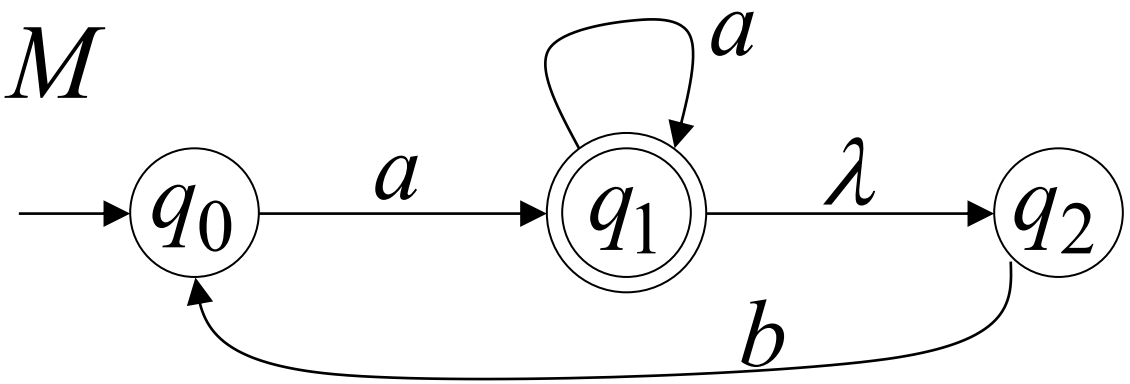


DFA M'

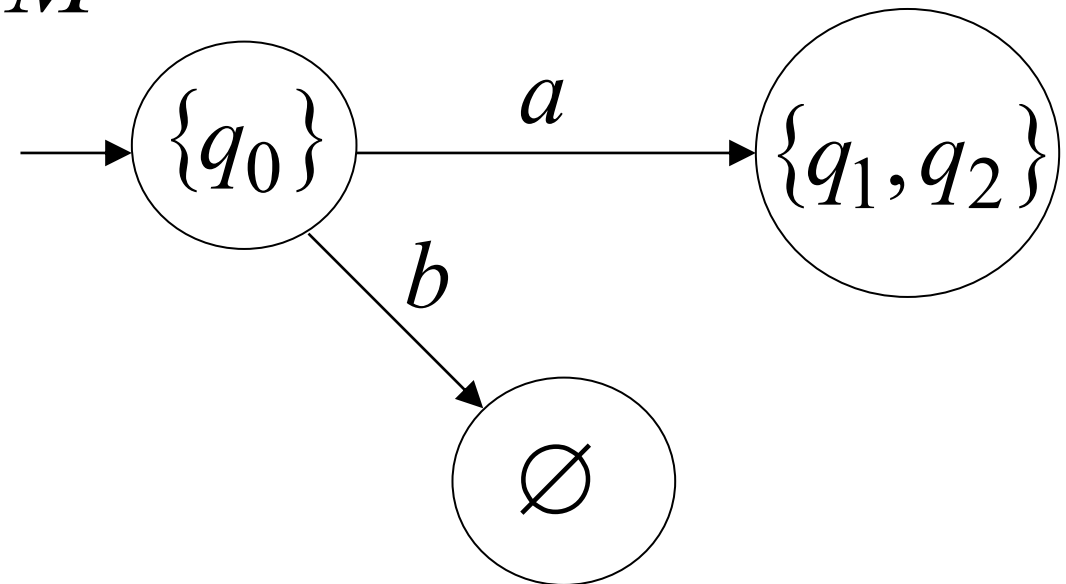


Convert NFA to DFA

NFA M

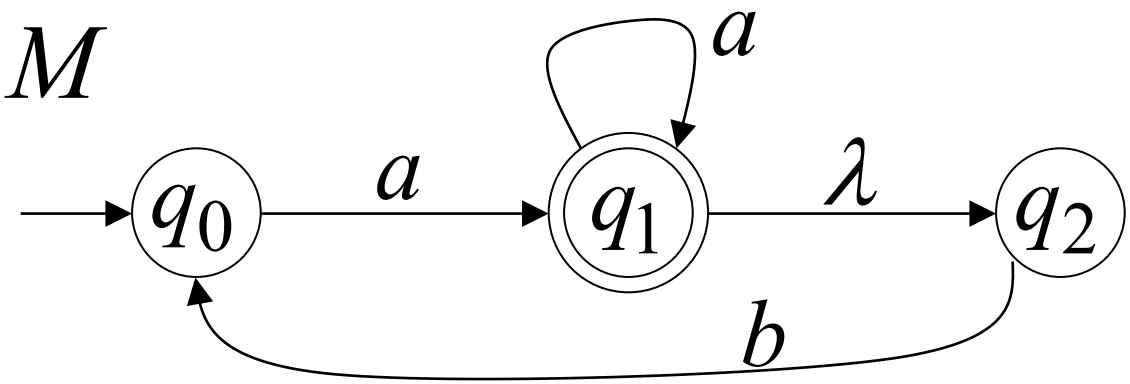


DFA M'

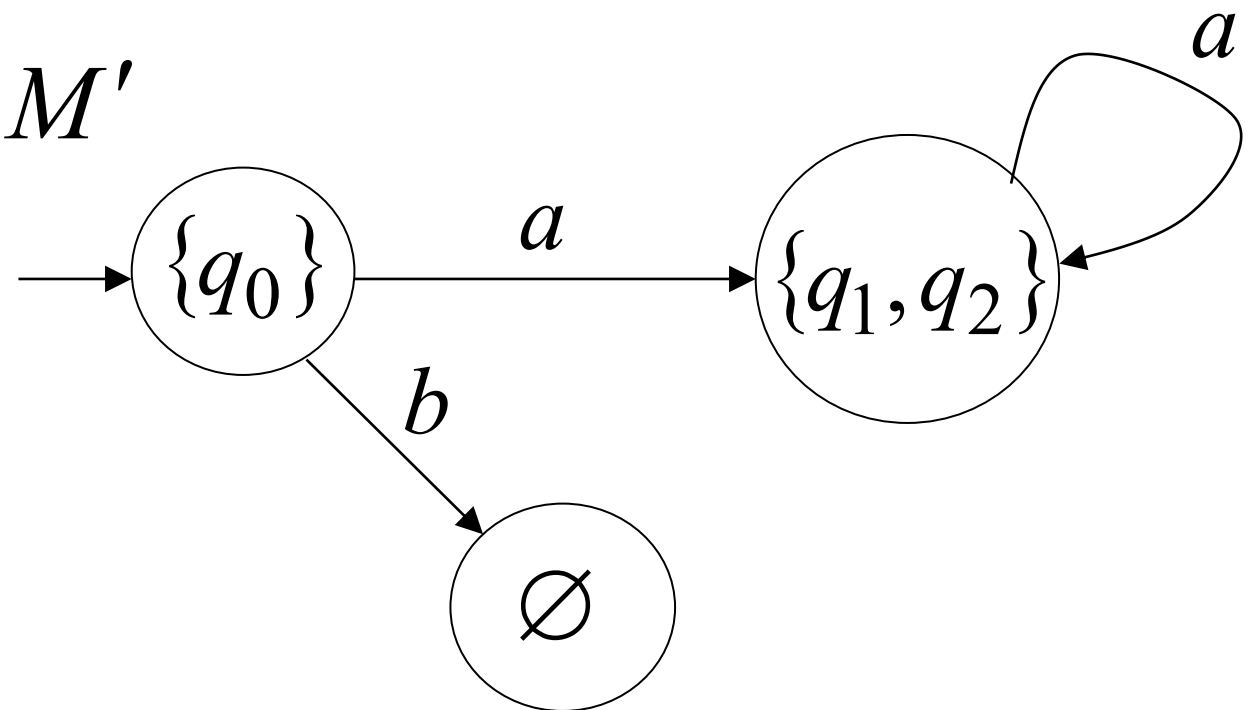


Convert NFA to DFA

NFA M

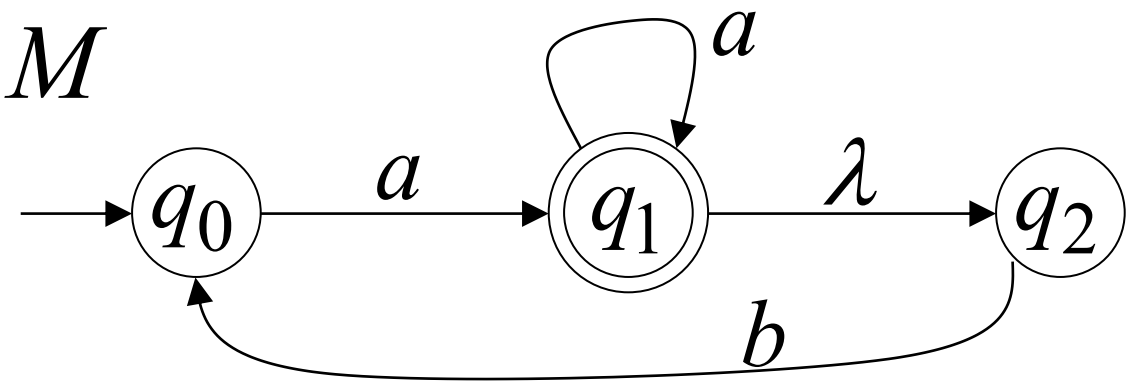


DFA M'

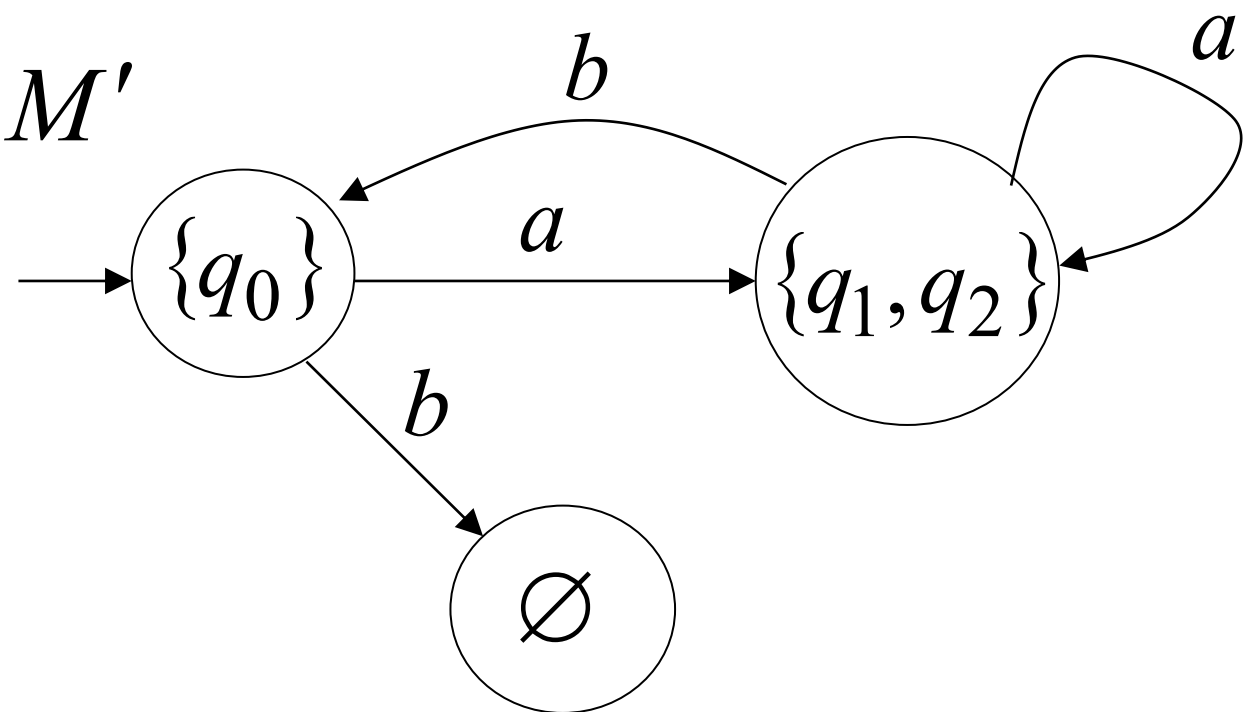


Convert NFA to DFA

NFA M

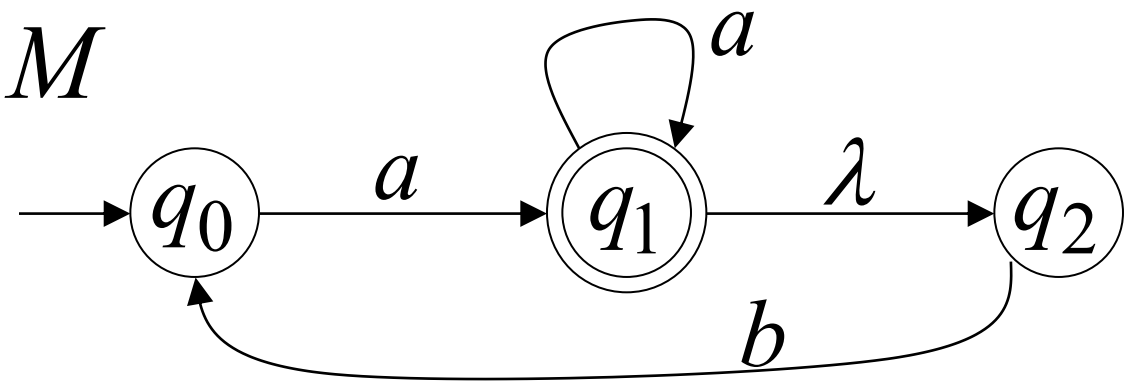


DFA M'

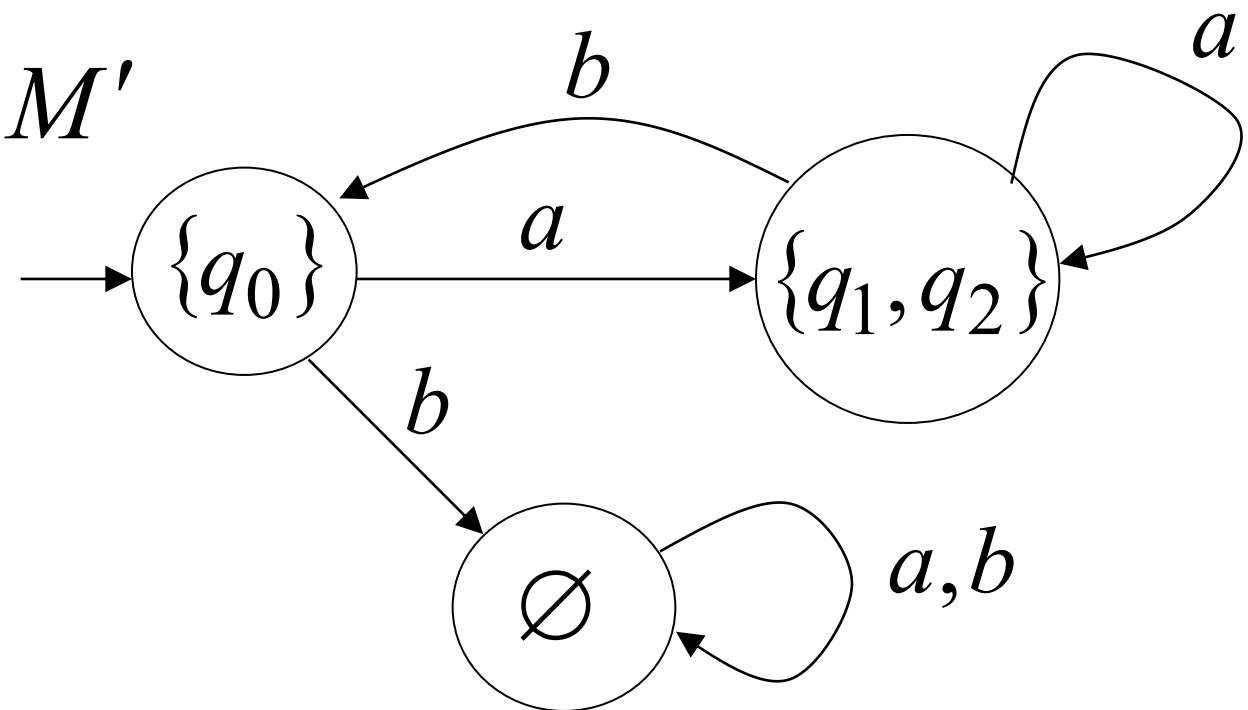


Convert NFA to DFA

NFA M

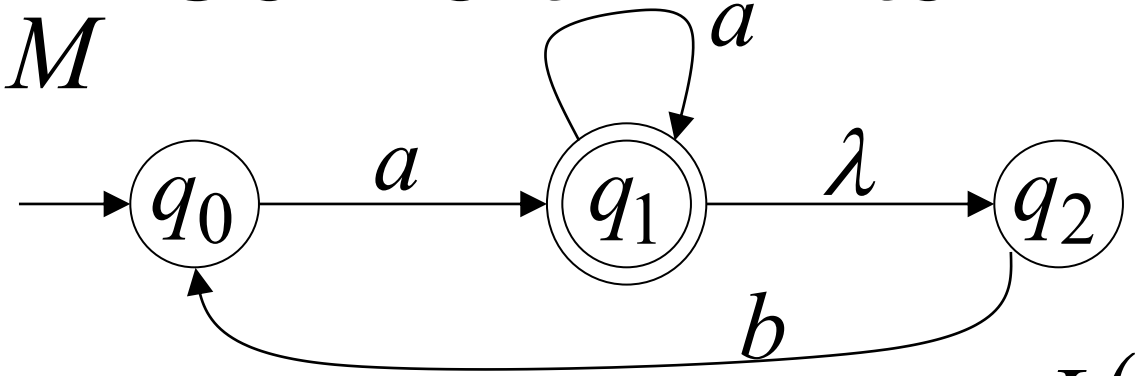


DFA M'



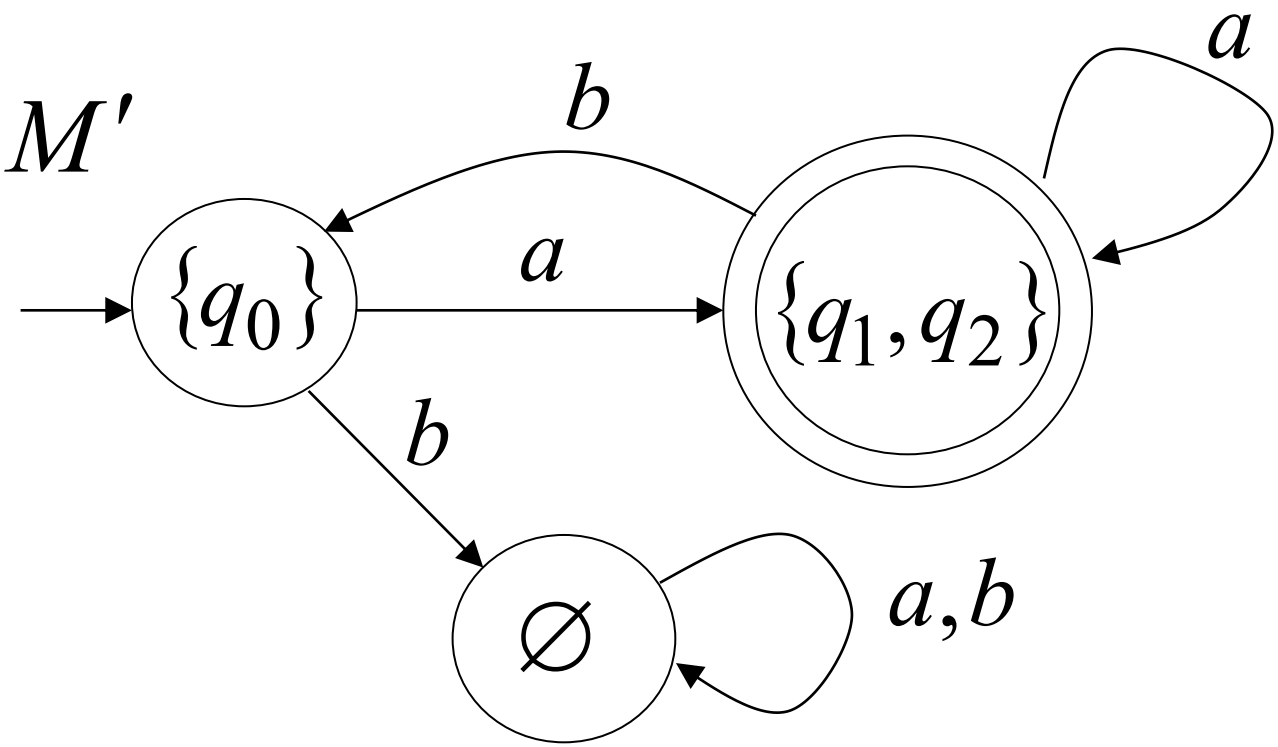
Convert NFA to DFA

NFA M



$$L(M) = L(M')$$

DFA M'



NFA to DFA: Remarks

We are given an NFA M

We want to convert it
to an equivalent DFA M'

With $L(M) = L(M')$

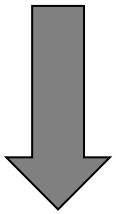
If the NFA has states $q_0, q_1, q_2, \ldots$

the DFA has states in the powerset

$$\varnothing, \{q_0\}, \{q_1\}, \{q_1, q_2\}, \{q_3, q_4, q_7\}, \ldots.$$

Procedure NFA to DFA

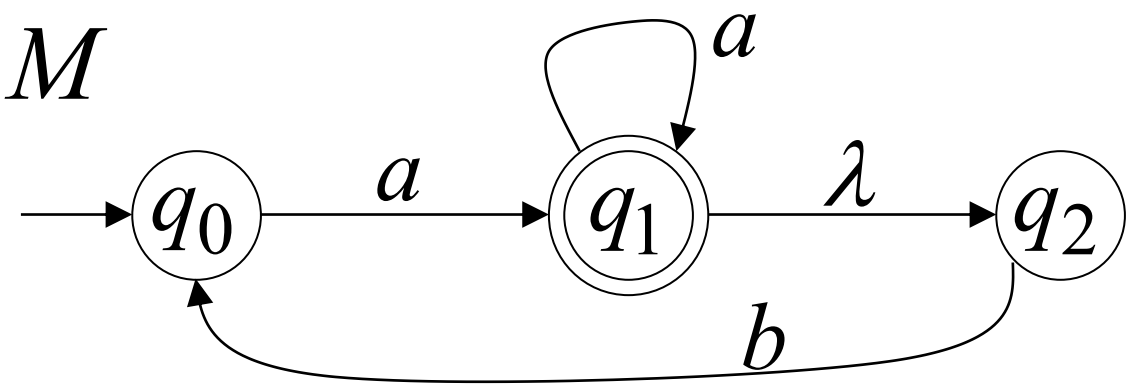
1. Initial state of NFA: q_0



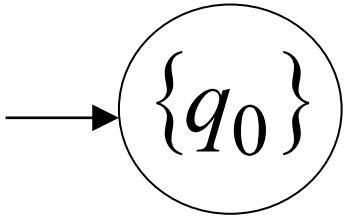
Initial state of DFA: $\{q_0\}$

Example 2.12

NFA M



DFA M'



Procedure NFA to DFA

2. For every DFA's state $\{q_i, q_j, \ldots, q_m\}$

Compute in the NFA

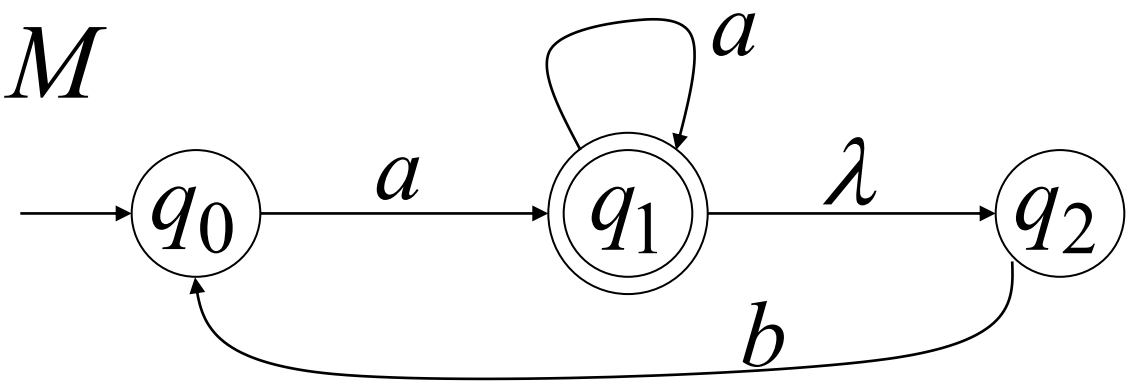
$$\left.\begin{array}{l} \delta^*(q_i, a), \\ \delta^*(q_j, a), \\ \\ \ldots \end{array}\right\} = \{q'_i, q'_j, \ldots, q'_m\}$$

Add transition to DFA

$$\delta(\{q_i, q_j, \ldots, q_m\},\ a) = \{q'_i, q'_j, \ldots, q'_m\}$$

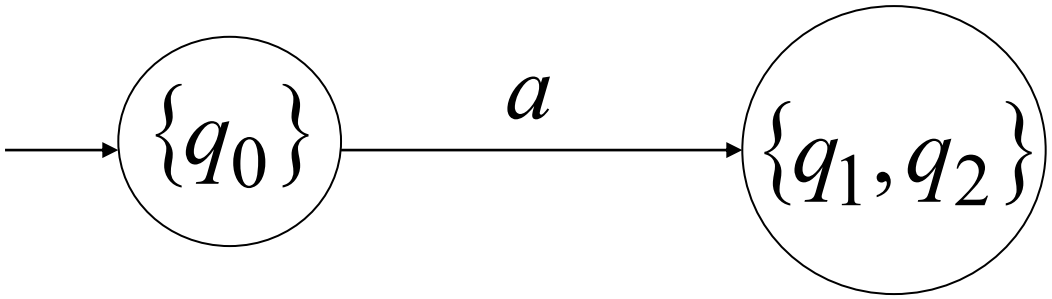
Example 2.12

NFA M



$$\delta^*(q_0, a) = \{q_1, q_2\}$$

DFA M'



$$\delta(\{q_0\}, a) = \{q_1, q_2\}$$

Procedure NFA to DFA

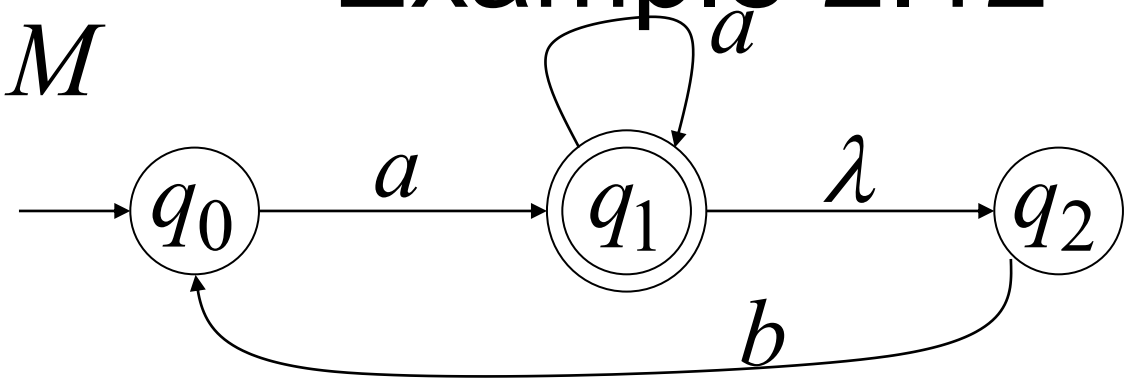
Repeat Step 2 for all letters in alphabet,

until

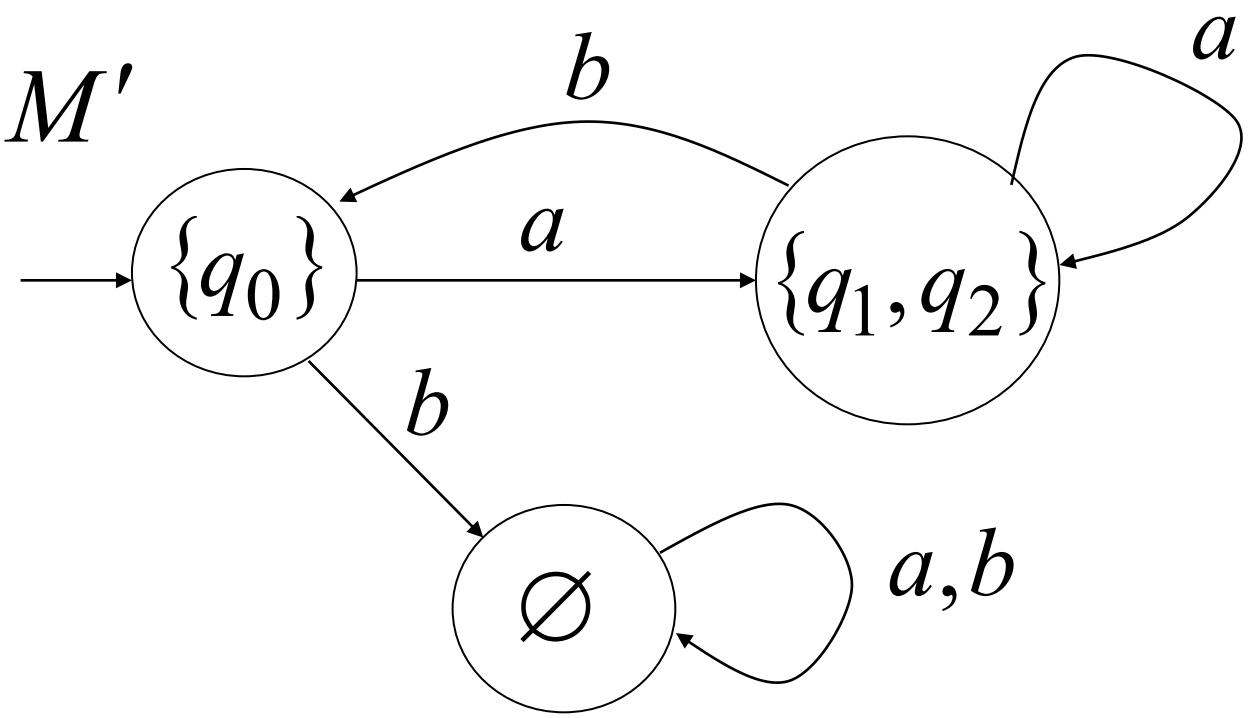
no more transitions can be added.

Example 2.12

NFA M



DFA M'



Procedure NFA to DFA

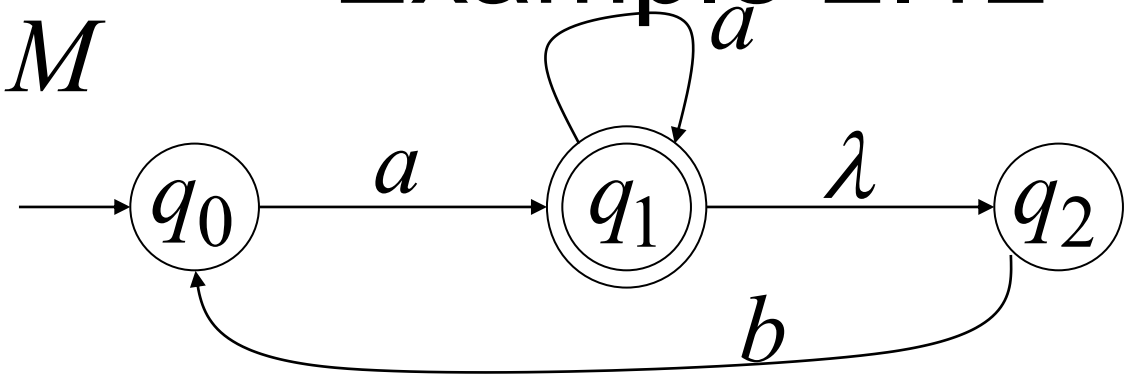
3. For any DFA state $\{q_i, q_j, \ldots, q_m\}$

If some q_j is a final state in the NFA

Then, $\{q_i, q_j, \ldots, q_m\}$
is a final state in the DFA

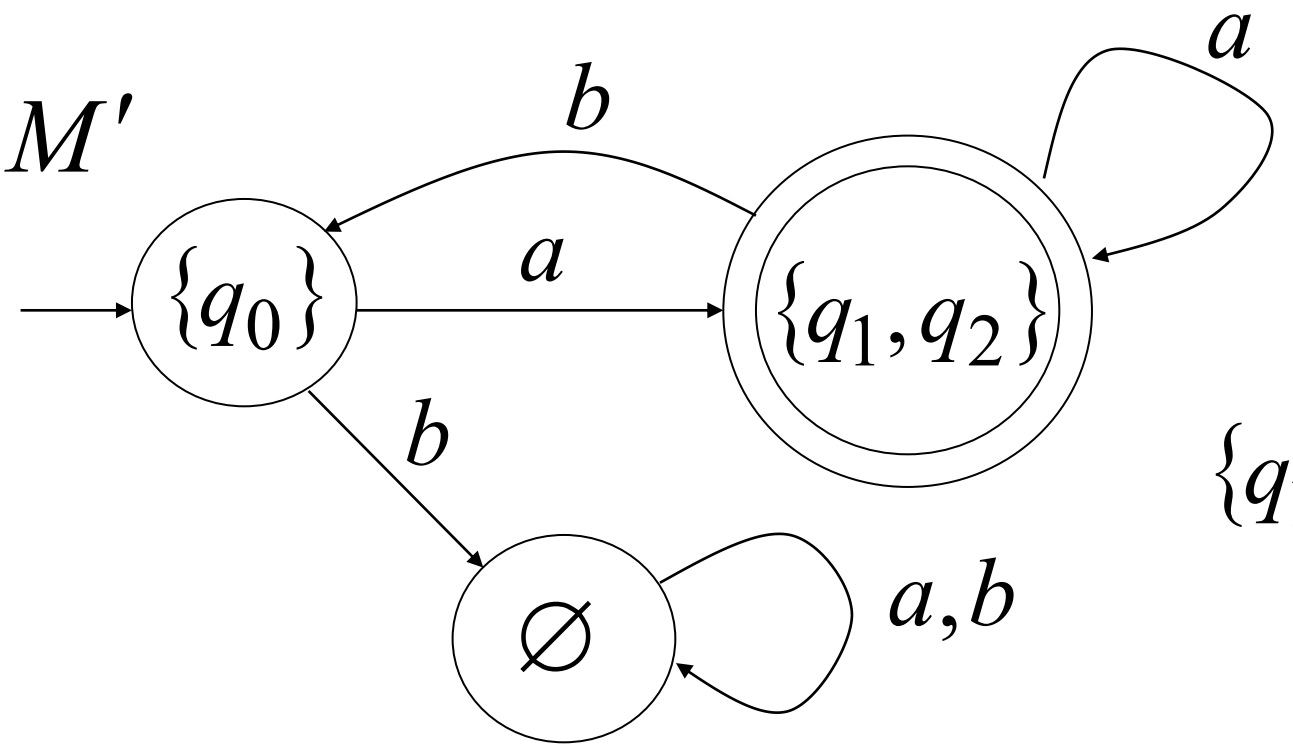
Example 2.12

NFA M



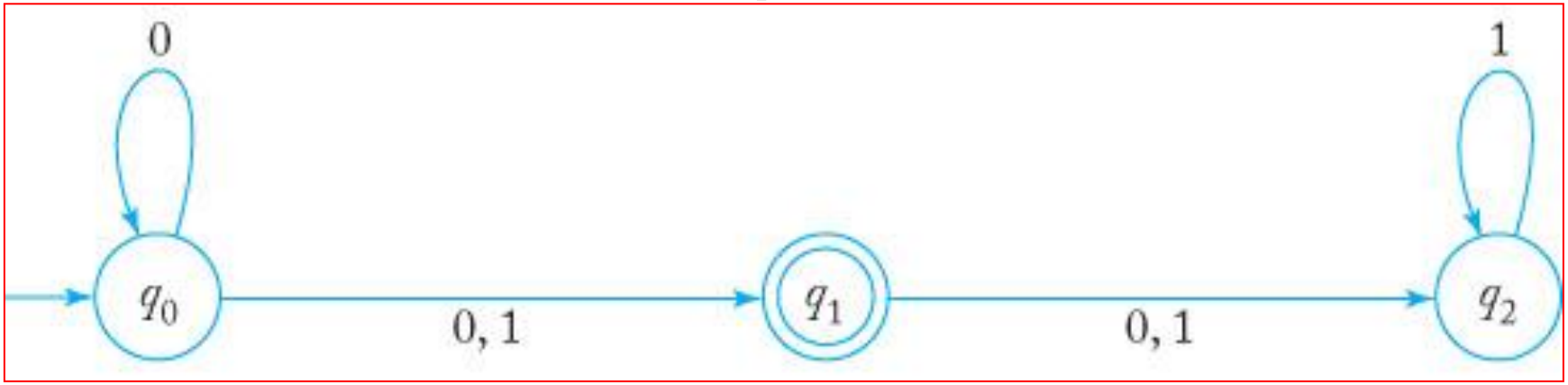
$q_1 \in F$

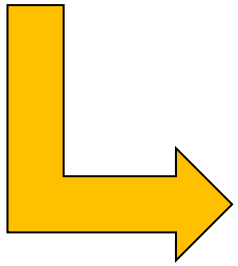
DFA M'

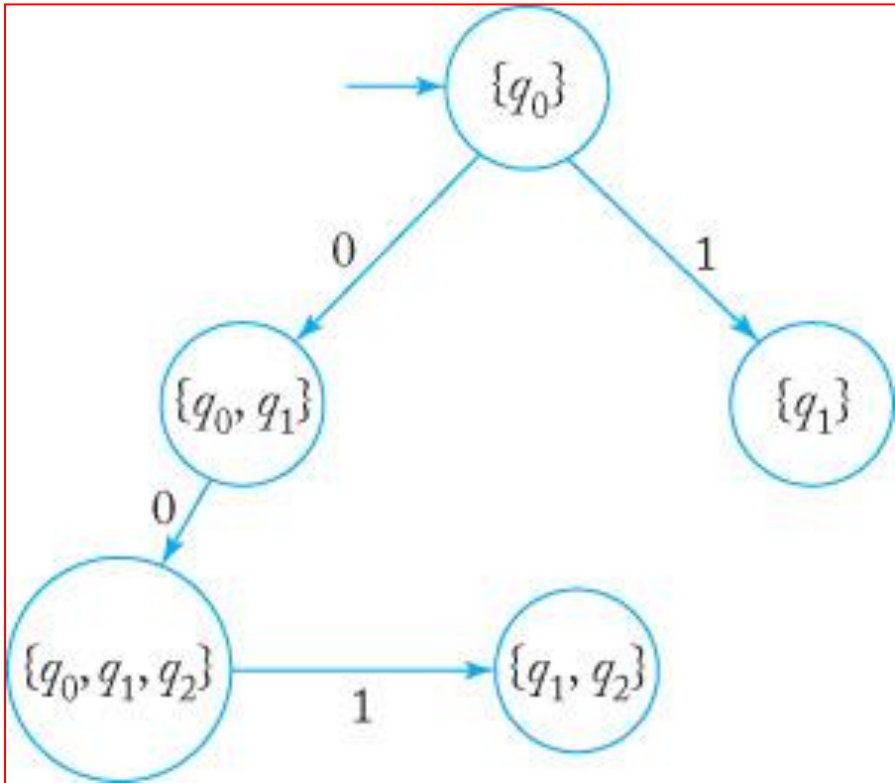


$\{q_1, q_2\} \in F'$

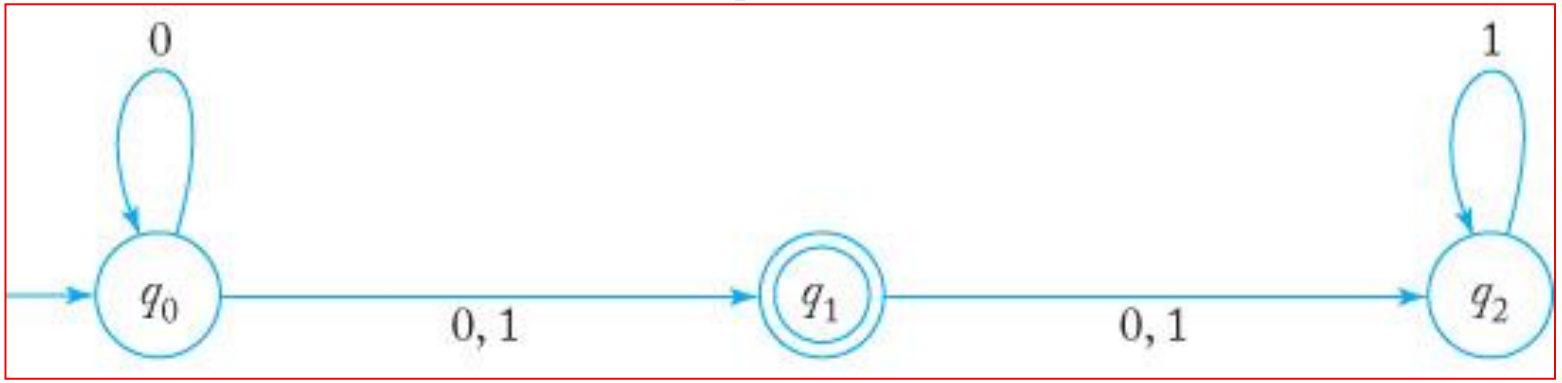
Example 2.13

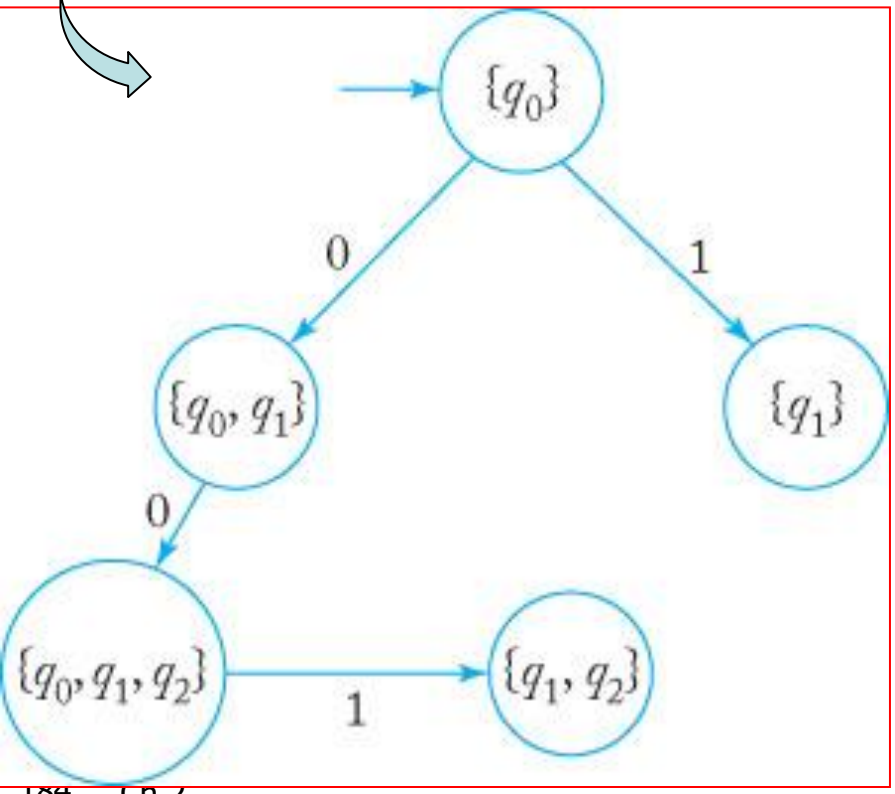


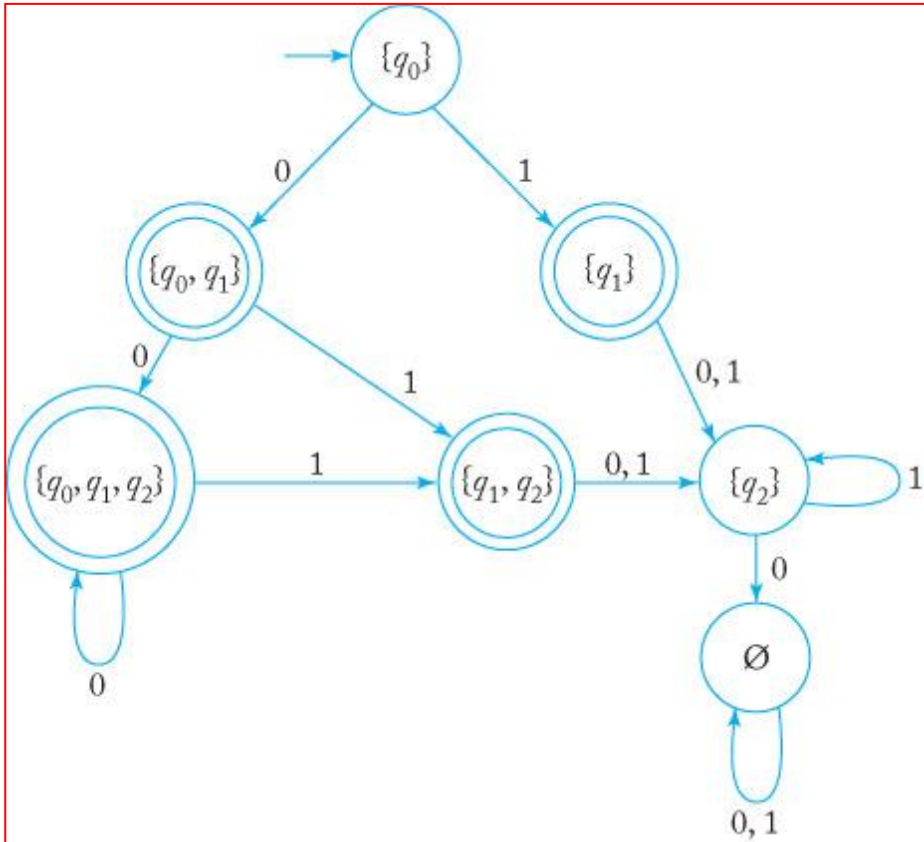




Example 2.13


0
1
q_0
0, 1
q_1
0, 1
q_2


$\{q_0\}$
0
1
$\{q_0, q_1\}$
$\{q_1\}$
0
$\{q_0, q_1, q_2\}$
1
$\{q_1, q_2\}$


$\{q_0\}$
0
1
$\{q_0, q_1\}$
$\{q_1\}$
0
1
0, 1
$\{q_0, q_1, q_2\}$
1
$\{q_1, q_2\}$
0, 1
$\{q_2\}$
1
0
0
Ø
0, 1

NFA M
DFA M'

Theorem 2.2

Take NFA M

Apply procedure to obtain DFA M'

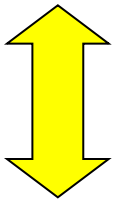
Then M and M' are equivalent :

$$L(M) = L(M')$$

Proof

NFA M
DFA M'

$$L(M) = L(M')$$



$$L(M) \subseteq L(M') \quad \text{AND} \quad L(M) \supseteq L(M')$$

NFA M
DFA M'

First we show: $L(M) \subseteq L(M')$

Take arbitrary: $w \in L(M)$

We will prove: $w \in L(M')$

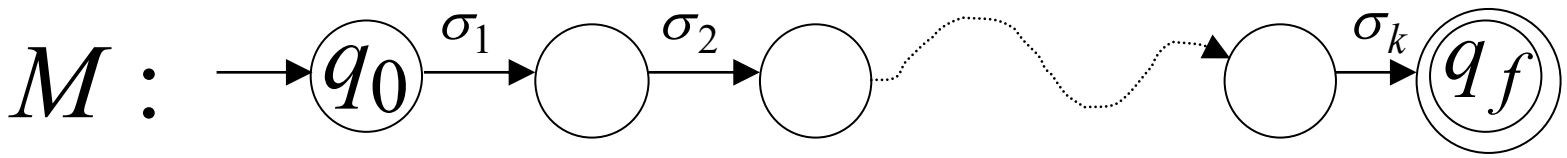
NFA M
DFA M'

$$w \in L(M)$$





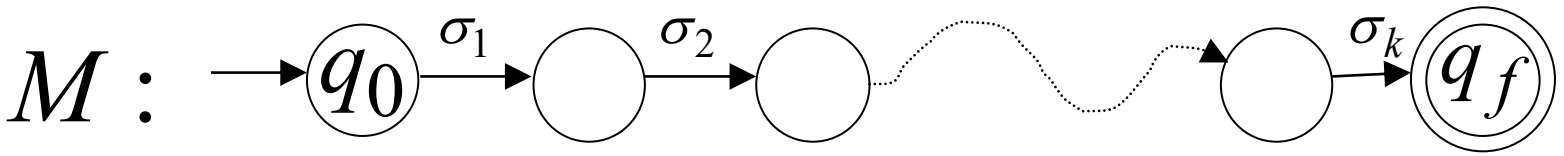
$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$

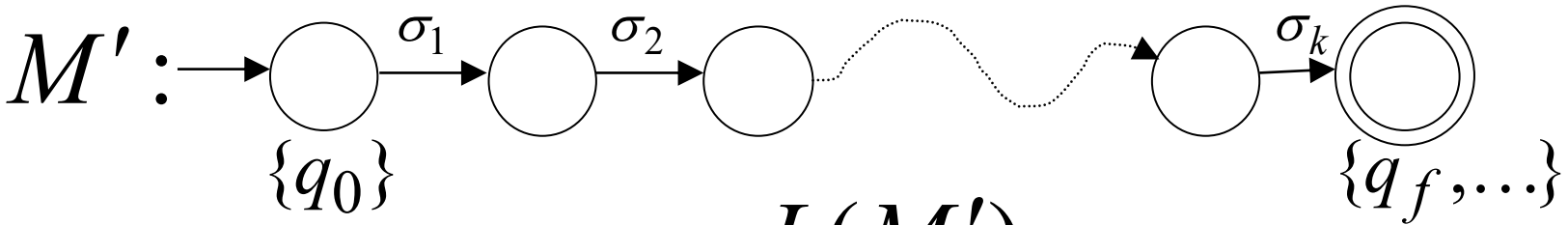


NFA M
DFA M'

We will show that if $w \in L(M)$

$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$



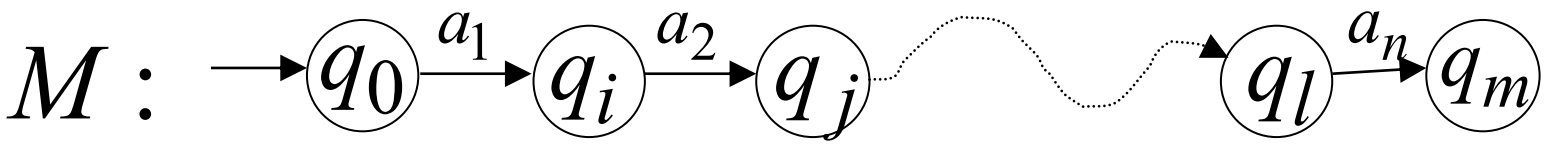


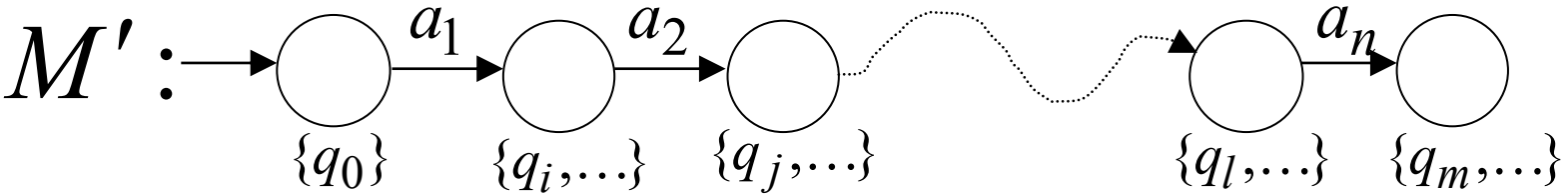
$$w \in L(M')$$

NFA M
DFA M'

More generally, we will show that if in M:

(arbitrary string) $v = a_1 a_2 \cdots a_n$



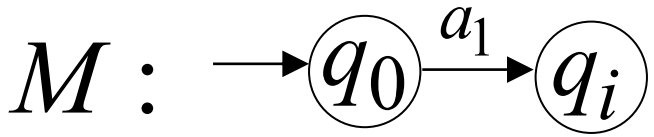


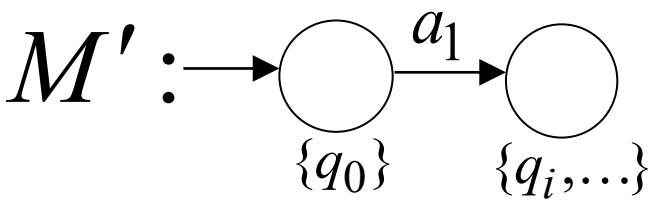
Proof by induction on $|v|$

NFA M
DFA M'

Induction Basis:

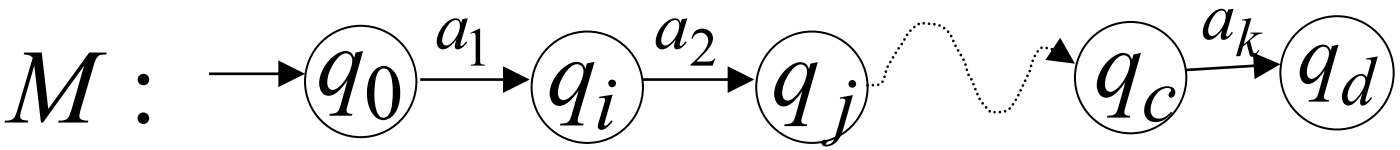
$$v = a_1$$

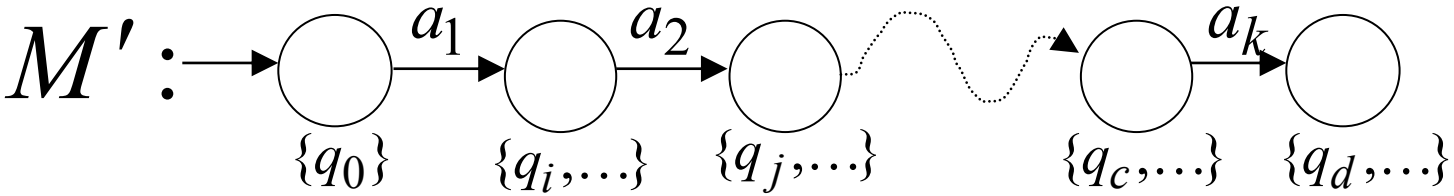




Induction hypothesis: $1 \leq |v| \leq k$

$$v = a_1 a_2 \cdots a_k$$

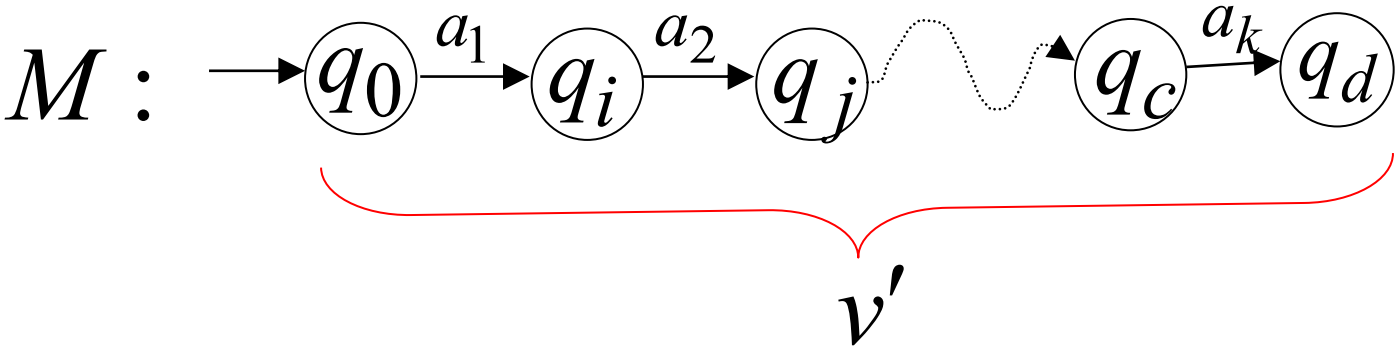


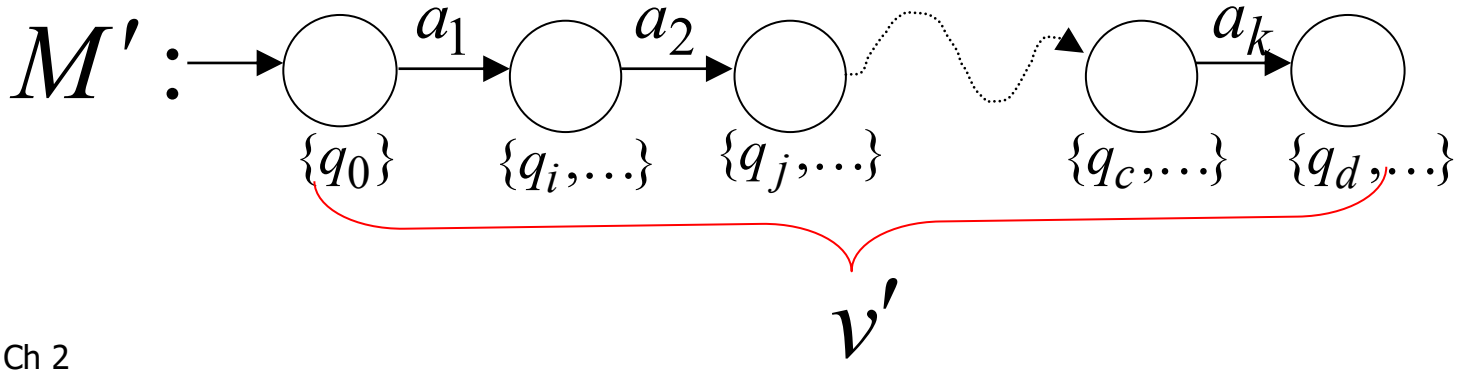


Induction Step: $|v| = k+1$

NFA M
DFA M'

$$v = \underbrace{a_1 a_2 \cdots a_k}_{v'} a_{k+1} = v' a_{k+1}$$

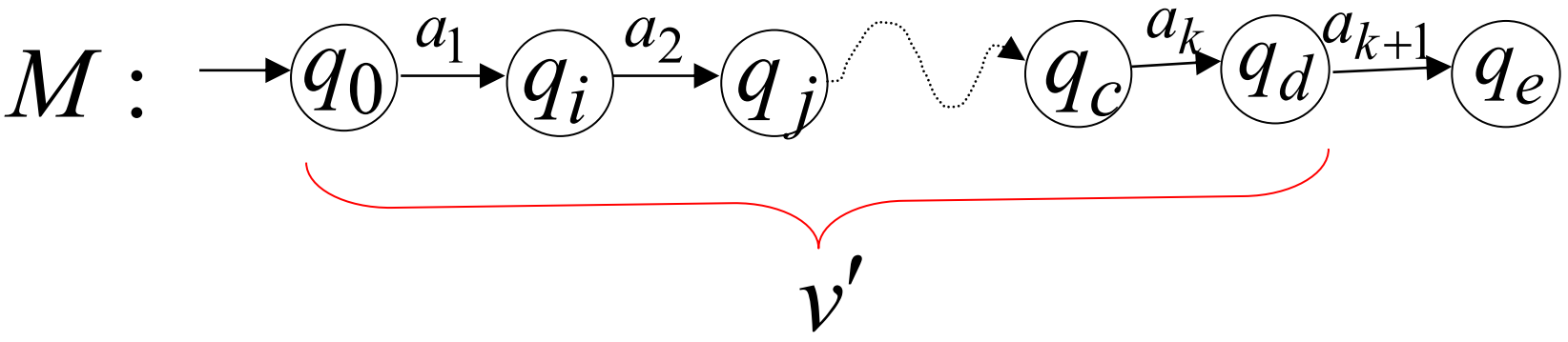


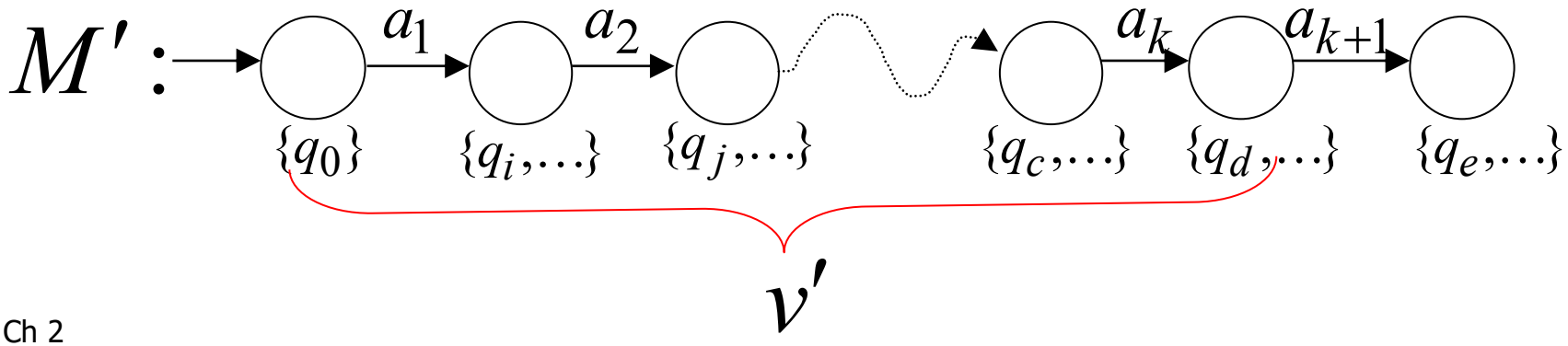


Induction Step: $|v| = k+1$

NFA M
DFA M'

$$v = \underbrace{a_1 a_2 \cdots a_k}_{v'} a_{k+1} = v' a_{k+1}$$

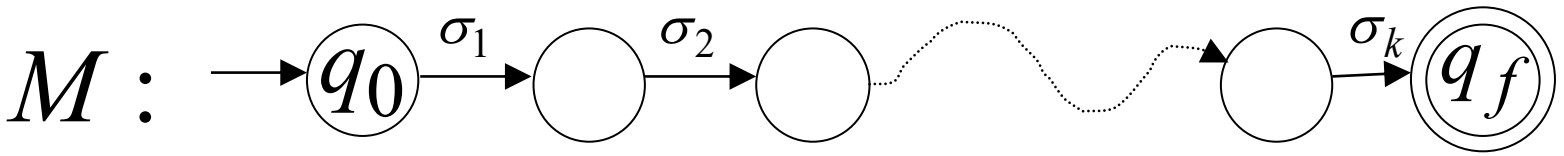


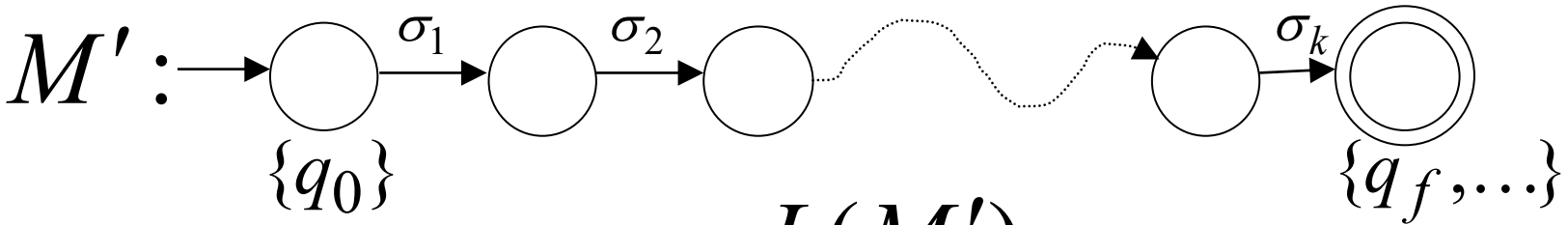


NFA M
DFA M'

Therefore if $w \in L(M)$

$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$





$w \in L(M')$

NFA M
DFA M'

We have shown: $L(M) \subseteq L(M')$

We also need to show: $L(M) \supseteq L(M')$

(proof is similar)

NFAs accept the Regular Languages

Exercise 2.3.7

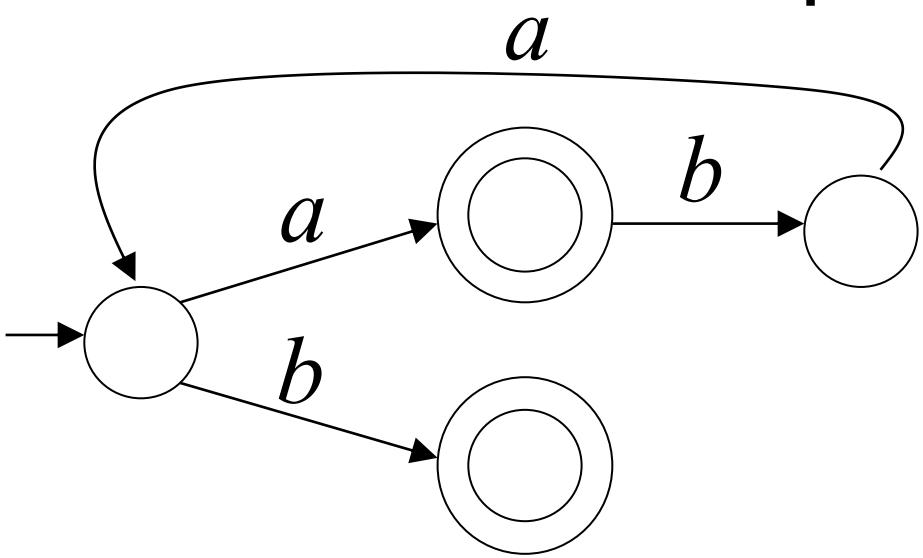
Any NFA can be converted

to an equivalent NFA

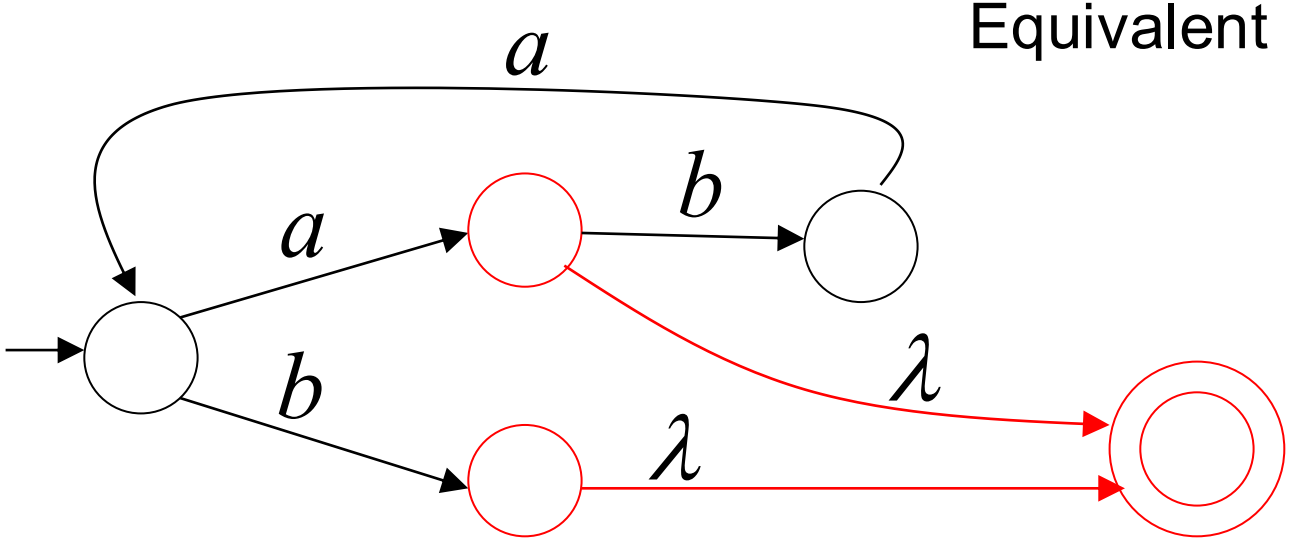
with a single final state

Example

NFA

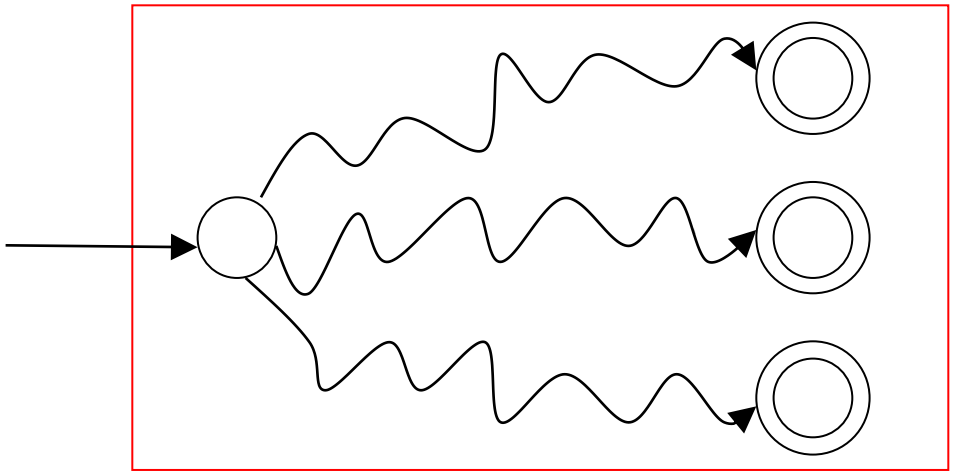


Equivalent NFA

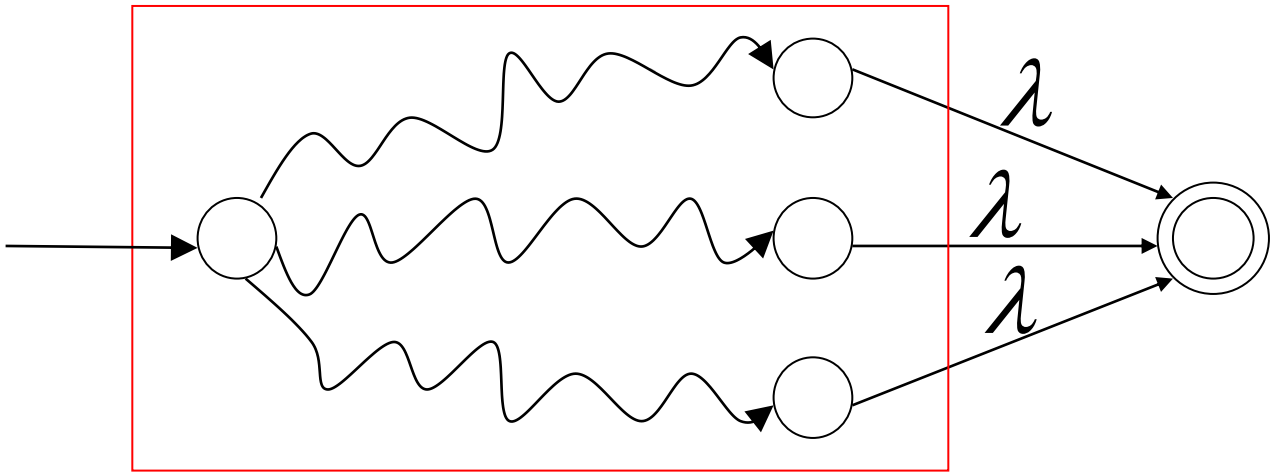


In General

NFA



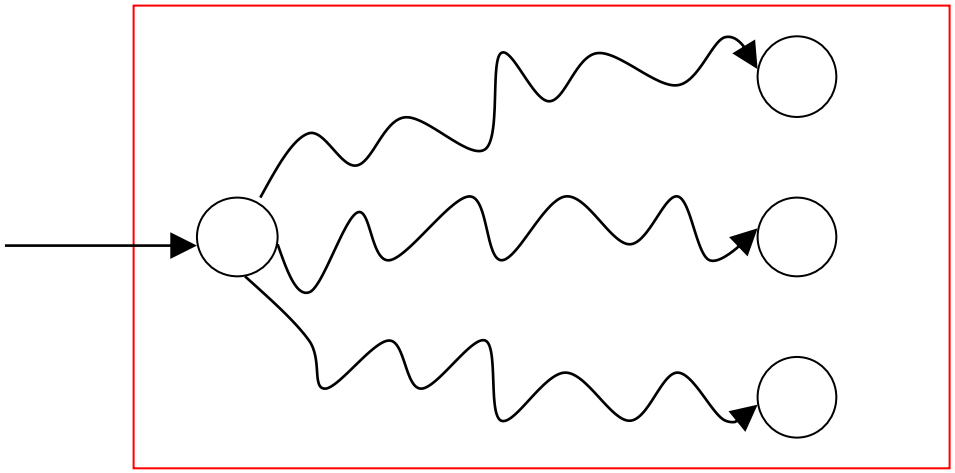
Equivalent NFA

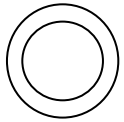


Single
final state

Extreme Case

NFA without final state (it accepts ϕ)





Add a final state
Without transitions

Outline



Deterministic Finite Accepters (DFA)



Nondeterministic Finite Accepters (NFA)

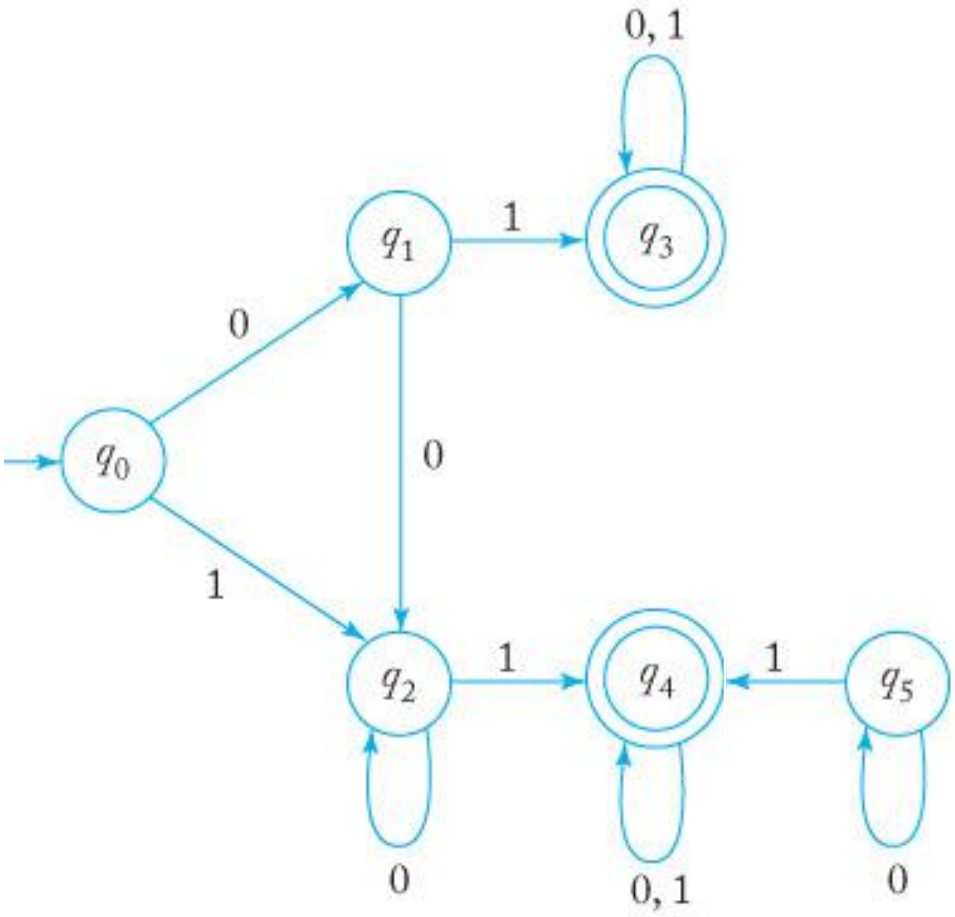


Equivalence of DFA and NFA



Reduction of the Number of States in FA*

Example 2.14



(a)

$\delta(q_0, 0) = q_1$

$\delta(q_0, 1) = q_2$

Inaccessible state

Definition 2.8

Two steps p and q of a DFA are called indistinguishable if

$$\delta^*(p,w) \in F \text{ implies } \delta^*(q,w) \in F,$$

And

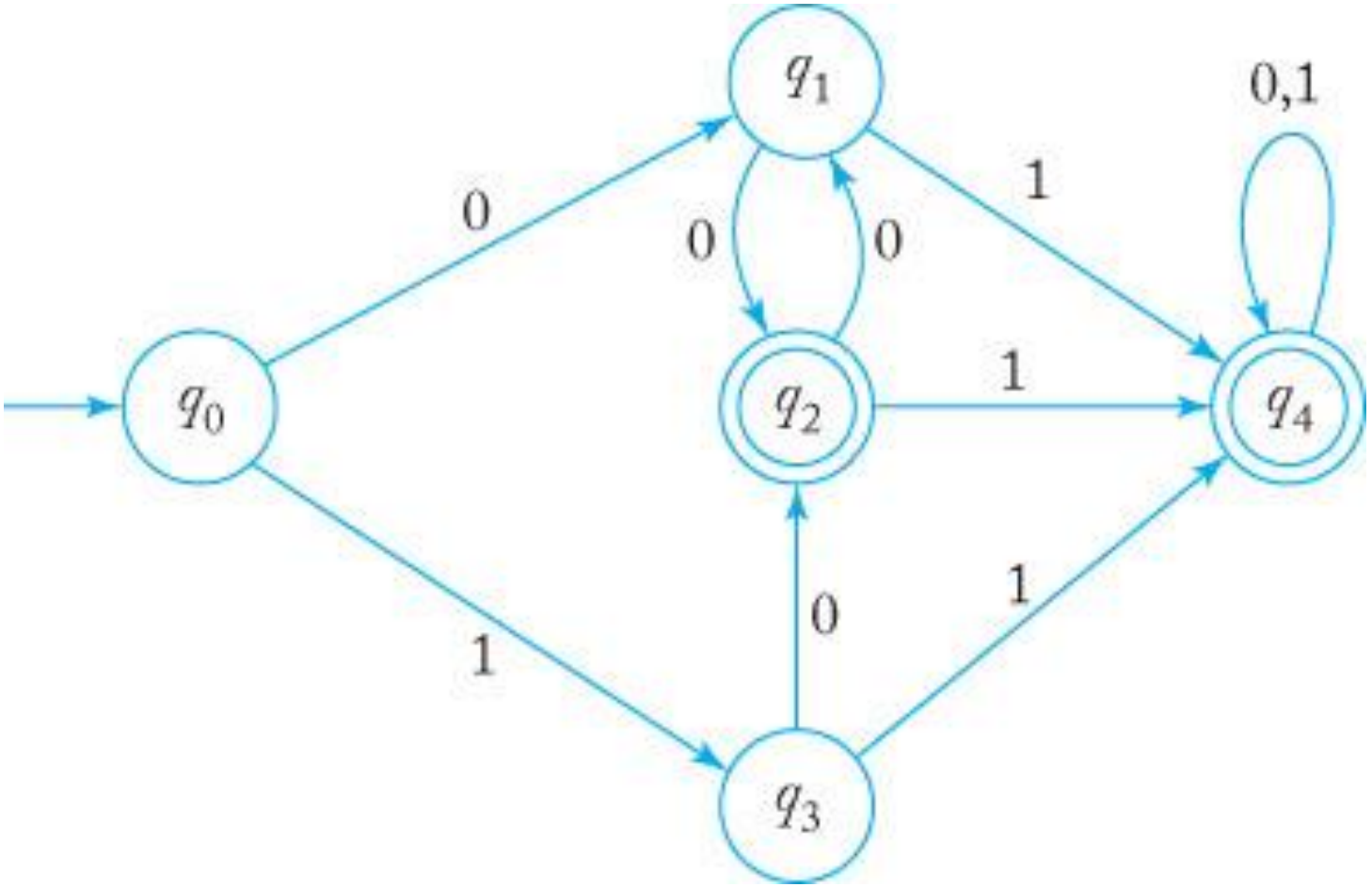
$$\delta^*(p,w) \notin F \text{ implies } \delta^*(q,w) \notin F,$$

For all w ϵ Σ*. If no the other hand, there exists some string w ϵ Σ* such that

$$\delta^*(p,w) \in F \text{ implies } \delta^*(q,w) \notin F,$$

Or vice versa, then the state p and q are said to be distinguishable by a string w.

Example 2.15

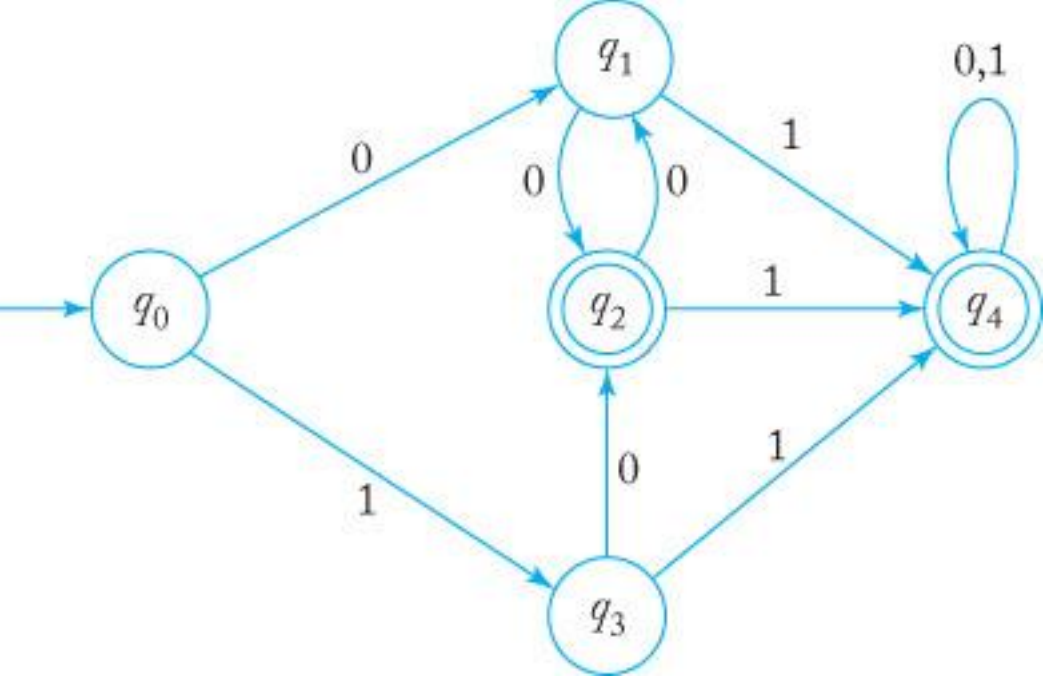


$\{q_0, q_1, q_3\} \notin F$

$\{q_2, q_4\} \in F$

$\delta(q_0, 0) = q_1$
$\delta(q_1, 0) = q_2$

Thus q_0 and q_1 are distinguishable
$\{q_0, q_1, q_3\} \rightarrow$
$\{q_0\}\ \{q_1, q_3\}$

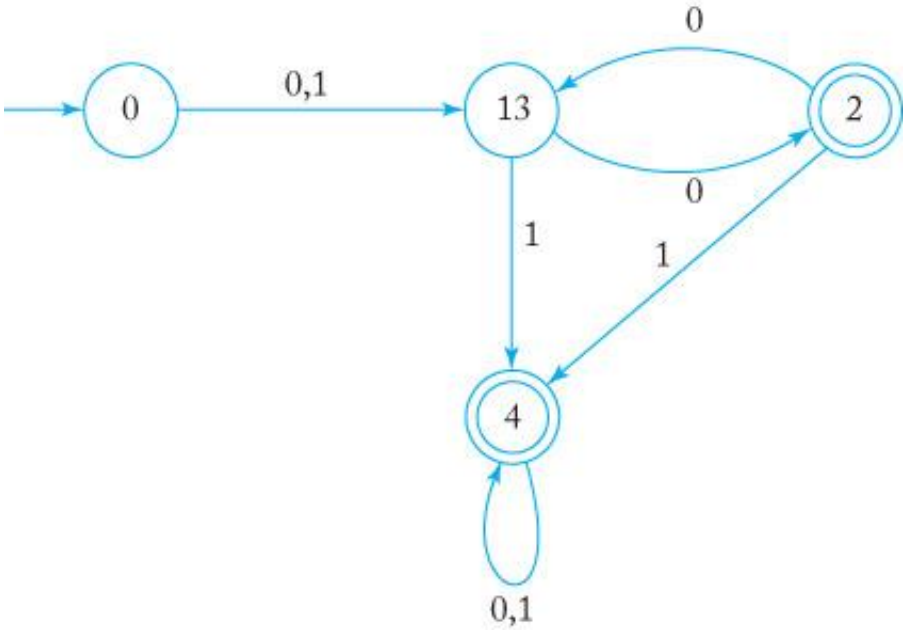


$\delta(q_0, 0) = q_1$
$\delta(q_1, 0) = q_2$

Thus q_0 and q_1 are distinguishable
$\{q_0, q_1, q_3\} \rightarrow$
$\{q_0\}\ \{q_1, q_3\}$

$\delta(q_2, 0) = q_1$
$\delta(q_4, 0) = q_4$

Thus q_2 and q_4 are distinguishable
$\{q_2, q_4\} \rightarrow$
$\{q_2\}\ \{q_4\}$



Questions?